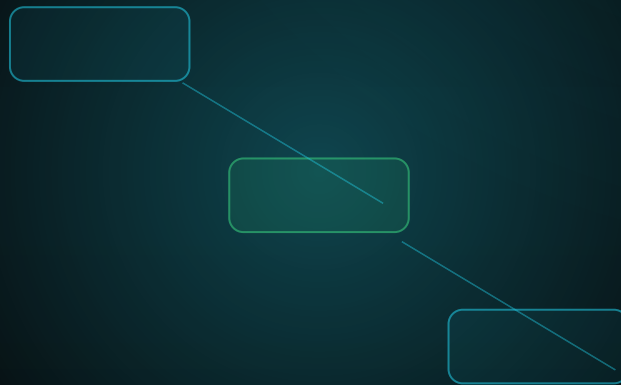


FROM ENTITY TO CONTEXT

The EF Core Configuration Atlas of the NEXUS-1 Digital Twin



TECHNICAL REFERENCE COMPANION

One entity - one mapping - clean context

Enterprise EF Core configuration files for the NEXUS-1 schema atlas.

From Entity to Context

The EF Core Configuration Atlas of the NEXUS-1 Digital Twin
Clean DbContext design, IEntityTypeConfiguration classes, schema
folders, explicit names, defaults, indexes, relationships, migrations,
and enterprise Code First discipline.

Grigorios Agathangelidis

A NEXUS-1 technical reference companion - complete build

This volume describes an EF Core mapping approach for a demonstrator and companion literature. It is not safety-class software, not certified operational code, and not connected to any real facility.

Preface

Why this book exists

From Schema to System wrote down the SQL truth of NEXUS-1. This book writes the C# and EF Core configuration truth above it.

In a small application, all mapping rules can hide inside `OnModelCreating`. In an enterprise project that style does not survive. The context becomes noisy, index names drift, defaults are forgotten, and migrations stop reading like a deliberate contract.

The approach here is one entity, one configuration file, one schema folder, one clean context. The entity remains readable. The configuration file carries persistence discipline. The migration becomes reviewable.

Reading order: Chapters 1-5 establish the configuration-class discipline. The bridge chapter refreshes EF Core Code First. The atlas chapters then map all 17 NEXUS-1 schemas.

Contents

PART I - FOUNDATIONS

1	Why Configuration Classes Matter
2	The Clean DbContext
3	Enterprise EF Core Mapping Conventions
4	Folder Structure by Schema
5	ReactorFleet.Unit Configuration Walkthrough
Bridge	Entity Framework Core Code-First Reminder

PART II - THE CONFIGURATION ATLAS

6	CorePlatform Configurations
7	Security Configurations
8	Organization Configurations
9	ReactorFleet Configurations
10	Instrumentation Configurations
11	DigitalTwin Configurations
12	AlarmManagement Configurations
13	EventManager Configurations
14	Maintenance Configurations
15	RootCause Configurations
16	ReinforcementLearning Configurations
17	Robotics Configurations
18	RadiationMonitoring Configurations
19	EmergencyPreparedness Configurations
20	Compliance Configurations
21	Reporting Configurations
22	Audit Configurations

BACK MATTER

-	Epilogue, references, and indexes
----------	-----------------------------------

1

Why configuration classes matter

EF Core can infer many things. Enterprise software should not make it infer the important things.

Conventions are useful, but they are not a substitute for architecture. If a table has a business code, the length should be visible. If an index is unique, its name should be stable. If a timestamp is created by SQL Server, the default should be written explicitly. If a relationship is restricted, the delete behavior should not be an accident.

For NEXUS-1, the scale makes this discipline unavoidable. The schema atlas contains **654 tables**. Treating all mappings as a single block inside `OnModelCreating` would hide the model behind noise. Treating each mapping as a separate file turns the model into a searchable body of evidence.

The rule

One database table should have one entity class and one configuration class unless there is a very strong reason to do otherwise.

CONCERN	WHERE IT BELONGS	REASON
Entity identity and navigation	Entity class	The C# model should express what the object is and how it connects.
Column length, precision, defaults	Configuration class	Persistence details belong in mapping, not in the business object.
Index names and FK names	Configuration class	Stable migration output requires explicit database names.
Schema-level grouping	Folder and namespace	The code should mirror the bounded contexts of the database.
Discovery	DbContext	The context should discover mappings, not contain all of them.

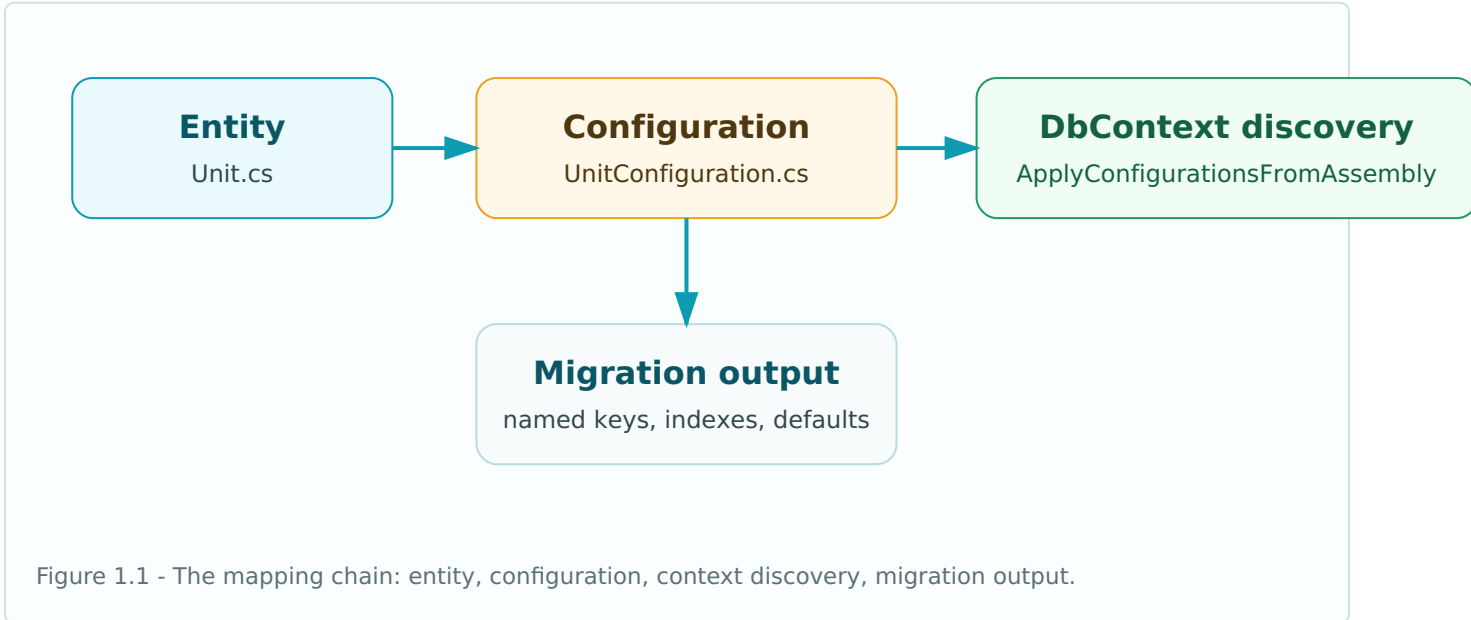
The anti-pattern

The anti-pattern is not using `OnModelCreating`. EF Core requires that method. The anti-pattern is allowing it to become the book of everything.

- A 2,000-line context cannot be reviewed safely.
- A generated migration with anonymous constraint names is difficult to reason about.

- A mapping change buried inside a giant method is easy to miss in code review.
- A developer working on ReactorFleet should not need to scroll through Security, Reporting, and Audit mappings.

A configuration atlas gives every entity a file, every file a clear purpose, and every schema a natural home.



2

The clean DbContext

The DbContext is the coordinator of persistence. It should not be the warehouse of every mapping rule.

The most important line in this book is not a property mapping. It is the line that tells EF Core to discover all configuration classes in the assembly.

```
public sealed class NexusContext : DbContext
{
    public NexusContext(DbContextOptions<NexusContext> options)
        : base(options)
    {
    }

    public DbSet<Unit> Units => Set<Unit>();
    public DbSet<Reactor> Reactors => Set<Reactor>();
    public DbSet<Signal> Signals => Set<Signal>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(NexusContext).Assembly);
    }
}
```

This keeps the context readable. The context exposes sets, participates in dependency injection, and applies all mappings. It does not need to know the details of every index, foreign key, precision, default value, and delete behavior.

Why the one-line discovery matters

BENEFIT	EFFECT IN A LARGE PROJECT
Reviewability	A change to <code>UnitConfiguration</code> can be reviewed without unrelated mappings.
Team ownership	Developers can work in schema folders that match their bounded context.
Migration stability	Explicit names prevent EF-generated names from becoming unreadable artifacts.
Testability	Configuration can be inspected through model metadata and migration assertions.
Scale	The same pattern works for 20 entities or 654 entities.

Assembly marker variation

If the context and configurations live in the same assembly, `typeof(NexusContext).Assembly` is enough. If the mappings live in a separate infrastructure project, an assembly marker class makes the intent clearer.

```
// Optional assembly marker if the context lives in another project.
public sealed class NexusPersistenceAssemblyMarker
{
    private NexusPersistenceAssemblyMarker() { }
}

protected override void OnModelCreating(ModelBuilder m)
{
    m.ApplyConfigurationsFromAssembly(
        typeof(NexusPersistenceAssemblyMarker).Assembly);
}
```

The important point is not which marker you choose. The important point is that discovery is centralized and configuration remains distributed.

3

Enterprise EF Core mapping conventions

A configuration atlas is useful only if every file follows the same grammar.

The following conventions are the starting contract for this book. They are deliberately practical and SQL Server friendly.

PATTERN	CONVENTION	EXAMPLE
Table mapping	Always declare table and schema	<code>b.ToTable("Unit", "ReactorFleet")</code>
Primary key name	<code>PK_Schema_Table</code>	<code>PK_ReactorFleet_Unit</code>
Unique index name	<code>UQ_Schema_Table_Column</code>	<code>UQ_ReactorFleet_Unit_Code</code>
Nonunique index name	<code>IX_Schema_Table_Column</code>	<code>IX_ReactorFleet_Unit_UnitStatusId</code>
Foreign key name	<code>FK_Schema_Table_Column</code>	<code>FK_ReactorFleet_Unit_UnitTypeId</code>
Date defaults	SQL Server UTC default	<code>SYSDATETIME()</code>
Concurrency	Use SQL Server rowversion	<code>IsRowVersion()</code>
Delete behavior	Restrict by default across lookups and passports	<code>OnDelete(DeleteBehavior.Restrict)</code>

String lengths

Strings should not be left as unlimited unless the column is genuinely narrative text or JSON. Codes, names, descriptions, notes, and hashes should have deliberate lengths.

COLUMN KIND	TYPICAL LENGTH	REASON
Short code	20-40	Business codes, type codes, status codes.
Name	120-200	Readable display names.
Description	500-1000	Short explanations, not long documents.
JSON payload	nvarchar(max)	Flexible payloads, snapshots, settings.
Hash	64 or binary(32)	Stable fingerprints and audit hashes.

Decimals and physical values

Engineering data should not rely on provider defaults. Power, pressure, flow, temperature, dose, and probability values should use explicit precision.

```
b.Property(e => e.NominalPowerMwe)
    .HasPrecision(10, 3);

b.Property(e => e.ThermalPowerMwth)
    .HasPrecision(10, 3);

b.Property(e => e.AvailabilityFactor)
    .HasPrecision(6, 5);
```

Lookup base configuration

Many lookup entities share the same shape: code, name, description, sort order, active flag. The atlas can use a base configuration where appropriate, while still letting each lookup file declare its own table and unique constraints.

```
public abstract class LookupConfiguration<TEntity> : IEntityTypeConfiguration<TEntity>
    where TEntity : LookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(40)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(160)
            .IsRequired();

        b.Property(e => e.SortOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);
    }
}
```

4

Folder structure by schema

The folder tree should read like the database boundary map.

For NEXUS-1, the database has seventeen schemas. The EF Core configuration layer should not flatten them. A developer opening the infrastructure project should see the same boundaries that appear in the SQL atlas.

```
Nexus.Infrastructure/  
  Persistence/  
    NexusContext.cs  
    Configurations/  
      CorePlatform/  
        AppSettingConfiguration.cs  
        EngineeringUnitConfiguration.cs  
      ReactorFleet/  
        UnitConfiguration.cs  
        ReactorConfiguration.cs  
        ReactorCoreConfiguration.cs  
    Instrumentation/  
      SignalConfiguration.cs  
      SignalReadingConfiguration.cs  
    DigitalTwin/  
      TwinModelConfiguration.cs  
      SignalBindingConfiguration.cs  
    Audit/  
      AuditEntryConfiguration.cs  
      EntityChangeConfiguration.cs
```

Namespace convention

LAYER	EXAMPLE NAMESPACE
Domain entity	<code>Nexus.Domain.ReactorFleet</code>
Configuration class	<code>Nexus.Infrastructure.Persistence.Configurations.ReactorFleet</code>
DbContext	<code>Nexus.Infrastructure.Persistence</code>
Migration assembly	<code>Nexus.Infrastructure.Migrations</code>

Configuration naming

The convention is simple:

Entity
Unit

Configuration
UnitConfiguration

File`UnitConfiguration.cs`**Folder**`Configurations/ReactorFleet`

This may look boring. That is the point. Good enterprise structure is often boring because it removes surprise.

5

ReactorFleet.Unit configuration walkthrough

The `Unit` table is a good first example because it sits near the core of the platform. Many other sectors will eventually refer to a physical unit.

Entity sketch

The entity carries identity, business code, status/type links, lifecycle columns, concurrency token, and navigation properties. It does not carry SQL-specific naming rules.

```
public sealed class Unit
{
    public int UnitId { get; set; }
    public string Code { get; set; } = null!;
    public string Name { get; set; } = null!;
    public int UnitTypeId { get; set; }
    public int UnitStatusId { get; set; }
    public DateTime CreatedDate { get; set; }
    public DateTime? UpdatedDate { get; set; }
    public byte[] RowVersion { get; set; } = Array.Empty<byte>();

    public UnitType UnitType { get; set; } = null!;
    public UnitStatus UnitStatus { get; set; } = null!;
    public ICollection<Reactor> Reactors { get; } = new List<Reactor>();
}
```

Configuration file

The configuration file makes the database contract explicit: schema, table, key name, string lengths, unique code index, defaults, and rowversion.

```
public sealed class UnitConfiguration : IEntityTypeConfiguration<Unit>
{
    public void Configure(EntityTypeBuilder<Unit> b)
    {
        b.ToTable("Unit", "ReactorFleet");

        b.HasKey(e => e.UnitId)
            .HasName("PK_ReactorFleet_Unit");

        b.Property(e => e.Code)
            .HasMaxLength(20)
            .IsRequired();

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_Unit_Code"); // no more hex

        b.Property(e => e.Name)
            .HasMaxLength(120)
            .IsRequired();
    }
}
```

```

        b.Property(e => e.CreatedDate)
            .HasDefaultValueSql("SYSUTCDATETIME()"); // default restored

        b.Property(e => e.RowVersion)
            .IsRowVersion()
            .IsConcurrencyToken();
    }
}

```

Lookup relationships

The `Unit` entity points to lookup tables for type and status. These relationships are required and restricted. A unit type should not disappear simply because a unit exists.

```

b.HasOne(e => e.UnitType)
    .WithMany()
    .HasForeignKey(e => e.UnitTypeId)
    .HasConstraintName("FK_ReactorFleet_Unit_UnitTypeId")
    .OnDelete(DeleteBehavior.Restrict);

b.HasOne(e => e.UnitStatus)
    .WithMany()
    .HasForeignKey(e => e.UnitStatusId)
    .HasConstraintName("FK_ReactorFleet_Unit_UnitStatusId")
    .OnDelete(DeleteBehavior.Restrict);

```

What this single file teaches

LINE OF MAPPING	ARCHITECTURAL MEANING
<code>ToTable("Unit", "ReactorFleet")</code>	The entity belongs to a bounded context, not to the default schema.
<code>HasName("PK_ReactorFleet_Unit")</code>	The primary key is named for humans and migrations.
<code>HasMaxLength(20).IsRequired()</code>	The business code has an explicit contract.
<code>HasDatabaseName("UQ_ReactorFleet_Unit_Code")</code>	The unique index has a stable name; there is no unreadable generated artifact.
<code>HasDefaultValueSql("SYSUTCDATETIME()")</code>	The database owns the creation timestamp.
<code>IsRowVersion()</code>	The table participates in optimistic concurrency.

Why this matters at NEXUS-1 scale

Doing this once is tidy. Doing it 654 times is architecture. The value of the atlas is not the existence of a single configuration file. The value is that every entity follows a comparable discipline. A programmer can open `SignalConfiguration`, `TwinModelConfiguration`, `AlarmEventConfiguration`, or `AuditEntryConfiguration` and know what to expect.

This starter chapter is intentionally small. The full atlas would repeat the pattern for every schema, every entity, every lookup, and every relationship in the NEXUS-1 data backbone.

Bridge Chapter

Entity Framework Core Code-First

Reminder

Before the atlas continues, this short chapter resets the essential Code First ideas. It is not a full EF Core course; it is the minimum needed to read the configuration files that follow.

What Code First means here

Code First does not mean the database is an afterthought. In a professional project, Code First means the model is expressed in C# first, but the generated SQL Server database is still treated as a contract. A table name is not accidental. A foreign key is not accidental. A default value is not accidental. A unique index is not decoration; it is a rule.

Entity

The C# object the application works with: properties, navigation properties, and domain-facing names.

Configuration

The persistence contract: table, schema, keys, lengths, indexes, defaults, constraints, relationships, and delete behavior.

Migration

The versioned change set that turns the current model into SQL and lets a reviewer see exactly what changed.

Entity classes stay readable

An entity class should be understandable before any database details are added. It should not become a crowded list of persistence attributes. In this book, the entity remains close to the application language, and the mapping file carries the database details.

```
public sealed class Unit
{
    public int UnitId { get; set; }
    public int PlantId { get; set; }
    public string Code { get; set; } = null!;
    public string Name { get; set; } = null!;
    public decimal RatedPowerMw { get; set; }
    public DateTime CreatedDate { get; set; }

    public Plant Plant { get; set; } = null!;
    public ICollection<SystemNode> Systems { get; set; } = new List<SystemNode>();
}
```

This class tells the programmer what a unit is. It does not carry the schema name, index name, decimal precision, SQL default, or relationship behavior. Those belong in the configuration file.

Configuration classes make persistence explicit

The configuration class implements `IEntityTypeConfiguration<TEntity>`. That one interface is the hinge of the book. It lets every table have a separate mapping file while the `DbContext` stays clean.

```
public sealed class UnitConfiguration : IEntityTypeConfiguration<Unit>
{
    public void Configure(EntityTypeBuilder<Unit> b)
    {
        b.ToTable("Unit", "ReactorFleet");

        b.HasKey(e => e.UnitId)
            .HasName("PK_ReactorFleet_Unit");

        b.Property(e => e.Code)
            .HasMaxLength(20)
            .IsRequired();

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_Unit_Code");

        b.Property(e => e.RatedPowerMw)
            .HasPrecision(10, 3);

        b.Property(e => e.CreatedDate)
            .HasDefaultValueSql("SYSUTCDATETIME()");
    }
}
```

The important point is visibility. If a migration creates an unexpected index name, cascade delete, or shadow column, the configuration file is where intent is corrected.

The clean DbContext

The context should coordinate persistence. It should not become the warehouse of every mapping rule. It declares sets and applies all mapping classes from the assembly.

```
public sealed class NexusContext : DbContext
{
    public DbSet<Unit> Units => Set<Unit>();
    public DbSet<Signal> Signals => Set<Signal>();
    public DbSet<AlarmEvent> AlarmEvents => Set<AlarmEvent>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(NexusContext).Assembly);
    }
}
```

Enterprise habit: when a mapping changes, open the entity configuration file, not the `DbContext`. The context should almost never be the place where table-level detail accumulates.

Relationships and delete behavior

EF Core can infer relationships from foreign key and navigation properties, but enterprise systems should not rely on inference for important relationships. The mapping should show who points to whom, which property is the foreign key, and what happens on delete.

```
b.HasOne(e => e.EngineeringUnit)
  .WithMany()
  .HasForeignKey(e => e.EngineeringUnitId)
  .OnDelete(DeleteBehavior.Restrict)
  .HasConstraintName("FK_Signal_EngineeringUnit");
```

In NEXUS-1, `Restrict` is usually the safest default across schemas. Shared reference rows such as engineering units, languages, countries, alarm severities, policy statuses, and audit action types should not disappear because one dependent row is removed.

Migrations are the audit trail of model intent

A migration is not only a mechanical artifact. In a Code First project, it is the reviewable history of schema intent. Stable names make that possible.

```
dotnet ef migrations add AddReactorFleetUnit

dotnet ef database update

dotnet ef migrations script --idempotent --output artifacts/sql/Nexus_Upgrade.sql
```

A clean migration should not surprise the reviewer. It should create the expected table, primary key, index names, default SQL, and foreign key constraints. If it creates a shadow column, random cascade, or unreadable generated name, the configuration is not complete yet.

Fluent API before attributes

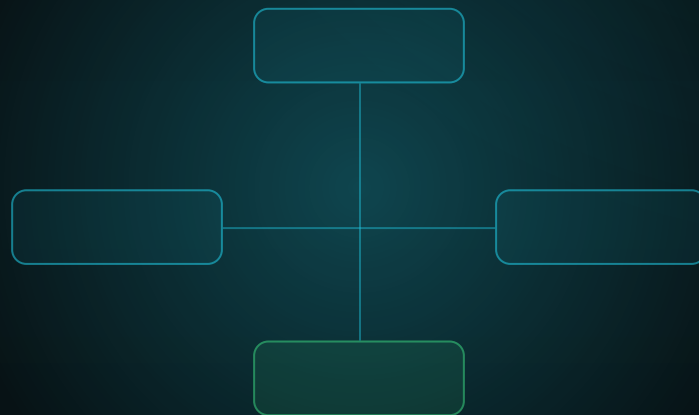
Attributes are convenient, but Fluent API is the language of this atlas. It keeps persistence detail in one place and supports rules that attributes cannot express cleanly: composite indexes, named foreign keys, delete behavior, precision, default SQL, conversion, and provider-specific mapping.

CONCERN	ENTITY CLASS	CONFIGURATION CLASS
Business property	Yes	Optional
Max length	Prefer no	Yes
Index name	No	Yes
Foreign-key name	No	Yes
Delete behavior	No	Yes
Default SQL	No	Yes
Precision	No	Yes

Honest boundary: Code First is not automatically better than Database First. It is better only when configuration is explicit, migrations are reviewed, and the generated SQL is treated as a contract.

FROM ENTITY TO CONTEXT

CorePlatform Configuration Files - The EF Core Configuration Atlas of the
NEXUS-1 Digital Twin



STEP 1 - COREPLATFORM SCHEMA

Reference data, settings, units, localization, and version metadata

Configuration classes that keep the DbContext clean and the SQL names stable.

From Entity to Context

CorePlatform Configuration Files
Step-by-step EF Core mapping atlas for the NEXUS-1 data
backbone.

Grigorios Agathangelidis

A NEXUS-1 technical reference companion - CorePlatform build

This document is a software architecture reference for EF Core mapping. It is not safety-class software, not certified operational code, and not connected to any real facility.

Chapter 6

CorePlatform Configurations

The CorePlatform schema is the smallest sector that touches almost everything. It holds settings, feature gates, languages, localization, geography, calendars, units of measure, and version metadata.

From Schema to System wrote this sector as SQL. This step writes the EF Core configuration layer for the same sector. The objective is not to decorate the entities with every database detail. The objective is to keep each mapping file honest, reviewable, and migration-friendly.

CorePlatform is where a programmer first sees the rule of this book: the entity stays readable, the configuration file carries persistence discipline, and the DbContext discovers the result.

What this schema teaches

Typed reference data

Language, Country, Region, TimeZone, Calendar, and EngineeringUnit are separate typed tables, not one generic lookup bucket.

Stable names

Keys, unique indexes, check constraints, and foreign keys get readable names so migrations are understandable.

Careful C# names

Tables named Version and TimeZone are mapped with clearer entity names such as PlatformVersion and PlatformTimeZone.

Configuration file catalogue

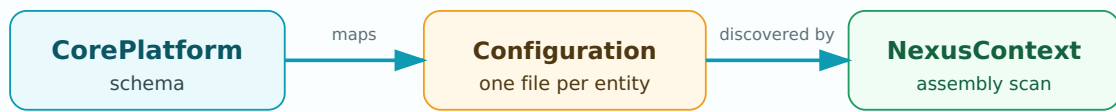
SQL TABLE	SUGGESTED ENTITY	CONFIGURATION FILE	IMPORTANT MAPPING
CorePlatform.AppSettings	AppSetting	AppSettingConfiguration.cs	Unique setting key; check constraint for value parser mode.
CorePlatform.SystemConfiguration	SystemConfiguration	SystemConfigurationConfiguration.cs	JSON check, versioned natural key, effective date window.
CorePlatform.FeatureFlag	FeatureFlag	FeatureFlagConfiguration.cs	Unique code per environment; expiry-window check.
CorePlatform.Language	Language	LanguageConfiguration.cs	BCP 47 language code as unique key.
CorePlatform.Localization	Localization	LocalizationConfiguration.cs	Resource key plus language is unique; FK to Language.
CorePlatform.Country	Country	CountryConfiguration.cs	ISO2 and ISO3 codes are unique fixed-length columns.
CorePlatform.Region	Region	RegionConfiguration.cs	Country plus region code is unique; FK to Country.
CorePlatform.TimeZone	PlatformTimeZone	PlatformTimeZoneConfiguration.cs	IANA name unique; UTC offset range constraint.
CorePlatform.Calendar	PlantCalendar	PlantCalendarConfiguration.cs	Calendar code unique; FK to PlatformTimeZone.
CorePlatform.EngineeringUnit	EngineeringUnit	EngineeringUnitConfiguration.cs	Unit symbol unique; decimal precision for SI conversion.
CorePlatform.Version	PlatformVersion	PlatformVersionConfiguration.cs	Component plus version unique; component type check.

Folder structure for this step

```
Nexus.Infrastructure/  
  Persistence/  
    NexusContext.cs  
  Configurations/  
    Shared/  
      NexusSql.cs  
      CorePlatformLookupConfiguration.cs  
    CorePlatform/  
      AppSettingConfiguration.cs  
      SystemConfigurationConfiguration.cs  
      FeatureFlagConfiguration.cs  
      LanguageConfiguration.cs  
      LocalizationConfiguration.cs  
      CountryConfiguration.cs  
      RegionConfiguration.cs  
      PlatformTimeZoneConfiguration.cs  
      PlantCalendarConfiguration.cs  
      EngineeringUnitConfiguration.cs  
      PlatformVersionConfiguration.cs
```

Why entity names may differ from table names. In SQL Server, CorePlatform.Version and CorePlatform.TimeZone are clear. In C#, Version and TimeZone can be confused with .NET framework types or general concepts. The configuration file solves this cleanly: PlatformVersion maps to table Version; PlatformTimeZone maps to table TimeZone.

CONFIGURATION DISCOVERY



INTERNAL FK PASSPORTS

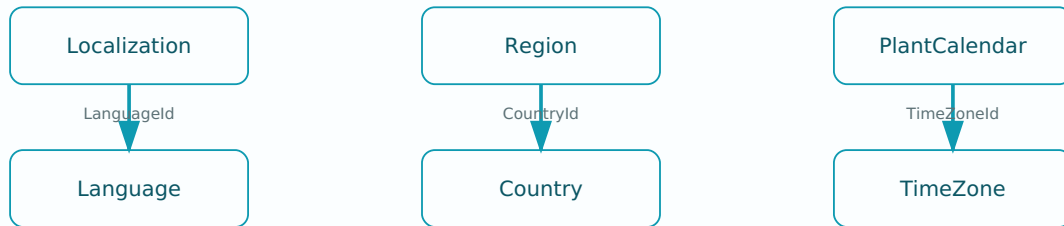


Figure 6.1 - CorePlatform configuration discovery and its three internal passports.

Shared configuration support

The shared file is deliberately modest. It provides the schema name, the UTC default expression, and a base configuration shape for simple lookup-style entities. It does not hide table-specific keys, relationships, or check constraints.

```
namespace Nexus.Infrastructure.Persistence.Configurations;

internal static class NexusSql
{
    public const string CorePlatform = "CorePlatform";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class CorePlatformLookupConfiguration<TEntity>
    : IEntityTypeConfiguration<TEntity>
    where TEntity : LookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(40)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(160)
            .IsRequired();

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}
```

Default constraint names. EF Core versions before native named default-constraint support usually generate provider names for default constraints. The configuration still restores the default SQL expression. If exact default constraint names are mandatory, enforce them in migrations or in the SQL-first script.

AppSettingConfiguration.cs

`AppSetting` is flexible but not shapeless. The mapping limits string lengths, restores UTC defaults, makes the key unique, and keeps the parser mode constrained.

```
namespace Nexus.Infrastructure.Persistence.Configurations.CorePlatform;

public sealed class AppSettingConfiguration
    : IEntityTypeConfiguration<AppSetting>
{
    public void Configure(EntityTypeBuilder<AppSetting> b)
    {
        b.ToTable("AppSetting", NexusSql.CorePlatform, t =>
        {
            t.HasCheckConstraint(
                "CK_CorePlatform_AppSetting_ValueType",
                "ValueType IN (N'String', N'Int', N'Bool', N'Json')");
        });

        b.HasKey(e => e.AppSettingId)
            .HasName("PK_CorePlatform_AppSetting");

        b.Property(e => e.Key)
            .HasMaxLength(200)
            .IsRequired();

        b.HasIndex(e => e.Key)
            .IsUnique()
            .HasDatabaseName("UQ_CorePlatform_AppSetting_Key");

        b.Property(e => e.Value)
            .HasMaxLength(2000)
            .IsRequired();

        b.Property(e => e.ValueType)
            .HasMaxLength(20)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.IsEncrypted)
            .HasDefaultValue(false);

        b.Property(e => e.IsSystem)
            .HasDefaultValue(false);

        b.Property(e => e.IsDeleted)
            .HasDefaultValue(false);

        b.Property(e => e.CreatedBy)
            .HasMaxLength(100)
            .IsRequired();

        b.Property(e => e.ModifiedBy)
            .HasMaxLength(100)
            .IsRequired();

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}
```

SystemConfigurationConfiguration.cs

`SystemConfiguration` stores JSON contracts by module, key, and schema version. The configuration makes that natural key unique and guards the JSON and effective-date window.

```
public sealed class SystemConfigurationConfiguration
    : IEntityTypeConfiguration<SystemConfiguration>
{
    public void Configure(EntityTypeBuilder<SystemConfiguration> b)
    {
        b.ToTable("SystemConfiguration", NexusSql.CorePlatform, t =>
        {
            t.HasCheckConstraint(
                "CK_CorePlatform_SystemConfiguration_SchemaVersion_Positive",
                "SchemaVersion > 0");

            t.HasCheckConstraint(
                "CK_CorePlatform_SystemConfiguration_EffectiveWindow",
                "EffectiveToUtc IS NULL OR EffectiveToUtc > EffectiveFromUtc");

            t.HasCheckConstraint(
                "CK_CorePlatform_SystemConfiguration_Json",
                "ISJSON(ConfigurationJson) = 1");
        });

        b.HasKey(e => e.SystemConfigurationId)
            .HasName("PK_CorePlatform_SystemConfiguration");

        b.Property(e => e.ModuleName)
            .HasMaxLength(100)
            .IsRequired();

        b.Property(e => e.ConfigurationKey)
            .HasMaxLength(150)
            .IsRequired();

        b.Property(e => e.ConfigurationJson)
            .IsRequired();

        b.HasIndex(e => new { e.ModuleName, e.ConfigurationKey, e.SchemaVersion })
            .IsUnique()
            .HasDatabaseName("UQ_CorePlatform_SystemConfiguration_Module_Key_Version");

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.EffectiveFromUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.EffectiveToUtc)
            .HasColumnType("datetime2(7)");

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.CreatedBy).HasMaxLength(100).IsRequired();
        b.Property(e => e.ModifiedBy).HasMaxLength(100).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.IsDeleted).HasDefaultValue(false);
        b.Property(e => e.RowVersion).IsRowVersion();
    }
}
```

FeatureFlagConfiguration.cs

`FeatureFlag` is environment-aware. The same feature code may exist in multiple environments, but not twice in the same one.

```
public sealed class FeatureFlagConfiguration
    : IEntityTypeConfiguration<FeatureFlag>
{
    public void Configure(EntityTypeBuilder<FeatureFlag> b)
    {
        b.ToTable("FeatureFlag", NexusSql.CorePlatform, t =>
        {
            t.HasCheckConstraint(
                "CK_CorePlatform_FeatureFlag_ExpiryWindow",
                "ExpiresAtUtc IS NULL OR EnabledFromUtc IS NULL OR ExpiresAtUtc > EnabledFromUtc");
        });

        b.HasKey(e => e.FeatureFlagId)
            .HasName("PK_CorePlatform_FeatureFlag");

        b.Property(e => e.Code).HasMaxLength(80).IsRequired();
        b.Property(e => e.Name).HasMaxLength(150).IsRequired();
        b.Property(e => e.EnvironmentName).HasMaxLength(50).HasDefaultValue("All").IsRequired();
        b.Property(e => e.Description).HasMaxLength(500);
        b.Property(e => e.Owner).HasMaxLength(150);
        b.Property(e => e.IsEnabled).HasDefaultValue(false);
        b.Property(e => e.IsDeleted).HasDefaultValue(false);

        b.Property(e => e.EnabledFromUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ExpiresAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.CreatedBy).HasMaxLength(100).IsRequired();
        b.Property(e => e.ModifiedBy).HasMaxLength(100).IsRequired();

        b.HasIndex(e => new { e.Code, e.EnvironmentName })
            .IsUnique()
            .HasDatabaseName("UQ_CorePlatform_FeatureFlag_Code_Environment");

        b.Property(e => e.RowVersion).IsRowVersion();
    }
}
```

LanguageConfiguration.cs and LocalizationConfiguration.cs

The localization pair shows the first internal passport in this book: every translated resource points to one language. The FK is named, restricted, and reviewable.

```
public sealed class LanguageConfiguration
    : IEntityTypeConfiguration<Language>
{
    public void Configure(EntityTypeBuilder<Language> b)
    {
        b.ToTable("Language", NexusSql.CorePlatform);

        b.HasKey(e => e.LanguageId)
            .HasName("PK_CorePlatform_Language");

        b.Property(e => e.Code).HasMaxLength(10).IsRequired();
        b.Property(e => e.Name).HasMaxLength(100).IsRequired();
        b.Property(e => e.NativeName).HasMaxLength(100).IsRequired();
        b.Property(e => e.IsRightToLeft).HasDefaultValue(false);
        b.Property(e => e.IsDefault).HasDefaultValue(false);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_CorePlatform_Language_Code");
    }
}
```

```
public sealed class LocalizationConfiguration
    : IEntityTypeConfiguration<Localization>
{
    public void Configure(EntityTypeBuilder<Localization> b)
    {
        b.ToTable("Localization", NexusSql.CorePlatform);

        b.HasKey(e => e.LocalizationId)
            .HasName("PK_CorePlatform_Localization");

        b.Property(e => e.ResourceKey).HasMaxLength(300).IsRequired();
        b.Property(e => e.Value).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.ResourceKey, e.LanguageId })
            .IsUnique()
            .HasDatabaseName("UQ_CorePlatform_Localization_Key_Language");

        b.HasOne(e => e.Language)
            .WithMany(e => e.Localizations)
            .HasForeignKey(e => e.LanguageId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_CorePlatform_Localization_LanguageId");
    }
}
```

CountryConfiguration.cs and RegionConfiguration.cs

The geography pair uses fixed-length ISO codes and a dependent region catalogue. The configuration keeps ISO codes unique and makes the region code unique only inside its country.

```
public sealed class CountryConfiguration
    : IEntityTypeConfiguration<Country>
{
    public void Configure(EntityTypeBuilder<Country> b)
    {
        b.ToTable("Country", NexusSql.CorePlatform);
        b.HasKey(e => e.CountryId).HasName("PK_CorePlatform_Country");

        b.Property(e => e.Iso2Code).HasColumnType("nchar(2)").IsRequired();
        b.Property(e => e.Iso3Code).HasColumnType("nchar(3)").IsRequired();
        b.Property(e => e.NumericCode).HasColumnType("nchar(3)");
        b.Property(e => e.Name).HasMaxLength(150).IsRequired();
        b.Property(e => e.OfficialName).HasMaxLength(250);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Iso2Code).IsUnique().HasDatabaseName("UQ_CorePlatform_Country_Iso2Code");
        b.HasIndex(e => e.Iso3Code).IsUnique().HasDatabaseName("UQ_CorePlatform_Country_Iso3Code");
    }
}

public sealed class RegionConfiguration
    : IEntityTypeConfiguration<Region>
{
    public void Configure(EntityTypeBuilder<Region> b)
    {
        b.ToTable("Region", NexusSql.CorePlatform);
        b.HasKey(e => e.RegionId).HasName("PK_CorePlatform_Region");

        b.Property(e => e.Code).HasMaxLength(40).IsRequired();
        b.Property(e => e.Name).HasMaxLength(150).IsRequired();
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.CountryId, e.Code })
            .IsUnique()
            .HasDatabaseName("UQ_CorePlatform_Region_Country_Code");

        b.HasOne(e => e.Country)
            .WithMany(e => e.Regions)
            .HasForeignKey(e => e.CountryId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_CorePlatform_Region_CountryId");
    }
}
```

PlatformTimeZoneConfiguration.cs and PlantCalendarConfiguration.cs

The table names remain `TimeZone` and `Calendar`, but the C# entity names are more explicit. This avoids ambiguity while preserving the SQL atlas exactly.

```
public sealed class PlatformTimeZoneConfiguration
    : IEntityTypeConfiguration<PlatformTimeZone>
{
    public void Configure(EntityTypeBuilder<PlatformTimeZone> b)
    {
        b.ToTable("TimeZone", NexusSql.CorePlatform, t =>
        {
            t.HasCheckConstraint(
                "CK_CorePlatform_TimeZone_OffsetRange",
                "CurrentUtcOffsetMinutes BETWEEN -840 AND 840");
        });

        b.HasKey(e => e.TimeZoneId).HasName("PK_CorePlatform_TimeZone");
        b.Property(e => e.IanaName).HasMaxLength(100).IsRequired();
        b.Property(e => e.DisplayName).HasMaxLength(150).IsRequired();
        b.Property(e => e.WindowsName).HasMaxLength(100);
        b.Property(e => e.ObservesDst).HasDefaultValue(false);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.IanaName)
            .IsUnique()
            .HasDatabaseName("UQ_CorePlatform_TimeZone_IanaName");
    }
}

public sealed class PlantCalendarConfiguration
    : IEntityTypeConfiguration<PlantCalendar>
{
    public void Configure(EntityTypeBuilder<PlantCalendar> b)
    {
        b.ToTable("Calendar", NexusSql.CorePlatform);
        b.HasKey(e => e.CalendarId).HasName("PK_CorePlatform_Calendar");

        b.Property(e => e.Code).HasMaxLength(60).IsRequired();
        b.Property(e => e.Name).HasMaxLength(150).IsRequired();
        b.Property(e => e.CalendarType).HasMaxLength(50).IsRequired();
        b.Property(e => e.Description).HasMaxLength(500);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_CorePlatform_Calendar_Code");

        b.HasOne(e => e.TimeZone)
            .WithMany(e => e.Calendars)
            .HasForeignKey(e => e.TimeZoneId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_CorePlatform_Calendar_TimeZoneId");
    }
}
```

EngineeringUnitConfiguration.cs and PlatformVersionConfiguration.cs

`EngineeringUnit` is one of the most reused reference tables in the platform. `PlatformVersion` is the operational breadcrumb that says what build, schema, seed data, or service version is installed.

```
public sealed class EngineeringUnitConfiguration
    : IEntityConfiguration<EngineeringUnit>
{
    public void Configure(EntityTypeBuilder<EngineeringUnit> b)
    {
        b.ToTable("EngineeringUnit", NexusSql.CorePlatform, t =>
        {
            t.HasCheckConstraint(
                "CK_CorePlatform_EngineeringUnit_QuantityType",
                "QuantityType IN (N'Power', N'ThermalPower', N'Temperature', " +
                "N'Pressure', N'Reactivity', N'Flow', N'Frequency', N'Percentage', " +
                "N'RadiationDoseRate', N'RadiationDose', N'CountRate', N'Mass', " +
                "N'Time', N'Other')");
        });

        b.HasKey(e => e.EngineeringUnitId)
            .HasName("PK_CorePlatform_EngineeringUnit");

        b.Property(e => e.Symbol).HasMaxLength(20).IsRequired();
        b.Property(e => e.Name).HasMaxLength(100).IsRequired();
        b.Property(e => e.QuantityType).HasMaxLength(50).IsRequired();
        b.Property(e => e.SiConversionFactor).HasPrecision(28, 12);
        b.Property(e => e.SiConversionOffset).HasPrecision(28, 12);
        b.Property(e => e.IsDimensionless).HasDefaultValue(false);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Symbol)
            .IsUnique()
            .HasDatabaseName("UQ_CorePlatform_EngineeringUnit_Symbol");
    }
}

public sealed class PlatformVersionConfiguration
    : IEntityConfiguration<PlatformVersion>
{
    public void Configure(EntityTypeBuilder<PlatformVersion> b)
    {
        b.ToTable("Version", NexusSql.CorePlatform, t =>
        {
            t.HasCheckConstraint(
                "CK_CorePlatform_Version_ComponentType",
                "ComponentType IN (N'Console', N'Schema', N'SeedData', " +
                "N'ApiService', N'Worker', N'Documentation')");
        });

        b.HasKey(e => e.VersionId)
            .HasName("PK_CorePlatform_Version");

        b.Property(e => e.ComponentName).HasMaxLength(120).IsRequired();
        b.Property(e => e.ComponentType).HasMaxLength(50).IsRequired();
        b.Property(e => e.VersionNumber).HasMaxLength(50).IsRequired();
        b.Property(e => e.BuildSignature).HasMaxLength(100);
        b.Property(e => e.GitCommit).HasColumnType("char(40)");
        b.Property(e => e.SchemaMigration).HasMaxLength(150);
        b.Property(e => e.ReleaseDateUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ChangeLogSummary).HasMaxLength(1000);
        b.Property(e => e.IsCurrent).HasDefaultValue(false);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.CreatedBy).HasMaxLength(100).IsRequired();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.ComponentName, e.VersionNumber })
            .IsUnique()
    }
}
```



```
        .HasDatabaseName("UQ_CorePlatform_Version_Component_Version");  
    }  
}
```

Index and constraint map

OBJECT	NAME IN CONFIGURATION	REASON
AppSetting key	UQ_CorePlatform_AppSetting_Key	One setting key means one runtime value.
SystemConfiguration natural key	UQ_CorePlatform_SystemConfiguration_Module_Key_Version	Structured config is versioned by module and key.
FeatureFlag natural key	UQ_CorePlatform_FeatureFlag_Code_Environment	Feature gates are environment-specific.
Localization passport	FK_CorePlatform_Localization_LanguageId	Translated resources cannot reference an unknown language.
Region passport	FK_CorePlatform_Region_CountryId	Regions exist inside countries.
Calendar passport	FK_CorePlatform_Calendar_TimeZoneId	Local schedules require an explicit time zone.
EngineeringUnit symbol	UQ_CorePlatform_EngineeringUnit_Symbol	Signals and thresholds must not split on free-text unit spellings.
PlatformVersion natural key	UQ_CorePlatform_Version_Component_Version	A deployed component should not have duplicate version rows.

Model metadata verification tests

Configuration classes should not only compile. They should be inspectable. A few tests can prevent silent drift in schema names, table names, index names, and delete behavior.

```
[Fact]
public void CorePlatform_model_has_expected_schema_and_table_names()
{
    using var db = CreateContext();

    var unit = db.Model.FindEntityType(typeof(EngineeringUnit));

    Assert.Equal("CorePlatform", unit.GetSchema());
    Assert.Equal("EngineeringUnit", unit.GetTableName());
}

[Fact]
public void EngineeringUnit_symbol_index_has_stable_database_name()
{

```

```

using var db = CreateContext();

var entity = db.Model.FindEntityType(typeof(EngineeringUnit));
var index = entity.GetIndexes()
    .Single(i => i.Properties.Select(p => p.Name).SequenceEqual(new[] { "Symbol" }));

Assert.True(index.IsUnique);
Assert.Equal("UQ_CorePlatform_EngineeringUnit_Symbol", index.GetDatabaseName());
}

[Fact]
public void Localization_language_fk_has_stable_constraint_name()
{
    using var db = CreateContext();

    var entity = db.Model.FindEntityType(typeof(Localization));
    var fk = entity.GetForeignKeys()
        .Single(x => x.Properties.Single().Name == "LanguageId");

    Assert.Equal("FK_CorePlatform_Localization_LanguageId", fk.GetConstraintName());
    Assert.Equal>DeleteBehavior.Restrict, fk.DeleteBehavior);
}

```

Migration sanity checks

After generating the migration, run simple database queries to confirm that the CorePlatform tables landed in the right schema and that the named relationships are present.

```

-- Generated migration should contain stable names that humans can read.
SELECT name, type_desc
FROM sys.objects
WHERE schema_id = SCHEMA_ID(N'CorePlatform')
AND name LIKE N'%CorePlatform%'
ORDER BY type_desc, name;

-- No CorePlatform table should be created outside the CorePlatform schema.
SELECT s.name AS schema_name, t.name AS table_name
FROM sys.tables AS t
JOIN sys.schemas AS s ON s.schema_id = t.schema_id
WHERE t.name IN (
    N'AppSetting', N'SystemConfiguration', N'FeatureFlag', N'Language',
    N'Localization', N'Country', N'Region', N'TimeZone', N'Calendar',
    N'EngineeringUnit', N'Version')
ORDER BY s.name, t.name;

-- Cross-check the most important passports inside CorePlatform.
SELECT fk.name AS foreign_key_name,
    OBJECT_SCHEMA_NAME(fk.parent_object_id) AS from_schema,
    OBJECT_NAME(fk.parent_object_id) AS from_table,
    OBJECT_SCHEMA_NAME(fk.referenced_object_id) AS to_schema,
    OBJECT_NAME(fk.referenced_object_id) AS to_table
FROM sys.foreign_keys AS fk
WHERE fk.name IN (
    N'FK_CorePlatform_Localization_LanguageId',
    N'FK_CorePlatform_Region_CountryId',
    N'FK_CorePlatform_Calendar_TimeZoneId')
ORDER BY fk.name;

```

What this step adds to the book

The starter edition established the principle: one entity, one mapping, clean context. This step applies that principle to the first complete schema sector. The pattern now exists in concrete form and can be repeated for Security, Organization, ReactorFleet, and the remaining NEXUS-1 bounded contexts.

Honest boundary

This chapter describes the EF Core mapping layer for a demonstrator-grade schema atlas. It is professional software architecture material, but it is not a claim that the NEXUS-1 schema is a certified industry data model or that these configuration classes have been validated for plant operation. The value here is clarity: every table, key, relationship, and database name can be read and reviewed before a migration is trusted.

Step index

AppSettingConfiguration - runtime settings, parser check, key uniqueness

Calendar - mapped as `PlantCalendar`

Check constraints - value type, JSON, effective window, offset range

CountryConfiguration - ISO codes

EngineeringUnitConfiguration - units, quantity type, decimal precision

FeatureFlagConfiguration - environment-scoped flags

Foreign-key passports - Language, Country, TimeZone

LanguageConfiguration - BCP 47 language codes

LocalizationConfiguration - resource key plus language

PlatformTimeZone - entity name for table `TimeZone`

PlatformVersion - entity name for table `Version`

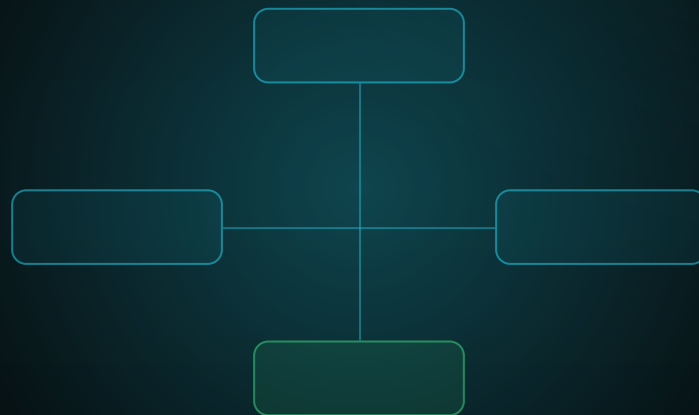
RegionConfiguration - country-scoped subdivisions

RowVersion - concurrency token for mutable reference data

SystemConfigurationConfiguration - JSON contracts and schema versions

FROM ENTITY TO CONTEXT

Security Configuration Files - The EF Core Configuration Atlas of the NEXUS-1 Digital Twin



STEP 2 - SECURITY SCHEMA

Identity, roles, permissions, policies, sessions, tokens, API clients, and access reviews

Configuration classes that seat ASP.NET Core Identity inside the NEXUS-1 bounded-context database.

From Entity to Context

Security Configuration Files

Step-by-step EF Core mapping atlas for the NEXUS-1 data backbone.

Companion to *From Schema to System* and *From Table to Twin*.
Generated as a separate chapter build so the full book can grow one schema at a time.

Chapter 7

Security Configurations

Security is the first schema where EF Core configuration has to do two jobs at once: respect the database names of the NEXUS-1 atlas and remain compatible with the habits of ASP.NET Core Identity. Users, roles, claims, logins, tokens, sessions, permissions and policies all belong in one bounded context, not scattered across an accidental membership database.

CorePlatform taught the shape of configuration. Security teaches ownership: Identity tables are not magic tables. They are domain tables with names, keys, constraints, indexes, and relationships that must be written down like every other table.

What this schema teaches

Identity inside a sector

Map users and roles into `Security`, not `dbo`.

Names over guesses

Pin primary keys, unique indexes, and FK constraint names.

RBAC plus policy

Roles grant permissions; policies add conditional rules and effective windows.

Operational evidence

Login attempts, sessions, tokens and access reviews become queryable records.

Configuration file catalogue

GROUP	SQL TABLE	SUGGESTED ENTITY	CONFIGURATION FILE
Lookup	<code>Security.UserStatus</code>	<code>UserStatus</code>	<code>UserStatusConfiguration.cs</code>
Lookup	<code>Security.RoleType</code>	<code>RoleType</code>	<code>RoleTypeConfiguration.cs</code>
Lookup	<code>Security.PermissionCategory</code>	<code>PermissionCategory</code>	<code>PermissionCategoryConfiguration.cs</code>
Lookup	<code>Security.PolicyType</code>	<code>PolicyType</code>	<code>PolicyTypeConfiguration.cs</code>
Lookup	<code>Security.TokenType</code>	<code>TokenType</code>	<code>TokenTypeConfiguration.cs</code>
Lookup	<code>Security.SessionStatus</code>	<code>SessionStatus</code>	<code>SessionStatusConfiguration.cs</code>
Lookup	<code>Security.LoginResult</code>	<code>LoginResult</code>	<code>LoginResultConfiguration.cs</code>
Lookup	<code>Security.AccessReviewStatus</code>	<code>AccessReviewStatus</code>	<code>AccessReviewStatusConfiguration.cs</code>
Identity	<code>Security.ApplicationUser</code>	<code>ApplicationUser</code>	<code>ApplicationUserConfiguration.cs</code>

GROUP	SQL TABLE	SUGGESTED ENTITY	CONFIGURATION FILE
Identity	Security.ApplicationRole	ApplicationRole	ApplicationRoleConfiguration.cs
Identity	Security.UserRole	UserRole	UserRoleConfiguration.cs
Identity	Security.UserClaim	UserClaim	UserClaimConfiguration.cs
Identity	Security.RoleClaim	RoleClaim	RoleClaimConfiguration.cs
Identity	Security.ExternalLogin	ExternalLogin	ExternalLoginConfiguration.cs
Permission	Security.Permission	Permission	PermissionConfiguration.cs
Permission	Security.RolePermission	RolePermission	RolePermissionConfiguration.cs
Permission	Security.UserPermissionOverride	UserPermissionOverride	UserPermissionOverrideConfiguration.cs
Policy	Security.Policy	Policy	PolicyConfiguration.cs
Policy	Security.PolicyPermission	PolicyPermission	PolicyPermissionConfiguration.cs
Policy	Security.RolePolicy	RolePolicy	RolePolicyConfiguration.cs
Policy	Security.UserPolicyAssignment	UserPolicyAssignment	UserPolicyAssignmentConfiguration.cs
Session	Security.UserSession	UserSession	UserSessionConfiguration.cs
Session	Security.SecurityToken	SecurityToken	SecurityTokenConfiguration.cs
Session	Security.LoginAttempt	LoginAttempt	LoginAttemptConfiguration.cs
Client	Security.UserPreference	UserPreference	UserPreferenceConfiguration.cs
Client	Security.ApiClient	ApiClient	ApiClientConfiguration.cs
Client	Security.ApiClientSecret	ApiClientSecret	ApiClientSecretConfiguration.cs
Review	Security.AccessReview	AccessReview	AccessReviewConfiguration.cs
Review	Security.AccessReviewItem	AccessReviewItem	AccessReviewItemConfiguration.cs

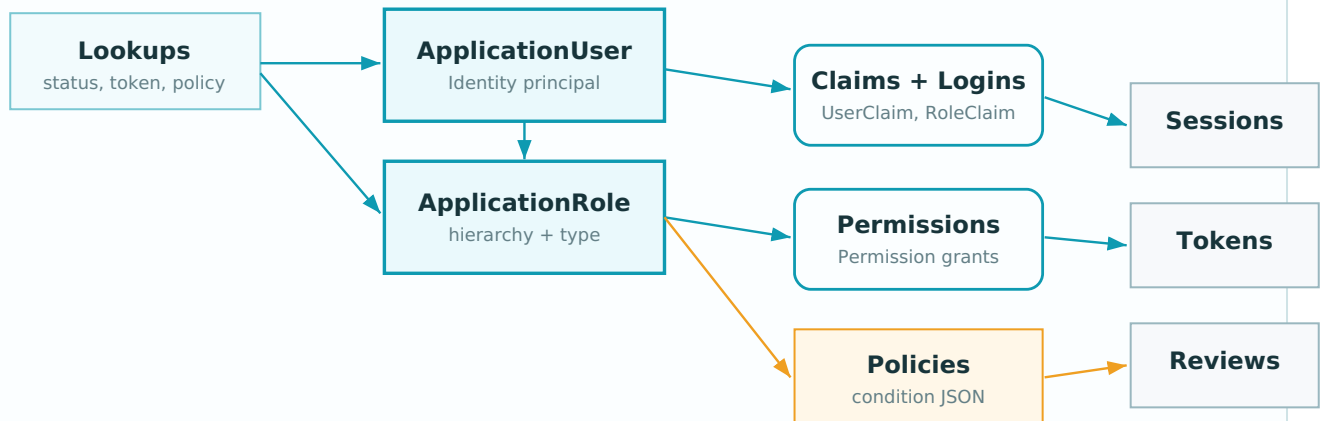
Identity users, roles, claims and external logins.

Authorization permissions, role grants, user overrides and policies.

Runtime evidence sessions, tokens and login attempts.

Governance API clients, secret rotation and access review campaigns.

Security mapping overview



Identity is the centre; permissions, policies, sessions, tokens, API clients, and reviews hang from stable user and role keys.

Figure 7.1 - Security configuration dependency map. The user and role tables form the centre. Permissions, policies, claims, sessions, tokens, API clients and access reviews depend on those stable identifiers.

Folder structure for this step

```
Nexus.Infrastructure/
  Persistence/
    Configurations/
      NexusSql.cs
    Security/
      SecurityLookupConfiguration.cs
      UserStatusConfiguration.cs
      RoleTypeConfiguration.cs
      PermissionCategoryConfiguration.cs
      PolicyTypeConfiguration.cs
      TokenTypeConfiguration.cs
      SessionStatusConfiguration.cs
      LoginResultConfiguration.cs
      AccessReviewStatusConfiguration.cs
      ApplicationUserConfiguration.cs
      ApplicationRoleConfiguration.cs
      UserRoleConfiguration.cs
      UserClaimConfiguration.cs
      RoleClaimConfiguration.cs
      ExternalLoginConfiguration.cs
      PermissionConfiguration.cs
      RolePermissionConfiguration.cs
      UserPermissionOverrideConfiguration.cs
      PolicyConfiguration.cs
      PolicyPermissionConfiguration.cs
      RolePolicyConfiguration.cs
      UserPolicyAssignmentConfiguration.cs
      UserSessionConfiguration.cs
      SecurityTokenConfiguration.cs
      LoginAttemptConfiguration.cs
      UserPreferenceConfiguration.cs
      ApiClientConfiguration.cs
      ApiClientSecretConfiguration.cs
      AccessReviewConfiguration.cs
      AccessReviewItemConfiguration.cs
```

One line still discovers the whole sector. The point of this atlas is not to make NexusContext longer. It is to make the context almost empty while each table's intent lives in one small file.

```
// One line in NexusContext discovers every Security configuration class.
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(NexusContext).Assembly);
}

// A smoke test that protects the atlas from accidental omissions.
[Fact]
public void Security_tables_are_mapped_to_security_schema()
{
    using var db = new NexusContext(_options);

    var securityEntities = db.Model.GetEntityTypes()
        .Where(t => t.GetSchema() == NexusSql.Security)
        .Select(t => t.GetTableName())
        .OrderBy(x => x)
        .ToArray();

    Assert.Contains("ApplicationUser", securityEntities);
    Assert.Contains("ApplicationRole", securityEntities);
    Assert.Contains("Permission", securityEntities);
    Assert.Contains("Policy", securityEntities);
    Assert.Contains("UserSession", securityEntities);
    Assert.Contains("AccessReview", securityEntities);
}
```

Shared configuration support

The lookup tables in Security share the same shape as the lookup tables in the SQL atlas: code, name, description, display order, active flag, audit timestamps, and rowversion. A base configuration avoids twenty near-identical mapping blocks while each table still gets its own explicit primary key and unique index name.

```
namespace Nexus.Infrastructure.Persistence.Configurations;

internal static class NexusSql
{
    public const string CorePlatform = "CorePlatform";
    public const string Security = "Security";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class SecurityLookupConfiguration<TEntity>
    : IEntityTypeConfiguration<TEntity>
    where TEntity : SecurityLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(150)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

internal static class SecurityModelBuilderExtensions
{
    public static PropertyBuilder<string> HasAuditName(
        this PropertyBuilder<string> property) => property
        .HasMaxLength(100)
        .IsRequired();

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);
}
```

Security lookup configurations

The lookup configuration files are intentionally boring. In enterprise systems boring is a feature: every lookup table is predictable, every code is unique, and every migration diff is small.

```
public sealed class UserStatusConfiguration
    : SecurityLookupConfiguration<UserStatus>
{
    public override void Configure(EntityTypeBuilder<UserStatus> b)
    {
        b.ToTable("UserStatus", NexusSql.Security);
        b.HasKey(e => e.UserId).HasName("PK_Security_UserStatus");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_UserStatus_Code");
    }
}

public sealed class RoleTypeConfiguration
    : SecurityLookupConfiguration<RoleType>
{
    public override void Configure(EntityTypeBuilder<RoleType> b)
    {
        b.ToTable("RoleType", NexusSql.Security);
        b.HasKey(e => e.RoleTypeId).HasName("PK_Security_RoleType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_RoleType_Code");
    }
}

public sealed class PermissionCategoryConfiguration
    : SecurityLookupConfiguration<PermissionCategory>
{
    public override void Configure(EntityTypeBuilder<PermissionCategory> b)
    {
        b.ToTable("PermissionCategory", NexusSql.Security);
        b.HasKey(e => e.PermissionCategoryId)
            .HasName("PK_Security_PermissionCategory");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_PermissionCategory_Code");
    }
}

// The remaining security lookups follow the same pattern.
// Keep each file separate so merge conflicts stay small.
public sealed class PolicyTypeConfiguration
    : SecurityLookupConfiguration<PolicyType>
{
    public override void Configure(EntityTypeBuilder<PolicyType> b)
    {
        b.ToTable("PolicyType", NexusSql.Security);
        b.HasKey(e => e.PolicyTypeId).HasName("PK_Security_PolicyType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_PolicyType_Code");
    }
}

public sealed class TokenTypeConfiguration
    : SecurityLookupConfiguration<TokenType>
{
    public override void Configure(EntityTypeBuilder<TokenType> b)
    {
        b.ToTable("TokenType", NexusSql.Security);
        b.HasKey(e => e.TokenTypeId).HasName("PK_Security_TokenType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_TokenType_Code");
    }
}
```

```

    }
}

public sealed class SessionStatusConfiguration
    : SecurityLookupConfiguration<SessionStatus>
{
    public override void Configure(EntityTypeBuilder<SessionStatus> b)
    {
        b.ToTable("SessionStatus", NexusSql.Security);
        b.HasKey(e => e.SessionStatusId).HasName("PK_Security_SessionStatus");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_SessionStatus_Code");
    }
}

public sealed class LoginResultConfiguration
    : SecurityLookupConfiguration<LoginResult>
{
    public override void Configure(EntityTypeBuilder<LoginResult> b)
    {
        b.ToTable("LoginResult", NexusSql.Security);
        b.HasKey(e => e.LoginResultId).HasName("PK_Security_LoginResult");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_LoginResult_Code");
    }
}

public sealed class AccessReviewStatusConfiguration
    : SecurityLookupConfiguration<AccessReviewStatus>
{
    public override void Configure(EntityTypeBuilder<AccessReviewStatus> b)
    {
        b.ToTable("AccessReviewStatus", NexusSql.Security);
        b.HasKey(e => e.AccessReviewStatusId)
            .HasName("PK_Security_AccessReviewStatus");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_AccessReviewStatus_Code");
    }
}
}

```

ApplicationUserConfiguration.cs

The custom user table keeps the familiar Identity columns but gives them NEXUS-1 names, explicit constraints, UTC defaults, and a passport to `UserStatus`. The important rule is that usernames and normalized usernames are not left for EF to name by convention.

```
namespace Nexus.Infrastructure.Persistence.Configurations.Security;

public sealed class ApplicationUserConfiguration
    : IEntityTypeConfiguration<ApplicationUser>
{
    public void Configure(EntityTypeBuilder<ApplicationUser> b)
    {
        b.ToTable("ApplicationUser", NexusSql.Security, t =>
        {
            t.HasCheckConstraint(
                "CK_Security_ApplicationUser_AccessFailedCount",
                "AccessFailedCount >= 0");
        });

        b.HasKey(e => e.ApplicationUserId)
            .HasName("PK_Security_ApplicationUser");

        b.Property(e => e.UserName).HasMaxLength(256).IsRequired();
        b.Property(e => e.NormalizedUserName).HasMaxLength(256).IsRequired();
        b.Property(e => e.Email).HasMaxLength(256);
        b.Property(e => e.NormalizedEmail).HasMaxLength(256);
        b.Property(e => e.PhoneNumber).HasMaxLength(50);
        b.Property(e => e.SecurityStamp).HasMaxLength(100);
        b.Property(e => e.ConcurrencyStamp).HasMaxLength(100);
        b.Property(e => e.DisplayName).HasMaxLength(200).IsRequired();
        b.Property(e => e.GivenName).HasMaxLength(100);
        b.Property(e => e.FamilyName).HasMaxLength(100);

        b.Property(e => e.EmailConfirmed).HasDefaultValue(false);
        b.Property(e => e.PhoneNumberConfirmed).HasDefaultValue(false);
        b.Property(e => e.TwoFactorEnabled).HasDefaultValue(false);
        b.Property(e => e.LockoutEnabled).HasDefaultValue(true);
        b.Property(e => e.AccessFailedCount).HasDefaultValue(0);
        b.Property(e => e.IsServiceAccount).HasDefaultValue(false);
        b.Property(e => e.IsDeleted).HasDefaultValue(false);

        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.CreatedBy).HasAuditName();
        b.Property(e => e.ModifiedBy).HasAuditName();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.NormalizedUserName)
            .IsUnique()
            .HasDatabaseName("UQ_Security_ApplicationUser_NormalizedUserName");

        b.HasIndex(e => e.NormalizedEmail)
            .HasDatabaseName("IX_Security_ApplicationUser_NormalizedEmail");

        b.HasOne(e => e.UserStatus)
            .WithMany(s => s.Users)
            .HasForeignKey(e => e.UserStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_ApplicationUser_UserStatus");
    }
}
```

ApplicationRoleConfiguration.cs

Roles are hierarchical and typed. The self-reference is deliberately configured with restrictive delete behaviour; deleting a parent role should be an administrative decision, not an accidental cascade.

```
public sealed class ApplicationRoleConfiguration
    : IEntityTypeConfiguration<ApplicationRole>
{
    public void Configure(EntityTypeBuilder<ApplicationRole> b)
    {
        b.ToTable("ApplicationRole", NexusSql.Security);

        b.HasKey(e => e.ApplicationRoleId)
            .HasName("PK_Security_ApplicationRole");

        b.Property(e => e.Name).HasMaxLength(256).IsRequired();
        b.Property(e => e.NormalizedName).HasMaxLength(256).IsRequired();
        b.Property(e => e.Description).HasMaxLength(500);
        b.Property(e => e.IsBuiltIn).HasDefaultValue(false);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.IsDeleted).HasDefaultValue(false);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.CreatedBy).HasAuditName();
        b.Property(e => e.ModifiedBy).HasAuditName();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.NormalizedName)
            .IsUnique()
            .HasDatabaseName("UQ_Security_ApplicationRole_NormalizedName");

        b.HasOne(e => e.RoleType)
            .WithMany(t => t.Roles)
            .HasForeignKey(e => e.RoleTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_ApplicationRole_RoleType");

        b.HasOne(e => e.ParentRole)
            .WithMany(e => e.ChildRoles)
            .HasForeignKey(e => e.ParentRoleId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_ApplicationRole_ParentRole");
    }
}
```


UserRole, UserClaim, RoleClaim, ExternalLogin

These are the classic Identity relationship tables, but the atlas treats them as first-class tables. The join table carries assignment metadata; claims and external logins use stable length limits and named indexes.

```
public sealed class UserRoleConfiguration : IEntityTypeConfiguration<UserRole>
{
    public void Configure(EntityTypeBuilder<UserRole> b)
    {
        b.ToTable("UserRole", NexusSql.Security);
        b.HasKey(e => e.UserId).HasName("PK_Security_UserRole");
        b.Property(e => e.AssignedAtUtc).HasUtcDefault();
        b.Property(e => e.AssignedBy).HasAuditName();
        b.Property(e => e.ExpiresAtUtc).HasColumnType("datetime2(7)");

        b.HasIndex(e => new { e.ApplicationUserId, e.ApplicationRoleId })
            .IsUnique()
            .HasDatabaseName("UQ_Security_UserRole_User_Role");

        b.HasOne(e => e.ApplicationUser).WithMany(u => u.UserRoles)
            .HasForeignKey(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_UserRole_ApplicationUser");

        b.HasOne(e => e.ApplicationRole).WithMany(r => r.UserRoles)
            .HasForeignKey(e => e.ApplicationRoleId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_UserRole_ApplicationRole");
    }
}

public sealed class UserClaimConfiguration : IEntityTypeConfiguration<UserClaim>
{
    public void Configure(EntityTypeBuilder<UserClaim> b)
    {
        b.ToTable("UserClaim", NexusSql.Security);
        b.HasKey(e => e.UserId).HasName("PK_Security_UserClaim");
        b.Property(e => e.ClaimType).HasMaxLength(250).IsRequired();
        b.Property(e => e.ClaimValue).HasMaxLength(1000).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.CreatedBy).HasAuditName();

        b.HasIndex(e => new { e.ApplicationUserId, e.ClaimType })
            .HasDatabaseName("IX_Security_UserClaim_User_Type");

        b.HasOne(e => e.ApplicationUser).WithMany(u => u.Claims)
            .HasForeignKey(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_UserClaim_ApplicationUser");
    }
}

public sealed class RoleClaimConfiguration : IEntityTypeConfiguration<RoleClaim>
{
    public void Configure(EntityTypeBuilder<RoleClaim> b)
    {
        b.ToTable("RoleClaim", NexusSql.Security);
        b.HasKey(e => e.RoleClaimId).HasName("PK_Security_RoleClaim");
        b.Property(e => e.ClaimType).HasMaxLength(250).IsRequired();
        b.Property(e => e.ClaimValue).HasMaxLength(1000).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.CreatedBy).HasAuditName();

        b.HasIndex(e => new { e.ApplicationRoleId, e.ClaimType })
            .HasDatabaseName("IX_Security_RoleClaim_Role_Type");

        b.HasOne(e => e.ApplicationRole).WithMany(r => r.Claims)
            .HasForeignKey(e => e.ApplicationRoleId)
            .OnDelete(DeleteBehavior.Restrict)
    }
}
```

```

        .HasConstraintName("FK_Security_RoleClaim_ApplicationRole");
    }
}

public sealed class ExternalLoginConfiguration
    : IEntityTypeConfiguration<ExternalLogin>
{
    public void Configure(EntityTypeBuilder<ExternalLogin> b)
    {
        b.ToTable("ExternalLogin", NexusSql.Security);
        b.HasKey(e => e.ExternalLoginId).HasName("PK_Security_ExternalLogin");
        b.Property(e => e.LoginProvider).HasMaxLength(128).IsRequired();
        b.Property(e => e.ProviderKey).HasMaxLength(256).IsRequired();
        b.Property(e => e.ProviderDisplayName).HasMaxLength(256);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.LastUsedAtUtc).HasColumnType("datetime2(7)");

        b.HasIndex(e => new { e.LoginProvider, e.ProviderKey })
            .IsUnique()
            .HasDatabaseName("UQ_Security_ExternalLogin_Provider_Key");

        b.HasOne(e => e.ApplicationUser).WithMany(u => u.ExternalLogins)
            .HasForeignKey(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_ExternalLogin_ApplicationUser");
    }
}

```

Permission and permission grants

`Permission` is the catalogue. `RolePermission` is the normal path. `UserPermissionOverride` is the exceptional path and therefore carries a reason and an effective window.

```
public sealed class PermissionConfiguration : IEntityTypeConfiguration<Permission>
{
    public void Configure(EntityTypeBuilder<Permission> b)
    {
        b.ToTable("Permission", NexusSql.Security);
        b.HasKey(e => e.PermissionId).HasName("PK_Security_Permission");
        b.Property(e => e.Code).HasMaxLength(120).IsRequired();
        b.Property(e => e.Name).HasMaxLength(180).IsRequired();
        b.Property(e => e.Description).HasMaxLength(700);
        b.Property(e => e.ResourceType).HasMaxLength(80);
        b.Property(e => e.ActionName).HasMaxLength(80).IsRequired();
        b.Property(e => e.IsSafetyRelevant).HasDefaultValue(false);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.IsDeleted).HasDefaultValue(false);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.CreatedBy).HasAuditName();
        b.Property(e => e.ModifiedBy).HasAuditName();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_Permission_Code");

        b.HasOne(e => e.PermissionCategory)
            .WithMany(c => c.Permissions)
            .HasForeignKey(e => e.PermissionCategoryId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_Permission_PermissionCategory");
    }
}

public sealed class RolePermissionConfiguration
    : IEntityTypeConfiguration<RolePermission>
{
    public void Configure(EntityTypeBuilder<RolePermission> b)
    {
        b.ToTable("RolePermission", NexusSql.Security);
        b.HasKey(e => e.RolePermissionId).HasName("PK_Security_RolePermission");
        b.Property(e => e.GrantedAtUtc).HasUtcDefault();
        b.Property(e => e.GrantedBy).HasAuditName();

        b.HasIndex(e => new { e.ApplicationRoleId, e.PermissionId })
            .IsUnique()
            .HasDatabaseName("UQ_Security_RolePermission_Role_Permission");

        b.HasOne(e => e.ApplicationRole).WithMany(r => r.RolePermissions)
            .HasForeignKey(e => e.ApplicationRoleId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_RolePermission_ApplicationRole");

        b.HasOne(e => e.Permission).WithMany(p => p.RolePermissions)
            .HasForeignKey(e => e.PermissionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_RolePermission_Permission");
    }
}

public sealed class UserPermissionOverrideConfiguration
    : IEntityTypeConfiguration<UserPermissionOverride>
{
    public void Configure(EntityTypeBuilder<UserPermissionOverride> b)
    {
        b.ToTable("UserPermissionOverride", NexusSql.Security);
        b.HasKey(e => e.UserPermissionOverrideId)
            .HasName("PK_Security_UserPermissionOverride");
    }
}
```

```

        b.Property(e => e.IsAllowed).HasDefaultValue(true);
        b.Property(e => e.Reason).HasMaxLength(700).IsRequired();
        b.Property(e => e.EffectiveFromUtc).HasUtcDefault();
        b.Property(e => e.EffectiveToUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.CreatedBy).HasAuditName();

        b.HasIndex(e => new { e.ApplicationUserId, e.PermissionId })
            .HasDatabaseName("IX_Security_UserPermissionOverride_User_Permission");

        b.HasOne(e => e.ApplicationUser).WithMany(u => u.PermissionOverrides)
            .HasForeignKey(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_UserPermissionOverride_ApplicationUser");

        b.HasOne(e => e.Permission).WithMany(p => p.UserOverrides)
            .HasForeignKey(e => e.PermissionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_UserPermissionOverride_Permission");
    }
}

```

Policies and policy assignments

Policies introduce conditions, priority, effective dates, and JSON validation. They do not replace permissions; they govern how permissions are interpreted in a given context.

```
public sealed class PolicyConfiguration : IEntityTypeConfiguration<Policy>
{
    public void Configure(EntityTypeBuilder<Policy> b)
    {
        b.ToTable("Policy", NexusSql.Security, t =>
        {
            t.HasCheckConstraint(
                "CK_Security_Policy_EffectiveWindow",
                "EffectiveToUtc IS NULL OR EffectiveToUtc > EffectiveFromUtc");
            t.HasCheckConstraint(
                "CK_Security_Policy_ConditionJson",
                "ConditionJson IS NULL OR ISJSON(ConditionJson) = 1");
        });

        b.HasKey(e => e.PolicyId).HasName("PK_Security_Policy");
        b.Property(e => e.Code).HasMaxLength(100).IsRequired();
        b.Property(e => e.Name).HasMaxLength(180).IsRequired();
        b.Property(e => e.Description).HasMaxLength(700);
        b.Property(e => e.ConditionJson).HasColumnType("nvarchar(max)");
        b.Property(e => e.Priority).HasDefaultValue(100);
        b.Property(e => e.IsEnabled).HasDefaultValue(true);
        b.Property(e => e.IsDeleted).HasDefaultValue(false);
        b.Property(e => e.EffectiveFromUtc).HasUtcDefault();
        b.Property(e => e.EffectiveToUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.CreatedBy).HasAuditName();
        b.Property(e => e.ModifiedBy).HasAuditName();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_Policy_Code");

        b.HasIndex(e => new { e.IsEnabled, e.Priority })
            .HasDatabaseName("IX_Security_Policy_Enabled_Priority");

        b.HasOne(e => e.PolicyType).WithMany(t => t.Policies)
            .HasForeignKey(e => e.PolicyTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_Policy_PolicyType");
    }
}

public sealed class PolicyPermissionConfiguration
    : IEntityTypeConfiguration<PolicyPermission>
{
    public void Configure(EntityTypeBuilder<PolicyPermission> b)
    {
        b.ToTable("PolicyPermission", NexusSql.Security);
        b.HasKey(e => e.PolicyPermissionId)
            .HasName("PK_Security_PolicyPermission");

        b.HasIndex(e => new { e.PolicyId, e.PermissionId })
            .IsUnique()
            .HasDatabaseName("UQ_Security_PolicyPermission_Policy_Permission");

        b.HasOne(e => e.Policy).WithMany(p => p.PolicyPermissions)
            .HasForeignKey(e => e.PolicyId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_PolicyPermission_Policy");

        b.HasOne(e => e.Permission).WithMany(p => p.PolicyPermissions)
            .HasForeignKey(e => e.PermissionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_PolicyPermission_Permission");
    }
}
```

```

    }
}

public sealed class RolePolicyConfiguration : IEntityTypeConfiguration<RolePolicy>
{
    public void Configure(EntityTypeBuilder<RolePolicy> b)
    {
        b.ToTable("RolePolicy", NexusSql.Security);
        b.HasKey(e => e.RolePolicyId).HasName("PK_Security_RolePolicy");
        b.Property(e => e.AssignedAtUtc).HasUtcDefault();
        b.Property(e => e.AssignedBy).HasAuditName();

        b.HasIndex(e => new { e.ApplicationRoleId, e.PolicyId })
            .IsUnique()
            .HasDatabaseName("UQ_Security_RolePolicy_Role_Policy");

        b.HasOne(e => e.ApplicationRole).WithMany(r => r.RolePolicies)
            .HasForeignKey(e => e.ApplicationRoleId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_RolePolicy_ApplicationRole");

        b.HasOne(e => e.Policy).WithMany(p => p.RolePolicies)
            .HasForeignKey(e => e.PolicyId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_RolePolicy_Policy");
    }
}

public sealed class UserPolicyAssignmentConfiguration
    : IEntityTypeConfiguration<UserPolicyAssignment>
{
    public void Configure(EntityTypeBuilder<UserPolicyAssignment> b)
    {
        b.ToTable("UserPolicyAssignment", NexusSql.Security);
        b.HasKey(e => e.UserPolicyAssignmentId)
            .HasName("PK_Security_UserPolicyAssignment");
        b.Property(e => e.AssignedAtUtc).HasUtcDefault();
        b.Property(e => e.AssignedBy).HasAuditName();
        b.Property(e => e.ExpiresAtUtc).HasColumnType("datetime2(7)");

        b.HasIndex(e => new { e.ApplicationUserId, e.PolicyId })
            .HasDatabaseName("IX_Security_UserPolicyAssignment_User_Policy");

        b.HasOne(e => e.ApplicationUser).WithMany(u => u.PolicyAssignments)
            .HasForeignKey(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_UserPolicyAssignment_ApplicationUser");

        b.HasOne(e => e.Policy).WithMany(p => p.UserAssignments)
            .HasForeignKey(e => e.PolicyId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_UserPolicyAssignment_Policy");
    }
}

```

Sessions, tokens, and login attempts

This group turns authentication from a black box into a queryable trail. Tokens store hashes only. Login attempts are append-only evidence. Sessions expose the active security surface of the platform.

```
public sealed class UserSessionConfiguration
    : IEntityTypeConfiguration<UserSession>
{
    public void Configure(EntityTypeBuilder<UserSession> b)
    {
        b.ToTable("UserSession", NexusSql.Security);
        b.HasKey(e => e.UserId).HasName("PK_Security_UserSession");
        b.Property(e => e.SessionKey).HasMaxLength(120).IsRequired();
        b.Property(e => e.IpAddress).HasMaxLength(64);
        b.Property(e => e.UserAgent).HasMaxLength(512);
        b.Property(e => e.StartedAtUtc).HasUtcDefault();
        b.Property(e => e.LastSeenAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.EndedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.SessionKey).IsUnique()
            .HasDatabaseName("UQ_Security_UserSession_SessionKey");
        b.HasIndex(e => new { e.ApplicationUserId, e.SessionStatusId, e.LastSeenAtUtc })
            .HasDatabaseName("IX_Security_UserSession_User_Status_LastSeen");

        b.HasOne(e => e.ApplicationUser).WithMany(u => u.Sessions)
            .HasForeignKey(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_UserSession_ApplicationUser");

        b.HasOne(e => e.SessionStatus).WithMany(s => s.UserSessions)
            .HasForeignKey(e => e.SessionStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_UserSession_SessionStatus");
    }
}

public sealed class SecurityTokenConfiguration
    : IEntityTypeConfiguration<SecurityToken>
{
    public void Configure(EntityTypeBuilder<SecurityToken> b)
    {
        b.ToTable("SecurityToken", NexusSql.Security);
        b.HasKey(e => e.SecurityTokenId).HasName("PK_Security_SecurityToken");
        b.Property(e => e.TokenHash).HasMaxLength(256).IsRequired();
        b.Property(e => e.Purpose).HasMaxLength(100).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ExpiresAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RevokedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedByIp).HasMaxLength(64);

        b.HasIndex(e => e.TokenHash).IsUnique()
            .HasDatabaseName("UQ_Security_SecurityToken_TokenHash");
        b.HasIndex(e => new { e.ApplicationUserId, e.TokenTypeId, e.ExpiresAtUtc })
            .HasDatabaseName("IX_Security_SecurityToken_User_Type_Expires");

        b.HasOne(e => e.ApplicationUser).WithMany(u => u.SecurityTokens)
            .HasForeignKey(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_SecurityToken_ApplicationUser");

        b.HasOne(e => e.TokenType).WithMany(t => t.SecurityTokens)
            .HasForeignKey(e => e.TokenTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_SecurityToken_TokenType");
    }
}

public sealed class LoginAttemptConfiguration
```

```

: IEntityTypeConfiguration<LoginAttempt>
{
    public void Configure(EntityTypeBuilder<LoginAttempt> b)
    {
        b.ToTable("LoginAttempt", NexusSql.Security);
        b.HasKey(e => e.LoginAttemptId).HasName("PK_Security_LoginAttempt");
        b.Property(e => e.UserName).HasMaxLength(256).IsRequired();
        b.Property(e => e.IpAddress).HasMaxLength(64);
        b.Property(e => e.UserAgent).HasMaxLength(512);
        b.Property(e => e.AttemptedAtUtc).HasUtcDefault();
        b.Property(e => e.FailureReason).HasMaxLength(500);

        b.HasIndex(e => new { e.UserName, e.AttemptedAtUtc })
            .HasDatabaseName("IX_Security_LoginAttempt_UserName_AttemptedAt");
        b.HasIndex(e => new { e.IpAddress, e.AttemptedAtUtc })
            .HasDatabaseName("IX_Security_LoginAttempt_Ip_AttemptedAt");

        b.HasOne(e => e.LoginResult).WithMany(r => r.LoginAttempts)
            .HasForeignKey(e => e.LoginResultId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_LoginAttempt_LoginResult");

        b.HasOne(e => e.ApplicationUser).WithMany(u => u.LoginAttempts)
            .HasForeignKey(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_LoginAttempt_ApplicationUser");
    }
}

```


User preferences, API clients, and access reviews

Security is not only human login. API clients need identities, secrets need rotation, and permissions need periodic review. These mappings make those concerns visible in the same bounded context.

```
public sealed class UserPreferenceConfiguration
    : IEntityTypeConfiguration<UserPreference>
{
    public void Configure(EntityTypeBuilder<UserPreference> b)
    {
        b.ToTable("UserPreference", NexusSql.Security);
        b.HasKey(e => e.UserId).HasName("PK_Security_UserPreference");
        b.Property(e => e.PreferenceKey).HasMaxLength(150).IsRequired();
        b.Property(e => e.PreferenceValue).HasMaxLength(2000).IsRequired();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();

        b.HasIndex(e => new { e.ApplicationUserId, e.PreferenceKey })
            .IsUnique()
            .HasDatabaseName("UQ_Security_UserPreference_User_Key");

        b.HasOne(e => e.ApplicationUser).WithMany(u => u.Preferences)
            .HasForeignKey(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_UserPreference_ApplicationUser");
    }
}

public sealed class ApiClientConfiguration : IEntityTypeConfiguration<ApiClient>
{
    public void Configure(EntityTypeBuilder<ApiClient> b)
    {
        b.ToTable("ApiClient", NexusSql.Security);
        b.HasKey(e => e.ApiClientId).HasName("PK_Security_ApiClient");
        b.Property(e => e.ClientId).HasMaxLength(120).IsRequired();
        b.Property(e => e.DisplayName).HasMaxLength(200).IsRequired();
        b.Property(e => e.Description).HasMaxLength(500);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.CreatedBy).HasAuditName();
        b.Property(e => e.ModifiedBy).HasAuditName();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.ClientId).IsUnique()
            .HasDatabaseName("UQ_Security_ApiClient_ClientId");
    }
}

public sealed class ApiClientSecretConfiguration
    : IEntityTypeConfiguration<ApiClientSecret>
{
    public void Configure(EntityTypeBuilder<ApiClientSecret> b)
    {
        b.ToTable("ApiClientSecret", NexusSql.Security);
        b.HasKey(e => e.ApiClientSecretId)
            .HasName("PK_Security_ApiClientSecret");
        b.Property(e => e.SecretHash).HasMaxLength(256).IsRequired();
        b.Property(e => e.Description).HasMaxLength(300);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ExpiresAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RevokedAtUtc).HasColumnType("datetime2(7)");

        b.HasIndex(e => e.SecretHash).IsUnique()
            .HasDatabaseName("UQ_Security_ApiClientSecret_SecretHash");

        b.HasOne(e => e.ApiClient).WithMany(c => c.Secrets)
            .HasForeignKey(e => e.ApiClientId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_ApiClientSecret_ApiClient");
    }
}
```

```

    }
}

public sealed class AccessReviewConfiguration
    : IEntityTypeConfiguration<AccessReview>
{
    public void Configure(EntityTypeBuilder<AccessReview> b)
    {
        b.ToTable("AccessReview", NexusSql.Security);
        b.HasKey(e => e.AccessReviewId).HasName("PK_Security_AccessReview");
        b.Property(e => e.Code).HasMaxLength(80).IsRequired();
        b.Property(e => e.Name).HasMaxLength(200).IsRequired();
        b.Property(e => e.Description).HasMaxLength(700);
        b.Property(e => e.StartedAtUtc).HasUtcDefault();
        b.Property(e => e.CompletedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedBy).HasAuditName();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Security_AccessReview_Code");

        b.HasOne(e => e.AccessReviewStatus).WithMany(s => s.AccessReviews)
            .HasForeignKey(e => e.AccessReviewStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_AccessReview_Status");
    }
}

public sealed class AccessReviewItemConfiguration
    : IEntityTypeConfiguration<AccessReviewItem>
{
    public void Configure(EntityTypeBuilder<AccessReviewItem> b)
    {
        b.ToTable("AccessReviewItem", NexusSql.Security);
        b.HasKey(e => e.AccessReviewItemId)
            .HasName("PK_Security_AccessReviewItem");
        b.Property(e => e.ItemType).HasMaxLength(80).IsRequired();
        b.Property(e => e.Decision).HasMaxLength(50);
        b.Property(e => e.DecisionReason).HasMaxLength(700);
        b.Property(e => e.DecidedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.DecidedBy).HasMaxLength(100);

        b.HasIndex(e => new { e.AccessReviewId, e.ApplicationUserId, e.ItemType })
            .HasDatabaseName("IX_Security_AccessReviewItem_Review_User_Type");

        b.HasOne(e => e.AccessReview).WithMany(r => r.Items)
            .HasForeignKey(e => e.AccessReviewId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_AccessReviewItem_AccessReview");

        b.HasOne(e => e.ApplicationUser).WithMany(u => u.AccessReviewItems)
            .HasForeignKey(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Security_AccessReviewItem_ApplicationUser");
    }
}

```

Migration and verification notes

Migration order. Run Security after CorePlatform and before Organization/ReactorFleet consumers. Security is early because users, roles, permissions and audit actors are referenced by later schemas.

Expected table count

29 mapped tables in the Security schema.

Lookup count

8 lookup/reference tables.

Most important unique names

UQ_Security_ApplicationUser_NormalizedUserName
UQ_Security_ApplicationRole_NormalizedName
UQ_Security_Permission_Code

Most important checks

access failed count cannot be negative; policy effective date windows must make sense; condition JSON must be valid.

Verification queries

```
-- 1. Every Security table should live in the Security schema.
SELECT s.name AS schema_name, t.name AS table_name
FROM sys.tables AS t
JOIN sys.schemas AS s ON s.schema_id = t.schema_id
WHERE s.name = N'Security'
ORDER BY t.name;

-- 2. User and role uniqueness must be explicit, not EF-generated noise.
SELECT i.name, OBJECT_SCHEMA_NAME(i.object_id) AS schema_name,
       OBJECT_NAME(i.object_id) AS table_name
FROM sys.indexes AS i
WHERE i.name IN (
    N'UQ_Security_ApplicationUser_NormalizedUserName',
    N'UQ_Security_ApplicationRole_NormalizedName',
    N'UQ_Security_Permission_Code'
);

-- 3. The role hierarchy must not cascade-delete silently.
SELECT fk.name, delete_referential_action_desc
FROM sys.foreign_keys AS fk
WHERE fk.name = N'FK_Security_ApplicationRole_ParentRole';
```

Index of configuration files in this step

UserStatusConfiguration.cs - Security.UserStatus

RoleTypeConfiguration.cs - Security.RoleType

PermissionCategoryConfiguration.cs -
Security.PermissionCategory

PolicyTypeConfiguration.cs - Security.PolicyType

TokenTypeConfiguration.cs - Security.TokenType

SessionStatusConfiguration.cs -
Security.SessionStatus

LoginResultConfiguration.cs - Security.LoginResult

AccessReviewStatusConfiguration.cs -
Security.AccessReviewStatus

ApplicationUserConfiguration.cs -
Security.ApplicationUser

ApplicationRoleConfiguration.cs -

Security.ApplicationRole

UserRoleConfiguration.cs - Security.UserRole

UserClaimConfiguration.cs - Security.UserClaim

RoleClaimConfiguration.cs - Security.RoleClaim

ExternalLoginConfiguration.cs -

Security.ExternalLogin

PermissionConfiguration.cs - Security.Permission

RolePermissionConfiguration.cs -

Security.RolePermission

UserPermissionOverrideConfiguration.cs -

Security.UserPermissionOverride

PolicyConfiguration.cs - Security.Policy

PolicyPermissionConfiguration.cs -

Security.PolicyPermission

RolePolicyConfiguration.cs - Security.RolePolicy

UserPolicyAssignmentConfiguration.cs -

Security.UserPolicyAssignment

UserSessionConfiguration.cs - Security.UserSession

SecurityTokenConfiguration.cs -

Security.SecurityToken

LoginAttemptConfiguration.cs -

Security.LoginAttempt

UserPreferenceConfiguration.cs -

Security.UserPreference

ApiClientConfiguration.cs - Security.ApiClient

ApiClientSecretConfiguration.cs -

Security.ApiClientSecret

AccessReviewConfiguration.cs -

Security.AccessReview

AccessReviewItemConfiguration.cs -

Security.AccessReviewItem

Honest boundary

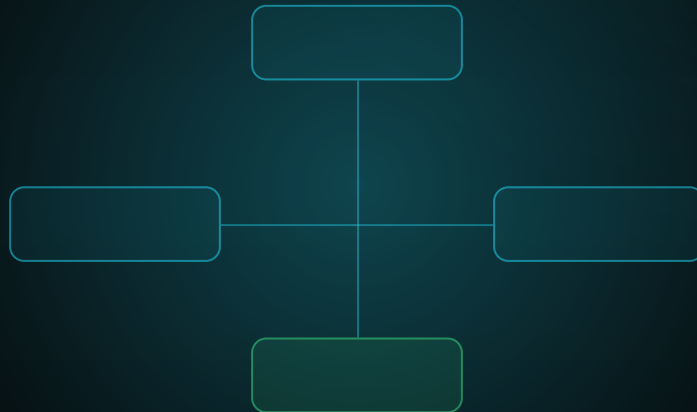
Security mapping is not security by itself. These configuration classes make the data model explicit. They do not implement password policy, MFA, authorization handlers, token issuance, secret rotation, or operational procedures. Those live in the application and infrastructure layers. The database provides durable structure; it does not replace security design.

Next step: **Chapter 8 - Organization Configurations**, where people, teams, sites, departments and operational rosters become mapped entities rather than loose identity metadata.

FROM ENTITY TO CONTEXT

Chapter 8 - Organization Configurations

The human, site, shift, qualification, and staffing model mapped professionally in EF Core.



CONFIGURATION CHAPTER

37 entity configuration files - 10 lookups - 27 substantive tables

Legal entities, sites, plants, departments, people, shifts, qualifications, and staffing scenarios.

Chapter 8

Organization Configurations

The third schema chapter of *From Entity to Context*. This chapter translates the Organization SQL schema into enterprise EF Core configuration classes: one entity, one mapping file, one stable database contract.

Security answers who can log in. Organization answers who the person is, where that person belongs, what position they hold, which shifts they serve, which qualifications they carry, and how staffing gaps are measured.

A login account is not an employee. A department is not a role. A shift assignment is not a permission. EF Core must keep those facts separate while still connecting them by explicit foreign-key passports.

8.1 Purpose of this schema chapter

The Organization schema is the human and institutional layer under NEXUS-1. It is the place where the physical plant becomes an accountable organization: legal entities operate sites, sites contain plants and buildings, departments own teams, people hold positions, shifts require coverage, and qualifications decide who is allowed to perform work.

In EF Core this is exactly where a large project can become messy if the mappings are left implicit. The configuration files in this chapter make every important modelling decision visible: stable table names, maximum lengths, unique indexes, restricted delete behaviour, row-version concurrency, UTC defaults, and cross-schema relationships to [CorePlatform](#) and [Security](#).

Schema
Organization

Configuration files
37 separate classes

Primary dependency
CorePlatform + Security passports

8.2 Configuration file catalogue

The catalogue below is the implementation checklist for the chapter. Each SQL table receives one EF Core configuration class. Lookup tables share a base configuration, but each keeps its own file so names, keys, seeds, and migrations remain explicit.

GROUP	SQL TABLE	ENTITY	CONFIGURATION FILE
Lookup	Organization.LegalEntityType	LegalEntityType	LegalEntityTypeConfiguration.cs
Lookup	Organization.SiteType	SiteType	SiteTypeConfiguration.cs
Lookup	Organization.PlantType	PlantType	PlantTypeConfiguration.cs
Lookup	Organization.DepartmentType	DepartmentType	DepartmentTypeConfiguration.cs
Lookup	Organization.TeamType	TeamType	TeamTypeConfiguration.cs
Lookup	Organization.PersonType	PersonType	PersonTypeConfiguration.cs
Lookup	Organization.EmploymentStatus	EmploymentStatus	EmploymentStatusConfiguration.cs
Lookup	Organization.ShiftType	ShiftType	ShiftTypeConfiguration.cs
Lookup	Organization.QualificationStatus	QualificationStatus	QualificationStatusConfiguration.cs
Lookup	Organization.ContactMethodType	ContactMethodType	ContactMethodTypeConfiguration.cs
Structure	Organization.LegalEntity	LegalEntity	LegalEntityConfiguration.cs
Structure	Organization.Site	Site	SiteConfiguration.cs
Structure	Organization.Plant	Plant	PlantConfiguration.cs
Structure	Organization.Building	Building	BuildingConfiguration.cs
Structure	Organization.Department	Department	DepartmentConfiguration.cs
Structure	Organization.Team	Team	TeamConfiguration.cs
Personnel	Organization.Position	Position	PositionConfiguration.cs
Personnel	Organization.Person	Person	PersonConfiguration.cs
Personnel	Organization.PersonContact	PersonContact	PersonContactConfiguration.cs
Personnel	Organization.PersonAddress	PersonAddress	PersonAddressConfiguration.cs
Personnel	Organization.Employment	Employment	EmploymentConfiguration.cs
Personnel	Organization.ContractorEngagement	ContractorEngagement	ContractorEngagementConfiguration.cs
Assignment	Organization.DepartmentAssignment	DepartmentAssignment	DepartmentAssignmentConfiguration.cs

GROUP	SQL TABLE	ENTITY	CONFIGURATION FILE
Assignment	Organization.TeamMembershi p	TeamMembership	TeamMembershipConfiguration.c s
Shift	Organization.ShiftPattern	ShiftPattern	ShiftPatternConfiguration.cs
Shift	Organization.Shift	Shift	ShiftConfiguration.cs
Shift	Organization.ShiftAssignme nt	ShiftAssignment	ShiftAssignmentConfiguration.c s
Shift	Organization.OnCallRoster	OnCallRoster	OnCallRosterConfiguration.cs
Competence	Organization.Qualification	Qualification	QualificationConfiguration.cs
Competence	Organization.PersonQualifi cation	PersonQualification	PersonQualificationConfigurati on.cs
Competence	Organization.Certification	Certification	CertificationConfiguration.cs
Competence	Organization.PersonCertifi cation	PersonCertification	PersonCertificationConfigurati on.cs
Staffing	Organization.PersonnelRequ irement	PersonnelRequirement	PersonnelRequirementConfigurat ion.cs
Staffing	Organization.StaffingScena rio	StaffingScenario	StaffingScenarioConfiguration. cs
Staffing	Organization.StaffingScena rioRequirement	StaffingScenarioRequ irement	StaffingScenarioRequirementCon figuration.cs
Staffing	Organization.StaffingScena rioResult	StaffingScenarioResu lt	StaffingScenarioResultConfigur ation.cs
Staffing	Organization.StaffingScena rioGap	StaffingScenarioGap	StaffingScenarioGapConfigurati on.cs

8.3 Folder structure and discovery

The folder structure mirrors the schema. The point is not decoration; it prevents the `DbContext` from becoming a dumping ground and makes pull-request review small enough for a real team.

```
/Nexus.Infrastructure/Persistence
NexusContext.cs
/Configurations
NexusSql.cs
/Organization
  Lookups/
    LegalEntityTypeConfiguration.cs
    SiteTypeConfiguration.cs
    PlantTypeConfiguration.cs
    DepartmentTypeConfiguration.cs
    TeamTypeConfiguration.cs
    PersonTypeConfiguration.cs
    EmploymentStatusConfiguration.cs
    ShiftTypeConfiguration.cs
    QualificationStatusConfiguration.cs
    ContactMethodTypeConfiguration.cs
  Structure/
    LegalEntityConfiguration.cs
    SiteConfiguration.cs
    PlantConfiguration.cs
    BuildingConfiguration.cs
    DepartmentConfiguration.cs
    TeamConfiguration.cs
  Personnel/
    PositionConfiguration.cs
    PersonConfiguration.cs
    PersonContactConfiguration.cs
    PersonAddressConfiguration.cs
    EmploymentConfiguration.cs
    ContractorEngagementConfiguration.cs
  Shifts/
    ShiftPatternConfiguration.cs
    ShiftConfiguration.cs
    ShiftAssignmentConfiguration.cs
    OnCallRosterConfiguration.cs
  Competence/
    QualificationConfiguration.cs
    PersonQualificationConfiguration.cs
    CertificationConfiguration.cs
    PersonCertificationConfiguration.cs
  Staffing/
    PersonnelRequirementConfiguration.cs
    StaffingScenarioConfiguration.cs
    StaffingScenarioRequirementConfiguration.cs
    StaffingScenarioResultConfiguration.cs
    StaffingScenarioGapConfiguration.cs
```

Discovery rule. The `DbContext` still uses one line:
`modelBuilder.ApplyConfigurationsFromAssembly(typeof(NexusContext).Assembly);`
The chapter adds many files, but not many lines inside `OnModelCreating`.

8.4 EF Core dependency diagram

Organization configuration dependency map

Solid teal edges stay inside Organization; violet edges are passports to CorePlatform and Security.

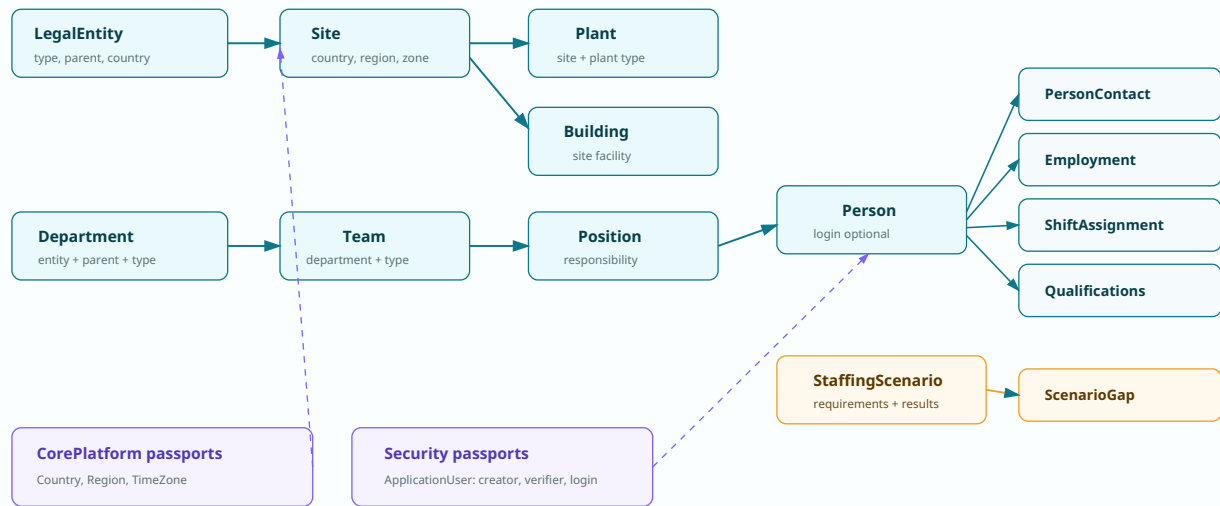


Figure 8.1 - Organization configuration dependency map. The EF model preserves the difference between institution, site, plant, department, person, shift, qualification, and staffing scenario. Cross-schema passports are intentionally drawn as separate dashed links.

8.5 Shared mapping base for Organization lookups

The ten lookup tables share the same shape: `Code`, `Name`, optional description, display order, active flag, timestamps, and row version. The base class teaches the shape once without hiding each table's identity.

```
namespace Nexus.Infrastructure.Persistence.Configurations;

internal static partial class NexusSql
{
    public const string Organization = "Organization";
    public const string CorePlatform = "CorePlatform";
    public const string Security = "Security";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class OrganizationLookupConfiguration<TEntity>
    : IEntityConfiguration<TEntity>
    where TEntity : OrganizationLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(160)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
```

```

        .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

internal static class OrganizationMappingExtensions
{
    public static PropertyBuilder<string> HasCode(
        this PropertyBuilder<string> property, int length = 40) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<string> HasName(
        this PropertyBuilder<string> property, int length = 180) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);
}

```

8.6 Lookup configuration files

The lookup list is: `LegalEntityType`, `SiteType`, `PlantType`, `DepartmentType`, `TeamType`, `PersonType`, `EmploymentStatus`, `ShiftType`, `QualificationStatus`, `ContactMethodType`. Each receives a separate configuration class. The examples below show the file pattern.

```

public sealed class LegalEntityTypeConfiguration
    : OrganizationLookupConfiguration<LegalEntityType>
{
    public override void Configure(EntityTypeBuilder<LegalEntityType> b)
    {
        b.ToTable("LegalEntityType", NexusSql.Organization);
        b.HasKey(e => e.LegalEntityTypeId)
            .HasName("PK_Organization_LegalEntityType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Organization_LegalEntityType_Code");
    }
}

public sealed class SiteTypeConfiguration
    : OrganizationLookupConfiguration<SiteType>
{
    public override void Configure(EntityTypeBuilder<SiteType> b)
    {
        b.ToTable("SiteType", NexusSql.Organization);
        b.HasKey(e => e.SiteTypeId).HasName("PK_Organization_SiteType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Organization_SiteType_Code");
    }
}

public sealed class PersonTypeConfiguration
    : OrganizationLookupConfiguration<PersonType>
{
    public override void Configure(EntityTypeBuilder<PersonType> b)
    {
        b.ToTable("PersonType", NexusSql.Organization);
        b.HasKey(e => e.PersonTypeId).HasName("PK_Organization_PersonType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Organization_PersonType_Code");
    }
}

// PlantType, DepartmentType, TeamType, EmploymentStatus,
// ShiftType, QualificationStatus and ContactMethodType use the same file pattern.
// Keep the classes separate: one entity, one configuration file, one small diff.

```

8.7 Structure configurations

The structural configurations define the institutional tree. Delete behaviour is deliberately restrictive: deleting a country, legal entity, site, or department should not silently delete operational history.

```
public sealed class LegalEntityConfiguration
    : IEntityTypeConfiguration<LegalEntity>
{
    public void Configure(EntityTypeBuilder<LegalEntity> b)
    {
        b.ToTable("LegalEntity", NexusSql.Organization);

        b.HasKey(e => e.LegalEntityId)
            .HasName("PK_Organization_LegalEntity");

        b.Property(e => e.Code).HasCode(40);
        b.Property(e => e.Name).HasName(200);
        b.Property(e => e.RegistrationNumber).HasMaxLength(80);
        b.Property(e => e.WebsiteUrl).HasMaxLength(300);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Organization_LegalEntity_Code");

        b.HasOne(e => e.LegalEntityType)
            .WithMany()
            .HasForeignKey(e => e.LegalEntityTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_LegalEntity_LegalEntityType");

        b.HasOne(e => e.ParentLegalEntity)
            .WithMany(e => e.Children)
            .HasForeignKey(e => e.ParentLegalEntityId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_LegalEntity_ParentLegalEntity");

        b.HasOne(e => e.Country)
            .WithMany()
            .HasForeignKey(e => e.CountryId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_LegalEntity_CorePlatform_Country");
    }
}
```

```
public sealed class SiteConfiguration : IEntityTypeConfiguration<Site>
{
    public void Configure(EntityTypeBuilder<Site> b)
    {
        b.ToTable("Site", NexusSql.Organization);
        b.HasKey(e => e.SiteId).HasName("PK_Organization_Site");
        b.Property(e => e.Code).HasCode(40);
        b.Property(e => e.Name).HasName(200);
        b.Property(e => e.AddressLine1).HasMaxLength(250);
        b.Property(e => e.City).HasMaxLength(120);
        b.Property(e => e.PostalCode).HasMaxLength(30);
        b.Property(e => e.Latitude).HasPrecision(9, 6);
        b.Property(e => e.Longitude).HasPrecision(9, 6);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Organization_Site_Code");

        b.HasOne(e => e.LegalEntity).WithMany(e => e.Sites)
            .HasForeignKey(e => e.LegalEntityId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Site_LegalEntity");
        b.HasOne(e => e.SiteType).WithMany()
            .HasForeignKey(e => e.SiteTypeId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Site_SiteType");
        b.HasOne(e => e.Country).WithMany()
            .HasForeignKey(e => e.CountryId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Site_CorePlatform_Country");
        b.HasOne(e => e.Region).WithMany()
            .HasForeignKey(e => e.RegionId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Site_CorePlatform_Region");
        b.HasOne(e => e.TimeZone).WithMany()
            .HasForeignKey(e => e.TimeZoneId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Site_CorePlatform_TimeZone");
    }
}
```

```

}

public sealed class PlantConfiguration : IEntityTypeConfiguration<Plant>
{
    public void Configure(EntityTypeBuilder<Plant> b)
    {
        b.ToTable("Plant", NexusSql.Organization);
        b.HasKey(e => e.PlantId).HasName("PK_Organization_Plant");
        b.Property(e => e.Code).HasCode(40);
        b.Property(e => e.Name).HasName(200);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.SiteId, e.Code }).IsUnique()
            .HasDatabaseName("UQ_Organization_Plant_SiteId_Code");

        b.HasOne(e => e.Site).WithMany(e => e.Plants)
            .HasForeignKey(e => e.SiteId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Plant_Site");
        b.HasOne(e => e.PlantType).WithMany()
            .HasForeignKey(e => e.PlantTypeId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Plant_PlantType");
    }
}

public sealed class DepartmentConfiguration : IEntityTypeConfiguration<Department>
{
    public void Configure(EntityTypeBuilder<Department> b)
    {
        b.ToTable("Department", NexusSql.Organization);
        b.HasKey(e => e.DepartmentId).HasName("PK_Organization_Department");
        b.Property(e => e.Code).HasCode(40);
        b.Property(e => e.Name).HasName(180);
        b.HasIndex(e => new { e.LegalEntityId, e.Code }).IsUnique()
            .HasDatabaseName("UQ_Organization_Department_LegalEntityId_Code");
        b.HasOne(e => e.LegalEntity).WithMany(e => e.Departments)
            .HasForeignKey(e => e.LegalEntityId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Department_LegalEntity");
        b.HasOne(e => e.ParentDepartment).WithMany(e => e.Children)
            .HasForeignKey(e => e.ParentDepartmentId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Department_ParentDepartment");
        b.HasOne(e => e.DepartmentType).WithMany()
            .HasForeignKey(e => e.DepartmentTypeId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Department_DepartmentType");
    }
}

```

8.8 Person and contact configurations

The person row is not the login row. `ApplicationUserId` is optional and unique when present; the person can exist without a login account, while a login account can be linked to only one person record.

```

public sealed class PersonConfiguration : IEntityTypeConfiguration<Person>
{
    public void Configure(EntityTypeBuilder<Person> b)
    {
        b.ToTable("Person", NexusSql.Organization);
        b.HasKey(e => e.PersonId).HasName("PK_Organization_Person");

        b.Property(e => e.EmployeeNumber).HasMaxLength(40);
        b.Property(e => e.FirstName).HasMaxLength(120).IsRequired();
        b.Property(e => e.LastName).HasMaxLength(120).IsRequired();
        b.Property(e => e.DisplayName).HasMaxLength(240).IsRequired();
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.EmployeeNumber).IsUnique()
            .HasFilter("[EmployeeNumber] IS NOT NULL")
            .HasDatabaseName("UQ_Organization_Person_EmployeeNumber");

        b.HasIndex(e => e.ApplicationUserId).IsUnique()
            .HasFilter("[ApplicationUserId] IS NOT NULL")
            .HasDatabaseName("UQ_Organization_Person_ApplicationUserId");

        b.HasOne(e => e.PersonType).WithMany()
            .HasForeignKey(e => e.PersonTypeId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Person_PersonType");
        b.HasOne(e => e.LegalEntity).WithMany()
            .HasForeignKey(e => e.LegalEntityId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Person_LegalEntity");
    }
}

```

```

        b.HasOne(e => e.ApplicationUser).WithOne()
            .HasForeignKey<Person>(e => e.ApplicationUserId)
            .OnDelete(DeleteBehavior.SetNull)
            .HasConstraintName("FK_Organization_Person_Security_ApplicationUser");
    }
}

public sealed class PersonContactConfiguration
    : IEntityTypeConfiguration<PersonContact>
{
    public void Configure(EntityTypeBuilder<PersonContact> b)
    {
        b.ToTable("PersonContact", NexusSql.Organization);
        b.HasKey(e => e.PersonContactId)
            .HasName("PK_Organization_PersonContact");
        b.Property(e => e.Value).HasMaxLength(300).IsRequired();
        b.Property(e => e.IsPrimary).HasDefaultValue(false);
        b.Property(e => e.IsEmergency).HasDefaultValue(false);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasIndex(e => new { e.PersonId, e.ContactMethodTypeId, e.Value })
            .IsUnique()
            .HasDatabaseName("UQ_Organization_PersonContact_Person_Method_Value");

        b.HasOne(e => e.Person).WithMany(e => e.Contacts)
            .HasForeignKey(e => e.PersonId).OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_Organization_PersonContact_Person");
        b.HasOne(e => e.ContactMethodType).WithMany()
            .HasForeignKey(e => e.ContactMethodTypeId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_PersonContact_ContactMethodType");
    }
}

```

8.9 Employment, assignments, and team membership

Employment and assignment tables are time-bounded relationship tables. They should not be modelled as a few nullable columns on `Person`, because people move between departments, positions, vendors, teams, and shifts over time.

```

public sealed class EmploymentConfiguration : IEntityTypeConfiguration<Employment>
{
    public void Configure(EntityTypeBuilder<Employment> b)
    {
        b.ToTable("Employment", NexusSql.Organization);
        b.HasKey(e => e.EmploymentId).HasName("PK_Organization_Employment");
        b.Property(e => e.StartDate).HasColumnType("date");
        b.Property(e => e.EndDate).HasColumnType("date");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.PersonId, e.StartDate })
            .HasDatabaseName("IX_Organization_Employment_Person_StartDate");
        b.HasIndex(e => new { e.LegalEntityId, e.EmploymentStatusId })
            .HasDatabaseName("IX_Organization_Employment_Entity_Status");

        b.HasOne(e => e.Person).WithMany(e => e.Employments)
            .HasForeignKey(e => e.PersonId).OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_Organization_Employment_Person");
        b.HasOne(e => e.LegalEntity).WithMany()
            .HasForeignKey(e => e.LegalEntityId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Employment_LegalEntity");
        b.HasOne(e => e.Position).WithMany()
            .HasForeignKey(e => e.PositionId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Employment_Position");
        b.HasOne(e => e.EmploymentStatus).WithMany()
            .HasForeignKey(e => e.EmploymentStatusId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Employment_EmploymentStatus");
        b.HasOne(e => e.Manager).WithMany()
            .HasForeignKey(e => e.ManagerPersonId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Employment_ManagerPerson");
    }
}

public sealed class DepartmentAssignmentConfiguration
    : IEntityTypeConfiguration<DepartmentAssignment>
{
    public void Configure(EntityTypeBuilder<DepartmentAssignment> b)
    {
        b.ToTable("DepartmentAssignment", NexusSql.Organization);
        b.HasKey(e => e.DepartmentAssignmentId)
            .HasName("PK_Organization_DepartmentAssignment");
        b.Property(e => e.StartDate).HasColumnType("date");
        b.Property(e => e.EndDate).HasColumnType("date");
    }
}

```

```

        b.HasIndex(e => new { e.PersonId, e.DepartmentId, e.StartDate })
            .IsUnique()
            .HasDatabaseName("UQ_Organization_DepartmentAssignment_Person_Department_Start");
        b.HasOne(e => e.Person).WithMany(e => e.DepartmentAssignments)
            .HasForeignKey(e => e.PersonId).OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_Organization_DepartmentAssignment_Person");
        b.HasOne(e => e.Department).WithMany()
            .HasForeignKey(e => e.DepartmentId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_DepartmentAssignment_Department");
        b.HasOne(e => e.Position).WithMany()
            .HasForeignKey(e => e.PositionId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_DepartmentAssignment_Position");
    }
}

```

8.10 Shift mapping

Shift configuration uses UTC timestamps, stable indexes by site and start time, and a unique person-per-shift assignment constraint. The `ConfirmedByUserId` passport points to `Security.ApplicationUser` because confirmation is a security/accountability action, not a person attribute.

```

public sealed class ShiftPatternConfiguration
    : IEntityTypeConfiguration<ShiftPattern>
{
    public void Configure(EntityTypeBuilder<ShiftPattern> b)
    {
        b.ToTable("ShiftPattern", NexusSql.Organization);
        b.HasKey(e => e.ShiftPatternId)
            .HasName("PK_Organization_ShiftPattern");
        b.Property(e => e.Code).HasCode(40);
        b.Property(e => e.Name).HasName(160);
        b.Property(e => e.DurationHours).HasPrecision(5, 2);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Organization_ShiftPattern_Code");
        b.HasOne(e => e.ShiftType).WithMany()
            .HasForeignKey(e => e.ShiftTypeId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_ShiftPattern_ShiftType");
    }
}

public sealed class ShiftConfiguration : IEntityTypeConfiguration<Shift>
{
    public void Configure(EntityTypeBuilder<Shift> b)
    {
        b.ToTable("Shift", NexusSql.Organization);
        b.HasKey(e => e.ShiftId).HasName("PK_Organization_Shift");
        b.Property(e => e.StartAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.EndAtUtc).HasColumnType("datetime2(7)");
        b.HasIndex(e => new { e.SiteId, e.StartAtUtc })
            .HasDatabaseName("IX_Organization_Shift_Site_StartAtUtc");
        b.HasOne(e => e.Site).WithMany()
            .HasForeignKey(e => e.SiteId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Shift_Site");
        b.HasOne(e => e.ShiftPattern).WithMany()
            .HasForeignKey(e => e.ShiftPatternId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_Shift_ShiftPattern");
    }
}

public sealed class ShiftAssignmentConfiguration
    : IEntityTypeConfiguration<ShiftAssignment>
{
    public void Configure(EntityTypeBuilder<ShiftAssignment> b)
    {
        b.ToTable("ShiftAssignment", NexusSql.Organization);
        b.HasKey(e => e.ShiftAssignmentId)
            .HasName("PK_Organization_ShiftAssignment");
        b.Property(e => e.IsConfirmed).HasDefaultValue(false);
        b.Property(e => e.ConfirmedAtUtc).HasColumnType("datetime2(7)");
        b.HasIndex(e => new { e.ShiftId, e.PersonId })
            .IsUnique()
            .HasDatabaseName("UQ_Organization_ShiftAssignment_Shift_Person");
        b.HasOne(e => e.Shift).WithMany(e => e.Assignments)
            .HasForeignKey(e => e.ShiftId).OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_Organization_ShiftAssignment_Shift");
        b.HasOne(e => e.Person).WithMany()
            .HasForeignKey(e => e.PersonId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_ShiftAssignment_Person");
        b.HasOne(e => e.ConfirmedByUser).WithMany()
            .HasForeignKey(e => e.ConfirmedByUserId).OnDelete(DeleteBehavior.Restrict)
    }
}

```

```

        .HasConstraintName("FK_Organization_ShiftAssignment_Security_ConfirmedByUser");
    }
}

```

8.11 Qualifications and certifications

Qualifications and certifications are split intentionally. A qualification can be an internal authorisation; a certification can be an external credential. The person-owned rows carry issue, expiry, status, and verifier information.

```

public sealed class QualificationConfiguration
    : IEntityTypeConfiguration<Qualification>
{
    public void Configure(EntityTypeBuilder<Qualification> b)
    {
        b.ToTable("Qualification", NexusSql.Organization);
        b.HasKey(e => e.QualificationId)
            .HasName("PK_Organization_Qualification");
        b.Property(e => e.Code).HasCode(50);
        b.Property(e => e.Name).HasName(200);
        b.Property(e => e.Description).HasMaxLength(1000);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Organization_Qualification_Code");
    }
}

public sealed class PersonQualificationConfiguration
    : IEntityTypeConfiguration<PersonQualification>
{
    public void Configure(EntityTypeBuilder<PersonQualification> b)
    {
        b.ToTable("PersonQualification", NexusSql.Organization);
        b.HasKey(e => e.PersonQualificationId)
            .HasName("PK_Organization_PersonQualification");
        b.Property(e => e.IssuedDate).HasColumnType("date");
        b.Property(e => e.ExpiresDate).HasColumnType("date");
        b.Property(e => e.VerifiedAtUtc).HasColumnType("datetime2(7)");
        b.HasIndex(e => new { e.PersonId, e.QualificationId })
            .HasDatabaseName("IX_Organization_PersonQualification_Person_Qualification");
        b.HasOne(e => e.Person).WithMany(e => e.Qualifications)
            .HasForeignKey(e => e.PersonId).OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_Organization_PersonQualification_Person");
        b.HasOne(e => e.Qualification).WithMany()
            .HasForeignKey(e => e.QualificationId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_PersonQualification_Qualification");
        b.HasOne(e => e.QualificationStatus).WithMany()
            .HasForeignKey(e => e.QualificationStatusId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_PersonQualification_QualificationStatus");
        b.HasOne(e => e.VerifiedByUser).WithMany()
            .HasForeignKey(e => e.VerifiedById).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_PersonQualification_Security_VerifiedByUser");
    }
}

// Certification and PersonCertification use the same structure as Qualification.
// Keep external certificate details in Certification; keep person ownership in PersonCertification.

```

8.12 Staffing scenario configurations

Staffing scenarios are the bridge to the NEXUS-1 personnel stress-test screens. The model records the scenario definition, requirements, saved evaluation, and gap rows rather than leaving the result as a transient UI calculation.

```

public sealed class PersonnelRequirementConfiguration
    : IEntityTypeConfiguration<PersonnelRequirement>
{
    public void Configure(EntityTypeBuilder<PersonnelRequirement> b)
    {
        b.ToTable("PersonnelRequirement", NexusSql.Organization);
        b.HasKey(e => e.PersonnelRequirementId)
            .HasName("PK_Organization_PersonnelRequirement");
        b.Property(e => e.RequiredCount).HasDefaultValue(1);
        b.Property(e => e.ValidFromUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ValidToUtc).HasColumnType("datetime2(7)");
    }
}

```



```

        b.HasIndex(e => new { e.SiteId, e.PlantId, e.PositionId })
        .HasDatabaseName("IX_Organization_PersonnelRequirement_Scope_Position");
    b.HasOne(e => e.Site).WithMany()
        .HasForeignKey(e => e.SiteId).OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_Organization_PersonnelRequirement_Site");
    b.HasOne(e => e.Plant).WithMany()
        .HasForeignKey(e => e.PlantId).OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_Organization_PersonnelRequirement_Plant");
    b.HasOne(e => e.Department).WithMany()
        .HasForeignKey(e => e.DepartmentId).OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_Organization_PersonnelRequirement_Department");
    b.HasOne(e => e.Position).WithMany()
        .HasForeignKey(e => e.PositionId).OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_Organization_PersonnelRequirement_Position");
    }
}

public sealed class StaffingScenarioConfiguration
    : IEntityTypeConfiguration<StaffingScenario>
{
    public void Configure(EntityTypeBuilder<StaffingScenario> b)
    {
        b.ToTable("StaffingScenario", NexusSql.Organization);
        b.HasKey(e => e.StaffingScenarioId)
            .HasName("PK_Organization_StaffingScenario");
        b.Property(e => e.Code).HasCode(50);
        b.Property(e => e.Name).HasName(200);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Organization_StaffingScenario_Code");
        b.HasOne(e => e.Site).WithMany()
            .HasForeignKey(e => e.SiteId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_StaffingScenario_Site");
        b.HasOne(e => e.CreatedByUser).WithMany()
            .HasForeignKey(e => e.CreatedById).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_StaffingScenario_Security_CreatedByUser");
    }
}

public sealed class StaffingScenarioGapConfiguration
    : IEntityTypeConfiguration<StaffingScenarioGap>
{
    public void Configure(EntityTypeBuilder<StaffingScenarioGap> b)
    {
        b.ToTable("StaffingScenarioGap", NexusSql.Organization);
        b.HasKey(e => e.StaffingScenarioGapId)
            .HasName("PK_Organization_StaffingScenarioGap");
        b.Property(e => e.RequiredCount).HasDefaultValue(0);
        b.Property(e => e.AvailableCount).HasDefaultValue(0);
        b.Property(e => e.MissingCount).HasDefaultValue(0);
        b.HasOne(e => e.StaffingScenarioResult).WithMany(e => e.Gaps)
            .HasForeignKey(e => e.StaffingScenarioResultId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_Organization_StaffingScenarioGap_Result");
        b.HasOne(e => e.Position).WithMany()
            .HasForeignKey(e => e.PositionId).OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Organization_StaffingScenarioGap_Position");
    }
}

```

8.13 Cross-schema passport rules

PASSPORT	USED BY	EF CORE RULE
CorePlatform.Country	LegalEntity, Site, PersonAddress	OnDelete(DeleteBehavior.Restrict)
CorePlatform.Region	Site, PersonAddress	Optional when the country has no region subdivision.
CorePlatform.TimeZone	Site	Required for shift scheduling and local display.
Security.ApplicationUser	Person, verifiers, creators, evaluators	Set null for optional login link; restrict for accountability rows.

PASSPORT	USED BY	EF CORE RULE
Organization.Plant	Later consumed by ReactorFleet.Unit	Organization owns the plant container; ReactorFleet owns the machine.

8.14 Migration and verification notes

The migration for Organization should run after `CorePlatform` and `Security`. A generated migration is acceptable only if the generated names match the stable naming discipline of the atlas.

```
// 1. The model discovers Organization configuration classes.
var entityNames = context.Model.GetEntityTypes()
    .Where(t => t.GetSchema() == "Organization")
    .Select(t => t.GetTableName())
    .OrderBy(t => t)
    .ToList();

entityNames.Should().Contain(new[]
{
    "LegalEntity", "Site", "Plant", "Department", "Person", "Shift", "StaffingScenario"
});

// 2. Every Organization table has a stable schema name.
context.Model.GetEntityTypes()
    .Where(t => t.ClrType.Namespace!.Contains("Organization"))
    .Should().OnlyContain(t => t.GetSchema() == "Organization");

// 3. No accidental cascade deletes across CorePlatform or Security passports.
context.Model.GetEntityTypes()
    .SelectMany(t => t.GetForeignKeys())
    .Where(fk => fk.PrincipalEntityType.GetSchema() is "CorePlatform" or "Security")
    .Should().OnlyContain(fk => fk.DeleteBehavior == DeleteBehavior.Restrict
        || fk.DeleteBehavior == DeleteBehavior.SetNull);

// 4. Names are stable: no generated hexadecimal names in migrations.
var indexes = context.Model.GetEntityTypes()
    .SelectMany(e => e.GetIndexes())
    .Select(i => i.GetDatabaseName());

indexes.Should().OnlyContain(n => n is not null && !n.Contains("IX_"));
```

Honest boundary. These configuration files are an enterprise-grade mapping pattern for the NEXUS-1 companion system. They are not a licensed HR system, not a nuclear staffing compliance package, and not a replacement for site-specific personnel governance. Their job is to make the demonstration's human and organizational model explicit, keyed, auditable, and reviewable.

8.15 Configuration index

LegalEntityTypeConfiguration - maps

Organization.LegalEntityType

SiteTypeConfiguration - maps

Organization.SiteType

PlantTypeConfiguration - maps

Organization.PlantType

DepartmentTypeConfiguration - maps

Organization.DepartmentType

TeamTypeConfiguration - maps

Organization.TeamType

PersonTypeConfiguration - maps

Organization.PersonType

EmploymentStatusConfiguration - maps

Organization.EmploymentStatus

ShiftTypeConfiguration - maps

Organization.ShiftType

QualificationStatusConfiguration - maps
Organization.QualificationStatus

ContactMethodTypeConfiguration - maps
Organization.ContactMethodType

LegalEntityConfiguration - maps
Organization.LegalEntity

SiteConfiguration - maps Organization.Site

PlantConfiguration - maps Organization.Plant

BuildingConfiguration - maps
Organization.Building

DepartmentConfiguration - maps
Organization.Department

TeamConfiguration - maps Organization.Team

PositionConfiguration - maps
Organization.Position

PersonConfiguration - maps Organization.Person

PersonContactConfiguration - maps
Organization.PersonContact

PersonAddressConfiguration - maps
Organization.PersonAddress

EmploymentConfiguration - maps
Organization.Employment

ContractorEngagementConfiguration - maps
Organization.ContractorEngagement

DepartmentAssignmentConfiguration - maps
Organization.DepartmentAssignment

TeamMembershipConfiguration - maps
Organization.TeamMembership

ShiftPatternConfiguration - maps
Organization.ShiftPattern

ShiftConfiguration - maps Organization.Shift

ShiftAssignmentConfiguration - maps
Organization.ShiftAssignment

OnCallRosterConfiguration - maps
Organization.OnCallRoster

QualificationConfiguration - maps
Organization.Qualification

PersonQualificationConfiguration - maps
Organization.PersonQualification

CertificationConfiguration - maps
Organization.Certification

PersonCertificationConfiguration - maps
Organization.PersonCertification

PersonnelRequirementConfiguration - maps
Organization.PersonnelRequirement

StaffingScenarioConfiguration - maps
Organization.StaffingScenario

StaffingScenarioRequirementConfiguration - maps
Organization.StaffingScenarioRequirement

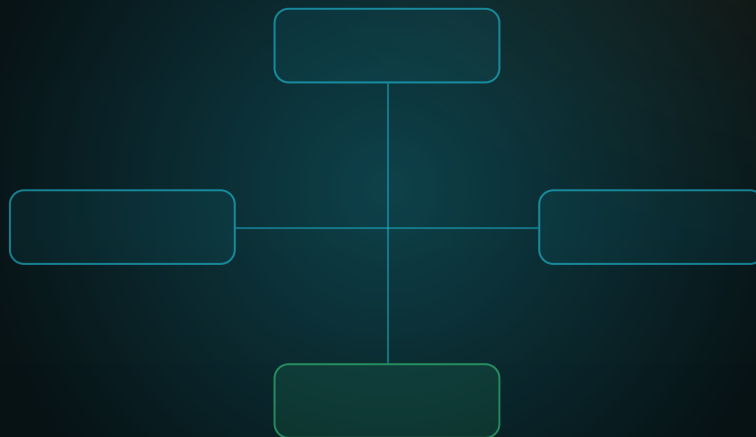
StaffingScenarioResultConfiguration - maps
Organization.StaffingScenarioResult

StaffingScenarioGapConfiguration - maps
Organization.StaffingScenarioGap

FROM ENTITY TO CONTEXT

Chapter 9 - ReactorFleet Configurations

The configuration backbone for units, systems, equipment, reactor core, power conversion and snapshots.



CONFIGURATION ATLAS CHAPTER

One physical identity - many EF Core mappings.

ReactorFleet provides the UnitId, EquipmentId and PlantSystemId passports that later schemas depend on.

Chapter 9

ReactorFleet Configurations

The EF Core configuration files for the NEXUS-1 ReactorFleet schema: the schema that turns physical plant identity into stable C# mappings, stable SQL names, and reusable foreign-key passports.

Part II - The Configuration Atlas
Follows Chapter 8 - Organization Configurations.

ReactorFleet is the sector most other sectors trust. If Unit, PlantSystem and Equipment are mapped badly, everything above them inherits ambiguity.

Chapter 9 - ReactorFleet Configurations

This chapter maps the physical backbone of NEXUS-1 into EF Core configuration files. It is the C# companion to the ReactorFleet SQL schema: one configuration class per table, explicit table names, explicit keys, explicit constraint names, and deliberate delete behaviour.

Schema

ReactorFleet

Tables

48 configuration files

Lookup files

16 typed reference
configs

Principal passports

UnitId, EquipmentId,
PlantSystemId

Why this schema matters

The ReactorFleet schema is not just another bounded context. It is the physical vocabulary of the platform. A digital twin cannot bind a signal, an alarm cannot point to a unit, a maintenance work order cannot reference a component, and a root-cause graph cannot traverse equipment unless the fleet model gives those things stable identities.

The EF Core mapping therefore has one job above all others: preserve identity without surprise. Code must not invent table names, relationship names, index names or cascade behaviour silently. A future migration should look like a deliberate engineering change, not like EF Core guessing in the dark.

Rule for this chapter. ReactorFleet may contain rich navigation properties, but the database vocabulary stays authoritative: stable SQL table names, stable primary-key names, stable foreign-key names and explicit indexes.

Configuration file catalogue

The SQL atlas contains forty-eight ReactorFleet tables. In this EF Core atlas that becomes forty-eight configuration files. The point is not file count for its own sake; the point is that a developer can open exactly one mapping file for the table they are changing.

GROUP	TABLE	CONFIGURATION FILE	ROLE IN MODEL
Lookup	ReactorFleet.UnitType	UnitTypeConfiguration.cs	Classifies the unit as large PWR, compact PWR, SMR module, simulator unit, or support unit.
Lookup	ReactorFleet.UnitStatus	UnitStatusConfiguration.cs	Lifecycle and operating status: planned, online, offline, outage, shutdown, decommissioning.
Lookup	ReactorFleet.OperationalMode	OperationalModeConfiguration.cs	Console operating mode for the unit: power, load-follow, startup, shutdown, trip, maintenance, training.
Lookup	ReactorFleet.ReactorType	ReactorTypeConfiguration.cs	Classifies the reactor model: PWR, compact PWR, SMR-PWR demonstrator, simulator.
Lookup	ReactorFleet.CoolantLoopType	CoolantLoopTypeConfiguration.cs	Classifies primary loops and integral compact loops.
Lookup	ReactorFleet.SystemType	SystemTypeConfiguration.cs	Classifies plant systems: primary, secondary, electrical, safety, support, I&C.
Lookup	ReactorFleet.EquipmentType	EquipmentTypeConfiguration.cs	Classifies equipment: pump, valve, steam generator, pressurizer, turbine, generator, transformer.
Lookup	ReactorFleet.EquipmentStatus	EquipmentStatusConfiguration.cs	Classifies equipment condition: available, running, standby, degraded, unavailable, retired.
Lookup	ReactorFleet.EquipmentCriticality	EquipmentCriticalityConfiguration.cs	Classifies operational criticality for maintenance, RCA, and dashboards.
Lookup	ReactorFleet.FuelAssemblyType	FuelAssemblyTypeConfiguration.cs	Classifies fuel assembly design families and lattice arrangements.
Lookup	ReactorFleet.FuelMaterialType	FuelMaterialTypeConfiguration.cs	Classifies illustrative fuel material families used by the demonstration.

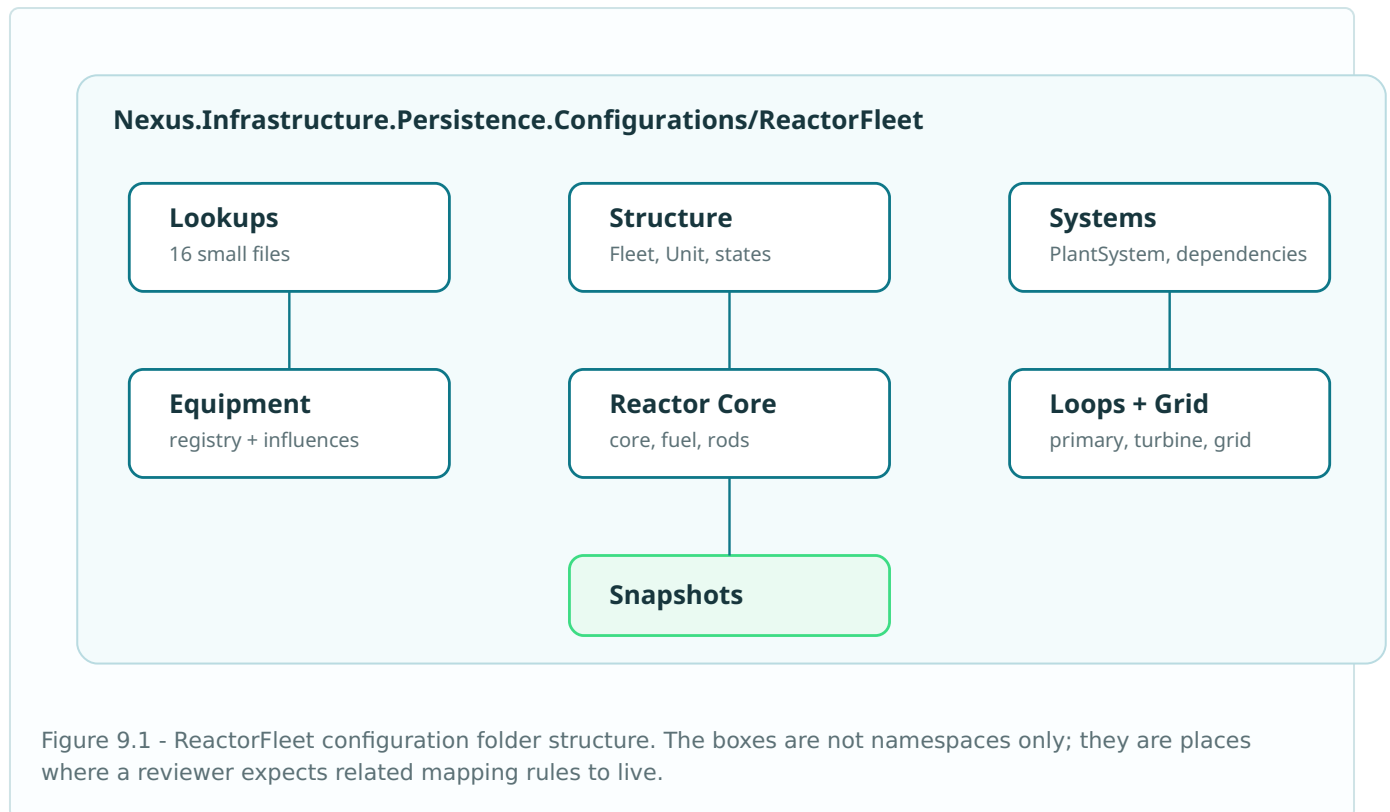
GROUP	TABLE	CONFIGURATION FILE	ROLE IN MODEL
Lookup	ReactorFleet.ControlRodMaterial	ControlRodMaterialConfiguration.cs	Classifies absorber materials such as B4C or Ag-In-Cd in the demonstration vocabulary.
Lookup	ReactorFleet.RodBankType	RodBankTypeConfiguration.cs	Classifies regulating, shutdown, safety, and calibration rod banks.
Lookup	ReactorFleet.SteamGeneratorType	SteamGeneratorTypeConfiguration.cs	Classifies U-tube, once-through, compact, or integral steam-generator representations.
Lookup	ReactorFleet.PumpType	PumpTypeConfiguration.cs	Classifies reactor coolant, feedwater, condensate, auxiliary, and service-water pumps.
Lookup	ReactorFleet.GridConnectionStatus	GridConnectionStatusConfiguration.cs	Classifies synchronized, isolated, trip, islanded, test, or disconnected grid state.
Structure	ReactorFleet.Fleet	FleetConfiguration.cs	Top-level grouping of plant units for the fleet dashboard and aggregate views.
Structure	ReactorFleet.FleetPlant	FleetPlantConfiguration.cs	Connects a fleet to the organization plants it contains without duplicating plant identity.
Structure	ReactorFleet.Unit	UnitConfiguration.cs	One physical or simulated reactor unit; the most important passport provider for later sectors.
Structure	ReactorFleet.UnitDesignParameter	UnitDesignParameterConfiguration.cs	Named design constants: thermal rating, pressure, loop count, fuel mass, design limits.
Structure	ReactorFleet.UnitOperationalState	UnitOperationalStateConfiguration.cs	Append-style operating-state history for the unit.
Systems	ReactorFleet.PlantSystem	PlantSystemConfiguration.cs	Functional system tree: primary loop, feedwater, turbine train, electrical, safety, service.
Systems	ReactorFleet.SystemDependency	SystemDependencyConfiguration.cs	Directed dependency between systems, used by System Dependencies and RCA preparation.
Equipment	ReactorFleet.EquipmentLocation	EquipmentLocationConfiguration.cs	Physical or logical location used by component registry, maintenance, radiation and access views.

GROUP	TABLE	CONFIGURATION FILE	ROLE IN MODEL
Equipment	<code>ReactorFleet.Equipment</code>	<code>EquipmentConfiguration.cs</code>	The central component registry table. Later sectors reference equipment instead of inventing component IDs.
Equipment	<code>ReactorFleet.EquipmentDependency</code>	<code>EquipmentDependencyConfiguration.cs</code>	Directed component-to-component influence with delay and signed weight.
Equipment	<code>ReactorFleet.EquipmentExternalReference</code>	<code>EquipmentExternalReferenceConfiguration.cs</code>	External tag, vendor, drawing, procedure, or document reference tied to one equipment row.
Reactor	<code>ReactorFleet.Reactor</code>	<code>ReactorConfiguration.cs</code>	The reactor vessel and reactor-level design anchor for a unit.
Reactor	<code>ReactorFleet.ReactorCore</code>	<code>ReactorCoreConfiguration.cs</code>	Core loading and neutronic design envelope for a reactor.
Reactor	<code>ReactorFleet.CoreRegion</code>	<code>CoreRegionConfiguration.cs</code>	Named core regions used by fuel assembly and flux-map tables.
Reactor	<code>ReactorFleet.FuelAssembly</code>	<code>FuelAssemblyConfiguration.cs</code>	Fuel assembly identity, position and design information for the illustrative core.
Reactor	<code>ReactorFleet.ControlRodBank</code>	<code>ControlRodBankConfiguration.cs</code>	Bank-level rod grouping used by operator controls, RL policy and snapshots.
Reactor	<code>ReactorFleet.ControlRod</code>	<code>ControlRodConfiguration.cs</code>	Individual rods inside a bank. Equipment link is optional but useful for maintenance.
Reactor	<code>ReactorFleet.RodPositionSnapshot</code>	<code>RodPositionSnapshotConfiguration.cs</code>	Time-series snapshots of rod-bank position.
Reactor	<code>ReactorFleet.BoronChemistrySnapshot</code>	<code>BoronChemistrySnapshotConfiguration.cs</code>	Time-series snapshots of soluble absorber and chemistry values.
Reactor	<code>ReactorFleet.NeutronicsParameterSet</code>	<code>NeutronicsParameterSetConfiguration.cs</code>	Versioned point-kinetics parameter set for the demonstration solver.
Primary/ Secondary	<code>ReactorFleet.PrimaryLoop</code>	<code>PrimaryLoopConfiguration.cs</code>	Primary loop or compact/integral loop representation.
Primary/ Secondary	<code>ReactorFleet.ReactorCoolantPump</code>	<code>ReactorCoolantPumpConfiguration.cs</code>	Reactor coolant pump linked to equipment registry and primary loop.
Primary/ Secondary	<code>ReactorFleet.Pressurizer</code>	<code>PressurizerConfiguration.cs</code>	Pressurizer representation and design envelope.
Primary/ Secondary	<code>ReactorFleet.SteamGenerator</code>	<code>SteamGeneratorConfiguration.cs</code>	Steam-generator object bridging primary and secondary sides.

GROUP	TABLE	CONFIGURATION FILE	ROLE IN MODEL
Power Conversion	ReactorFleet.TurbineTrain	TurbineTrainConfiguration.cs	Main turbine train for the unit.
Power Conversion	ReactorFleet.TurbineStage	TurbineStageConfiguration.cs	HP, IP, LP or compact turbine stage.
Power Conversion	ReactorFleet.Generator	GeneratorConfiguration.cs	Electrical generator connected to the turbine train and grid interface.
Power Conversion	ReactorFleet.Transformer	TransformerConfiguration.cs	Generator step-up or auxiliary transformer.
Power Conversion	ReactorFleet.SwitchyardBay	SwitchyardBayConfiguration.cs	Switchyard bay or electrical connection point.
Power Conversion	ReactorFleet.GridConnection	GridConnectionConfiguration.cs	Grid interface state and design values.
Snapshots	ReactorFleet.FleetSnapshot	FleetSnapshotConfiguration.cs	Fleet-level aggregate output, online count and demand snapshot.
Snapshots	ReactorFleet.UnitPowerSnapshot	UnitPowerSnapshotConfiguration.cs	Unit-level output, load, frequency and power-factor snapshot.

Folder structure

The folder structure mirrors the domain groups. Lookup configurations stay small and regular. The main physical hierarchy starts with fleet and unit, then moves through systems, equipment, reactor core, loops, power conversion and snapshots.



Entity relationship focus

The C# model is allowed to be comfortable to navigate, but the map below is the dependency discipline the configuration files protect.

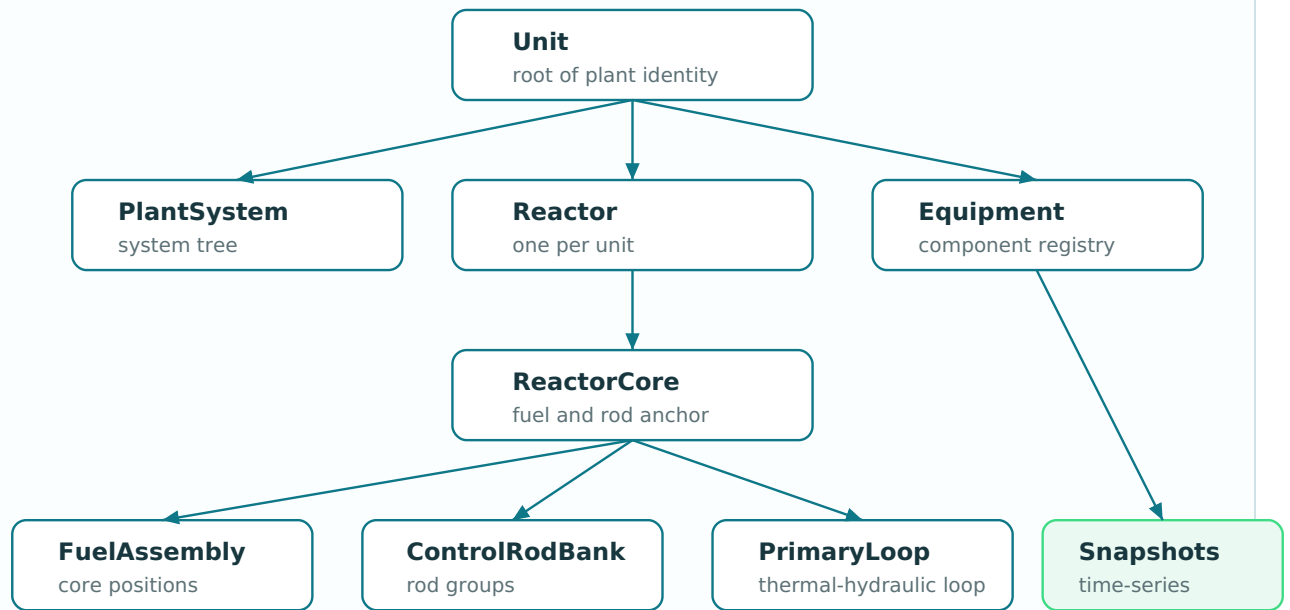
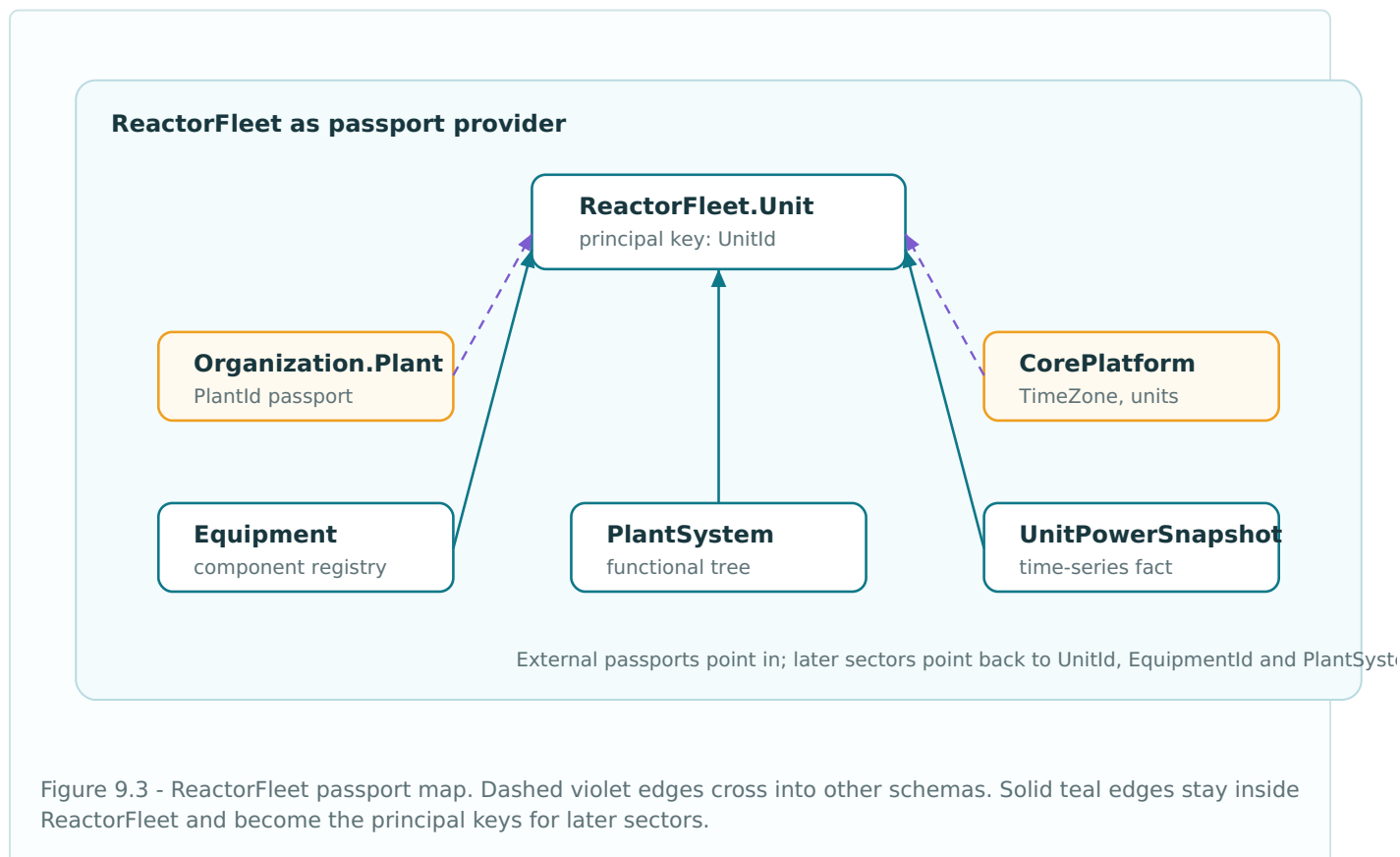


Figure 9.2 - Entity mapping focus. Unit anchors the plant, ReactorCore anchors nuclear structure, and Equipment anchors the component registry.

Passport map

ReactorFleet imports a small number of external passports and exports many more. Organization provides the plant identity; CorePlatform provides time-zone and engineering-unit reference rows; Security can identify the user who recorded an operating state. Later sectors depend heavily on ReactorFleet principals.



Delete behaviour. Principal operational tables use **Restrict** where deleting would erase meaning in other sectors. Append-style snapshots and child rows may cascade only when the parent is a local aggregate root and not a shared reference.

Shared mapping infrastructure

The lookup configuration base is intentionally boring. Every typed lookup table uses the same columns, the same defaults and the same row-version discipline. This keeps the schema predictable and makes seed-data scripts easier to audit.

```
namespace Nexus.Infrastructure.Persistence.Configurations;

internal static partial class NexusSql
{
    public const string ReactorFleet = "ReactorFleet";
    public const string Organization = "Organization";
    public const string CorePlatform = "CorePlatform";
    public const string Security = "Security";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class ReactorFleetLookupConfiguration<TEntity>
    : IEntityConfiguration<TEntity>
    where TEntity : ReactorFleetLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(160)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

internal static class ReactorFleetMappingExtensions
{
    public static PropertyBuilder<string> HasCode(
        this PropertyBuilder<string> property, int length = 40) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<string> HasName(
        this PropertyBuilder<string> property, int length = 180) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);
}
```

Lookup configurations

ReactorFleet contains sixteen lookup tables. The files are separate so each lookup can receive comments, seed data and future rules without touching a monolithic lookup mapper.

```
public sealed class UnitTypeConfiguration
    : ReactorFleetLookupConfiguration<UnitType>
{
    public override void Configure(EntityTypeBuilder<UnitType> b)
    {
        b.ToTable("UnitType", NexusSql.ReactorFleet);
        b.HasKey(e => e.UnitTypeId).HasName("PK_ReactorFleet_UnitType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_UnitType_Code");
    }
}

public sealed class ReactorTypeConfiguration
    : ReactorFleetLookupConfiguration<ReactorType>
{
    public override void Configure(EntityTypeBuilder<ReactorType> b)
    {
        b.ToTable("ReactorType", NexusSql.ReactorFleet);
        b.HasKey(e => e.ReactorTypeId).HasName("PK_ReactorFleet_ReactorType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_ReactorType_Code");
    }
}

public sealed class EquipmentTypeConfiguration
    : ReactorFleetLookupConfiguration<EquipmentType>
{
    public override void Configure(EntityTypeBuilder<EquipmentType> b)
    {
        b.ToTable("EquipmentType", NexusSql.ReactorFleet);
        b.HasKey(e => e.EquipmentTypeId).HasName("PK_ReactorFleet_EquipmentType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_EquipmentType_Code");
    }
}

// The remaining lookup files are deliberately separate but identical in shape:
// UnitStatusConfiguration, OperationalModeConfiguration, CoolantLoopTypeConfiguration,
// SystemTypeConfiguration, EquipmentStatusConfiguration, EquipmentCriticalityConfiguration,
// FuelAssemblyTypeConfiguration, FuelMaterialTypeConfiguration, ControlRodMaterialConfiguration,
// RodBankTypeConfiguration, SteamGeneratorTypeConfiguration, PumpTypeConfiguration,
// GridConnectionStatusConfiguration.
```

Fleet and fleet-plant mapping

The fleet layer connects the NEXUS-1 dashboard view to Organization. It imports legal-entity identity and plant identity rather than duplicating them.

```
public sealed class FleetConfiguration : IEntityTypeConfiguration<Fleet>
{
    public void Configure(EntityTypeBuilder<Fleet> b)
    {
        b.ToTable("Fleet", NexusSql.ReactorFleet);

        b.HasKey(e => e.FleetId)
            .HasName("PK_ReactorFleet_Fleet");

        b.Property(e => e.Code).HasCode(30);
        b.Property(e => e.Name).HasName(180);
        b.Property(e => e.Description).HasMaxLength(800);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_Fleet_Code");

        b.HasOne(e => e.OperatorLegalEntity)
            .WithMany()
            .HasForeignKey(e => e.OperatorLegalEntityId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Fleet_Organization_LegalEntity");

        b.HasOne(e => e.TimeZone)
            .WithMany()
            .HasForeignKey(e => e.TimeZoneId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Fleet_CorePlatform_TimeZone");
    }
}

public sealed class FleetPlantConfiguration : IEntityTypeConfiguration<FleetPlant>
{
    public void Configure(EntityTypeBuilder<FleetPlant> b)
    {
        b.ToTable("FleetPlant", NexusSql.ReactorFleet);
        b.HasKey(e => e.FleetPlantId).HasName("PK_ReactorFleet_FleetPlant");
        b.Property(e => e.JoinedAtUtc).HasUtcDefault();

        b.HasIndex(e => new { e.FleetId, e.PlantId }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_FleetPlant_FleetId_PlantId");

        b.HasOne(e => e.Fleet).WithMany(e => e.FleetPlants)
            .HasForeignKey(e => e.FleetId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_FleetPlant_Fleet");

        b.HasOne(e => e.Plant).WithMany()
            .HasForeignKey(e => e.PlantId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_FleetPlant_Organization_Plant");
    }
}
```


Unit mapping

`UnitConfiguration` is the most important mapping in the chapter. Many later schemas refer to `ReactorFleet.Unit`. The code and index names must therefore stay clean, short and human-readable.

```
public sealed class UnitConfiguration : IEntityTypeConfiguration<Unit>
{
    public void Configure(EntityTypeBuilder<Unit> b)
    {
        b.ToTable("Unit", NexusSql.ReactorFleet);

        b.HasKey(e => e.UnitId)
            .HasName("PK_ReactorFleet_Unit");

        b.Property(e => e.Code).HasCode(20);
        b.Property(e => e.Name).HasName(160);
        b.Property(e => e.ShortName).HasMaxLength(40);
        b.Property(e => e.DesignNetMwe).HasPrecision(10, 2);
        b.Property(e => e.DesignThermalMwt).HasPrecision(10, 2);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_Unit_Code");

        b.HasIndex(e => new { e.PlantId, e.Code }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_Unit_PlantId_Code");

        b.HasOne(e => e.Plant).WithMany()
            .HasForeignKey(e => e.PlantId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Unit_Organization_Plant");

        b.HasOne(e => e.UnitType).WithMany()
            .HasForeignKey(e => e.UnitTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Unit_UnitType");

        b.HasOne(e => e.UnitStatus).WithMany()
            .HasForeignKey(e => e.UnitStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Unit_UnitStatus");

        b.HasOne(e => e.ReactorType).WithMany()
            .HasForeignKey(e => e.ReactorTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Unit_ReactorType");
    }
}

public sealed class UnitDesignParameterConfiguration
: IEntityTypeConfiguration<UnitDesignParameter>
{
    public void Configure(EntityTypeBuilder<UnitDesignParameter> b)
    {
        b.ToTable("UnitDesignParameter", NexusSql.ReactorFleet);
        b.HasKey(e => e.UnitDesignParameterId)
            .HasName("PK_ReactorFleet_UnitDesignParameter");

        b.Property(e => e.Code).HasCode(80);
        b.Property(e => e.Name).HasName(180);
        b.Property(e => e.Value).HasPrecision(18, 6);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasIndex(e => new { e.UnitId, e.Code }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_UnitDesignParameter_UnitId_Code");

        b.HasOne(e => e.Unit).WithMany(e => e.DesignParameters)
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_UnitDesignParameter_Unit");

        b.HasOne(e => e.EngineeringUnit).WithMany()
            .HasForeignKey(e => e.EngineeringUnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_UnitDesignParameter_CorePlatform_EngineeringUnit");
    }
}

public sealed class UnitOperationalStateConfiguration
: IEntityTypeConfiguration<UnitOperationalState>
{

```

```

public void Configure(EntityTypeBuilder<UnitOperationalState> b)
{
    b.ToTable("UnitOperationalState", NexusSql.ReactorFleet);
    b.HasKey(e => e.UnitOperationalStateId)
        .HasName("PK_ReactorFleet_UnitOperationalState");

    b.Property(e => e.RecordedAtUtc).HasColumnType("datetime2(7)");
    b.Property(e => e.Reason).HasMaxLength(500);

    b.HasIndex(e => new { e.UnitId, e.RecordedAtUtc })
        .HasDatabaseName("IX_ReactorFleet_UnitOperationalState_UnitId_RecordedAtUtc");

    b.HasOne(e => e.Unit).WithMany(e => e.OperationalStates)
        .HasForeignKey(e => e.UnitId)
        .OnDelete(DeleteBehavior.Cascade)
        .HasConstraintName("FK_ReactorFleet_UnitOperationalState_Unit");

    b.HasOne(e => e.OperationalMode).WithMany()
        .HasForeignKey(e => e.OperationalModeId)
        .OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_ReactorFleet_UnitOperationalState_OperationalMode");

    b.HasOne(e => e.RecordedByUser).WithMany()
        .HasForeignKey(e => e.RecordedByUserId)
        .OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_ReactorFleet_UnitOperationalState_Security_ApplicationUser");
}
}

```

Systems and equipment mapping

PlantSystem models functional hierarchy. **Equipment** models physical or logical components. Both become relationship hubs for alarms, maintenance, instrumentation, root-cause analysis and reporting.

```
public sealed class PlantSystemConfiguration : IEntityTypeConfiguration<PlantSystem>
{
    public void Configure(EntityTypeBuilder<PlantSystem> b)
    {
        b.ToTable("PlantSystem", NexusSql.ReactorFleet);
        b.HasKey(e => e.PlantSystemId).HasName("PK_ReactorFleet_PlantSystem");

        b.Property(e => e.Code).HasCode(50);
        b.Property(e => e.Name).HasName(180);
        b.Property(e => e.Path).HasMaxLength(500);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.UnitId, e.Code }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_PlantSystem_UnitId_Code");

        b.HasOne(e => e.Unit).WithMany(e => e.PlantSystems)
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_PlantSystem_Unit");

        b.HasOne(e => e.SystemType).WithMany()
            .HasForeignKey(e => e.SystemTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_PlantSystem_SystemType");

        b.HasOne(e => e.ParentPlantSystem).WithMany(e => e.Children)
            .HasForeignKey(e => e.ParentPlantSystemId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_PlantSystem_Parent");
    }
}

public sealed class SystemDependencyConfiguration : IEntityTypeConfiguration<SystemDependency>
{
    public void Configure(EntityTypeBuilder<SystemDependency> b)
    {
        b.ToTable("SystemDependency", NexusSql.ReactorFleet);
        b.HasKey(e => e.SystemDependencyId).HasName("PK_ReactorFleet_SystemDependency");
        b.Property(e => e.DependencyKind).HasMaxLength(50).IsRequired();
        b.Property(e => e.Weight).HasPrecision(9, 6);
        b.Property(e => e.PropagationDelaySeconds).HasPrecision(10, 3);

        b.HasIndex(e => new { e.SourcePlantSystemId, e.TargetPlantSystemId }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_SystemDependency_Source_Target");

        b.HasOne(e => e.SourcePlantSystem).WithMany(e => e.OutgoingDependencies)
            .HasForeignKey(e => e.SourcePlantSystemId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_SystemDependency_SourcePlantSystem");

        b.HasOne(e => e.TargetPlantSystem).WithMany(e => e.IncomingDependencies)
            .HasForeignKey(e => e.TargetPlantSystemId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_SystemDependency_TargetPlantSystem");
    }
}

public sealed class EquipmentConfiguration : IEntityTypeConfiguration<Equipment>
{
    public void Configure(EntityTypeBuilder<Equipment> b)
    {
        b.ToTable("Equipment", NexusSql.ReactorFleet);
        b.HasKey(e => e.EquipmentId).HasName("PK_ReactorFleet_Equipment");

        b.Property(e => e.Tag).HasMaxLength(80).IsRequired();
        b.Property(e => e.Name).HasName(200);
        b.Property(e => e.SerialNumber).HasMaxLength(80);
        b.Property(e => e.InstallationDate).HasColumnType("date");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.UnitId, e.Tag }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_Equipment_UnitId_Tag");

        b.HasIndex(e => new { e.PlantSystemId, e.EquipmentTypeId })
    }
}
```

```

        .HasDatabaseName("IX_ReactorFleet_Equipment_System_Type");

        b.HasOne(e => e.Unit).WithMany(e => e.Equipment)
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_Equipment_Unit");

        b.HasOne(e => e.PlantSystem).WithMany(e => e.Equipment)
            .HasForeignKey(e => e.PlantSystemId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Equipment_PlantSystem");

        b.HasOne(e => e.EquipmentType).WithMany()
            .HasForeignKey(e => e.EquipmentTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Equipment_EquipmentType");

        b.HasOne(e => e.EquipmentStatus).WithMany()
            .HasForeignKey(e => e.EquipmentStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Equipment_EquipmentStatus");
    }
}

public sealed class EquipmentDependencyConfiguration
    : IEntityTypeConfiguration<EquipmentDependency>
{
    public void Configure(EntityTypeBuilder<EquipmentDependency> b)
    {
        b.ToTable("EquipmentDependency", NexusSql.ReactorFleet);
        b.HasKey(e => e.EquipmentDependencyId)
            .HasName("PK_ReactorFleet_EquipmentDependency");

        b.Property(e => e.DependencyKind).HasMaxLength(50).IsRequired();
        b.Property(e => e.SignedWeight).HasPrecision(9, 6);
        b.Property(e => e.PropagationDelaySeconds).HasPrecision(10, 3);

        b.HasIndex(e => new { e.SourceEquipmentId, e.TargetEquipmentId }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_EquipmentDependency_Source_Target");

        b.HasOne(e => e.SourceEquipment).WithMany(e => e.OutgoingDependencies)
            .HasForeignKey(e => e.SourceEquipmentId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_EquipmentDependency_SourceEquipment");

        b.HasOne(e => e.TargetEquipment).WithMany(e => e.IncomingDependencies)
            .HasForeignKey(e => e.TargetEquipmentId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_EquipmentDependency_TargetEquipment");
    }
}

```

Reactor and core mapping

The reactor-core tables are not intended to be licensed reactor design data. They are a demonstrator-grade structure that gives the platform stable rows for reactor, core, regions, fuel assemblies, rods and neutronics parameters.

```
public sealed class ReactorConfiguration : IEntityConfiguration<Reactor>
{
    public void Configure(EntityTypeBuilder<Reactor> b)
    {
        b.ToTable("Reactor", NexusSql.ReactorFleet);
        b.HasKey(e => e.ReactorId).HasName("PK_ReactorFleet_Reactor");

        b.Property(e => e.Code).HasCode(40);
        b.Property(e => e.Name).HasName(160);
        b.Property(e => e.DesignPressureBar).HasPrecision(10, 4);
        b.Property(e => e.DesignInletTemperatureC).HasPrecision(10, 4);
        b.Property(e => e.DesignOutletTemperatureC).HasPrecision(10, 4);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();

        b.HasIndex(e => e.UnitId).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_Reactor_UnitId");

        b.HasOne(e => e.Unit).WithOne(e => e.Reactor)
            .HasForeignKey<Reactor>(e => e.UnitId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_Reactor_Unit");

        b.HasOne(e => e.ReactorType).WithMany()
            .HasForeignKey(e => e.ReactorTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Reactor_ReactorType");

        b.HasOne(e => e.ReactorVesselEquipment).WithMany()
            .HasForeignKey(e => e.ReactorVesselEquipmentId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_Reactor_ReactorVesselEquipment");
    }
}

public sealed class ReactorCoreConfiguration : IEntityConfiguration<ReactorCore>
{
    public void Configure(EntityTypeBuilder<ReactorCore> b)
    {
        b.ToTable("ReactorCore", NexusSql.ReactorFleet);
        b.HasKey(e => e.ReactorCoreId).HasName("PK_ReactorFleet_ReactorCore");
        b.Property(e => e.CoreName).HasName(120);
        b.Property(e => e.CycleNumber).HasDefaultValue(1);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.ReactorId, e.CycleNumber }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_ReactorCore_ReactorId_CycleNumber");

        b.HasOne(e => e.Reactor).WithMany(e => e.Cores)
            .HasForeignKey(e => e.ReactorId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_ReactorCore_Reactor");
    }
}

public sealed class CoreRegionConfiguration : IEntityConfiguration<CoreRegion>
{
    public void Configure(EntityTypeBuilder<CoreRegion> b)
    {
        b.ToTable("CoreRegion", NexusSql.ReactorFleet);
        b.HasKey(e => e.CoreRegionId).HasName("PK_ReactorFleet_CoreRegion");
        b.Property(e => e.Code).HasCode(30);
        b.Property(e => e.Name).HasName(100);
        b.Property(e => e.RadialZone).HasMaxLength(40);

        b.HasIndex(e => new { e.ReactorCoreId, e.Code }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_CoreRegion_CoreId_Code");

        b.HasOne(e => e.ReactorCore).WithMany(e => e.Regions)
            .HasForeignKey(e => e.ReactorCoreId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_CoreRegion_ReactorCore");
    }
}

public sealed class FuelAssemblyConfiguration : IEntityConfiguration<FuelAssembly>
{

```

```

public void Configure(EntityTypeBuilder<FuelAssembly> b)
{
    b.ToTable("FuelAssembly", NexusSql.ReactorFleet);
    b.HasKey(e => e.FuelAssemblyId).HasName("PK_ReactorFleet_FuelAssembly");
    b.Property(e => e.AssemblyCode).HasMaxLength(50).IsRequired();
    b.Property(e => e.CorePosition).HasMaxLength(20).IsRequired();
    b.Property(e => e.EnrichmentPercent).HasPrecision(8, 5);
    b.Property(e => e.BurnupGwdPerT).HasPrecision(10, 4);

    b.HasIndex(e => new { e.ReactorCoreId, e.CorePosition }).IsUnique()
        .HasDatabaseName("UQ_ReactorFleet_FuelAssembly_Core_Position");

    b.HasOne(e => e.ReactorCore).WithMany(e => e.FuelAssemblies)
        .HasForeignKey(e => e.ReactorCoreId)
        .OnDelete(DeleteBehavior.Cascade)
        .HasConstraintName("FK_ReactorFleet_FuelAssembly_ReactorCore");

    b.HasOne(e => e.CoreRegion).WithMany(e => e.FuelAssemblies)
        .HasForeignKey(e => e.CoreRegionId)
        .OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_ReactorFleet_FuelAssembly_CoreRegion");

    b.HasOne(e => e.FuelAssemblyType).WithMany()
        .HasForeignKey(e => e.FuelAssemblyTypeId)
        .OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_ReactorFleet_FuelAssembly_FuelAssemblyType");
}
}

```

Rod and neutronics mapping

Control rods are mapped as their own aggregate around a bank. The reinforcement-learning and operator-console layers can reference a bank without needing to know every individual rod row.

```
public sealed class ControlRodBankConfiguration
    : IEntityTypeConfiguration<ControlRodBank>
{
    public void Configure(EntityTypeBuilder<ControlRodBank> b)
    {
        b.ToTable("ControlRodBank", NexusSql.ReactorFleet);
        b.HasKey(e => e.ControlRodBankId).HasName("PK_ReactorFleet_ControlRodBank");
        b.Property(e => e.BankCode).HasMaxLength(30).IsRequired();
        b.Property(e => e.Name).HasMaxLength(120);
        b.Property(e => e.MinPositionPercent).HasPrecision(6, 3);
        b.Property(e => e.MaxPositionPercent).HasPrecision(6, 3);

        b.HasIndex(e => new { e.ReactorCoreId, e.BankCode }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_ControlRodBank_Core_BankCode");

        b.HasOne(e => e.ReactorCore).WithMany(e => e.ControlRodBanks)
            .HasForeignKey(e => e.ReactorCoreId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_ControlRodBank_ReactorCore");

        b.HasOne(e => e.RodBankType).WithMany()
            .HasForeignKey(e => e.RodBankTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_ControlRodBank_RodBankType");

        b.HasOne(e => e.ControlRodMaterial).WithMany()
            .HasForeignKey(e => e.ControlRodMaterialId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_ControlRodBank_ControlRodMaterial");
    }
}

public sealed class ControlRodConfiguration : IEntityTypeConfiguration<ControlRod>
{
    public void Configure(EntityTypeBuilder<ControlRod> b)
    {
        b.ToTable("ControlRod", NexusSql.ReactorFleet);
        b.HasKey(e => e.ControlRodId).HasName("PK_ReactorFleet_ControlRod");
        b.Property(e => e.RodCode).HasMaxLength(40).IsRequired();
        b.Property(e => e.CoreCoordinate).HasMaxLength(20);

        b.HasIndex(e => new { e.ControlRodBankId, e.RodCode }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_ControlRod_Bank_RodCode");

        b.HasOne(e => e.ControlRodBank).WithMany(e => e.ControlRods)
            .HasForeignKey(e => e.ControlRodBankId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_ControlRod_ControlRodBank");

        b.HasOne(e => e.Equipment).WithMany()
            .HasForeignKey(e => e.EquipmentId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_ControlRod_Equipment");
    }
}

public sealed class RodPositionSnapshotConfiguration
    : IEntityTypeConfiguration<RodPositionSnapshot>
{
    public void Configure(EntityTypeBuilder<RodPositionSnapshot> b)
    {
        b.ToTable("RodPositionSnapshot", NexusSql.ReactorFleet);
        b.HasKey(e => e.RodPositionSnapshotId)
            .HasName("PK_ReactorFleet_RodPositionSnapshot");
        b.Property(e => e.RecordedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.PositionPercent).HasPrecision(8, 4);
        b.Property(e => e.ReactivityWorthPcm).HasPrecision(12, 4);

        b.HasIndex(e => new { e.ControlRodBankId, e.RecordedAtUtc })
            .HasDatabaseName("IX_ReactorFleet_RodPositionSnapshot_Bank_Time");

        b.HasOne(e => e.ControlRodBank).WithMany(e => e.PositionSnapshots)
            .HasForeignKey(e => e.ControlRodBankId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_RodPositionSnapshot_ControlRodBank");
    }
}

public sealed class NeutronicsParameterSetConfiguration
```

```

: IEntityTypeConfiguration<NeutronicsParameterSet>
{
    public void Configure(EntityTypeBuilder<NeutronicsParameterSet> b)
    {
        b.ToTable("NeutronicsParameterSet", NexusSql.ReactorFleet);
        b.HasKey(e => e.NeutronicsParameterSetId)
            .HasName("PK_ReactorFleet_NeutronicsParameterSet");
        b.Property(e => e.Code).HasCode(60);
        b.Property(e => e.EffectiveDelayedNeutronFraction).HasPrecision(12, 9);
        b.Property(e => e.PromptNeutronLifetimeSeconds).HasPrecision(18, 12);
        b.Property(e => e.IsActive).HasDefaultValue(false);

        b.HasIndex(e => new { e.ReactorCoreId, e.Code }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_NeutronicsParameterSet_Core_Code");

        b.HasOne(e => e.ReactorCore).WithMany(e => e.NeutronicsParameterSets)
            .HasForeignKey(e => e.ReactorCoreId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_NeutronicsParameterSet_ReactorCore");
    }
}

```


Primary, secondary and grid mapping

Primary loop, steam generator and grid connection mappings bridge the physical plant to power-conversion views. Equipment links remain explicit so maintenance and RCA can refer to the same component registry.

```
public sealed class PrimaryLoopConfiguration : IEntityTypeConfiguration<PrimaryLoop>
{
    public void Configure(EntityTypeBuilder<PrimaryLoop> b)
    {
        b.ToTable("PrimaryLoop", NexusSql.ReactorFleet);
        b.HasKey(e => e.PrimaryLoopId).HasName("PK_ReactorFleet_PrimaryLoop");
        b.Property(e => e.LoopCode).HasMaxLength(20).IsRequired();
        b.Property(e => e.DesignFlowKgPerSecond).HasPrecision(14, 4);
        b.Property(e => e.DesignHotLegTemperatureC).HasPrecision(10, 4);
        b.Property(e => e.DesignColdLegTemperatureC).HasPrecision(10, 4);

        b.HasIndex(e => new { e.ReactorId, e.LoopCode }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_PrimaryLoop_Reactor_LoopCode");

        b.HasOne(e => e.Reactor).WithMany(e => e.PrimaryLoops)
            .HasForeignKey(e => e.ReactorId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_PrimaryLoop_Reactor");

        b.HasOne(e => e.CoolantLoopType).WithMany()
            .HasForeignKey(e => e.CoolantLoopTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_PrimaryLoop_CoolantLoopType");
    }
}

public sealed class ReactorCoolantPumpConfiguration
    : IEntityTypeConfiguration<ReactorCoolantPump>
{
    public void Configure(EntityTypeBuilder<ReactorCoolantPump> b)
    {
        b.ToTable("ReactorCoolantPump", NexusSql.ReactorFleet);
        b.HasKey(e => e.ReactorCoolantPumpId)
            .HasName("PK_ReactorFleet_ReactorCoolantPump");
        b.Property(e => e.DesignFlowKgPerSecond).HasPrecision(14, 4);
        b.Property(e => e.DesignHeadMeters).HasPrecision(12, 4);

        b.HasOne(e => e.PrimaryLoop).WithMany(e => e.ReactorCoolantPumps)
            .HasForeignKey(e => e.PrimaryLoopId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_ReactorCoolantPump_PrimaryLoop");

        b.HasOne(e => e.Equipment).WithMany()
            .HasForeignKey(e => e.EquipmentId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_ReactorCoolantPump_Equipment");

        b.HasOne(e => e.PumpType).WithMany()
            .HasForeignKey(e => e.PumpTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_ReactorCoolantPump_PumpType");
    }
}

public sealed class SteamGeneratorConfiguration : IEntityTypeConfiguration<SteamGenerator>
{
    public void Configure(EntityTypeBuilder<SteamGenerator> b)
    {
        b.ToTable("SteamGenerator", NexusSql.ReactorFleet);
        b.HasKey(e => e.SteamGeneratorId).HasName("PK_ReactorFleet_SteamGenerator");
        b.Property(e => e.Code).HasCode(40);
        b.Property(e => e.TubeCount).HasDefaultValue(0);
        b.Property(e => e.DesignSteamPressureBar).HasPrecision(10, 4);

        b.HasIndex(e => new { e.PrimaryLoopId, e.Code }).IsUnique()
            .HasDatabaseName("UQ_ReactorFleet_SteamGenerator_Loop_Code");

        b.HasOne(e => e.PrimaryLoop).WithMany(e => e.SteamGenerators)
            .HasForeignKey(e => e.PrimaryLoopId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_SteamGenerator_PrimaryLoop");

        b.HasOne(e => e.Equipment).WithMany()
            .HasForeignKey(e => e.EquipmentId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_SteamGenerator_Equipment");

        b.HasOne(e => e.SteamGeneratorType).WithMany()
    }
}
```

```

        .HasForeignKey(e => e.SteamGeneratorTypeId)
        .OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_ReactorFleet_SteamGenerator_SteamGeneratorType");
    }
}

public sealed class GridConnectionConfiguration : IEntityTypeConfiguration<GridConnection>
{
    public void Configure(EntityTypeBuilder<GridConnection> b)
    {
        b.ToTable("GridConnection", NexusSql.ReactorFleet);
        b.HasKey(e => e.GridConnectionId).HasName("PK_ReactorFleet_GridConnection");
        b.Property(e => e.NominalVoltageKv).HasPrecision(10, 4);
        b.Property(e => e.FrequencyHz).HasPrecision(8, 4);
        b.Property(e => e.SynchronizedAtUtc).HasColumnType("datetime2(7)");

        b.HasIndex(e => e.UnitId).HasDatabaseName("IX_ReactorFleet_GridConnection_UnitId");

        b.HasOne(e => e.Unit).WithMany(e => e.GridConnections)
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_GridConnection_Unit");

        b.HasOne(e => e.Generator).WithMany()
            .HasForeignKey(e => e.GeneratorId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_GridConnection_Generator");

        b.HasOne(e => e.Transformer).WithMany()
            .HasForeignKey(e => e.TransformerId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_GridConnection_Transformer");

        b.HasOne(e => e.GridConnectionStatus).WithMany()
            .HasForeignKey(e => e.GridConnectionStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_GridConnection_GridConnectionStatus");
    }
}

```

Snapshot mapping

Snapshot tables are time-oriented fact tables. Their indexes start with the principal key and then time, because most operational questions ask for a unit or fleet over a window.

```
public sealed class FleetSnapshotConfiguration : IEntityTypeConfiguration<FleetSnapshot>
{
    public void Configure(EntityTypeBuilder<FleetSnapshot> b)
    {
        b.ToTable("FleetSnapshot", NexusSql.ReactorFleet);
        b.HasKey(e => e.FleetSnapshotId).HasName("PK_ReactorFleet_FleetSnapshot");
        b.Property(e => e.RecordedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.TotalNetMwe).HasPrecision(14, 4);
        b.Property(e => e.OnlineUnitCount).HasDefaultValue(0);
        b.Property(e => e.GridDemandMwe).HasPrecision(14, 4);

        b.HasIndex(e => new { e.FleetId, e.RecordedAtUtc })
            .HasDatabaseName("IX_ReactorFleet_FleetSnapshot_FleetId_Time");

        b.HasOne(e => e.Fleet).WithMany(e => e.FleetSnapshots)
            .HasForeignKey(e => e.FleetId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_FleetSnapshot_Fleet");
    }
}

public sealed class UnitPowerSnapshotConfiguration
    : IEntityTypeConfiguration<UnitPowerSnapshot>
{
    public void Configure(EntityTypeBuilder<UnitPowerSnapshot> b)
    {
        b.ToTable("UnitPowerSnapshot", NexusSql.ReactorFleet);
        b.HasKey(e => e.UnitPowerSnapshotId)
            .HasName("PK_ReactorFleet_UnitPowerSnapshot");
        b.Property(e => e.RecordedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.NetMwe).HasPrecision(14, 4);
        b.Property(e => e.ThermalMwt).HasPrecision(14, 4);
        b.Property(e => e.PowerFactor).HasPrecision(8, 6);
        b.Property(e => e.GridFrequencyHz).HasPrecision(8, 4);

        b.HasIndex(e => new { e.UnitId, e.RecordedAtUtc })
            .HasDatabaseName("IX_ReactorFleet_UnitPowerSnapshot_UnitId_Time");

        b.HasOne(e => e.Unit).WithMany(e => e.PowerSnapshots)
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ReactorFleet_UnitPowerSnapshot_Unit");

        b.HasOne(e => e.OperationalMode).WithMany()
            .HasForeignKey(e => e.OperationalModeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReactorFleet_UnitPowerSnapshot_OperationalMode");
    }
}
```

Migration and verification notes

A clean ReactorFleet migration should read like a plant model being assembled. Lookup tables appear first, then fleet and unit, then systems and equipment, then reactor-core structures, loops, power conversion and snapshots.

Index names

Use `UQ_ReactorFleet_...` for alternate keys and `IX_ReactorFleet_...` for query indexes. Avoid anonymous EF-generated names.

Relationship names

Name every FK, especially self-references such as `PlantSystem.ParentPlantSystemId` and dependency tables.

Precision

Thermal, electrical, chemistry and neutronics numbers should use explicit precision. Do not let decimal defaults vary between migrations.

Snapshots

Use time-window indexes. Append-only facts should not rely on mutable row-order semantics.

```
-- 1. Every configuration class is discovered by assembly scanning.
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(NexusContext).Assembly);
}

-- 2. Migration review: no random EF-generated names.
SELECT name
FROM sys.objects
WHERE name LIKE 'FK_%%' OR name LIKE 'IX_%%';

-- 3. ReactorFleet passport check: Unit must be a visible principal table.
SELECT fk.name, OBJECT_SCHEMA_NAME(fk.parent_object_id) AS child_schema,
       OBJECT_NAME(fk.parent_object_id) AS child_table
FROM sys.foreign_keys fk
WHERE fk.referenced_object_id = OBJECT_ID('ReactorFleet.Unit')
ORDER BY child_schema, child_table;

-- 4. Snapshot indexes should be time-oriented.
SELECT i.name, OBJECT_NAME(i.object_id) AS table_name
FROM sys.indexes i
WHERE i.name LIKE 'IX_ReactorFleet_%%Snapshot%Time%'
ORDER BY table_name, i.name;
```

Configuration file index

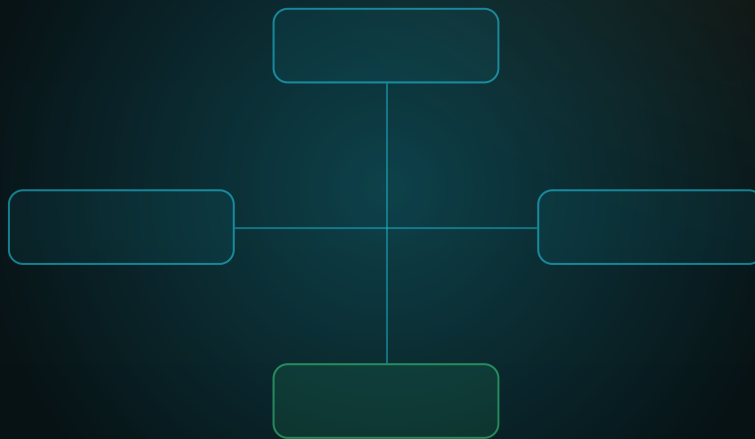
UnitType - UnitTypeConfiguration.cs	EquipmentLocation - EquipmentLocationConfiguration.cs
UnitStatus - UnitStatusConfiguration.cs	Equipment - EquipmentConfiguration.cs
OperationalMode - OperationalModeConfiguration.cs	EquipmentDependency - EquipmentDependencyConfiguration.cs
ReactorType - ReactorTypeConfiguration.cs	EquipmentExternalReference - EquipmentExternalReferenceConfiguration.cs
CoolantLoopType - CoolantLoopTypeConfiguration.cs	Reactor - ReactorConfiguration.cs
SystemType - SystemTypeConfiguration.cs	ReactorCore - ReactorCoreConfiguration.cs
EquipmentType - EquipmentTypeConfiguration.cs	CoreRegion - CoreRegionConfiguration.cs
EquipmentStatus - EquipmentStatusConfiguration.cs	FuelAssembly - FuelAssemblyConfiguration.cs
EquipmentCriticality - EquipmentCriticalityConfiguration.cs	ControlRodBank - ControlRodBankConfiguration.cs
FuelAssemblyType - FuelAssemblyTypeConfiguration.cs	ControlRod - ControlRodConfiguration.cs
FuelMaterialType - FuelMaterialTypeConfiguration.cs	RodPositionSnapshot - RodPositionSnapshotConfiguration.cs
ControlRodMaterial - ControlRodMaterialConfiguration.cs	BoronChemistrySnapshot - BoronChemistrySnapshotConfiguration.cs
RodBankType - RodBankTypeConfiguration.cs	NeutronicsParameterSet - NeutronicsParameterSetConfiguration.cs
SteamGeneratorType - SteamGeneratorTypeConfiguration.cs	PrimaryLoop - PrimaryLoopConfiguration.cs
PumpType - PumpTypeConfiguration.cs	ReactorCoolantPump - ReactorCoolantPumpConfiguration.cs
GridConnectionStatus - GridConnectionStatusConfiguration.cs	Pressurizer - PressurizerConfiguration.cs
Fleet - FleetConfiguration.cs	SteamGenerator - SteamGeneratorConfiguration.cs
FleetPlant - FleetPlantConfiguration.cs	TurbineTrain - TurbineTrainConfiguration.cs
Unit - UnitConfiguration.cs	TurbineStage - TurbineStageConfiguration.cs
UnitDesignParameter - UnitDesignParameterConfiguration.cs	Generator - GeneratorConfiguration.cs
UnitOperationalState - UnitOperationalStateConfiguration.cs	Transformer - TransformerConfiguration.cs
PlantSystem - PlantSystemConfiguration.cs	SwitchyardBay - SwitchyardBayConfiguration.cs
SystemDependency - SystemDependencyConfiguration.cs	GridConnection - GridConnectionConfiguration.cs
	FleetSnapshot - FleetSnapshotConfiguration.cs
	UnitPowerSnapshot - UnitPowerSnapshotConfiguration.cs

Honest boundary

This chapter maps the NEXUS-1 demonstrator data model. It is an enterprise EF Core configuration reference, not a licensed nuclear plant information model. The mappings are designed to be readable, auditable and migration-friendly; they are not a substitute for certified plant engineering data.

FROM ENTITY TO CONTEXT

Chapter 10 - Instrumentation Configurations
The signal, historian, calibration and acquisition mapping layer.



CONFIGURATION REFERENCE COMPANION

Instrumentation is where tags become trustworthy EF Core objects.

40 configuration files - 14 lookups - signal passport discipline.

Chapter 10

Instrumentation Configurations

The EF Core mapping atlas for `Instrumentation`: acquisition nodes, sensors, canonical signals, limits, calibration plans, high-volume measurements, data gaps, import batches and backfill jobs.

In the SQL atlas, Instrumentation is the sector that turns physical or simulated measurements into canonical tags. In the EF Core atlas, it is the place where those tags become disciplined C# entities with stable table names, explicit keys, named indexes and declared foreign keys.

One tag should mean one signal. One signal should carry its unit, quality, source and lineage. Every mapping class in this chapter protects that rule.

10.1 What this schema configures

The Instrumentation schema is the bridge between the physical plant and the data platform. It owns the canonical `Signal` table, the acquisition path that feeds it, the sensor and channel catalogue that explains it, and the measurement facts that make it useful to digital twin, alarms, root cause, reporting and audit.

Acquisition

nodes, connections and raw points before a canonical tag exists.

Signals

the unique tag space used by twin, alarms, RCA and historian.

Historian

raw measurements, aggregates, quality events and gaps.

Calibration

plans and records that make sensor trust auditable.

Standing rule. The DbContext does not become a 40-table mapping file. Every entity is configured in its own class and discovered by `ApplyConfigurationsFromAssembly`.

10.2 Configuration file catalogue

The first deliverable is the file map. The names below are intentionally boring: one entity, one configuration class, one table name, one place to look when a migration changes.

AREA	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	<code>Instrumentation.SignalType</code>	<code>SignalTypeConfiguration.cs</code>	Classifies measured, calculated, commanded, state, alarm-derived, and simulation mirror signals.
Lookup	<code>Instrumentation.SignalCategory</code>	<code>SignalCategoryConfiguration.cs</code>	Domain category such as power, pressure, temperature, flow, radiation, vibration, chemistry or electrical.
Lookup	<code>Instrumentation.SignalRole</code>	<code>SignalRoleConfiguration.cs</code>	Role of the signal in the platform: telemetry, control indication, diagnostic, twin input, RCA witness or training signal.
Lookup	<code>Instrumentation.SensorType</code>	<code>SensorTypeConfiguration.cs</code>	Physical sensor family: RTD, pressure transmitter, flow transmitter, neutron detector, vibration probe, radiation monitor.
Lookup	<code>Instrumentation.SensorStatus</code>	<code>SensorStatusConfiguration.cs</code>	Sensor status: available, maintenance, suspect, failed, retired.
Lookup	<code>Instrumentation.ChannelStatus</code>	<code>ChannelStatusConfiguration.cs</code>	Acquisition-channel status: online, stale, disabled, faulted, simulated.
Lookup	<code>Instrumentation.SignalQuality</code>	<code>SignalQualityConfiguration.cs</code>	Quality state attached to measurements: good, uncertain, bad, stale, substituted, simulated.
Lookup	<code>Instrumentation.SamplingMode</code>	<code>SamplingModeConfiguration.cs</code>	Sampling mode: periodic, event-driven, manual, derived, simulated.
Lookup	<code>Instrumentation.MeasurementSource</code>	<code>MeasurementSourceConfiguration.cs</code>	Origin of a value: historian, simulator, operator entry, import, twin backfill, test harness.
Lookup	<code>Instrumentation.CalibrationStatus</code>	<code>CalibrationStatusConfiguration.cs</code>	Calibration lifecycle: scheduled, due, overdue, passed, failed, waived.
Lookup	<code>Instrumentation.HistorianRetentionClass</code>	<code>HistorianRetentionClassConfiguration.cs</code>	Retention policy category for raw and aggregated measurements.
Lookup	<code>Instrumentation.ThresholdType</code>	<code>ThresholdTypeConfiguration.cs</code>	Threshold semantics: high, high-high, low, low-low, rate, deviation, deadband.

AREA	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	<code>Instrumentation.BackfillJobStatus</code>	<code>BackfillJobStatusConfiguration.cs</code>	Backfill job status: queued, running, completed, failed, cancelled.
Lookup	<code>Instrumentation.LineageRole</code>	<code>LineageRoleConfiguration.cs</code>	Role of a source signal in a derived signal: primary, witness, fallback, correction, normalization.
Acquisition	<code>Instrumentation.DataAcquisitionNode</code>	<code>DataAcquisitionNodeConfiguration.cs</code>	Logical acquisition node, gateway, simulator feed or historian bridge for one unit.
Acquisition	<code>Instrumentation.AcquisitionConnection</code>	<code>AcquisitionConnectionConfiguration.cs</code>	Named protocol connection or local feed into an acquisition node.
Acquisition	<code>Instrumentation.AcquisitionPoint</code>	<code>AcquisitionPointConfiguration.cs</code>	Protocol address, register, topic, JSON path or synthetic point before it becomes a signal.
Instruments	<code>Instrumentation.Instrument</code>	<code>InstrumentConfiguration.cs</code>	Installed instrument package or cabinet entry that may host one or more sensors.
Instruments	<code>Instrumentation.Sensor</code>	<code>SensorConfiguration.cs</code>	Physical or simulated sensor with range, accuracy and status.
Instruments	<code>Instrumentation.SensorChannel</code>	<code>SensorChannelConfiguration.cs</code>	A channel from a sensor; redundant channels become separate rows.
Signals	<code>Instrumentation.Signal</code>	<code>SignalConfiguration.cs</code>	The canonical tag. Digital twin, alarms, root cause and historian all reference this table.
Signals	<code>Instrumentation.SignalAlias</code>	<code>SignalAliasConfiguration.cs</code>	Alternative tag, legacy name, historian name, screen key or vendor tag.
Signals	<code>Instrumentation.SignalGroup</code>	<code>SignalGroupConfiguration.cs</code>	Named signal set for screens, diagnostics, export packs and data-quality views.
Signals	<code>Instrumentation.SignalGroupMember</code>	<code>SignalGroupMemberConfiguration.cs</code>	Membership of signals in a named group.
Signals	<code>Instrumentation.SignalMapping</code>	<code>SignalMappingConfiguration.cs</code>	Maps a canonical signal to a raw acquisition point.
Signals	<code>Instrumentation.SignalDependency</code>	<code>SignalDependencyConfiguration.cs</code>	Signal-level influence or derivation relationship with optional lag and weight.
Signals	<code>Instrumentation.SignalLineage</code>	<code>SignalLineageConfiguration.cs</code>	Explicit lineage for calculated and derived signals.
Limits	<code>Instrumentation.SignalLimit</code>	<code>SignalLimitConfiguration.cs</code>	Engineering, display, alarm-preparation or model-boundary limit for a signal.

AREA	TABLE	CONFIGURATION FILE	PURPOSE
Limits	Instrumentation.SignalDeadband	SignalDeadbandConfiguration.cs	Deadband or noise band for display, alarm rationalisation and digital-twin divergence.
Calibration	Instrumentation.CalibrationPlan	CalibrationPlanConfiguration.cs	Calibration obligation and interval for a signal/sensor pair.
Calibration	Instrumentation.CalibrationRecord	CalibrationRecordConfiguration.cs	Completed calibration result, as-found/as-left value and pass/fail state.
Historian	Instrumentation.Measurement	MeasurementConfiguration.cs	High-volume raw time-series fact table.
Historian	Instrumentation.MeasurementAggregate	MeasurementAggregateConfiguration.cs	Aggregated windows for fast charts and reporting.
Historian	Instrumentation.MeasurementAnnotation	MeasurementAnnotationConfiguration.cs	Human or system note attached to one measurement row.
Historian	Instrumentation.SignalQualityEvent	SignalQualityEventConfiguration.cs	Interval record explaining when a signal became stale, bad, substituted or simulated.
Historian	Instrumentation.DataGap	DataGapConfiguration.cs	Detected gap in historian coverage for one signal.
Historian	Instrumentation.HistorianRetentionPolicy	HistorianRetentionPolicyConfiguration.cs	Retention, aggregation and purge contract for signals in one class.
Historian	Instrumentation.HistorianImportBatch	HistorianImportBatchConfiguration.cs	Imported block of measurements from simulator, historian or CSV feed.
Historian	Instrumentation.HistorianImportBatchItem	HistorianImportBatchItemConfiguration.cs	Per-signal import counts and time range inside one batch.
Historian	Instrumentation.HistorianBackfillJob	HistorianBackfillJobConfiguration.cs	Job used to backfill missing measurements or recompute derived signals.

10.3 Folder structure

The folder layout follows the schema's own subdomains. The purpose is not aesthetic. It prevents the Historian mappings from drowning the lookup mappings, and it lets a developer open the folder that matches the business conversation.

```
Nexus.Infrastructure/  
  Persistence/  
    Configurations/  
      Instrumentation/  
        Lookups/  
          SignalTypeConfiguration.cs  
          SignalCategoryConfiguration.cs  
          SignalQualityConfiguration.cs  
          ThresholdTypeConfiguration.cs  
          ...  
        Acquisition/  
          DataAcquisitionNodeConfiguration.cs  
          AcquisitionConnectionConfiguration.cs  
          AcquisitionPointConfiguration.cs  
        Instruments/  
          InstrumentConfiguration.cs  
          SensorConfiguration.cs  
          SensorChannelConfiguration.cs  
        Signals/  
          SignalConfiguration.cs  
          SignalAliasConfiguration.cs  
          SignalGroupConfiguration.cs  
          SignalGroupMemberConfiguration.cs  
          SignalMappingConfiguration.cs  
          SignalDependencyConfiguration.cs  
          SignalLineageConfiguration.cs  
        Limits/  
          SignalLimitConfiguration.cs  
          SignalDeadbandConfiguration.cs  
        Calibration/  
          CalibrationPlanConfiguration.cs  
          CalibrationRecordConfiguration.cs  
        Historian/  
          MeasurementConfiguration.cs  
          MeasurementAggregateConfiguration.cs  
          SignalQualityEventConfiguration.cs  
          DataGapConfiguration.cs  
          HistorianImportBatchConfiguration.cs  
          HistorianBackfillJobConfiguration.cs
```

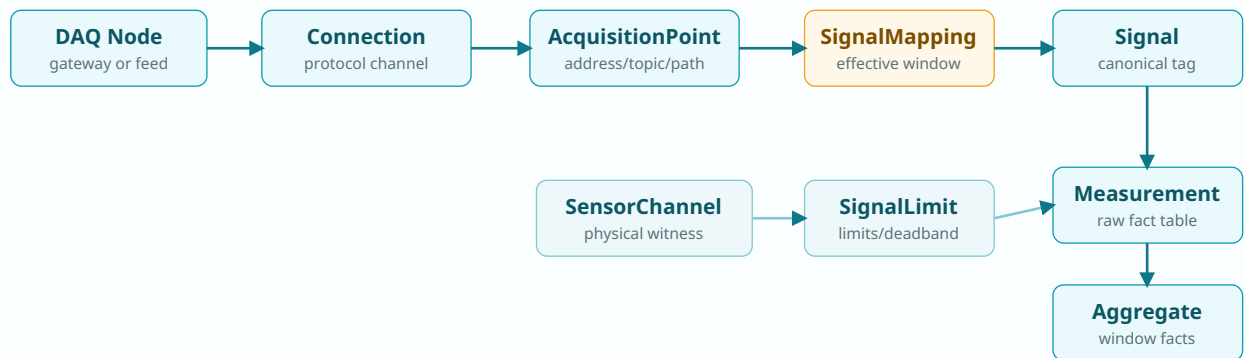


Figure 10.1 - Acquisition-to-historian pipeline. EF Core mappings should preserve this chain without hidden navigation magic: raw acquisition points are mapped to canonical signals, then measurements and aggregates attach to those signals.

10.4 Shared lookup configuration

Instrumentation contains many lookup tables because quality, role, source and status should be typed data, not strings spread through code. The common base keeps the shape identical.

```

namespace Nexus.Infrastructure.Persistence.Configurations;

internal static partial class NexusSql
{
    public const string Instrumentation = "Instrumentation";
    public const string ReactorFleet = "ReactorFleet";
    public const string CorePlatform = "CorePlatform";
    public const string Security = "Security";
    public const string.UtcNow = "SYSUTCDATETIME()";
}

public abstract class InstrumentationLookupConfiguration<TEntity>
    : IEntityTypeConfiguration<TEntity>
    where TEntity : InstrumentationLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(150)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

```

```

    }
}

internal static class InstrumentationMappingExtensions
{
    public static PropertyBuilder<string> HasTag(
        this PropertyBuilder<string> property, int length = 80) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<decimal?> HasEngineeringDecimal(
        this PropertyBuilder<decimal?> property) => property
        .HasColumnType("decimal(18,6)");

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);
}

```

```

public sealed class SignalTypeConfiguration
    : InstrumentationLookupConfiguration<SignalType>
{
    public override void Configure(EntityTypeBuilder<SignalType> b)
    {
        b.ToTable("SignalType", NexusSql.Instrumentation);
        b.HasKey(e => e.SignalTypeId)
            .HasName("PK_Instrumentation_SignalType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Instrumentation_SignalType_Code");
    }
}

public sealed class SignalQualityConfiguration
    : InstrumentationLookupConfiguration<SignalQuality>
{
    public override void Configure(EntityTypeBuilder<SignalQuality> b)
    {
        b.ToTable("SignalQuality", NexusSql.Instrumentation);
        b.HasKey(e => e.SignalQualityId)
            .HasName("PK_Instrumentation_SignalQuality");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Instrumentation_SignalQuality_Code");
    }
}

public sealed class ThresholdTypeConfiguration
    : InstrumentationLookupConfiguration<ThresholdType>
{
    public override void Configure(EntityTypeBuilder<ThresholdType> b)
    {
        b.ToTable("ThresholdType", NexusSql.Instrumentation);
        b.HasKey(e => e.ThresholdTypeId)
            .HasName("PK_Instrumentation_ThresholdType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Instrumentation_ThresholdType_Code");
    }
}

```

10.5 Signal as the central configuration

`SignalConfiguration` is the heart of the schema. It maps a canonical tag to its unit, physical unit, equipment, channel, signal type, category, role, sampling mode and retention class. The class is long because the table is the passport table for the entire platform.

```

public sealed class SignalConfiguration : IEntityConfiguration<Signal>
{
    public void Configure(EntityTypeBuilder<Signal> b)
    {
        b.ToTable("Signal", NexusSql.Instrumentation);

        b.HasKey(e => e.SignalId)
            .HasName("PK_Instrumentation_Signal");

        b.Property(e => e.Tag).HasTag(80);
        b.Property(e => e.Name).HasMaxLength(250).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1000);
        b.Property(e => e.ScanRateHz).HasColumnType("decimal(12,6)");
        b.Property(e => e.NormalMin).HasEngineeringDecimal();
    }
}

```

```

b.Property(e => e.NormalMax).HasEngineeringDecimal();
b.Property(e => e.CreatedAtUtc).HasUtcDefault();
b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
b.Property(e => e.CreatedBy).HasMaxLength(100).IsRequired();
b.Property(e => e.ModifiedBy).HasMaxLength(100).IsRequired();
b.Property(e => e.IsSafetyRelated).HasDefaultValue(false);
b.Property(e => e.IsHistorized).HasDefaultValue(true);
b.Property(e => e.IsDeleted).HasDefaultValue(false);
b.Property(e => e.RowVersion).IsRowVersion();

b.HasIndex(e => e.Tag).IsUnique()
    .HasDatabaseName("UQ_Instrumentation_Signal_Tag");
b.HasIndex(e => new { e.UnitId, e.SignalCategoryId })
    .HasDatabaseName("IX_Instrumentation_Signal_Unit_Category");
b.HasIndex(e => new { e.EquipmentId, e.SignalRoleId })
    .HasDatabaseName("IX_Instrumentation_Signal_Equipment_Role");

b.HasOne(e => e.Unit)
    .WithMany()
    .HasForeignKey(e => e.UnitId)
    .OnDelete(DeleteBehavior.Restrict)
    .HasConstraintName("FK_Instrumentation_Signal_Unit");

b.HasOne(e => e.Equipment)
    .WithMany()
    .HasForeignKey(e => e.EquipmentId)
    .OnDelete(DeleteBehavior.NoAction)
    .HasConstraintName("FK_Instrumentation_Signal_Equipment");

b.HasOne(e => e.EngineeringUnit)
    .WithMany()
    .HasForeignKey(e => e.EngineeringUnitId)
    .OnDelete(DeleteBehavior.Restrict)
    .HasConstraintName("FK_Instrumentation_Signal_EngineeringUnit");

b.HasOne(e => e.SignalType).WithMany()
    .HasForeignKey(e => e.SignalTypeId)
    .OnDelete(DeleteBehavior.Restrict)
    .HasConstraintName("FK_Instrumentation_Signal_SignalType");

b.HasOne(e => e.SignalCategory).WithMany()
    .HasForeignKey(e => e.SignalCategoryId)
    .OnDelete(DeleteBehavior.Restrict)
    .HasConstraintName("FK_Instrumentation_Signal_SignalCategory");
}
}

```

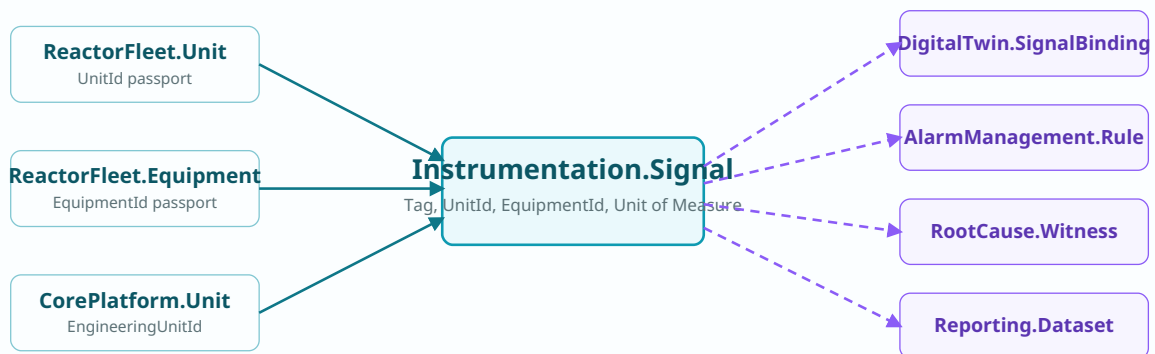


Figure 10.2 - Signal passport map. The signal table receives passports from ReactorFleet and CorePlatform, then becomes the shared identity used downstream by DigitalTwin, AlarmManagement, RootCause and Reporting.

10.6 Acquisition and mapping configurations

The acquisition classes keep raw transport concerns out of the canonical signal. A protocol register, OPC tag, JSON path or simulated point becomes useful only when `SignalMapping` gives it an effective-time relationship to `Signal`.

```
public sealed class DataAcquisitionNodeConfiguration
    : IEntityTypeConfiguration<DataAcquisitionNode>
{
    public void Configure(EntityTypeBuilder<DataAcquisitionNode> b)
    {
        b.ToTable("DataAcquisitionNode", NexusSql.Instrumentation);
        b.HasKey(e => e.DataAcquisitionNodeId)
            .HasName("PK_Instrumentation_DataAcquisitionNode");

        b.Property(e => e.Code).HasMaxLength(80).IsRequired();
        b.Property(e => e.Name).HasMaxLength(200).IsRequired();
        b.Property(e => e.HostName).HasMaxLength(200);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Instrumentation_DataAcquisitionNode_Code");
        b.HasIndex(e => new { e.UnitId, e.ChannelStatusId })
            .HasDatabaseName("IX_Instrumentation_DaqNode_Unit_Status");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Instrumentation_DataAcquisitionNode_Unit");

        b.HasOne(e => e.ChannelStatus)
            .WithMany()
            .HasForeignKey(e => e.ChannelStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Instrumentation_DataAcquisitionNode_ChannelStatus");
    }
}

public sealed class SignalMappingConfiguration
    : IEntityTypeConfiguration<SignalMapping>
{
    public void Configure(EntityTypeBuilder<SignalMapping> b)
    {
        b.ToTable("SignalMapping", NexusSql.Instrumentation);
        b.HasKey(e => e.SignalMappingId)
            .HasName("PK_Instrumentation_SignalMapping");

        b.Property(e => e.EffectiveFromUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.EffectiveToUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.CreatedBy).HasMaxLength(100).IsRequired();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.SignalId, e.AcquisitionPointId, e.EffectiveFromUtc })
            .IsUnique()
            .HasDatabaseName("UQ_Instrumentation_SignalMapping_Signal_Point_From");

        b.HasOne(e => e.Signal).WithMany(e => e.Mappings)
            .HasForeignKey(e => e.SignalId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_Instrumentation_SignalMapping_Signal");

        b.HasOne(e => e.AcquisitionPoint).WithMany()
            .HasForeignKey(e => e.AcquisitionPointId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Instrumentation_SignalMapping_AcquisitionPoint");
    }
}
```

10.7 Historian and measurement configurations

`Measurement` is a high-volume fact table. The mapping avoids cascade surprises across lookups, keeps time indexes visible, and makes signal/time queries cheap and explicit.

```

public sealed class MeasurementConfiguration
    : IEntityTypeConfiguration<Measurement>
{
    public void Configure(EntityTypeBuilder<Measurement> b)
    {
        b.ToTable("Measurement", NexusSql.Instrumentation);
        b.HasKey(e => e.MeasurementId)
            .HasName("PK_Instrumentation_Measurement");

        b.Property(e => e.MeasuredAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.Value).HasColumnType("decimal(18,6)");
        b.Property(e => e.RawValue).HasMaxLength(200);
        b.Property(e => e.IngestedAtUtc).HasUtcDefault();

        b.HasIndex(e => new { e.SignalId, e.MeasuredAtUtc })
            .HasDatabaseName("IX_Instrumentation_Measurement_Signal_Time");
        b.HasIndex(e => new { e.MeasuredAtUtc, e.SignalQualityId })
            .HasDatabaseName("IX_Instrumentation_Measurement_Time_Quality");

        b.HasOne(e => e.Signal).WithMany(e => e.Measurements)
            .HasForeignKey(e => e.SignalId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_Instrumentation_Measurement_Signal");

        b.HasOne(e => e.SignalQuality).WithMany()
            .HasForeignKey(e => e.SignalQualityId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Instrumentation_Measurement_SignalQuality");

        b.HasOne(e => e.MeasurementSource).WithMany()
            .HasForeignKey(e => e.MeasurementSourceId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Instrumentation_Measurement_MeasurementSource");
    }
}

public sealed class MeasurementAggregateConfiguration
    : IEntityTypeConfiguration<MeasurementAggregate>
{
    public void Configure(EntityTypeBuilder<MeasurementAggregate> b)
    {
        b.ToTable("MeasurementAggregate", NexusSql.Instrumentation);
        b.HasKey(e => e.MeasurementAggregateId)
            .HasName("PK_Instrumentation_MeasurementAggregate");

        b.Property(e => e.WindowStartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.WindowEndUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.MinValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.MaxValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.AvgValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasIndex(e => new { e.SignalId, e.WindowStartUtc, e.WindowEndUtc })
            .IsUnique()
            .HasDatabaseName("UQ_Instrumentation_MeasurementAggregate_Signal_Window");
    }
}

```

10.8 Calibration and quality configurations

Instrumentation is not trustworthy because it has values; it is trustworthy because it records calibration, quality intervals, gaps and substitutions. These mappings make that evidence queryable.

```

public sealed class CalibrationPlanConfiguration
    : IEntityTypeConfiguration<CalibrationPlan>
{
    public void Configure(EntityTypeBuilder<CalibrationPlan> b)
    {
        b.ToTable("CalibrationPlan", NexusSql.Instrumentation);
        b.HasKey(e => e.CalibrationPlanId)
            .HasName("PK_Instrumentation_CalibrationPlan");

        b.Property(e => e.IntervalDays).IsRequired();
        b.Property(e => e.NextDueAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.SignalId, e.SensorId })
            .HasDatabaseName("IX_Instrumentation_CalibrationPlan_Signal_Sensor");
        b.HasIndex(e => new { e.CalibrationStatusId, e.NextDueAtUtc })
            .HasDatabaseName("IX_Instrumentation_CalibrationPlan_Status_Due");
    }
}

```



```

        b.HasOne(e => e.Signal).WithMany()
            .HasForeignKey(e => e.SignalId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Instrumentation_CalibrationPlan_Signal");
        b.HasOne(e => e.Sensor).WithMany()
            .HasForeignKey(e => e.SensorId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Instrumentation_CalibrationPlan_Sensor");
    }
}

public sealed class CalibrationRecordConfiguration
    : IEntityTypeConfiguration<CalibrationRecord>
{
    public void Configure(EntityTypeBuilder<CalibrationRecord> b)
    {
        b.ToTable("CalibrationRecord", NexusSql.Instrumentation);
        b.HasKey(e => e.CalibrationRecordId)
            .HasName("PK_Instrumentation_CalibrationRecord");

        b.Property(e => e.PerformedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.AsFoundValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.AsLeftValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.ResultNotes).HasMaxLength(1500);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.CalibrationPlanId, e.PerformedAtUtc })
            .HasDatabaseName("IX_Instrumentation_CalibrationRecord_Plan_Time");
    }
}

```

```

public sealed class SignalQualityEventConfiguration
    : IEntityTypeConfiguration<SignalQualityEvent>
{
    public void Configure(EntityTypeBuilder<SignalQualityEvent> b)
    {
        b.ToTable("SignalQualityEvent", NexusSql.Instrumentation);
        b.HasKey(e => e.SignalQualityEventId)
            .HasName("PK_Instrumentation_SignalQualityEvent");

        b.Property(e => e.StartedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.EndedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.Reason).HasMaxLength(1000);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasIndex(e => new { e.SignalId, e.StartedAtUtc, e.EndedAtUtc })
            .HasDatabaseName("IX_Instrumentation_SignalQualityEvent_Signal_Window");
        b.HasOne(e => e.Signal).WithMany()
            .HasForeignKey(e => e.SignalId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_Instrumentation_SignalQualityEvent_Signal");
    }
}

public sealed class DataGapConfiguration : IEntityTypeConfiguration<DataGap>
{
    public void Configure(EntityTypeBuilder<DataGap> b)
    {
        b.ToTable("DataGap", NexusSql.Instrumentation);
        b.HasKey(e => e.DataGapId).HasName("PK_Instrumentation_DataGap");
        b.Property(e => e.StartedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.EndedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.DetectedAtUtc).HasUtcDefault();
        b.Property(e => e.Reason).HasMaxLength(1000);
        b.HasIndex(e => new { e.SignalId, e.StartedAtUtc })
            .HasDatabaseName("IX_Instrumentation_DataGap_Signal_Start");
    }
}

```

10.9 Import, retention and backfill configurations

Historian data often arrives in batches, may require backfill, and may need to be recomputed into aggregate windows. These configurations keep operational jobs as first-class rows rather than log-file rumours.

```

public sealed class HistorianImportBatchConfiguration
    : IEntityTypeConfiguration<HistorianImportBatch>
{
    public void Configure(EntityTypeBuilder<HistorianImportBatch> b)
    {
        b.ToTable("HistorianImportBatch", NexusSql.Instrumentation);
        b.HasKey(e => e.HistorianImportBatchId)
    }
}

```

```

        .HasName("PK_Instrumentation_HistorianImportBatch");

        b.Property(e => e.BatchKey).HasMaxLength(120).IsRequired();
        b.Property(e => e.SourceFileName).HasMaxLength(260);
        b.Property(e => e.StartedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CompletedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.BatchKey).IsUnique()
            .HasDatabaseName("UQ_Instrumentation_HistorianImportBatch_BatchKey");
    }
}

public sealed class HistorianBackfillJobConfiguration
    : IEntityTypeConfiguration<HistorianBackfillJob>
{
    public void Configure(EntityTypeBuilder<HistorianBackfillJob> b)
    {
        b.ToTable("HistorianBackfillJob", NexusSql.Instrumentation);
        b.HasKey(e => e.HistorianBackfillJobId)
            .HasName("PK_Instrumentation_HistorianBackfillJob");

        b.Property(e => e.RequestedFromUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RequestedToUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.StartedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CompletedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.SignalId, e.BackfillJobStatusId })
            .HasDatabaseName("IX_Instrumentation_BackfillJob_Signal_Status");
    }
}

```

10.10 DbContext integration

The context stays clean. It exposes DbSet's when useful for application code, but it does not host the mapping logic. The assembly scan is the contract.

```

public sealed class NexusContext : DbContext
{
    public DbSet<Signal> Signals => Set<Signal>();
    public DbSet<Measurement> Measurements => Set<Measurement>();
    public DbSet<DataAcquisitionNode> DataAcquisitionNodes => Set<DataAcquisitionNode>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(NexusContext).Assembly);
    }
}

```

10.11 Migration notes and verification queries

CONCERN	CONFIGURATION RULE	REASON
Signal identity	UQ_Instrumentation_Signal_Tag	One canonical tag per signal.
Time-series access	IX_Instrumentation_Measurement_Signal_Time	Most historian reads are signal + time-window queries.
Cross-schema passports	DeleteBehavior.Restrict or NoAction	Deleting a unit or equipment row must not erase history.
Lookup consistency	shared lookup base class	Status, quality, role and source tables all behave the same.
Effective mapping	unique signal/point/effective-from index	Prevents ambiguous raw-to-canonical mapping windows.

```
-- Verify canonical signal uniqueness.
SELECT Tag, COUNT(*) AS duplicates
FROM Instrumentation.Signal
GROUP BY Tag
HAVING COUNT(*) > 1;

-- Verify every measurement can resolve signal, quality and source.
SELECT TOP (50) m.MeasurementId
FROM Instrumentation.Measurement AS m
LEFT JOIN Instrumentation.Signal AS s ON s.SignalId = m.SignalId
LEFT JOIN Instrumentation.SignalQuality AS q ON q.SignalQualityId = m.SignalQualityId
LEFT JOIN Instrumentation.MeasurementSource AS src ON src.MeasurementSourceId = m.MeasurementSourceId
WHERE s.SignalId IS NULL OR q.SignalQualityId IS NULL OR src.MeasurementSourceId IS NULL;

-- Verify the main historian index exists after migration.
SELECT name
FROM sys.indexes
WHERE object_id = OBJECT_ID('Instrumentation.Measurement')
AND name = 'IX_Instrumentation_Measurement_Signal_Time';
```

10.12 Configuration index

DataAcquisitionNode -

DataAcquisitionNodeConfiguration.cs

AcquisitionConnection -

AcquisitionConnectionConfiguration.cs

AcquisitionPoint -

AcquisitionPointConfiguration.cs

Instrument - InstrumentConfiguration.cs

Sensor - SensorConfiguration.cs

SensorChannel - SensorChannelConfiguration.cs

Signal - SignalConfiguration.cs

SignalAlias - SignalAliasConfiguration.cs

SignalGroup - SignalGroupConfiguration.cs

SignalGroupMember -

SignalGroupMemberConfiguration.cs

SignalMapping - SignalMappingConfiguration.cs

SignalDependency -

SignalDependencyConfiguration.cs

SignalLineage - SignalLineageConfiguration.cs

SignalLimit - SignalLimitConfiguration.cs

SignalDeadband -

SignalDeadbandConfiguration.cs

CalibrationPlan -

CalibrationPlanConfiguration.cs

CalibrationRecord -

CalibrationRecordConfiguration.cs

Measurement - MeasurementConfiguration.cs

MeasurementAggregate -

MeasurementAggregateConfiguration.cs

MeasurementAnnotation -

MeasurementAnnotationConfiguration.cs

SignalQualityEvent -

SignalQualityEventConfiguration.cs

DataGap - DataGapConfiguration.cs

HistorianRetentionPolicy -

HistorianRetentionPolicyConfiguration.cs

HistorianImportBatch -

HistorianImportBatchConfiguration.cs

HistorianImportBatchItem -

HistorianImportBatchItemConfiguration.cs

HistorianBackfillJob -

HistorianBackfillJobConfiguration.cs

SignalType - lookup mapping

SignalCategory - lookup mapping

SignalRole - lookup mapping

SensorType - lookup mapping

SensorStatus - lookup mapping

ChannelStatus - lookup mapping

SignalQuality - lookup mapping

SamplingMode - lookup mapping

MeasurementSource - lookup mapping

CalibrationStatus - lookup mapping

HistorianRetentionClass - lookup mapping

ThresholdType - lookup mapping

BackfillJobStatus - lookup mapping

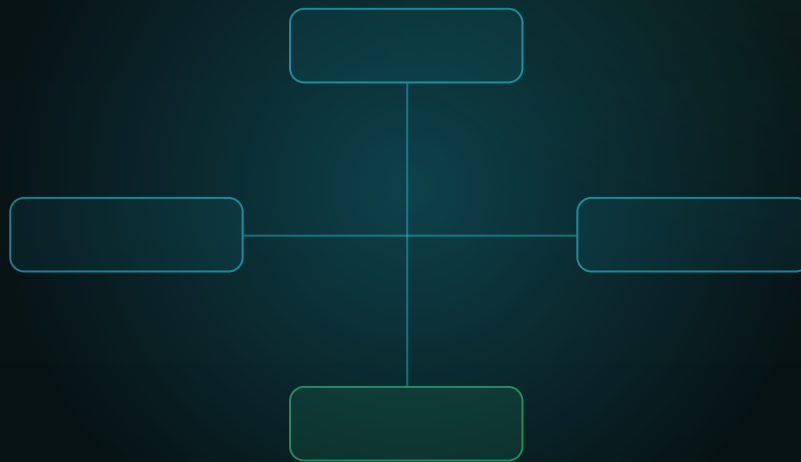
LineageRole - lookup mapping

Honest boundary. These configuration files are an enterprise mapping atlas for the NEXUS-1 demonstration schema. They are not safety-class software and they are not a certified plant historian. Their purpose is to show how a large EF Core model can make tags, measurements, quality, calibration and lineage explicit.

FROM ENTITY TO CONTEXT

Chapter 11 - DigitalTwin Configurations

The EF Core mapping layer that binds model variables to measured reality.



SCHEMA CONFIGURATION COMPANION

**DigitalTwin: model, signal binding,
snapshots, divergence, simulation**

One model boundary - explicit passports - readable Fluent API.

Chapter 11

DigitalTwin Configurations

The complete EF Core configuration pass for the NEXUS-1 **DigitalTwin** schema. This chapter follows Chapter 10, where Instrumentation made signals explicit. DigitalTwin now gives those signals a model: variables, bindings, snapshots, divergences, simulations, and what-if cases.

From Entity to Context - The EF Core Configuration Atlas of the NEXUS-1 Digital Twin

Chapter 11 - DigitalTwin Configurations

The DigitalTwin schema is the contract between the physical unit, the instrumentation layer, and the model that represents the unit. In SQL the contract is expressed as tables and foreign keys; in EF Core it becomes configuration files that state every table name, key name, index name, relationship, precision, and delete behavior.

Continuation rule. Chapters 1-5 taught the EF Core configuration method. Chapter 6 began the atlas with CorePlatform, Chapter 7 configured Security, Chapter 8 configured Organization, Chapter 9 configured ReactorFleet, and Chapter 10 configured Instrumentation. This chapter configures the first schema whose purpose is explicitly twin-like: it binds model variables to real signals and records where the model and the world diverge.

11.1 What This Schema Owns

Model identity

Twin models, versions, components, assumptions, parameters, and variables.

Signal binding

Explicit links between a model variable and the Instrumentation signal that feeds or witnesses it.

Runtime state

Sessions, synchronizations, snapshots, snapshot values, state vectors, and health checks.

Divergence and validation

Records where measured values and modelled values disagree, including review and validation results.

11.2 Configuration Catalogue

The catalogue is intentionally broad. Every table receives its own configuration class; lookup tables share a base configuration, but still get explicit table names, key names, and unique code indexes.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	DigitalTwin.TwinModelType	TwinModelTypeConfiguration.cs	Classifies a model as physics surrogate, point-kinetics model, thermal-hydraulic approximation, equipment health model, or visualization model.
Lookup	DigitalTwin.TwinModelStatus	TwinModelStatusConfiguration.cs	Lifecycle of a twin model: draft, validating, active, retired, superseded, failed.
Lookup	DigitalTwin.TwinFidelityLevel	TwinFidelityLevelConfiguration.cs	Declared fidelity band: illustrative, training, shadow, advisory-ready, validated.
Lookup	DigitalTwin.ModelVariableType	ModelVariableTypeConfiguration.cs	Classifies model variables: state, input, output, derived, diagnostic, safety-margin.
Lookup	DigitalTwin.ModelParameterType	ModelParameterTypeConfiguration.cs	Classifies model parameters: constant, calibration, solver, bound, tuning, assumption.
Lookup	DigitalTwin.BindingRole	BindingRoleConfiguration.cs	Role of a signal binding: input, output witness, calibration witness, divergence witness, fallback.
Lookup	DigitalTwin.BindingStatus	BindingStatusConfiguration.cs	Binding lifecycle: proposed, active, disabled, stale, rejected.
Lookup	DigitalTwin.SnapshotReason	SnapshotReasonConfiguration.cs	Why a snapshot was captured: scheduled, alarm, manual, simulation, divergence, training.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	DigitalTwin.DivergenceSeverity	DivergenceSeverityConfiguration.cs	Severity of model-vs-reality mismatch: info, warning, critical, invalid.
Lookup	DigitalTwin.DivergenceStatus	DivergenceStatusConfiguration.cs	Review state of a divergence: open, acknowledged, explained, corrected, waived, closed.
Lookup	DigitalTwin.SimulationScenarioType	SimulationScenarioTypeConfiguration.cs	Scenario family: what-if, replay, training, validation, regression, RL environment.
Lookup	DigitalTwin.SimulationRunStatus	SimulationRunStatusConfiguration.cs	Run status: queued, running, completed, failed, cancelled, invalidated.
Lookup	DigitalTwin.SynchronizationStatus	SynchronizationStatusConfiguration.cs	Synchronization health: locked, drift, diverging, stale, offline.
Lookup	DigitalTwin.ValidationStatus	ValidationStatusConfiguration.cs	Validation state: unvalidated, testing, passed, failed, expired, waived.
Lookup	DigitalTwin.SolverType	SolverTypeConfiguration.cs	Numerical solver or update strategy used by a model version.
Model	DigitalTwin.TwinModel	TwinModelConfiguration.cs	Names the physical unit mirrored by the twin and declares its model type, status and fidelity.
Model	DigitalTwin.TwinModelVersion	TwinModelVersionConfiguration.cs	Versioned model implementation, solver configuration, code reference, hash and validation state.
Model	DigitalTwin.TwinModelComponent	TwinModelComponentConfiguration.cs	Maps model components to physical equipment and plant systems where that relationship exists.
Model	DigitalTwin.TwinVariable	TwinVariableConfiguration.cs	Variable catalogue for model state, inputs, outputs and diagnostics.
Model	DigitalTwin.TwinParameter	TwinParameterConfiguration.cs	Versioned constants, bounds, solver parameters and calibration parameters.
Binding	DigitalTwin.SignalBinding	SignalBindingConfiguration.cs	Wires a model variable to a real instrumentation signal.
Binding	DigitalTwin.SignalBindingCalibration	SignalBindingCalibrationConfiguration.cs	Scale, offset, tolerance and calibration provenance for a binding.
Runtime	DigitalTwin.TwinRuntimeSession	TwinRuntimeSessionConfiguration.cs	Runtime instance of a twin model version, including mode, host and time window.
Runtime	DigitalTwin.TwinSynchronization	TwinSynchronizationConfiguration.cs	Records synchronization state between model and telemetry for one runtime session.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Runtime	DigitalTwin.TwinStateVector	TwinStateVectorConfiguration.cs	Serialized state vector used to restart, replay or inspect a model runtime.
Snapshot	DigitalTwin.TwinSnapshot	TwinSnapshotConfiguration.cs	Captures the model state at one instant.
Snapshot	DigitalTwin.TwinSnapshotValue	TwinSnapshotValueConfiguration.cs	One model variable value inside a snapshot.
Divergence	DigitalTwin.TwinDivergence	TwinDivergenceConfiguration.cs	Where modelled value and measured value disagree. This is the conscience table of the sector.
Divergence	DigitalTwin.TwinDivergenceReview	TwinDivergenceReviewConfiguration.cs	Human or system review of a divergence: explained, corrected, waived or closed.
Simulation	DigitalTwin.SimulationScenario	SimulationScenarioConfiguration.cs	Named what-if, replay, validation or training scenario.
Simulation	DigitalTwin.SimulationScenarioInput	SimulationScenarioInputConfiguration.cs	Input variable, initial condition or forcing term for a scenario.
Simulation	DigitalTwin.SimulationRun	SimulationRunConfiguration.cs	Concrete execution of a scenario against a model version.
Simulation	DigitalTwin.SimulationRunStep	SimulationRunStepConfiguration.cs	Time-step record for replay, validation and training traces.
Simulation	DigitalTwin.SimulationRunOutput	SimulationRunOutputConfiguration.cs	Output variable values emitted by a simulation step.
What-if	DigitalTwin.WhatIfCase	WhatIfCaseConfiguration.cs	A named operator or engineer what-if analysis case.
What-if	DigitalTwin.WhatIfCaseInput	WhatIfCaseInputConfiguration.cs	Input override or perturbation supplied to a what-if case.
What-if	DigitalTwin.WhatIfCaseResult	WhatIfCaseResultConfiguration.cs	Summarized result and conclusion of a what-if case.
Validation	DigitalTwin.TwinHealthCheck	TwinHealthCheckConfiguration.cs	Periodic operational health check of the running twin.
Validation	DigitalTwin.TwinModelValidation	TwinModelValidationConfiguration.cs	Validation campaign for a model version.
Validation	DigitalTwin.TwinValidationMetric	TwinValidationMetricConfiguration.cs	Metric result such as RMSE, max error, Brier score, coverage or timing error.
Governance	DigitalTwin.ModelAssumption	ModelAssumptionConfiguration.cs	Versioned assumption, simplification or validity boundary.
Governance	DigitalTwin.TwinAnnotation	TwinAnnotationConfiguration.cs	General annotation attached to a model, snapshot, divergence or simulation run.

11.3 Folder Structure

The folder mirrors the schema. Lookups are separated from operational and fact tables, while all configuration files remain discoverable by `ApplyConfigurationsFromAssembly`.

```
Nexus.Infrastructure/  
  Persistence/  
    NexusContext.cs  
  Configurations/  
    DigitalTwin/  
      Lookups/  
        TwinModelTypeConfiguration.cs  
        TwinModelStatusConfiguration.cs  
        TwinFidelityLevelConfiguration.cs  
        ModelVariableTypeConfiguration.cs  
        ModelParameterTypeConfiguration.cs  
        BindingRoleConfiguration.cs  
        BindingStatusConfiguration.cs  
        SnapshotReasonConfiguration.cs  
        DivergenceSeverityConfiguration.cs  
        DivergenceStatusConfiguration.cs  
        SynchronizationStatusConfiguration.cs  
        ValidationStatusConfiguration.cs  
        SolverTypeConfiguration.cs  
        SimulationScenarioTypeConfiguration.cs  
        SimulationRunStatusConfiguration.cs  
      TwinModelConfiguration.cs  
      TwinModelVersionConfiguration.cs  
      TwinModelComponentConfiguration.cs  
      TwinVariableConfiguration.cs  
      TwinParameterConfiguration.cs  
      SignalBindingConfiguration.cs  
      TwinSnapshotConfiguration.cs  
      TwinSnapshotValueConfiguration.cs  
      TwinDivergenceConfiguration.cs  
      SimulationRunConfiguration.cs  
      WhatIfCaseConfiguration.cs
```

11.4 EF Core Dependency Diagram

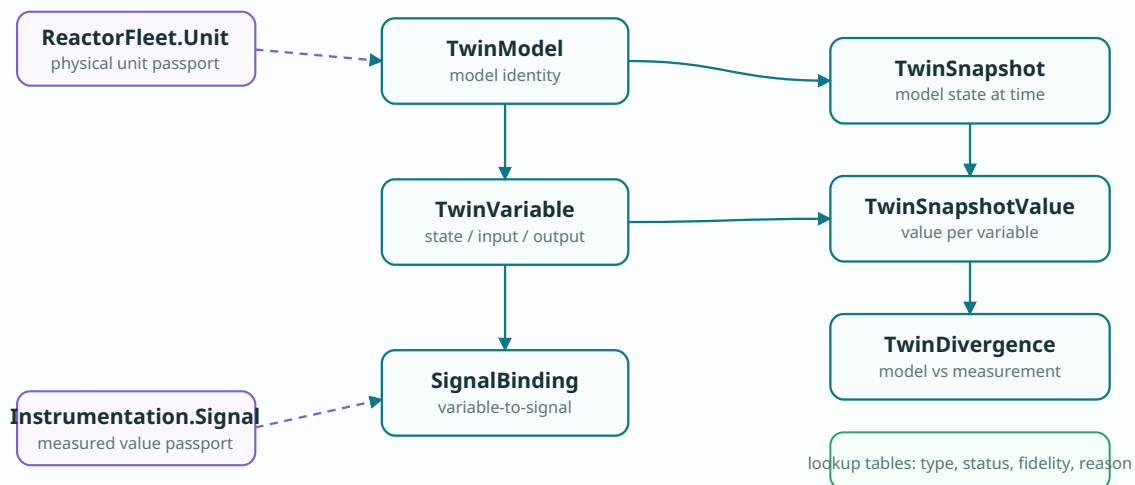


Figure 11.1 - DigitalTwin EF Core dependency diagram. Dashed violet arrows are cross-schema passports; solid teal arrows stay inside the DigitalTwin schema.

11.5 Shared Configuration Primitives

The schema has many typed lookup tables. The base configuration keeps the shape uniform while still allowing every concrete lookup to own its SQL table and constraint names.

```
namespace Nexus.Infrastructure.Persistence.Configurations;

internal static partial class NexusSql
{
    public const string DigitalTwin = "DigitalTwin";
    public const string ReactorFleet = "ReactorFleet";
    public const string Instrumentation = "Instrumentation";
    public const string CorePlatform = "CorePlatform";
    public const string Security = "Security";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class DigitalTwinLookupConfiguration<TEntity>
    : IEntityTypeConfiguration<TEntity>
    where TEntity : DigitalTwinLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(150)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

internal static class DigitalTwinMappingExtensions
{
    public static PropertyBuilder<string> HasTwinCode(
        this PropertyBuilder<string> property, int length = 80) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<decimal?> HasEngineeringValue(
        this PropertyBuilder<decimal?> property) => property
        .HasColumnType("decimal(18,6)");

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);
}
```

11.6 Lookup Configuration Examples

These examples show the repeated lookup pattern. The same form applies to: `TwinModelType`, `TwinModelStatus`, `TwinFidelityLevel`, `ModelVariableType`, `ModelParameterType`, `BindingRole`, `BindingStatus`, `SnapshotReason`, `DivergenceSeverity`, `DivergenceStatus`, `SimulationScenarioType`, `SimulationRunStatus`, `SynchronizationStatus`, `ValidationStatus`, `SolverType`.

```
public sealed class TwinModelTypeConfiguration
    : DigitalTwinLookupConfiguration<TwinModelType>
{
    public override void Configure(EntityTypeBuilder<TwinModelType> b)
    {
        b.ToTable("TwinModelType", NexusSql.DigitalTwin);
        b.HasKey(e => e.TwinModelTypeId)
            .HasName("PK_DigitalTwin_TwinModelType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_DigitalTwin_TwinModelType_Code");
    }
}

public sealed class TwinModelStatusConfiguration
    : DigitalTwinLookupConfiguration<TwinModelStatus>
{
    public override void Configure(EntityTypeBuilder<TwinModelStatus> b)
    {
        b.ToTable("TwinModelStatus", NexusSql.DigitalTwin);
        b.HasKey(e => e.TwinModelStatusId)
            .HasName("PK_DigitalTwin_TwinModelStatus");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_DigitalTwin_TwinModelStatus_Code");
    }
}

public sealed class DivergenceSeverityConfiguration
    : DigitalTwinLookupConfiguration<DivergenceSeverity>
{
    public override void Configure(EntityTypeBuilder<DivergenceSeverity> b)
    {
        b.ToTable("DivergenceSeverity", NexusSql.DigitalTwin);
        b.HasKey(e => e.DivergenceSeverityId)
            .HasName("PK_DigitalTwin_DivergenceSeverity");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_DigitalTwin_DivergenceSeverity_Code");
    }
}
```

11.7 Twin Model and Variable Configurations

`TwinModel` is the aggregate root of the schema: it names which physical unit is mirrored and what class of model is being used. `TwinVariable` makes each model variable addressable and joinable.

```
public sealed class TwinModelConfiguration : IEntityTypeConfiguration<TwinModel>
{
    public void Configure(EntityTypeBuilder<TwinModel> b)
    {
        b.ToTable("TwinModel", NexusSql.DigitalTwin);

        b.HasKey(e => e.TwinModelId)
            .HasName("PK_DigitalTwin_TwinModel");

        b.Property(e => e.Code).HasTwinCode(80);
        b.Property(e => e.Name).HasMaxLength(250).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1000);
        b.Property(e => e.ModelUri).HasMaxLength(500);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.CreatedBy).HasMaxLength(100).IsRequired();
        b.Property(e => e.ModifiedBy).HasMaxLength(100).IsRequired();
        b.Property(e => e.IsDeleted).HasDefaultValue(false);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_DigitalTwin_TwinModel_Code");
        b.HasIndex(e => new { e.UnitId, e.TwinModelTypeId, e.TwinModelStatusId })
            .HasDatabaseName("IX_DigitalTwin_TwinModel_Unit_Type_Status");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DigitalTwin_TwinModel_ReactorFleet_Unit");

        b.HasOne(e => e.ModelType)
            .WithMany()
            .HasForeignKey(e => e.TwinModelTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DigitalTwin_TwinModel_TwinModelType");

        b.HasOne(e => e.Status)
            .WithMany()
            .HasForeignKey(e => e.TwinModelStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DigitalTwin_TwinModel_TwinModelStatus");
    }
}
```

```
public sealed class TwinVariableConfiguration : IEntityTypeConfiguration<TwinVariable>
{
    public void Configure(EntityTypeBuilder<TwinVariable> b)
    {
        b.ToTable("TwinVariable", NexusSql.DigitalTwin);

        b.HasKey(e => e.TwinVariableId)
            .HasName("PK_DigitalTwin_TwinVariable");

        b.Property(e => e.Code).HasTwinCode(120);
        b.Property(e => e.Name).HasMaxLength(250).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1000);
        b.Property(e => e.DefaultValue).HasEngineeringValue();
        b.Property(e => e.MinValue).HasEngineeringValue();
        b.Property(e => e.MaxValue).HasEngineeringValue();
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.TwinModelId, e.Code }).IsUnique()
            .HasDatabaseName("UQ_DigitalTwin_TwinVariable_Model_Code");
        b.HasIndex(e => new { e.TwinModelId, e.ModelVariableTypeId })
            .HasDatabaseName("IX_DigitalTwin_TwinVariable_Model_Type");

        b.HasOne(e => e.TwinModel)
            .WithMany(e => e.Variables)
            .HasForeignKey(e => e.TwinModelId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_DigitalTwin_TwinVariable_TwinModel");

        b.HasOne(e => e.VariableType)
    }
}
```

```

        .WithMany()
        .HasForeignKey(e => e.ModelVariableTypeId)
        .OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_DigitalTwin_TwinVariable_ModelVariableType");

    b.HasOne(e => e.EngineeringUnit)
        .WithMany()
        .HasForeignKey(e => e.EngineeringUnitId)
        .OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_DigitalTwin_TwinVariable_CorePlatform_EngineeringUnit");
    }
}

```

11.8 Binding the Model to Signals

The most important DigitalTwin mapping is `SignalBinding`. It is the C# expression of the book's central rule: the model is not trusted because it looks elegant; it is trusted only where it is bound to measured reality.

```

public sealed class SignalBindingConfiguration : IEntityTypeConfiguration<SignalBinding>
{
    public void Configure(EntityTypeBuilder<SignalBinding> b)
    {
        b.ToTable("SignalBinding", NexusSql.DigitalTwin);

        b.HasKey(e => e.SignalBindingId)
            .HasName("PK_DigitalTwin_SignalBinding");

        b.Property(e => e.BindingExpression).HasMaxLength(1000);
        b.Property(e => e.ScaleFactor).HasColumnType("decimal(18,9)");
        b.Property(e => e.Offset).HasColumnType("decimal(18,9)");
        b.Property(e => e.ValidFromUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ValidToUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.TwinVariableId, e.SignalId, e.BindingRoleId }).IsUnique()
            .HasDatabaseName("UQ_DigitalTwin_SignalBinding_Variable_Signal_Role");
        b.HasIndex(e => new { e.SignalId, e.BindingStatusId })
            .HasDatabaseName("IX_DigitalTwin_SignalBinding_Signal_Status");

        b.HasOne(e => e.TwinVariable)
            .WithMany(e => e.SignalBindings)
            .HasForeignKey(e => e.TwinVariableId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_DigitalTwin_SignalBinding_TwinVariable");

        b.HasOne(e => e.Signal)
            .WithMany()
            .HasForeignKey(e => e.SignalId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DigitalTwin_SignalBinding_Instrumentation_Signal");

        b.HasOne(e => e.BindingRole)
            .WithMany()
            .HasForeignKey(e => e.BindingRoleId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DigitalTwin_SignalBinding_BindingRole");
    }
}

```

11.9 Snapshot and Snapshot Value Configurations

Snapshots record the model state at a specific instant. Values are stored separately so a snapshot can contain many variables without widening the table whenever a model changes.

```

public sealed class TwinSnapshotConfiguration : IEntityTypeConfiguration<TwinSnapshot>
{
    public void Configure(EntityTypeBuilder<TwinSnapshot> b)
    {
        b.ToTable("TwinSnapshot", NexusSql.DigitalTwin);

        b.HasKey(e => e.TwinSnapshotId)
            .HasName("PK_DigitalTwin_TwinSnapshot");
    }
}

```

```

        b.Property(e => e.CapturedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.SourceCorrelationId).HasMaxLength(100);
        b.Property(e => e.Notes).HasMaxLength(1000);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.TwinModelId, e.CapturedAtUtc })
            .HasDatabaseName("IX_DigitalTwin_TwinSnapshot_Model_Time");
        b.HasIndex(e => new { e.SnapshotReasonId, e.CapturedAtUtc })
            .HasDatabaseName("IX_DigitalTwin_TwinSnapshot_Reason_Time");

        b.HasOne(e => e.TwinModel)
            .WithMany(e => e.Snapshots)
            .HasForeignKey(e => e.TwinModelId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_DigitalTwin_TwinSnapshot_TwinModel");

        b.HasOne(e => e.SnapshotReason)
            .WithMany()
            .HasForeignKey(e => e.SnapshotReasonId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DigitalTwin_TwinSnapshot_SnapshotReason");
    }
}

public sealed class TwinSnapshotValueConfiguration
    : IEntityTypeConfiguration<TwinSnapshotValue>
{
    public void Configure(EntityTypeBuilder<TwinSnapshotValue> b)
    {
        b.ToTable("TwinSnapshotValue", NexusSql.DigitalTwin);
        b.HasKey(e => e.TwinSnapshotValueId)
            .HasName("PK_DigitalTwin_TwinSnapshotValue");

        b.Property(e => e.ModelledValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.MeasuredValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.QualityCode).HasMaxLength(50);

        b.HasIndex(e => new { e.TwinSnapshotId, e.TwinVariableId }).IsUnique()
            .HasDatabaseName("UQ_DigitalTwin_TwinSnapshotValue_Snapshot_Variable");

        b.HasOne(e => e.TwinSnapshot)
            .WithMany(e => e.Values)
            .HasForeignKey(e => e.TwinSnapshotId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_DigitalTwin_TwinSnapshotValue_TwinSnapshot");

        b.HasOne(e => e.TwinVariable)
            .WithMany()
            .HasForeignKey(e => e.TwinVariableId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DigitalTwin_TwinSnapshotValue_TwinVariable");
    }
}

```

11.10 Divergence Configuration

`TwinDivergence` is the table whose existence keeps the twin honest. The EF configuration makes divergence review queryable by model variable, severity, status, and time.

```
public sealed class TwinDivergenceConfiguration : IEntityTypeConfiguration<TwinDivergence>
{
    public void Configure(EntityTypeBuilder<TwinDivergence> b)
    {
        b.ToTable("TwinDivergence", NexusSql.DigitalTwin);

        b.HasKey(e => e.TwinDivergenceId)
            .HasName("PK_DigitalTwin_TwinDivergence");

        b.Property(e => e.DetectedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModelledValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.MeasuredValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.AbsoluteError).HasColumnType("decimal(18,6)");
        b.Property(e => e.RelativeErrorPercent).HasColumnType("decimal(9,4)");
        b.Property(e => e.Explanation).HasMaxLength(2000);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.TwinVariableId, e.DetectedAtUtc })
            .HasDatabaseName("IX_DigitalTwin_TwinDivergence_Variable_Time");
        b.HasIndex(e => new { e.DivergenceSeverityId, e.DivergenceStatusId })
            .HasDatabaseName("IX_DigitalTwin_TwinDivergence_Severity_Status");

        b.HasOne(e => e.TwinVariable)
            .WithMany()
            .HasForeignKey(e => e.TwinVariableId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DigitalTwin_TwinDivergence_TwinVariable");

        b.HasOne(e => e.Severity)
            .WithMany()
            .HasForeignKey(e => e.DivergenceSeverityId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DigitalTwin_TwinDivergence_DivergenceSeverity");

        b.HasOne(e => e.Status)
            .WithMany()
            .HasForeignKey(e => e.DivergenceStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DigitalTwin_TwinDivergence_DivergenceStatus");
    }
}
```

11.11 Simulation Run Configuration

Simulation runs, scenarios, steps, and outputs are treated as durable facts. They can be compared, reproduced, audited, and joined back to the model version that produced them.

```
public sealed class SimulationRunConfiguration : IEntityTypeConfiguration<SimulationRun>
{
    public void Configure(EntityTypeBuilder<SimulationRun> b)
    {
        b.ToTable("SimulationRun", NexusSql.DigitalTwin);

        b.HasKey(e => e.SimulationRunId)
            .HasName("PK_DigitalTwin_SimulationRun");

        b.Property(e => e.RunCode).HasTwinCode(80);
        b.Property(e => e.StartedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CompletedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RandomSeed).HasMaxLength(100);
        b.Property(e => e.ResultSummary).HasMaxLength(2000);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.RunCode).IsUnique()
            .HasDatabaseName("UQ_DigitalTwin_SimulationRun_RunCode");
        b.HasIndex(e => new { e.TwinModelId, e.SimulationRunStatusId, e.StartedAtUtc })
            .HasDatabaseName("IX_DigitalTwin_SimulationRun_Model_Status_Start");

        b.HasOne(e => e.TwinModel)
            .WithMany(e => e.SimulationRuns)
    }
}
```

```

        .HasForeignKey(e => e.TwinModelId)
        .OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_DigitalTwin_SimulationRun_TwinModel");

    b.HasOne(e => e.Status)
        .WithMany()
        .HasForeignKey(e => e.SimulationRunStatusId)
        .OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_DigitalTwin_SimulationRun_SimulationRunStatus");
    }
}

```

11.12 Passport Rules

CHILD CONFIGURATION	PARENT TABLE	DELETE BEHAVIOR	REASON
TwinModelConfiguration	ReactorFleet.Unit	Restrict	A physical unit must not disappear because a model record is deleted.
TwinVariableConfiguration	CorePlatform.EngineeringUnit	Restrict	Units are shared reference data and must remain stable.
SignalBindingConfiguration	Instrumentation.Signal	Restrict	The binding points to a measured signal; deleting the signal must be deliberate.
TwinSnapshotValueConfiguration	DigitalTwin.TwinSnapshot	Cascade	Snapshot values have no meaning without their parent snapshot.
SimulationRunConfiguration	DigitalTwin.TwinModel	Restrict	Simulation evidence remains attached to the model that produced it.

11.13 Migration and Verification Notes

Migration order. DigitalTwin depends on ReactorFleet, Instrumentation, CorePlatform, and Security. In the full build order it must be created after those schemas and before AlarmManagement, EventManagement, RootCause, Reporting, and Audit consume its outputs.

```

-- EF Core migration smoke tests for DigitalTwin.
SELECT name
FROM sys.tables
WHERE schema_id = SCHEMA_ID('DigitalTwin')
ORDER BY name;

-- Ensure model codes are unique and mapped as intended.
SELECT i.name, i.is_unique
FROM sys.indexes AS i
JOIN sys.tables AS t ON t.object_id = i.object_id
JOIN sys.schemas AS s ON s.schema_id = t.schema_id
WHERE s.name = 'DigitalTwin'
    AND i.name LIKE 'UQ_DigitalTwin%'
ORDER BY i.name;

-- Confirm the model-to-reality binding passports exist.
SELECT fk.name, OBJECT_SCHEMA_NAME(fk.parent_object_id) AS child_schema,
    OBJECT_NAME(fk.parent_object_id) AS child_table,
    OBJECT_SCHEMA_NAME(fk.referenced_object_id) AS parent_schema,
    OBJECT_NAME(fk.referenced_object_id) AS parent_table

```



```

FROM sys.foreign_keys AS fk
WHERE fk.name LIKE 'FK_DigitalTwin%'
ORDER BY fk.name;

-- The heart of the twin: variable + signal + unit + model.
SELECT TOP (20)
    m.Code AS model_code,
    v.Code AS variable_code,
    s.Tag AS signal_tag,
    eu.Symbol AS unit_symbol
FROM DigitalTwin.TwinVariable AS v
JOIN DigitalTwin.TwinModel AS m ON m.TwinModelId = v.TwinModelId
LEFT JOIN DigitalTwin.SignalBinding AS b ON b.TwinVariableId = v.TwinVariableId
LEFT JOIN Instrumentation.Signal AS s ON s.SignalId = b.SignalId
LEFT JOIN CorePlatform.EngineeringUnit AS eu ON eu.EngineeringUnitId = v.EngineeringUnitId
ORDER BY m.Code, v.Code;

```

11.14 Configuration File Index

TwinModelConfiguration - Configurations/
DigitalTwin/TwinModelConfiguration.cs

TwinModelVersionConfiguration - Configurations/
DigitalTwin/TwinModelVersionConfiguration.cs

TwinModelComponentConfiguration -
Configurations/DigitalTwin/
TwinModelComponentConfiguration.cs

TwinVariableConfiguration - Configurations/
DigitalTwin/TwinVariableConfiguration.cs

TwinParameterConfiguration - Configurations/
DigitalTwin/TwinParameterConfiguration.cs

SignalBindingConfiguration - Configurations/
DigitalTwin/SignalBindingConfiguration.cs

SignalBindingCalibrationConfiguration -
Configurations/DigitalTwin/
SignalBindingCalibrationConfiguration.cs

TwinSnapshotConfiguration - Configurations/
DigitalTwin/TwinSnapshotConfiguration.cs

TwinSnapshotValueConfiguration -
Configurations/DigitalTwin/
TwinSnapshotValueConfiguration.cs

TwinDivergenceConfiguration - Configurations/
DigitalTwin/TwinDivergenceConfiguration.cs

TwinDivergenceReviewConfiguration -
Configurations/DigitalTwin/
TwinDivergenceReviewConfiguration.cs

TwinRuntimeSessionConfiguration -
Configurations/DigitalTwin/
TwinRuntimeSessionConfiguration.cs

TwinSynchronizationConfiguration -
Configurations/DigitalTwin/
TwinSynchronizationConfiguration.cs

TwinHealthCheckConfiguration - Configurations/
DigitalTwin/TwinHealthCheckConfiguration.cs

TwinModelValidationConfiguration -
Configurations/DigitalTwin/
TwinModelValidationConfiguration.cs

SimulationRunConfiguration - Configurations/
DigitalTwin/SimulationRunConfiguration.cs

SimulationRunStepConfiguration -
Configurations/DigitalTwin/
SimulationRunStepConfiguration.cs

SimulationRunOutputConfiguration -
Configurations/DigitalTwin/
SimulationRunOutputConfiguration.cs

WhatIfCaseConfiguration - Configurations/
DigitalTwin/WhatIfCaseConfiguration.cs

WhatIfCaseInputConfiguration - Configurations/
DigitalTwin/WhatIfCaseInputConfiguration.cs

WhatIfCaseResultConfiguration - Configurations/
DigitalTwin/WhatIfCaseResultConfiguration.cs

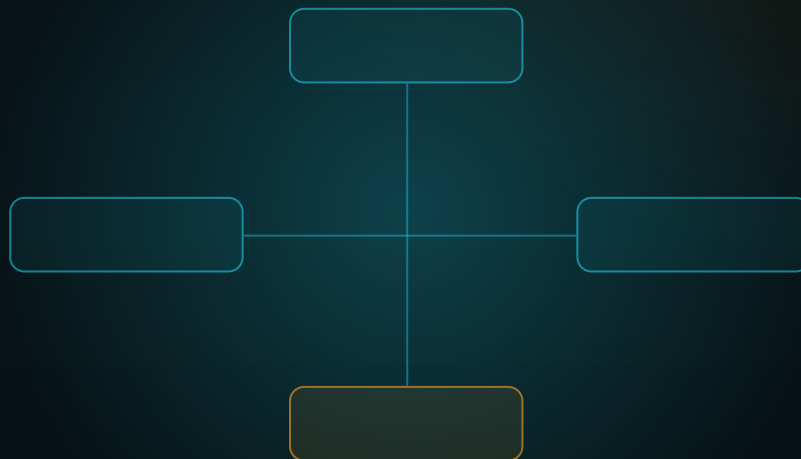
Honest Boundary

The EF Core configuration files make the DigitalTwin schema precise; they do not make a model validated. A table can say which signal feeds which variable, and a configuration can enforce that relationship, but calibration, fidelity, uncertainty, and certification remain engineering work outside the mapper. This chapter maps the contract; it does not certify the twin.

FROM ENTITY TO CONTEXT

Chapter 12 - AlarmManagement Configurations

The EF Core mapping layer for definitions, runtime alarm events, acknowledgements, floods, and operator response.



SCHEMA CONFIGURATION COMPANION

AlarmManagement: what lit up, when, for whom, and how it was handled

Definitions - runtime facts - flood windows - accountability.

Chapter 12

AlarmManagement Configurations

The complete EF Core configuration pass for the NEXUS-1 **AlarmManagement** schema. The schema records alarm definitions, condition logic, runtime alarm events, acknowledgements, comments, operator actions, notifications, suppressions, shelving, alarm floods, rankings, rationalization and KPIs.

From Entity to Context - The EF Core Configuration Atlas of the NEXUS-1 Digital Twin

Chapter 12 - AlarmManagement Configurations

AlarmManagement is where the flood becomes durable data. It does not decide the root cause. It preserves the operator-facing facts: what alarm definition fired, when it fired, which signal and unit it was attached to, how it changed state, who acknowledged it, which flood window it belonged to, and which naive rankings were shown before causal analysis.

Continuation rule. Chapters 6-11 configured CorePlatform, Security, Organization, ReactorFleet, Instrumentation and DigitalTwin. AlarmManagement is the first operations schema after the twin. It consumes passports from ReactorFleet, Instrumentation, CorePlatform and Security, then produces alarm events that RootCause, Reporting and Audit can read without rewriting them.

12.1 What This Schema Owns

Governed definitions

AlarmDefinition, conditions, limits, deadbands, routing, annunciator panels and tiles.

Runtime alarm facts

AlarmEvent, state history, acknowledgements, comments, evidence, notifications and escalation.

Operator controls

Suppression, shelving and standing orders are mapped as accountable records with time windows.

Flood capture

Flood windows, members, summaries and non-causal rankings preserve the flood as the operator saw it.

45

configuration files

18

lookup
configurations

27

substantive
configurations

4

main passport
sources

0

root-cause verdicts

12.2 Configuration Catalogue

Every table receives its own configuration class. This is especially important for alarms because definitions are governed and mutable, while runtime events and flood membership are append-only operational facts.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	AlarmManagement.AlarmSeverity	AlarmSeverityConfiguration.cs	Severity of the alarm condition: information, warning, high, critical, trip.
Lookup	AlarmManagement.AlarmPriority	AlarmPriorityConfiguration.cs	Operator handling priority after rationalization. Priority is not causality.
Lookup	AlarmManagement.AlarmState	AlarmStateConfiguration.cs	Runtime state: active, acknowledged, returned, cleared, suppressed, shelved.
Lookup	AlarmManagement.AlarmCategory	AlarmCategoryConfiguration.cs	Process, safety, security, equipment, instrumentation, digital-twin or advisory category.
Lookup	AlarmManagement.AlarmClass	AlarmClassConfiguration.cs	Annunciation class used for display, filtering and operator workflows.
Lookup	AlarmManagement.AlarmCondition Type	AlarmConditionTypeConfiguration.cs	High limit, low limit, rate-of-change, deviation, quality, state-change or composite rule.
Lookup	AlarmManagement.AlarmDirection	AlarmDirectionConfiguration.cs	Rising, falling, either, state-entered or state-exited.
Lookup	AlarmManagement.AlarmLifecycle Status	AlarmLifecycleStatusConfiguration.cs	Draft, rationalized, approved, active, retired or superseded.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	AlarmManagement.AlarmSourceType	AlarmSourceTypeConfiguration.cs	Signal, equipment, system, digital twin, manual entry, import or external service source.
Lookup	AlarmManagement.AlarmFloodStatus	AlarmFloodStatusConfiguration.cs	Detected, open, analyzed, handed-off, closed or archived flood.
Lookup	AlarmManagement.AlarmSuppressionReason	AlarmSuppressionReasonConfiguration.cs	Reasons for designed suppression: maintenance, startup mode, known nuisance, test, invalid source.
Lookup	AlarmManagement.AlarmShelvingReason	AlarmShelvingReasonConfiguration.cs	Operator shelving reason with expiry and accountability.
Lookup	AlarmManagement.AlarmAckMethod	AlarmAckMethodConfiguration.cs	How an alarm was acknowledged: console, mobile, API, batch, system, drill.
Lookup	AlarmManagement.NotificationChannel	NotificationChannelConfiguration.cs	Console, email, SMS, Teams, API, historian or pager channel.
Lookup	AlarmManagement.EscalationStatus	EscalationStatusConfiguration.cs	Pending, sent, accepted, failed, cancelled or timed-out escalation.
Lookup	AlarmManagement.RationalizationStatus	RationalizationStatusConfiguration.cs	Review state of a rationalized alarm: proposed, approved, rejected, expired.
Lookup	AlarmManagement.OperatorActionType	OperatorActionTypeConfiguration.cs	Operator response type: observe, diagnose, adjust, isolate, confirm, escalate.
Lookup	AlarmManagement.KpiMetricType	KpiMetricTypeConfiguration.cs	Alarm-rate, standing alarm count, flood count, acknowledgement latency and nuisance metrics.
Definition	AlarmManagement.AlarmDefinition	AlarmDefinitionConfiguration.cs	Authoritative definition of one alarmable condition, including source, classification and lifecycle.
Definition	AlarmManagement.AlarmCondition	AlarmConditionConfiguration.cs	One evaluable condition under a definition: high limit, low limit, deviation, quality or composite condition.
Definition	AlarmManagement.AlarmLimit	AlarmLimitConfiguration.cs	Numeric limit, limit band and engineering unit used by a condition.
Definition	AlarmManagement.AlarmDeadband	AlarmDeadbandConfiguration.cs	Return-to-normal hysteresis, delay and debounce configuration.
Definition	AlarmManagement.AlarmRoutingRule	AlarmRoutingRuleConfiguration.cs	Routing rule deciding who is notified for a class, priority, unit or time window.
Definition	AlarmManagement.AlarmRoutingRecipient	AlarmRoutingRecipientConfiguration.cs	Users, roles or channels that receive notification from a routing rule.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Definition	AlarmManagement.AnnunciatorPanel	AnnunciatorPanelConfiguration.cs	Logical alarm panel displayed by the console.
Definition	AlarmManagement.AnnunciatorTile	AnnunciatorTileConfiguration.cs	Position of an alarm definition on a panel or screen.
Runtime	AlarmManagement.AlarmEvent	AlarmEventConfiguration.cs	Append-only fact that an alarm condition fired, returned or was imported.
Runtime	AlarmManagement.AlarmEventStateHistory	AlarmEventStateHistoryConfiguration.cs	State-transition history for an event: active, acknowledged, returned, cleared.
Runtime	AlarmManagement.AlarmAcknowledgement	AlarmAcknowledgementConfiguration.cs	Who acknowledged the event, when, how and with which note.
Runtime	AlarmManagement.AlarmComment	AlarmCommentConfiguration.cs	Operator or engineer comment attached to an alarm event.
Runtime	AlarmManagement.AlarmOperatorAction	AlarmOperatorActionConfiguration.cs	Action taken or recommended in response to an alarm.
Runtime	AlarmManagement.AlarmEventEvidence	AlarmEventEvidenceConfiguration.cs	Evidence snapshot tying an event to the signal measurements that supported it.
Runtime	AlarmManagement.AlarmNotification	AlarmNotificationConfiguration.cs	Notification attempt generated by a routing or escalation rule.
Runtime	AlarmManagement.AlarmEscalation	AlarmEscalationConfiguration.cs	Escalation instance for unacknowledged or critical alarms.
Runtime	AlarmManagement.AlarmEscalationAction	AlarmEscalationActionConfiguration.cs	Individual action or delivery attempt inside an escalation.
Control	AlarmManagement.AlarmSuppression	AlarmSuppressionConfiguration.cs	Planned or system-approved suppression with reason and time boundary.
Control	AlarmManagement.AlarmShelving	AlarmShelvingConfiguration.cs	Temporary operator shelving with expiry and accountability.
Control	AlarmManagement.AlarmStandingOrder	AlarmStandingOrderConfiguration.cs	Approved handling instruction presented when the alarm fires.
Flood	AlarmManagement.AlarmFlood	AlarmFloodConfiguration.cs	Detected alarm flood window for a unit.
Flood	AlarmManagement.AlarmFloodMember	AlarmFloodMemberConfiguration.cs	One alarm event inside a flood, with order, strand and display grouping.
Flood	AlarmManagement.AlarmFloodSummary	AlarmFloodSummaryConfiguration.cs	Aggregate counts, rates and severity mix for the flood.
Flood	AlarmManagement.AlarmFloodRanking	AlarmFloodRankingConfiguration.cs	Naive ranking by count or priority, explicitly kept separate from root-cause ranking.
Governance	AlarmManagement.AlarmRationalization	AlarmRationalizationConfiguration.cs	Rationalization record defining purpose, consequence, priority and operator response.
Governance	AlarmManagement.AlarmRationalizationReview	AlarmRationalizationReviewConfiguration.cs	Review and approval trail for a rationalized alarm.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Governance	AlarmManagement.AlarmKpiSnapshot	AlarmKpiSnapshotConfiguration.cs	Periodic KPI row for alarm management performance.

12.3 Folder Structure

The folder structure separates definitions, runtime facts, control records, flood tables and governance records. The context still discovers all files with one line:

`ApplyConfigurationsFromAssembly.`

```
Nexus.Infrastructure/
  Persistence/
    NexusContext.cs
  Configurations/
    AlarmManagement/
      Lookups/
        AlarmSeverityConfiguration.cs
        AlarmPriorityConfiguration.cs
        AlarmStateConfiguration.cs
        AlarmCategoryConfiguration.cs
        AlarmClassConfiguration.cs
        AlarmConditionTypeConfiguration.cs
        AlarmDirectionConfiguration.cs
        AlarmLifecycleStatusConfiguration.cs
        AlarmSourceTypeConfiguration.cs
        AlarmFloodStatusConfiguration.cs
        AlarmSuppressionReasonConfiguration.cs
        AlarmShelvingReasonConfiguration.cs
        AlarmAckMethodConfiguration.cs
        NotificationChannelConfiguration.cs
        EscalationStatusConfiguration.cs
        RationalizationStatusConfiguration.cs
        OperatorActionTypeConfiguration.cs
        KpiMetricTypeConfiguration.cs
      Definitions/
        AlarmDefinitionConfiguration.cs
        AlarmConditionConfiguration.cs
        AlarmLimitConfiguration.cs
        AlarmDeadbandConfiguration.cs
        AlarmRoutingRuleConfiguration.cs
        AlarmRoutingRecipientConfiguration.cs
        AnnunciatorPanelConfiguration.cs
        AnnunciatorTileConfiguration.cs
      Runtime/
        AlarmEventConfiguration.cs
        AlarmEventStateHistoryConfiguration.cs
        AlarmAcknowledgementConfiguration.cs
        AlarmCommentConfiguration.cs
        AlarmOperatorActionConfiguration.cs
        AlarmEventEvidenceConfiguration.cs
        AlarmNotificationConfiguration.cs
        AlarmEscalationConfiguration.cs
      Control/
        AlarmSuppressionConfiguration.cs
        AlarmShelvingConfiguration.cs
        AlarmStandingOrderConfiguration.cs
      Flood/
        AlarmFloodConfiguration.cs
        AlarmFloodMemberConfiguration.cs
        AlarmFloodSummaryConfiguration.cs
        AlarmFloodRankingConfiguration.cs
      Governance/
        AlarmRationalizationConfiguration.cs
        AlarmRationalizationReviewConfiguration.cs
        AlarmKpiSnapshotConfiguration.cs
```


12.4 EF Core Dependency Diagram

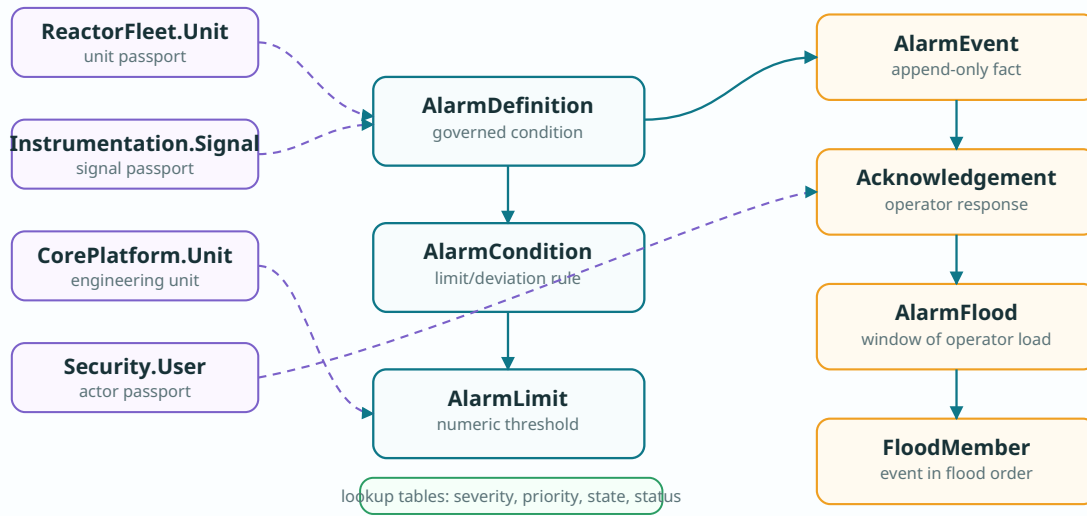


Figure 12.1 - AlarmManagement EF Core dependency diagram. Definitions consume unit/signal/unit-of-measure passports; events and operator response records preserve what happened after a definition fired.

12.5 Shared Configuration Primitives

Alarm lookup tables use the same enterprise shape: code, name, description, display order, active flag, created/modified timestamps and rowversion. The extension methods keep precision and naming consistent.

```
namespace Nexus.Infrastructure.Persistence.Configurations;

internal static partial class NexusSql
{
    public const string AlarmManagement = "AlarmManagement";
    public const string ReactorFleet = "ReactorFleet";
    public const string Instrumentation = "Instrumentation";
    public const string CorePlatform = "CorePlatform";
    public const string Security = "Security";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class AlarmLookupConfiguration<TEntity>
    : IEntityConfiguration<TEntity>
    where TEntity : AlarmLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(150)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

internal static class AlarmMappingExtensions
{
    public static PropertyBuilder<string> HasAlarmCode(
        this PropertyBuilder<string> property, int length = 80) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);

    public static PropertyBuilder<decimal?> HasAlarmValue(
        this PropertyBuilder<decimal?> property) => property
        .HasColumnType("decimal(18,6)");
}
```

12.6 Lookup Configuration Examples

The same pattern applies to: `AlarmSeverity`, `AlarmPriority`, `AlarmState`, `AlarmCategory`, `AlarmClass`, `AlarmConditionType`, `AlarmDirection`, `AlarmLifecycleStatus`, `AlarmSourceType`, `AlarmFloodStatus`, `AlarmSuppressionReason`, `AlarmShelvingReason`, `AlarmAckMethod`, `NotificationChannel`, `EscalationStatus`, `RationalizationStatus`, `OperatorActionType`, `KpiMetricType`.

```
public sealed class AlarmSeverityConfiguration
    : AlarmLookupConfiguration<AlarmSeverity>
{
    public override void Configure(EntityTypeBuilder<AlarmSeverity> b)
    {
        b.ToTable("AlarmSeverity", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmSeverityId)
            .HasName("PK_AlarmManagement_AlarmSeverity");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_AlarmManagement_AlarmSeverity_Code");
    }
}

public sealed class AlarmPriorityConfiguration
    : AlarmLookupConfiguration<AlarmPriority>
{
    public override void Configure(EntityTypeBuilder<AlarmPriority> b)
    {
        b.ToTable("AlarmPriority", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmPriorityId)
            .HasName("PK_AlarmManagement_AlarmPriority");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_AlarmManagement_AlarmPriority_Code");
    }
}

public sealed class AlarmStateConfiguration
    : AlarmLookupConfiguration<AlarmState>
{
    public override void Configure(EntityTypeBuilder<AlarmState> b)
    {
        b.ToTable("AlarmState", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmStateId)
            .HasName("PK_AlarmManagement_AlarmState");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_AlarmManagement_AlarmState_Code");
    }
}
```

12.7 Definition and Condition Configurations

`AlarmDefinition` maps the governed alarmable condition. Conditions, limits and deadbands are children of the definition and use cascade delete only inside the definition aggregate. Cross-schema passports use `Restrict`.

```
public sealed class AlarmDefinitionConfiguration
    : IEntityTypeConfiguration<AlarmDefinition>
{
    public void Configure(EntityTypeBuilder<AlarmDefinition> b)
    {
        b.ToTable("AlarmDefinition", NexusSql.AlarmManagement);

        b.HasKey(e => e.AlarmDefinitionId)
            .HasName("PK_AlarmManagement_AlarmDefinition");

        b.Property(e => e.Code).HasAlarmCode(80);
        b.Property(e => e.Name).HasMaxLength(250).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1000);
        b.Property(e => e.OperatorMessage).HasMaxLength(1000);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_AlarmManagement_AlarmDefinition_Code");
        b.HasIndex(e => new { e.UnitId, e.SignalId, e.AlarmLifecycleStatusId })
            .HasDatabaseName("IX_AlarmManagement_AlarmDefinition_Unit_Signal_Status");
        b.HasIndex(e => new { e.AlarmPriorityId, e.AlarmSeverityId })
            .HasDatabaseName("IX_AlarmManagement_AlarmDefinition_Priority_Severity");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmDefinition_ReactorFleet_Unit");

        b.HasOne(e => e.Signal)
            .WithMany()
            .HasForeignKey(e => e.SignalId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmDefinition_Instrumentation_Signal");

        b.HasOne(e => e.Priority)
            .WithMany()
            .HasForeignKey(e => e.AlarmPriorityId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmDefinition_AlarmPriority");
    }
}
```

```
public sealed class AlarmConditionConfiguration
    : IEntityTypeConfiguration<AlarmCondition>
{
    public void Configure(EntityTypeBuilder<AlarmCondition> b)
    {
        b.ToTable("AlarmCondition", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmConditionId)
            .HasName("PK_AlarmManagement_AlarmCondition");

        b.Property(e => e.Expression).HasMaxLength(2000);
        b.Property(e => e.EvaluationOrder).HasDefaultValue(0);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.AlarmDefinitionId, e.EvaluationOrder })
            .HasDatabaseName("IX_AlarmManagement_AlarmCondition_Definition_Order");

        b.HasOne(e => e.AlarmDefinition)
            .WithMany(e => e.Conditions)
            .HasForeignKey(e => e.AlarmDefinitionId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_AlarmManagement_AlarmCondition_AlarmDefinition");

        b.HasOne(e => e.ConditionType)
            .WithMany()
            .HasForeignKey(e => e.AlarmConditionTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmCondition_AlarmConditionType");
    }
}

public sealed class AlarmLimitConfiguration : IEntityTypeConfiguration<AlarmLimit>
```

```

{
    public void Configure(EntityTypeBuilder<AlarmLimit> b)
    {
        b.ToTable("AlarmLimit", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmLimitId)
            .HasName("PK_AlarmManagement_AlarmLimit");

        b.Property(e => e.LimitValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.WarningValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.TripValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.AlarmConditionId).IsUnique()
            .HasDatabaseName("UQ_AlarmManagement_AlarmLimit_Condition");

        b.HasOne(e => e.Condition)
            .WithOne(e => e.Limit)
            .HasForeignKey<AlarmLimit>(e => e.AlarmConditionId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_AlarmManagement_AlarmLimit_AlarmCondition");

        b.HasOne(e => e.EngineeringUnit)
            .WithMany()
            .HasForeignKey(e => e.EngineeringUnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmLimit_CorePlatform_EngineeringUnit");
    }
}

```

12.8 Runtime Alarm Event Configuration

`AlarmEvent` is the runtime fact. It is indexed by unit/time, definition/time and state/severity/time because those are the operational queries that dominate alarm screens.

```

public sealed class AlarmEventConfiguration : IEntityTypeConfiguration<AlarmEvent>
{
    public void Configure(EntityTypeBuilder<AlarmEvent> b)
    {
        b.ToTable("AlarmEvent", NexusSql.AlarmManagement);

        b.HasKey(e => e.AlarmEventId)
            .HasName("PK_AlarmManagement_AlarmEvent");

        b.Property(e => e.EventCode).HasAlarmCode(100);
        b.Property(e => e.RaisedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ReturnedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.MeasuredValue).HasAlarmValue();
        b.Property(e => e.ThresholdValue).HasAlarmValue();
        b.Property(e => e.Message).HasMaxLength(1000);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.EventCode).IsUnique()
            .HasDatabaseName("UQ_AlarmManagement_AlarmEvent_EventCode");
        b.HasIndex(e => new { e.UnitId, e.RaisedAtUtc })
            .HasDatabaseName("IX_AlarmManagement_AlarmEvent_Unit_Time");
        b.HasIndex(e => new { e.AlarmDefinitionId, e.RaisedAtUtc })
            .HasDatabaseName("IX_AlarmManagement_AlarmEvent_Definition_Time");
        b.HasIndex(e => new { e.AlarmStateId, e.AlarmSeverityId, e.RaisedAtUtc })
            .HasDatabaseName("IX_AlarmManagement_AlarmEvent_State_Severity_Time");

        b.HasOne(e => e.AlarmDefinition)
            .WithMany(e => e.Events)
            .HasForeignKey(e => e.AlarmDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmEvent_AlarmDefinition");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmEvent_ReactorFleet_Unit");

        b.HasOne(e => e.Signal)
            .WithMany()
            .HasForeignKey(e => e.SignalId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmEvent_Instrumentation_Signal");
    }
}

```

12.9 Operator Response Configurations

Acknowledgements, comments and operator actions are children of the event. User references remain restricted passports to Security; deleting a user must not delete the historical response.

```
public sealed class AlarmAcknowledgementConfiguration
    : IEntityTypeConfiguration<AlarmAcknowledgement>
{
    public void Configure(EntityTypeBuilder<AlarmAcknowledgement> b)
    {
        b.ToTable("AlarmAcknowledgement", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmAcknowledgementId)
            .HasName("PK_AlarmManagement_AlarmAcknowledgement");

        b.Property(e => e.AcknowledgedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.Note).HasMaxLength(1000);

        b.HasIndex(e => new { e.AlarmEventId, e.AcknowledgedAtUtc })
            .HasDatabaseName("IX_AlarmManagement_AlarmAck_Event_Time");

        b.HasOne(e => e.AlarmEvent)
            .WithMany(e => e.Acknowledgements)
            .HasForeignKey(e => e.AlarmEventId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_AlarmManagement_AlarmAcknowledgement_AlarmEvent");

        b.HasOne(e => e.AcknowledgedByUser)
            .WithMany()
            .HasForeignKey(e => e.AcknowledgedByUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmAcknowledgement_Security_User");

        b.HasOne(e => e.Method)
            .WithMany()
            .HasForeignKey(e => e.AlarmAckMethodId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmAcknowledgement_AlarmAckMethod");
    }
}

public sealed class AlarmOperatorActionConfiguration
    : IEntityTypeConfiguration<AlarmOperatorAction>
{
    public void Configure(EntityTypeBuilder<AlarmOperatorAction> b)
    {
        b.ToTable("AlarmOperatorAction", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmOperatorActionId)
            .HasName("PK_AlarmManagement_AlarmOperatorAction");

        b.Property(e => e.ActionText).HasMaxLength(2000).IsRequired();
        b.Property(e => e.ActionedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.AlarmEventId, e.ActionedAtUtc })
            .HasDatabaseName("IX_AlarmManagement_AlarmOperatorAction_Event_Time");

        b.HasOne(e => e.AlarmEvent)
            .WithMany(e => e.OperatorActions)
            .HasForeignKey(e => e.AlarmEventId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_AlarmManagement_AlarmOperatorAction_AlarmEvent");

        b.HasOne(e => e.ActionType)
            .WithMany()
            .HasForeignKey(e => e.OperatorActionTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmOperatorAction_OperatorActionType");
    }
}
```

12.10 Flood Configurations

Flood records keep the event order and grouping that operators saw. The flood tables do not store causal verdicts; those are the responsibility of the RootCause schema.

```

public sealed class AlarmFloodConfiguration : IEntityTypeConfiguration<AlarmFlood>
{
    public void Configure(EntityTypeBuilder<AlarmFlood> b)
    {
        b.ToTable("AlarmFlood", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmFloodId)
            .HasName("PK_AlarmManagement_AlarmFlood");

        b.Property(e => e.FloodCode).HasAlarmCode(100);
        b.Property(e => e.WindowStartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.WindowEndUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.Description).HasMaxLength(1000);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.FloodCode).IsUnique()
            .HasDatabaseName("UQ_AlarmManagement_AlarmFlood_FloodCode");
        b.HasIndex(e => new { e.UnitId, e.WindowStartUtc })
            .HasDatabaseName("IX_AlarmManagement_AlarmFlood_Unit_Start");
        b.HasIndex(e => new { e.AlarmFloodStatusId, e.WindowStartUtc })
            .HasDatabaseName("IX_AlarmManagement_AlarmFlood_Status_Start");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmFlood_ReactorFleet_Unit");

        b.HasOne(e => e.Status)
            .WithMany()
            .HasForeignKey(e => e.AlarmFloodStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmFlood_AlarmFloodStatus");
    }
}

public sealed class AlarmFloodMemberConfiguration
: IEntityTypeConfiguration<AlarmFloodMember>
{
    public void Configure(EntityTypeBuilder<AlarmFloodMember> b)
    {
        b.ToTable("AlarmFloodMember", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmFloodMemberId)
            .HasName("PK_AlarmManagement_AlarmFloodMember");

        b.Property(e => e.ArrivalOrder).IsRequired();
        b.Property(e => e.DisplayGroup).HasMaxLength(100);
        b.Property(e => e.StrandLabel).HasMaxLength(100);

        b.HasIndex(e => new { e.AlarmFloodId, e.AlarmEventId }).IsUnique()
            .HasDatabaseName("UQ_AlarmManagement_AlarmFloodMember_Flood_Event");
        b.HasIndex(e => new { e.AlarmFloodId, e.ArrivalOrder })
            .HasDatabaseName("IX_AlarmManagement_AlarmFloodMember_Flood_Order");

        b.HasOne(e => e.AlarmFlood)
            .WithMany(e => e.Members)
            .HasForeignKey(e => e.AlarmFloodId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_AlarmManagement_AlarmFloodMember_AlarmFlood");

        b.HasOne(e => e.AlarmEvent)
            .WithMany()
            .HasForeignKey(e => e.AlarmEventId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmFloodMember_AlarmEvent");
    }
}

```

12.11 Suppression, Shelving and Notification

Suppression and shelving are accountable control records. Notifications and escalations are delivery facts linked back to events.

```

public sealed class AlarmSuppressionConfiguration
: IEntityTypeConfiguration<AlarmSuppression>
{
    public void Configure(EntityTypeBuilder<AlarmSuppression> b)
    {
        b.ToTable("AlarmSuppression", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmSuppressionId)
            .HasName("PK_AlarmManagement_AlarmSuppression");

        b.Property(e => e.StartsAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.EndsAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.Justification).HasMaxLength(1000).IsRequired();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.AlarmDefinitionId, e.StartsAtUtc, e.EndsAtUtc })

```

```

        .HasDatabaseName("IX_AlarmManagement_AlarmSuppression_Definition_Window");

        b.HasOne(e => e.AlarmDefinition)
            .WithMany(e => e.Suppressions)
            .HasForeignKey(e => e.AlarmDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmSuppression_AlarmDefinition");

        b.HasOne(e => e.SuppressedByUser)
            .WithMany()
            .HasForeignKey(e => e.SuppressedByUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmSuppression_Security_User");
    }
}

public sealed class AlarmShelvingConfiguration
    : IEntityTypeConfiguration<AlarmShelving>
{
    public void Configure(EntityTypeBuilder<AlarmShelving> b)
    {
        b.ToTable("AlarmShelving", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmShelvingId)
            .HasName("PK_AlarmManagement_AlarmShelving");

        b.Property(e => e.ShelvedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ExpiresAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ReasonText).HasMaxLength(1000).IsRequired();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.AlarmDefinitionId, e.ExpiresAtUtc })
            .HasDatabaseName("IX_AlarmManagement_AlarmShelving_Definition_Expiry");

        b.HasOne(e => e.AlarmDefinition)
            .WithMany(e => e.Shelvings)
            .HasForeignKey(e => e.AlarmDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmShelving_AlarmDefinition");
    }
}

```

```

public sealed class AlarmNotificationConfiguration
    : IEntityTypeConfiguration<AlarmNotification>
{
    public void Configure(EntityTypeBuilder<AlarmNotification> b)
    {
        b.ToTable("AlarmNotification", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmNotificationId)
            .HasName("PK_AlarmManagement_AlarmNotification");

        b.Property(e => e.Recipient).HasMaxLength(250).IsRequired();
        b.Property(e => e.SentAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.DeliveryReference).HasMaxLength(200);
        b.Property(e => e.ErrorMessage).HasMaxLength(1000);

        b.HasIndex(e => new { e.AlarmEventId, e.NotificationChannelId, e.SentAtUtc })
            .HasDatabaseName("IX_AlarmManagement_AlarmNotification_Event_Channel_Time");

        b.HasOne(e => e.AlarmEvent)
            .WithMany(e => e.Notifications)
            .HasForeignKey(e => e.AlarmEventId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_AlarmManagement_AlarmNotification_AlarmEvent");

        b.HasOne(e => e.Channel)
            .WithMany()
            .HasForeignKey(e => e.NotificationChannelId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AlarmManagement_AlarmNotification_NotificationChannel");
    }
}

public sealed class AlarmEscalationConfiguration
    : IEntityTypeConfiguration<AlarmEscalation>
{
    public void Configure(EntityTypeBuilder<AlarmEscalation> b)
    {
        b.ToTable("AlarmEscalation", NexusSql.AlarmManagement);
        b.HasKey(e => e.AlarmEscalationId)
            .HasName("PK_AlarmManagement_AlarmEscalation");

        b.Property(e => e.StartedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CompletedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.AlarmEventId, e.EscalationStatusId })
            .HasDatabaseName("IX_AlarmManagement_AlarmEscalation_Event_Status");

        b.HasOne(e => e.AlarmEvent)
            .WithMany(e => e.Escalations)
            .HasForeignKey(e => e.AlarmEventId)
            .OnDelete(DeleteBehavior.Cascade)
    }
}

```



```

    }
    .HasConstraintName("FK_AlarmManagement_AlarmEscalation_AlarmEvent");
}

```

12.12 Passport Rules

CHILD CONFIGURATION	PARENT TABLE	DELETE BEHAVIOR	REASON
AlarmDefinitionConfiguration	ReactorFleet.Unit	Restrict	Definitions are scoped to units; deleting a unit cannot silently delete alarm definitions.
AlarmDefinitionConfiguration	Instrumentation.Signal	Restrict	The signal is the measured source; removal must be explicit and audited.
AlarmLimitConfiguration	CorePlatform.EngineeringUnit	Restrict	Alarm thresholds must keep a stable unit-of-measure reference.
AlarmAcknowledgementConfiguration	Security.ApplicationUser	Restrict	Operator accountability must survive user lifecycle changes.
AlarmFloodMemberConfiguration	AlarmManagement.AlarmFlood	Cascade	Membership rows have no meaning without their parent flood window.

12.13 Migration and Verification Notes

Migration order. AlarmManagement depends on CorePlatform, Security, ReactorFleet and Instrumentation. It must exist before RootCause, Reporting, Compliance and Audit consume alarm events and flood windows.

```

-- EF Core migration smoke tests for AlarmManagement.
SELECT name
FROM sys.tables
WHERE schema_id = SCHEMA_ID('AlarmManagement')
ORDER BY name;

-- Confirm unique natural-code indexes were generated with stable names.
SELECT i.name, i.is_unique
FROM sys.indexes AS i
JOIN sys.tables AS t ON t.object_id = i.object_id
JOIN sys.schemas AS s ON s.schema_id = t.schema_id
WHERE s.name = 'AlarmManagement'
AND (i.name LIKE 'UQ_AlarmManagement%' OR i.name LIKE 'IX_AlarmManagement%')
ORDER BY i.name;

-- Confirm cross-schema passports are declared, not implied.
SELECT fk.name,
       OBJECT_SCHEMA_NAME(fk.parent_object_id) AS child_schema,
       OBJECT_NAME(fk.parent_object_id) AS child_table,
       OBJECT_SCHEMA_NAME(fk.referenced_object_id) AS parent_schema,
       OBJECT_NAME(fk.referenced_object_id) AS parent_table
FROM sys.foreign_keys AS fk
WHERE fk.name LIKE 'FK_AlarmManagement%'
ORDER BY fk.name;

-- Follow a unit from alarm definition to event to flood membership.
SELECT TOP (20)
    u.Code AS unit_code,
    d.Code AS alarm_code,
    e.EventCode AS event_code,
    e.RaisedAtUtc,
    f.FloodCode,

```

```

        m.ArrivalOrder
FROM AlarmManagement.AlarmEvent AS e
JOIN AlarmManagement.AlarmDefinition AS d ON d.AlarmDefinitionId = e.AlarmDefinitionId
JOIN ReactorFleet.Unit AS u ON u.UnitId = e.UnitId
LEFT JOIN AlarmManagement.AlarmFloodMember AS m ON m.AlarmEventId = e.AlarmEventId
LEFT JOIN AlarmManagement.AlarmFlood AS f ON f.AlarmFloodId = m.AlarmFloodId
ORDER BY e.RaisedAtUtc DESC;

```

12.14 Configuration File Index

AlarmSeverityConfiguration - Configurations/
AlarmManagement/AlarmSeverityConfiguration.cs

AlarmPriorityConfiguration - Configurations/
AlarmManagement/AlarmPriorityConfiguration.cs

AlarmStateConfiguration - Configurations/
AlarmManagement/AlarmStateConfiguration.cs

AlarmCategoryConfiguration - Configurations/
AlarmManagement/AlarmCategoryConfiguration.cs

AlarmClassConfiguration - Configurations/
AlarmManagement/AlarmClassConfiguration.cs

AlarmConditionTypeConfiguration -
Configurations/AlarmManagement/
AlarmConditionTypeConfiguration.cs

AlarmDirectionConfiguration - Configurations/
AlarmManagement/AlarmDirectionConfiguration.cs

AlarmLifecycleStatusConfiguration -
Configurations/AlarmManagement/
AlarmLifecycleStatusConfiguration.cs

AlarmSourceTypeConfiguration - Configurations/
AlarmManagement/
AlarmSourceTypeConfiguration.cs

AlarmFloodStatusConfiguration - Configurations/
AlarmManagement/
AlarmFloodStatusConfiguration.cs

AlarmSuppressionReasonConfiguration -
Configurations/AlarmManagement/
AlarmSuppressionReasonConfiguration.cs

AlarmShelvingReasonConfiguration -
Configurations/AlarmManagement/
AlarmShelvingReasonConfiguration.cs

AlarmAckMethodConfiguration - Configurations/
AlarmManagement/AlarmAckMethodConfiguration.cs

NotificationChannelConfiguration -
Configurations/AlarmManagement/
NotificationChannelConfiguration.cs

EscalationStatusConfiguration - Configurations/
AlarmManagement/
EscalationStatusConfiguration.cs

RationalizationStatusConfiguration -
Configurations/AlarmManagement/
RationalizationStatusConfiguration.cs

OperatorActionTypeConfiguration -
Configurations/AlarmManagement/
OperatorActionTypeConfiguration.cs

KpiMetricTypeConfiguration - Configurations/
AlarmManagement/KpiMetricTypeConfiguration.cs

AlarmDefinitionConfiguration - Configurations/
AlarmManagement/
AlarmDefinitionConfiguration.cs

AlarmConditionConfiguration - Configurations/
AlarmManagement/AlarmConditionConfiguration.cs

AlarmLimitConfiguration - Configurations/
AlarmManagement/AlarmLimitConfiguration.cs

AlarmDeadbandConfiguration - Configurations/
AlarmManagement/AlarmDeadbandConfiguration.cs

AlarmRoutingRuleConfiguration - Configurations/
AlarmManagement/
AlarmRoutingRuleConfiguration.cs

AlarmRoutingRecipientConfiguration -
Configurations/AlarmManagement/
AlarmRoutingRecipientConfiguration.cs

AnnunciatorPanelConfiguration - Configurations/
AlarmManagement/
AnnunciatorPanelConfiguration.cs

AnnunciatorTileConfiguration - Configurations/
AlarmManagement/
AnnunciatorTileConfiguration.cs

AlarmEventConfiguration - Configurations/
AlarmManagement/AlarmEventConfiguration.cs

AlarmEventStateHistoryConfiguration -
Configurations/AlarmManagement/
AlarmEventStateHistoryConfiguration.cs

AlarmAcknowledgementConfiguration -
Configurations/AlarmManagement/
AlarmAcknowledgementConfiguration.cs

AlarmCommentConfiguration - Configurations/
AlarmManagement/AlarmCommentConfiguration.cs

AlarmOperatorActionConfiguration -
Configurations/AlarmManagement/
AlarmOperatorActionConfiguration.cs

AlarmEventEvidenceConfiguration -
Configurations/AlarmManagement/
AlarmEventEvidenceConfiguration.cs

AlarmNotificationConfiguration - Configurations/
AlarmManagement/
AlarmNotificationConfiguration.cs

AlarmEscalationConfiguration - Configurations/
AlarmManagement/
AlarmEscalationConfiguration.cs

AlarmEscalationActionConfiguration -

Configurations/AlarmManagement/
AlarmEscalationActionConfiguration.cs

AlarmSuppressionConfiguration - Configurations/

AlarmManagement/
AlarmSuppressionConfiguration.cs

AlarmShelvingConfiguration - Configurations/
AlarmManagement/AlarmShelvingConfiguration.cs

AlarmStandingOrderConfiguration -

Configurations/AlarmManagement/
AlarmStandingOrderConfiguration.cs

AlarmFloodConfiguration - Configurations/
AlarmManagement/AlarmFloodConfiguration.cs

AlarmFloodMemberConfiguration -

Configurations/AlarmManagement/
AlarmFloodMemberConfiguration.cs

AlarmFloodSummaryConfiguration -

Configurations/AlarmManagement/
AlarmFloodSummaryConfiguration.cs

AlarmFloodRankingConfiguration -

Configurations/AlarmManagement/
AlarmFloodRankingConfiguration.cs

AlarmRationalizationConfiguration -

Configurations/AlarmManagement/
AlarmRationalizationConfiguration.cs

AlarmRationalizationReviewConfiguration -

Configurations/AlarmManagement/
AlarmRationalizationReviewConfiguration.cs

AlarmKpiSnapshotConfiguration -

Configurations/AlarmManagement/
AlarmKpiSnapshotConfiguration.cs

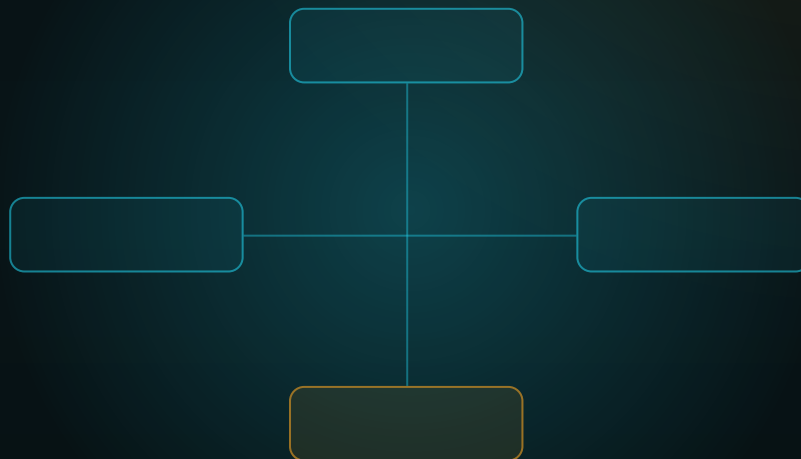
Honest Boundary

The mapping layer makes alarm facts durable and queryable; it does not make alarm rationalization correct by itself. EF Core can enforce keys, constraints, time columns, named indexes and delete behavior. It cannot decide whether an alarm should exist, whether a priority is appropriate, or whether a flood ranking is causal. That engineering judgement remains outside the mapper.

FROM ENTITY TO CONTEXT

Chapter 13 - EventManagement Configurations

The EF Core mapping layer for operational events, timelines, near misses, incidents, actions, and reviews.



SCHEMA CONFIGURATION COMPANION

EventManagement: from alarms to operational records

One event record - timeline - evidence links - incident actions.

Chapter 13

EventManagement Configurations

The complete EF Core configuration pass for the NEXUS-1 **EventManagement** schema. The schema turns alarm-created, manually reported, twin-detected, drill, maintenance and security occurrences into durable operational records.

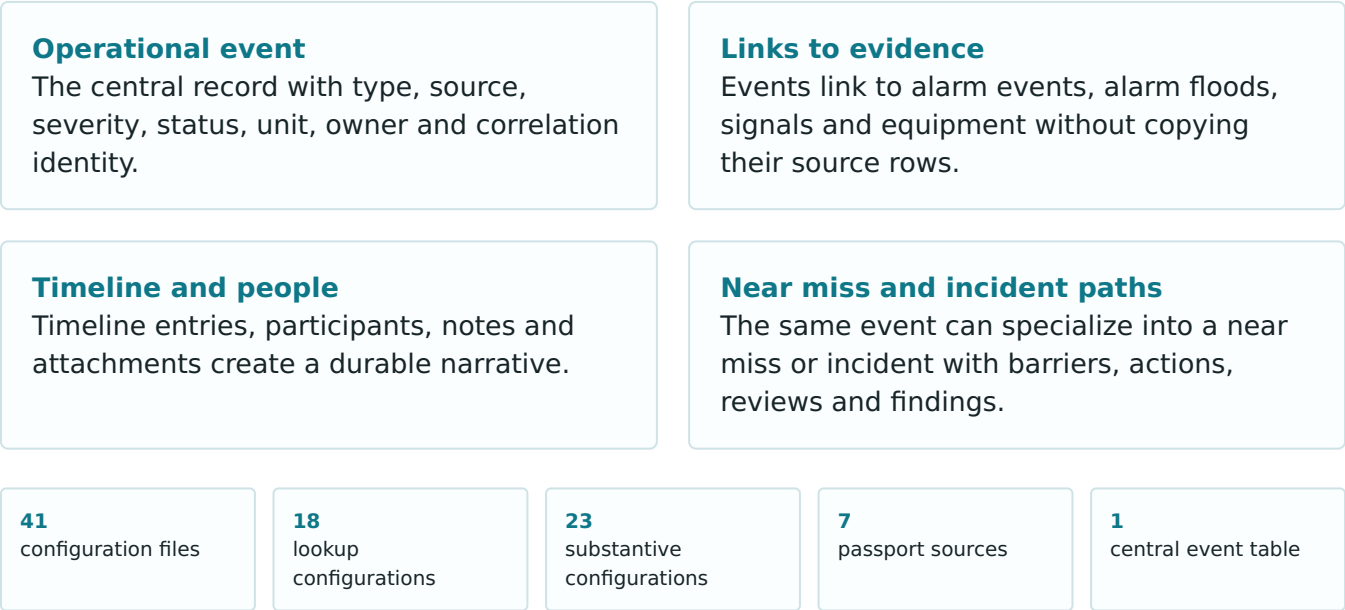
From Entity to Context - The EF Core Configuration Atlas of the NEXUS-1 Digital Twin

Chapter 13 - EventManagement Configurations

EventManagement is the bridge between raw operational signals and formal records. AlarmManagement knows what lit up; EventManagement decides which of those facts belong to an operational event, a near miss, an incident, a timeline, an action list, or a formal review.

Continuation rule. Chapter 12 mapped alarm definitions and runtime alarm facts. Chapter 13 consumes those facts through explicit passports and gives them lifecycle: an operational event, a timeline, participants, notes, attachments, impact assessments, near-miss barriers, incident actions and review findings.

13.1 What This Schema Owns



13.2 Configuration Catalogue

Each table receives its own configuration class. The catalogue is long because the schema is intentionally not a single incident table with text columns. It separates identity, lifecycle, relationships, timeline, impact, near-miss barriers and incident actions.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	EventManagement.EventType	EventTypeConfiguration.cs	Classifies operational events: incident, near miss, transient, drill, maintenance handoff, security or digital-twin divergence.
Lookup	EventManagement.EventStatus	EventStatusConfiguration.cs	Lifecycle of an operational event: new, triaged, investigating, actioned, closed, archived.
Lookup	EventManagement.EventSeverity	EventSeverityConfiguration.cs	Severity of the operational record, separate from alarm priority and root-cause confidence.
Lookup	EventManagement.EventSourceType	EventSourceTypeConfiguration.cs	How the event was created: alarm flood, manual report, drill, import, digital twin, maintenance or security.
Lookup	EventManagement.EventRelationshipType	EventRelationshipTypeConfiguration.cs	Relationship between events: duplicate, parent, child, related, follow-up, supersedes.
Lookup	EventManagement.EventTimelineEntryType	EventTimelineEntryTypeConfiguration.cs	Classifies timeline entries: detected, alarmed, acknowledged, action, decision, handoff, closure.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	EventManagement.EventParticipantRole	EventParticipantRoleConfiguration.cs	Role of a person or user in an event: owner, reporter, investigator, reviewer, observer.
Lookup	EventManagement.EventAttachmentType	EventAttachmentTypeConfiguration.cs	Classifies attachments: log export, screenshot, procedure, photo, report, historian extract.
Lookup	EventManagement.EventNoteType	EventNoteTypeConfiguration.cs	Classifies notes: operator note, engineer note, investigation note, closure note, system note.
Lookup	EventManagement.NearMissType	NearMissTypeConfiguration.cs	Classifies a near miss by the barrier or condition that prevented escalation.
Lookup	EventManagement.BarrierType	BarrierTypeConfiguration.cs	Physical, procedural, human, software, interlock, alarm, training or administrative barrier.
Lookup	EventManagement.IncidentType	IncidentTypeConfiguration.cs	Classifies incidents: operational, equipment, safety, radiation, security, cyber, environmental, training.
Lookup	EventManagement.IncidentStatus	IncidentStatusConfiguration.cs	Incident lifecycle: opened, investigating, awaiting action, review, closed, reopened.
Lookup	EventManagement.IncidentClassification	IncidentClassificationConfiguration.cs	Regulatory, internal, training, outage, reliability, quality or reporting classification.
Lookup	EventManagement.IncidentActionType	IncidentActionTypeConfiguration.cs	Corrective, preventive, compensatory, investigation, training, inspection or reporting action.
Lookup	EventManagement.IncidentActionStatus	IncidentActionStatusConfiguration.cs	Action state: proposed, assigned, in progress, completed, verified, cancelled, overdue.
Lookup	EventManagement.IncidentReviewOutcome	IncidentReviewOutcomeConfiguration.cs	Review outcome: accepted, returned, escalated, reopened, closed with actions.
Lookup	EventManagement.EventImpactType	EventImpactTypeConfiguration.cs	Classifies impact: safety, production, equipment, dose, compliance, availability, training.
Core	EventManagement.OperationalEvent	OperationalEventConfiguration.cs	The central event record: a structured operational occurrence with ownership and lifecycle.
Core	EventManagement.EventStatusHistory	EventStatusHistoryConfiguration.cs	Append-only status transition history for the event.
Core	EventManagement.EventRelationship	EventRelationshipConfiguration.cs	Directed relationship between events.
Core	EventManagement.EventTag	EventTagConfiguration.cs	Controlled tag catalogue for searching and reporting.
Core	EventManagement.EventTagAssignment	EventTagAssignmentConfiguration.cs	Many-to-many assignment of controlled tags to events.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Links	EventManagement.EventAlarmLink	EventAlarmLinkConfiguration.cs	Connects an event to one or more alarm events that triggered or supported it.
Links	EventManagement.EventFloodLink	EventFloodLinkConfiguration.cs	Connects an event to an alarm flood window.
Links	EventManagement.EventSignalLink	EventSignalLinkConfiguration.cs	Signals relevant to the event, including witnesses and context signals.
Links	EventManagement.EventEquipmentLink	EventEquipmentLinkConfiguration.cs	Equipment involved, suspected, affected, or explicitly ruled out at event level.
Timeline	EventManagement.EventTimelineEntry	EventTimelineEntryConfiguration.cs	Chronological narrative of what happened, who acted, and what changed.
Timeline	EventManagement.EventParticipant	EventParticipantConfiguration.cs	People or users involved in the event.
Timeline	EventManagement.EventNote	EventNoteConfiguration.cs	Notes attached to the event lifecycle.
Timeline	EventManagement.EventAttachment	EventAttachmentConfiguration.cs	Metadata for stored attachments; binary content is outside the normalized schema.
Context	EventManagement.EventImpactAssessment	EventImpactAssessmentConfiguration.cs	Structured impact row: safety, production, dose, equipment, compliance or availability.
Context	EventManagement.EventUnitStateSnapshot	EventUnitStateSnapshotConfiguration.cs	Event-time unit context: power, mode, trip status and grid state.
Near miss	EventManagement.NearMiss	NearMissConfiguration.cs	Specialized near-miss record tied to an operational event.
Near miss	EventManagement.NearMissBarrier	NearMissBarrierConfiguration.cs	Barrier that prevented escalation, and whether it succeeded, weakened or failed.
Incident	EventManagement.Incident	IncidentConfiguration.cs	Specialized incident record with investigation and closure fields.
Incident	EventManagement.IncidentClassificationAssignment	IncidentClassificationAssignmentConfiguration.cs	Many-to-many classification of an incident.
Incident	EventManagement.IncidentAction	IncidentActionConfiguration.cs	Corrective or preventive action arising from an incident.
Incident	EventManagement.IncidentActionOwner	IncidentActionOwnerConfiguration.cs	Ownership assignment for one incident action.
Incident	EventManagement.IncidentReview	IncidentReviewConfiguration.cs	Formal review record for an incident.
Incident	EventManagement.IncidentReviewFinding	IncidentReviewFindingConfiguration.cs	Finding or observation from a formal review.

13.3 Folder Structure

The folder structure follows the event lifecycle. The context remains clean because `ApplyConfigurationsFromAssembly` discovers every class.

```
Nexus.Infrastructure/
  Persistence/
    NexusContext.cs
  Configurations/
    EventManagement/
      Lookups/
        EventTypeConfiguration.cs
        EventStatusConfiguration.cs
        EventSeverityConfiguration.cs
        EventSourceTypeConfiguration.cs
        EventRelationshipTypeConfiguration.cs
        EventTimelineEntryTypeConfiguration.cs
        EventParticipantRoleConfiguration.cs
        EventAttachmentTypeConfiguration.cs
        EventNoteTypeConfiguration.cs
        NearMissTypeConfiguration.cs
        BarrierTypeConfiguration.cs
        IncidentTypeConfiguration.cs
        IncidentStatusConfiguration.cs
        IncidentClassificationConfiguration.cs
        IncidentActionTypeConfiguration.cs
        IncidentActionStatusConfiguration.cs
        IncidentReviewOutcomeConfiguration.cs
        EventImpactTypeConfiguration.cs
      Core/
        OperationalEventConfiguration.cs
        EventStatusHistoryConfiguration.cs
        EventRelationshipConfiguration.cs
        EventTagConfiguration.cs
        EventTagAssignmentConfiguration.cs
      Links/
        EventAlarmLinkConfiguration.cs
        EventFloodLinkConfiguration.cs
        EventSignalLinkConfiguration.cs
        EventEquipmentLinkConfiguration.cs
      Timeline/
        EventTimelineEntryConfiguration.cs
        EventParticipantConfiguration.cs
        EventNoteConfiguration.cs
        EventAttachmentConfiguration.cs
      Context/
        EventImpactAssessmentConfiguration.cs
        EventUnitStateSnapshotConfiguration.cs
      NearMiss/
        NearMissConfiguration.cs
        NearMissBarrierConfiguration.cs
      Incident/
        IncidentConfiguration.cs
        IncidentClassificationAssignmentConfiguration.cs
        IncidentActionConfiguration.cs
        IncidentActionOwnerConfiguration.cs
        IncidentReviewConfiguration.cs
        IncidentReviewFindingConfiguration.cs
```

13.4 EF Core Dependency Diagram

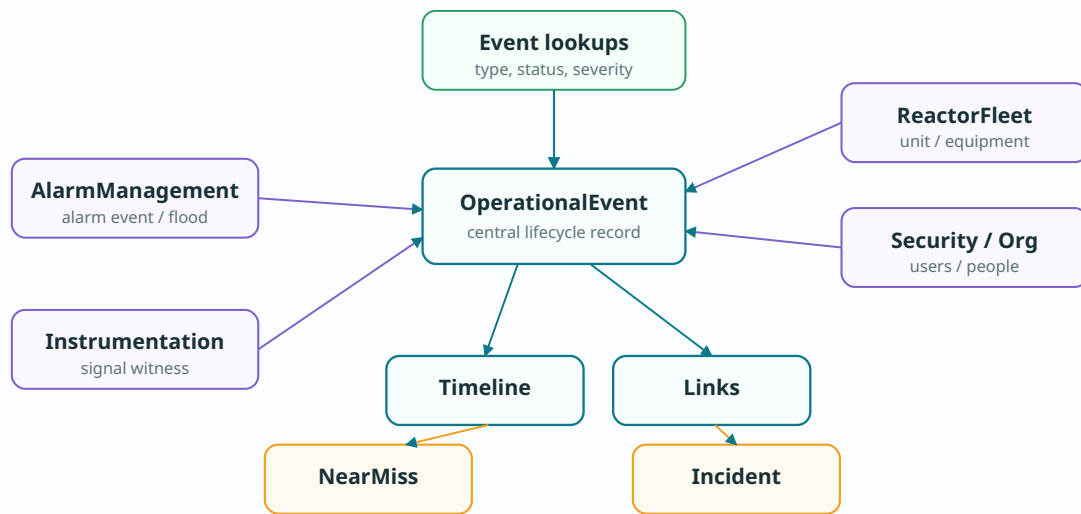
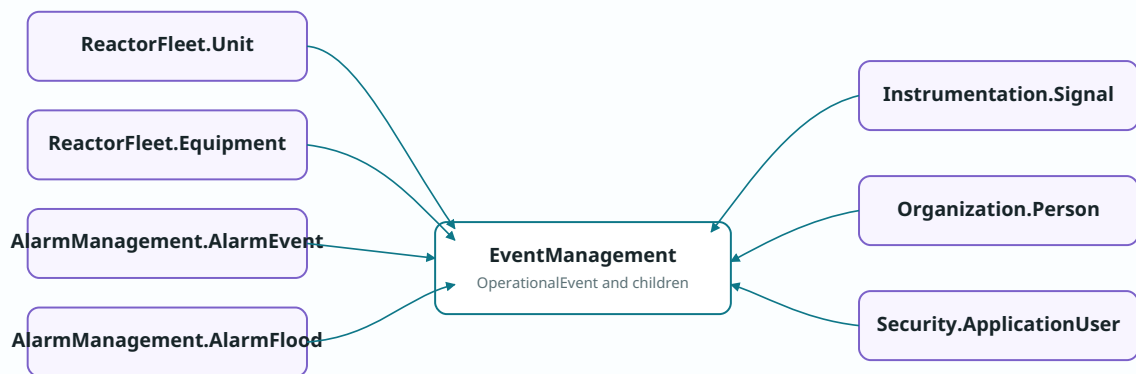


Figure 13.1 - **OperationalEvent** is the center. Alarm, signal, unit, equipment, user and person records stay in their owning schemas and are reached through explicit EF Core relationships.

13.5 Passport Map



DeleteBehavior.Restrict for external passports; Cascade only for true event-owned children.

Figure 13.2 - **EventManager** is a hub, not an owner of every fact. External rows remain in their original bounded contexts and are joined by named FK constraints.

13.6 Shared Lookup Configuration

The lookup family uses one base class. The concrete classes only set table, key and unique code index. Lookup tables in this chapter include: `EventType`, `EventStatus`, `EventSeverity`, `EventSourceType`, `EventRelationshipType`, `EventTimelineEntryType`, `EventParticipantRole`, `EventAttachmentType`, `EventNoteType`, `NearMissType`, `BarrierType`, `IncidentType`, `IncidentStatus`, `IncidentClassification`, `IncidentActionType`, `IncidentActionStatus`, `IncidentReviewOutcome`, `EventImpactType`.

```
namespace Nexus.Infrastructure.Persistence.Configurations;

internal static partial class NexusSql
{
    public const string EventManagement = "EventManagement";
    public const string ReactorFleet = "ReactorFleet";
    public const string AlarmManagement = "AlarmManagement";
    public const string Instrumentation = "Instrumentation";
    public const string Organization = "Organization";
    public const string Security = "Security";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class EventLookupConfiguration<TEntity>
    : IEntityConfiguration<TEntity>
    where TEntity : EventLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(150)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

internal static class EventMappingExtensions
{
    public static PropertyBuilder<string> HasEventCode(
        this PropertyBuilder<string> property, int length = 100) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);

    public static PropertyBuilder<decimal?> HasEventMetric(
        this PropertyBuilder<decimal?> property) => property
        .HasColumnType("decimal(18,6)");
}
```

```
public sealed class EventTypeConfiguration
    : EventLookupConfiguration<EventType>
{
    public override void Configure(EntityTypeBuilder<EventType> b)
    {
        b.ToTable("EventType", NexusSql.EventManagement);
        b.HasKey(e => e.EventTypeId)
            .HasName("PK_EventManagement_EventType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique();
    }
}
```

```

        .HasDatabaseName("UQ_EventManagement_EventType_Code");
    }
}

public sealed class EventStatusConfiguration
    : EventLookupConfiguration<EventStatus>
{
    public override void Configure(EntityTypeBuilder<EventStatus> b)
    {
        b.ToTable("EventStatus", NexusSql.EventManagement);
        b.HasKey(e => e.EventStatusId)
            .HasName("PK_EventManagement_EventStatus");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_EventManagement_EventStatus_Code");
    }
}

public sealed class EventSeverityConfiguration
    : EventLookupConfiguration<EventSeverity>
{
    public override void Configure(EntityTypeBuilder<EventSeverity> b)
    {
        b.ToTable("EventSeverity", NexusSql.EventManagement);
        b.HasKey(e => e.EventSeverityId)
            .HasName("PK_EventManagement_EventSeverity");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_EventManagement_EventSeverity_Code");
    }
}

```

13.7 Core Event Mapping

`OperationalEvent` is the aggregate root for the schema. It owns children such as status history, timeline entries, notes and incident branches; it only references external objects through passports.

```

public sealed class OperationalEventConfiguration
    : IEntityTypeConfiguration<OperationalEvent>
{
    public void Configure(EntityTypeBuilder<OperationalEvent> b)
    {
        b.ToTable("OperationalEvent", NexusSql.EventManagement);

        b.HasKey(e => e.OperationalEventId)
            .HasName("PK_EventManagement_OperationalEvent");

        b.Property(e => e.EventCode).HasEventCode(100);
        b.Property(e => e.Title).HasMaxLength(250).IsRequired();
        b.Property(e => e.Description).HasMaxLength(2000);
        b.Property(e => e.DetectedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.OpenedAtUtc).HasUtcDefault();
        b.Property(e => e.ClosedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CorrelationId).HasMaxLength(100);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.EventCode).IsUnique()
            .HasDatabaseName("UQ_EventManagement_OperationalEvent_EventCode");
        b.HasIndex(e => new { e.UnitId, e.DetectedAtUtc })
            .HasDatabaseName("IX_EventManagement_OperationalEvent_Unit_DetectedAt");
        b.HasIndex(e => new { e.EventStatusId, e.EventSeverityId, e.OpenedAtUtc })
            .HasDatabaseName("IX_EventManagement_OperationalEvent_Status_Severity_Open");
        b.HasIndex(e => e.CorrelationId)
            .HasDatabaseName("IX_EventManagement_OperationalEvent_CorrelationId");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_OperationalEvent_ReactorFleet_Unit");

        b.HasOne(e => e.EventType)
            .WithMany()
            .HasForeignKey(e => e.EventTypeid)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_OperationalEvent_EventType");

        b.HasOne(e => e.EventStatus)
            .WithMany()
            .HasForeignKey(e => e.EventStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_OperationalEvent_EventStatus");

        b.HasOne(e => e.EventSeverity)

```

```

        .WithMany()
        .HasForeignKey(e => e.EventSeverityId)
        .OnDelete(DeleteBehavior.Restrict)
        .HasConstraintName("FK_EventManagement_OperationalEvent_EventSeverity");
    }
}

```

```

public sealed class EventStatusHistoryConfiguration
    : IEntityTypeConfiguration<EventStatusHistory>
{
    public void Configure(EntityTypeBuilder<EventStatusHistory> b)
    {
        b.ToTable("EventStatusHistory", NexusSql.EventManagement);
        b.HasKey(e => e.EventStatusHistoryId)
            .HasName("PK_EventManagement_EventStatusHistory");

        b.Property(e => e.ChangedAtUtc).HasUtcDefault();
        b.Property(e => e.Reason).HasMaxLength(1000);

        b.HasIndex(e => new { e.OperationalEventId, e.ChangedAtUtc })
            .HasDatabaseName("IX_EventManagement_EventStatusHistory_Event_Time");

        b.HasOne(e => e.OperationalEvent)
            .WithMany(e => e.StatusHistory)
            .HasForeignKey(e => e.OperationalEventId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_EventManagement_EventStatusHistory_OperationalEvent");

        b.HasOne(e => e.EventStatus)
            .WithMany()
            .HasForeignKey(e => e.EventStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_EventStatusHistory_EventStatus");
    }
}

public sealed class EventRelationshipConfiguration
    : IEntityTypeConfiguration<EventRelationship>
{
    public void Configure(EntityTypeBuilder<EventRelationship> b)
    {
        b.ToTable("EventRelationship", NexusSql.EventManagement);
        b.HasKey(e => e.EventRelationshipId)
            .HasName("PK_EventManagement_EventRelationship");

        b.Property(e => e.Note).HasMaxLength(1000);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasIndex(e => new { e.SourceOperationalEventId, e.TargetOperationalEventId, e.EventRelationshipTypeId })
            .IsUnique()
            .HasDatabaseName("UQ_EventManagement_EventRelationship_Source_Target_Type");

        b.HasOne(e => e.SourceOperationalEvent)
            .WithMany(e => e.OutgoingRelationships)
            .HasForeignKey(e => e.SourceOperationalEventId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_EventRelationship_SourceEvent");

        b.HasOne(e => e.TargetOperationalEvent)
            .WithMany(e => e.IncomingRelationships)
            .HasForeignKey(e => e.TargetOperationalEventId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_EventRelationship_TargetEvent");
    }
}

```

13.8 Links to Alarms, Signals and Equipment

Links are separate tables because one operational event can have many witnesses and one alarm may support more than one event interpretation over time. Deleting an event deletes its links; deleting a source alarm, signal or equipment row is restricted.

```

public sealed class EventAlarmLinkConfiguration
    : IEntityTypeConfiguration<EventAlarmLink>
{
    public void Configure(EntityTypeBuilder<EventAlarmLink> b)
    {
        b.ToTable("EventAlarmLink", NexusSql.EventManagement);
        b.HasKey(e => e.EventAlarmLinkId)
            .HasName("PK_EventManagement_EventAlarmLink");

        b.Property(e => e.Role).HasMaxLength(80);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
    }
}

```

```

        b.HasIndex(e => new { e.OperationalEventId, e.AlarmEventId }).IsUnique()
            .HasDatabaseName("UQ_EventManagement_EventAlarmLink_Event_Alarm");

        b.HasOne(e => e.OperationalEvent)
            .WithMany(e => e.AlarmLinks)
            .HasForeignKey(e => e.OperationalEventId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_EventManagement_EventAlarmLink_OperationalEvent");

        b.HasOne(e => e.AlarmEvent)
            .WithMany()
            .HasForeignKey(e => e.AlarmEventId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_EventAlarmLink_AlarmManagement_AlarmEvent");
    }
}

public sealed class EventSignalLinkConfiguration
    : IEntityTypeConfiguration<EventSignalLink>
{
    public void Configure(EntityTypeBuilder<EventSignalLink> b)
    {
        b.ToTable("EventSignalLink", NexusSql.EventManagement);
        b.HasKey(e => e.EventSignalLinkId)
            .HasName("PK_EventManagement_EventSignalLink");

        b.Property(e => e.LinkRole).HasMaxLength(80);
        b.Property(e => e.Note).HasMaxLength(1000);

        b.HasIndex(e => new { e.OperationalEventId, e.SignalId }).IsUnique()
            .HasDatabaseName("UQ_EventManagement_EventSignalLink_Event_Signal");

        b.HasOne(e => e.OperationalEvent)
            .WithMany(e => e.SignalLinks)
            .HasForeignKey(e => e.OperationalEventId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_EventManagement_EventSignalLink_OperationalEvent");

        b.HasOne(e => e.Signal)
            .WithMany()
            .HasForeignKey(e => e.SignalId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_EventSignalLink_Instrumentation_Signal");
    }
}

```

13.9 Timeline, Participants and Narrative

The timeline is append-friendly and ordered by event time and sequence number. Participants preserve both the structured person/user reference and a display-name snapshot for audit readability.

```

public sealed class EventTimelineEntryConfiguration
    : IEntityTypeConfiguration<EventTimelineEntry>
{
    public void Configure(EntityTypeBuilder<EventTimelineEntry> b)
    {
        b.ToTable("EventTimelineEntry", NexusSql.EventManagement);
        b.HasKey(e => e.EventTimelineEntryId)
            .HasName("PK_EventManagement_EventTimelineEntry");

        b.Property(e => e.EntryAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.Title).HasMaxLength(250).IsRequired();
        b.Property(e => e.Body).HasMaxLength(4000);
        b.Property(e => e.SequenceNumber).HasDefaultValue(0);

        b.HasIndex(e => new { e.OperationalEventId, e.EntryAtUtc, e.SequenceNumber })
            .HasDatabaseName("IX_EventManagement_EventTimelineEntry_Event_Time_Seq");

        b.HasOne(e => e.OperationalEvent)
            .WithMany(e => e.Timeline)
            .HasForeignKey(e => e.OperationalEventId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_EventManagement_EventTimelineEntry_OperationalEvent");

        b.HasOne(e => e.EventType)
            .WithMany()
            .HasForeignKey(e => e.EventTimelineEntryTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_EventTimelineEntry_EventTimelineEntryType");
    }
}

public sealed class EventParticipantConfiguration
    : IEntityTypeConfiguration<EventParticipant>
{
    public void Configure(EntityTypeBuilder<EventParticipant> b)

```

```

{
    b.ToTable("EventParticipant", NexusSql.EventManagement);
    b.HasKey(e => e.EventParticipantId)
      .HasName("PK_EventManagement_EventParticipant");

    b.Property(e => e.DisplayNameSnapshot).HasMaxLength(200);
    b.Property(e => e.CreatedAtUtc).HasUtcDefault();

    b.HasIndex(e => new { e.OperationalEventId, e.EventParticipantRoleId })
      .HasDatabaseName("IX_EventManagement_EventParticipant_Event_Role");

    b.HasOne(e => e.OperationalEvent)
      .WithMany(e => e.Participants)
      .HasForeignKey(e => e.OperationalEventId)
      .OnDelete(DeleteBehavior.Cascade)
      .HasConstraintName("FK_EventManagement_EventParticipant_OperationalEvent");

    b.HasOne(e => e.Person)
      .WithMany()
      .HasForeignKey(e => e.PersonId)
      .OnDelete(DeleteBehavior.Restrict)
      .HasConstraintName("FK_EventManagement_EventParticipant_Organization_Person");

    b.HasOne(e => e.ApplicationUser)
      .WithMany()
      .HasForeignKey(e => e.ApplicationUserId)
      .OnDelete(DeleteBehavior.Restrict)
      .HasConstraintName("FK_EventManagement_EventParticipant_Security_ApplicationUser");
}
}

```

13.10 Near Miss and Incident Branches

A near miss and an incident are specializations of an operational event. EF Core maps them as one-to-one branch tables, keeping the central event record stable while allowing domain-specific fields.

```

public sealed class NearMissConfiguration : IEntityTypeConfiguration<NearMiss>
{
    public void Configure(EntityTypeBuilder<NearMiss> b)
    {
        b.ToTable("NearMiss", NexusSql.EventManagement);
        b.HasKey(e => e.NearMissId)
          .HasName("PK_EventManagement_NearMiss");

        b.Property(e => e.NearMissCode).HasEventCode(100);
        b.Property(e => e.PotentialOutcome).HasMaxLength(1000);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.OperationalEventId).IsUnique()
          .HasDatabaseName("UQ_EventManagement_NearMiss_OperationalEvent");

        b.HasOne(e => e.OperationalEvent)
          .WithOne(e => e.NearMiss)
          .HasForeignKey<NearMiss>(e => e.OperationalEventId)
          .OnDelete(DeleteBehavior.Cascade)
          .HasConstraintName("FK_EventManagement_NearMiss_OperationalEvent");

        b.HasOne(e => e.NearMissType)
          .WithMany()
          .HasForeignKey(e => e.NearMissTypeId)
          .OnDelete(DeleteBehavior.Restrict)
          .HasConstraintName("FK_EventManagement_NearMiss_NearMissType");
    }
}

public sealed class IncidentConfiguration : IEntityTypeConfiguration<Incident>
{
    public void Configure(EntityTypeBuilder<Incident> b)
    {
        b.ToTable("Incident", NexusSql.EventManagement);
        b.HasKey(e => e.IncidentId)
          .HasName("PK_EventManagement_Incident");

        b.Property(e => e.IncidentCode).HasEventCode(100);
        b.Property(e => e.Summary).HasMaxLength(2000);
        b.Property(e => e.OpenedAtUtc).HasUtcDefault();
        b.Property(e => e.ClosedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.IncidentCode).IsUnique()
          .HasDatabaseName("UQ_EventManagement_Incident_IncidentCode");
        b.HasIndex(e => e.OperationalEventId).IsUnique()
          .HasDatabaseName("UQ_EventManagement_Incident_OperationalEvent");
        b.HasIndex(e => new { e.IncidentStatusId, e.OpenedAtUtc })
          .HasDatabaseName("IX_EventManagement_Incident_Status_Opened");
    }
}

```

```

        b.HasOne(e => e.OperationalEvent)
            .WithOne(e => e.Incident)
            .HasForeignKey<Incident>(e => e.OperationalEventId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_EventManagement_Incident_OperationalEvent");

        b.HasOne(e => e.IncidentType)
            .WithMany()
            .HasForeignKey(e => e.IncidentTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_Incident_IncidentType");

        b.HasOne(e => e.Status)
            .WithMany()
            .HasForeignKey(e => e.IncidentStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_Incident_IncidentStatus");
    }
}

```

```

public sealed class IncidentActionConfiguration
    : IEntityTypeConfiguration<IncidentAction>
{
    public void Configure(EntityTypeBuilder<IncidentAction> b)
    {
        b.ToTable("IncidentAction", NexusSql.EventManagement);
        b.HasKey(e => e.IncidentActionId)
            .HasName("PK_EventManagement_IncidentAction");

        b.Property(e => e.ActionCode).HasEventCode(100);
        b.Property(e => e.Description).HasMaxLength(2000).IsRequired();
        b.Property(e => e.DueAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CompletedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.ActionCode).IsUnique()
            .HasDatabaseName("UQ_EventManagement_IncidentAction_ActionCode");
        b.HasIndex(e => new { e.IncidentId, e.IncidentActionStatusId, e.DueAtUtc })
            .HasDatabaseName("IX_EventManagement_IncidentAction_Incident_Status_Due");

        b.HasOne(e => e.Incident)
            .WithMany(e => e.Actions)
            .HasForeignKey(e => e.IncidentId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_EventManagement_IncidentAction_Incident");

        b.HasOne(e => e.ActionType)
            .WithMany()
            .HasForeignKey(e => e.IncidentActionTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_IncidentAction_IncidentActionType");
    }
}

public sealed class IncidentReviewConfiguration
    : IEntityTypeConfiguration<IncidentReview>
{
    public void Configure(EntityTypeBuilder<IncidentReview> b)
    {
        b.ToTable("IncidentReview", NexusSql.EventManagement);
        b.HasKey(e => e.IncidentReviewId)
            .HasName("PK_EventManagement_IncidentReview");

        b.Property(e => e.ReviewedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ReviewSummary).HasMaxLength(2000);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.IncidentId, e.ReviewedAtUtc })
            .HasDatabaseName("IX_EventManagement_IncidentReview_Incident_Time");

        b.HasOne(e => e.Incident)
            .WithMany(e => e.Reviews)
            .HasForeignKey(e => e.IncidentId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_EventManagement_IncidentReview_Incident");

        b.HasOne(e => e.Outcome)
            .WithMany()
            .HasForeignKey(e => e.IncidentReviewOutcomeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EventManagement_IncidentReview_IncidentReviewOutcome");
    }
}

```


13.11 Migration and Verification Notes

- Run EventManagement after AlarmManagement, ReactorFleet, Instrumentation, Organization and Security because it references all of them.
- Use stable index and FK names to avoid unreadable migration diffs.
- Use `DeleteBehavior.Restrict` across schema boundaries and `Cascade` only for event-owned children.
- Keep incident actions and reviews separate from notes; they are workflow records, not prose comments.

```
-- EF Core migration smoke tests for EventManagement.
SELECT name
FROM sys.tables
WHERE schema_id = SCHEMA_ID('EventManagement')
ORDER BY name;

-- Confirm stable indexes and unique natural codes.
SELECT i.name, i.is_unique
FROM sys.indexes AS i
JOIN sys.tables AS t ON t.object_id = i.object_id
JOIN sys.schemas AS s ON s.schema_id = t.schema_id
WHERE s.name = 'EventManagement'
AND (i.name LIKE 'UQ_EventManagement%' OR i.name LIKE 'IX_EventManagement%')
ORDER BY i.name;

-- Confirm cross-schema passports are declared, not implied.
SELECT fk.name,
       OBJECT_SCHEMA_NAME(fk.parent_object_id) AS child_schema,
       OBJECT_NAME(fk.parent_object_id) AS child_table,
       OBJECT_SCHEMA_NAME(fk.referenced_object_id) AS parent_schema,
       OBJECT_NAME(fk.referenced_object_id) AS parent_table
FROM sys.foreign_keys AS fk
WHERE fk.name LIKE 'FK_EventManagement%'
ORDER BY fk.name;

-- Follow an alarm-created operational event into incident actions.
SELECT TOP (20)
       oe.EventCode,
       oe.Title,
       ae.EventCode AS alarm_event_code,
       i.IncidentCode,
       ia.ActionCode,
       ia.DueAtUtc
FROM EventManagement.OperationalEvent AS oe
LEFT JOIN EventManagement.EventAlarmLink AS eal ON eal.OperationalEventId = oe.OperationalEventId
LEFT JOIN AlarmManagement.AlarmEvent AS ae ON ae.AlarmEventId = eal.AlarmEventId
LEFT JOIN EventManagement.Incident AS i ON i.OperationalEventId = oe.OperationalEventId
LEFT JOIN EventManagement.IncidentAction AS ia ON ia.IncidentId = i.IncidentId
ORDER BY oe.OpenedAtUtc DESC;
```

Honest boundary. These mappings express enterprise software structure, not operating authority. EventManagement records and organizes operational occurrences inside the NEXUS-1 demonstration model; it does not certify an incident investigation, issue a real corrective action, or replace a plant procedure.

13.12 Configuration File Index

EventTypeConfiguration - Configurations/
EventManagement/EventTypeConfiguration.cs

EventStatusConfiguration - Configurations/
EventManagement/EventStatusConfiguration.cs

EventSeverityConfiguration - Configurations/
EventManagement/EventSeverityConfiguration.cs

EventSourceTypeConfiguration - Configurations/
EventManagement/
EventSourceTypeConfiguration.cs

EventRelationshipTypeConfiguration -
Configurations/EventManagement/
EventRelationshipTypeConfiguration.cs

EventTimelineEntryTypeConfiguration -
Configurations/EventManagement/
EventTimelineEntryTypeConfiguration.cs

EventParticipantRoleConfiguration -
Configurations/EventManagement/
EventParticipantRoleConfiguration.cs

EventAttachmentTypeConfiguration -
Configurations/EventManagement/
EventAttachmentTypeConfiguration.cs

EventNoteTypeConfiguration - Configurations/
EventManagement/EventNoteTypeConfiguration.cs

NearMissTypeConfiguration - Configurations/
EventManagement/NearMissTypeConfiguration.cs

BarrierTypeConfiguration - Configurations/
EventManagement/BarrierTypeConfiguration.cs

IncidentTypeConfiguration - Configurations/
EventManagement/IncidentTypeConfiguration.cs

IncidentStatusConfiguration - Configurations/
EventManagement/IncidentStatusConfiguration.cs

IncidentClassificationConfiguration -
Configurations/EventManagement/
IncidentClassificationConfiguration.cs

IncidentActionTypeConfiguration -
Configurations/EventManagement/
IncidentActionTypeConfiguration.cs

IncidentActionStatusConfiguration -
Configurations/EventManagement/
IncidentActionStatusConfiguration.cs

IncidentReviewOutcomeConfiguration -
Configurations/EventManagement/
IncidentReviewOutcomeConfiguration.cs

EventImpactTypeConfiguration - Configurations/
EventManagement/
EventImpactTypeConfiguration.cs

OperationalEventConfiguration - Configurations/
EventManagement/
OperationalEventConfiguration.cs

EventStatusHistoryConfiguration -
Configurations/EventManagement/
EventStatusHistoryConfiguration.cs

EventRelationshipConfiguration - Configurations/
EventManagement/
EventRelationshipConfiguration.cs

EventTagConfiguration - Configurations/
EventManagement/EventTagConfiguration.cs

EventTagAssignmentConfiguration -
Configurations/EventManagement/
EventTagAssignmentConfiguration.cs

EventAlarmLinkConfiguration - Configurations/
EventManagement/EventAlarmLinkConfiguration.cs

EventFloodLinkConfiguration - Configurations/
EventManagement/EventFloodLinkConfiguration.cs

EventSignalLinkConfiguration - Configurations/
EventManagement/
EventSignalLinkConfiguration.cs

EventEquipmentLinkConfiguration -
Configurations/EventManagement/
EventEquipmentLinkConfiguration.cs

EventTimelineEntryConfiguration -
Configurations/EventManagement/
EventTimelineEntryConfiguration.cs

EventParticipantConfiguration - Configurations/
EventManagement/
EventParticipantConfiguration.cs

EventNoteConfiguration - Configurations/
EventManagement/EventNoteConfiguration.cs

EventAttachmentConfiguration - Configurations/
EventManagement/
EventAttachmentConfiguration.cs

EventImpactAssessmentConfiguration -
Configurations/EventManagement/
EventImpactAssessmentConfiguration.cs

EventUnitStateSnapshotConfiguration -
Configurations/EventManagement/
EventUnitStateSnapshotConfiguration.cs

NearMissConfiguration - Configurations/
EventManagement/NearMissConfiguration.cs

NearMissBarrierConfiguration - Configurations/
EventManagement/
NearMissBarrierConfiguration.cs

IncidentConfiguration - Configurations/
EventManagement/IncidentConfiguration.cs

IncidentClassificationAssignmentConfiguration -
Configurations/EventManagement/
IncidentClassificationAssignmentConfiguration.cs

IncidentActionConfiguration - Configurations/
EventManagement/IncidentActionConfiguration.cs

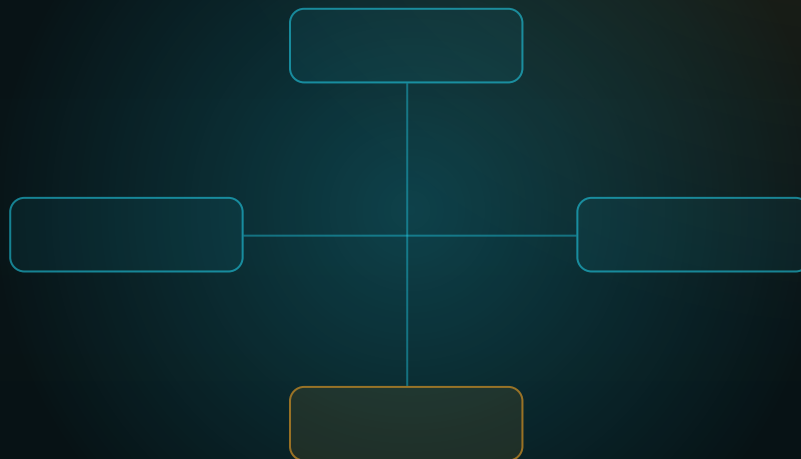
IncidentActionOwnerConfiguration -
Configurations/EventManager/
IncidentActionOwnerConfiguration.cs

IncidentReviewConfiguration - Configurations/
EventManager/IncidentReviewConfiguration.cs

IncidentReviewFindingConfiguration -
Configurations/EventManager/
IncidentReviewFindingConfiguration.cs

FROM ENTITY TO CONTEXT

Chapter 14 - Maintenance Configurations
Asset health, degradation, inventory, plans, work orders, inspections and
evidence mapped with Fluent API.



TECHNICAL REFERENCE COMPANION

**One schema chapter - 45 configuration files -
fully mapped**

SQL Server names stay stable. EF Core discovers the mappings
automatically.

Chapter 14

Maintenance Configurations

The Maintenance schema is where physical assets become maintainable assets: condition records, degradation trends, suppliers, spare parts, plans, schedules, work orders, inspections, findings, documents and approvals.

This chapter continues the EF Core atlas pattern: every table in the SQL schema receives one `IEntityTypeConfiguration<T>` class, every important key/index/constraint receives a stable name, and every cross-schema relationship is mapped intentionally as a passport.

Maintenance is the bridge between the live plant and the durable work record. In code, that bridge should not be a thousand anonymous Fluent API lines inside `OnModelCreating`; it should be a folder of named configuration files.

14.1 Scope of the Maintenance configuration chapter

The Maintenance sector is a high-value mapping chapter because it touches almost every design pattern used in an enterprise EF Core model: lookup tables, self-referencing components, append-only status history, condition measurements, inventory, many-to-many asset/part suitability, generated schedule occurrences, work-order tasks, operational-event links, documents and approvals.

Standing rule. The configuration class is the code-side authority for EF Core mapping: table name, schema, key, index name, delete behavior, precision, default SQL, rowversion and relationship name.

14.2 Configuration file catalogue

The catalogue below is generated from the Maintenance schema atlas and then expressed as EF Core configuration files. Long tables are intentionally allowed to flow across pages; no figure or table is allowed to overflow the page boundary.

GROUP	SQL TABLE	CONFIGURATION FILE	PURPOSE
Lookup	Maintenance.AssetCategory	AssetCategoryConfiguration.cs	Groups maintainable items: pump, valve, rod drive, turbine, generator, sensor, building service, robot support.
Lookup	Maintenance.AssetStatus	AssetStatusConfiguration.cs	Runtime maintenance status: in service, degraded, out of service, under maintenance, retired.
Lookup	Maintenance.AssetCriticality	AssetCriticalityConfiguration.cs	Business and engineering criticality used for priority, inspection frequency and reporting.
Lookup	Maintenance.ConditionGrade	ConditionGradeConfiguration.cs	Condition grade from excellent to failed. Used by asset condition and degradation records.
Lookup	Maintenance.DegradationMechanism	DegradationMechanismConfiguration.cs	Corrosion, wear, fatigue, thermal cycling, radiation embrittlement, fouling, vibration or obsolescence.
Lookup	Maintenance.InspectionType	InspectionTypeConfiguration.cs	Visual, vibration, thermography, ultrasonic, radiography, functional, calibration or document inspection.
Lookup	Maintenance.FindingSeverity	FindingSeverityConfiguration.cs	Severity assigned to inspection findings and active degradation records.
Lookup	Maintenance.WorkOrderType	WorkOrderTypeConfiguration.cs	Corrective, preventive, predictive, inspection, calibration, modification or emergent work.
Lookup	Maintenance.WorkOrderStatus	WorkOrderStatusConfiguration.cs	Draft, requested, approved, planned, in progress, blocked, complete, closed or cancelled.
Lookup	Maintenance.WorkPriority	WorkPriorityConfiguration.cs	Priority used by the planner: routine, low, normal, high, urgent, outage-critical.
Lookup	Maintenance.TaskStatus	TaskStatusConfiguration.cs	Task-level status independent of the parent work order lifecycle.
Lookup	Maintenance.MaintenancePlanType	MaintenancePlanTypeConfiguration.cs	Preventive, predictive, condition-based, outage, vendor-recommended or regulatory plan.
Lookup	Maintenance.ScheduleBasis	ScheduleBasisConfiguration.cs	Calendar interval, running hours, cycles, condition threshold, outage window or manual trigger.
Lookup	Maintenance.MaterialUsageType	MaterialUsageTypeConfiguration.cs	Planned, reserved, issued, consumed, returned, scrapped or substituted material usage.
Lookup	Maintenance.ApprovalStatus	ApprovalStatusConfiguration.cs	Approval lifecycle for work, documents and maintenance plans.

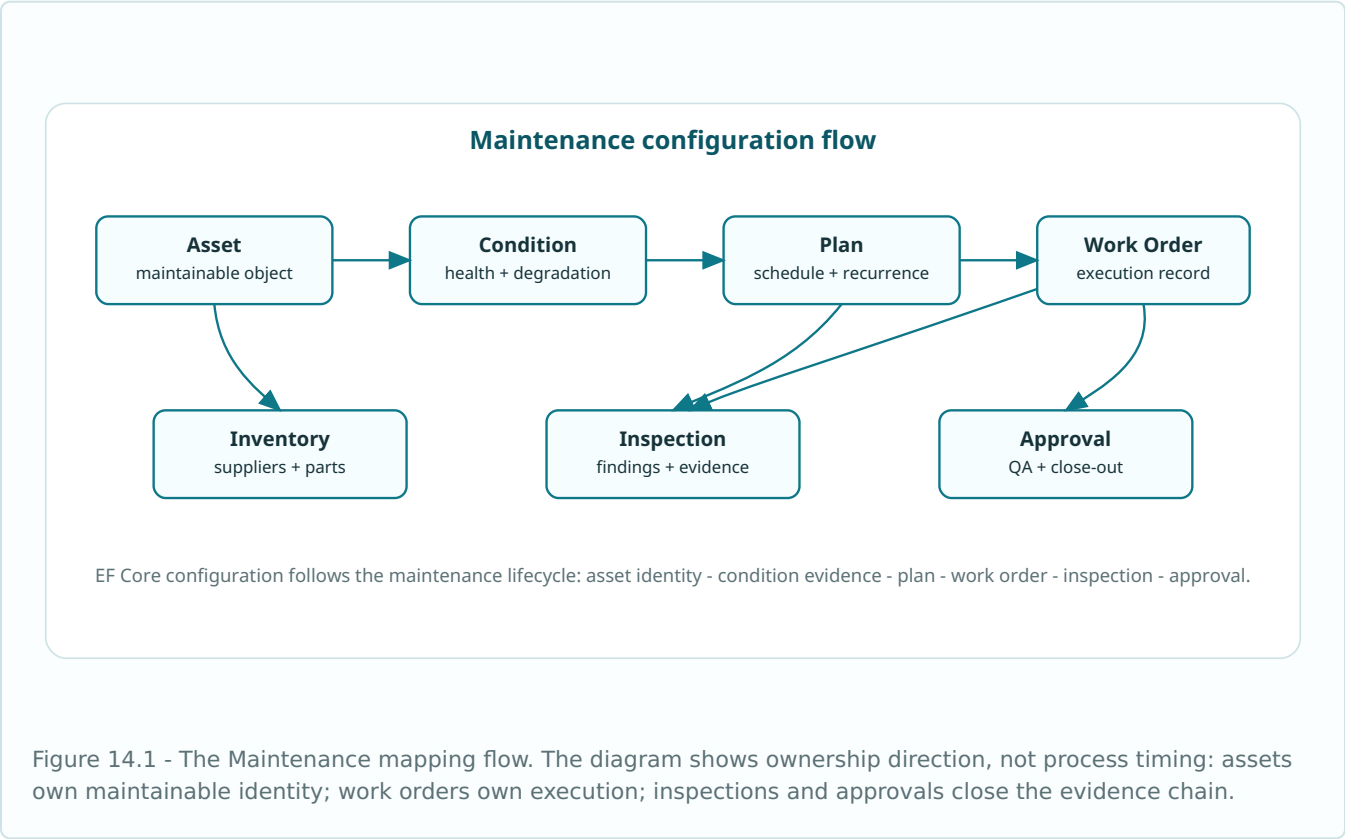
GROUP	SQL TABLE	CONFIGURATION FILE	PURPOSE
Lookup	Maintenance.SupplierStatus	SupplierStatusConfiguration.cs	Approved, conditional, suspended, retired or evaluation supplier state.
Lookup	Maintenance.DocumentType	DocumentTypeConfiguration.cs	Procedure, vendor manual, work package, photo, certificate, calibration sheet or close-out report.
Lookup	Maintenance.LabourRole	LabourRoleConfiguration.cs	Planner, technician, engineer, supervisor, radiation protection, QA reviewer or contractor role.
Asset	Maintenance.Asset	AssetConfiguration.cs	Maintainable representation of a physical component or equipment item.
Asset	Maintenance.AssetComponent	AssetComponentConfiguration.cs	Subcomponent tree below a maintainable asset: seal, bearing, actuator, motor winding, module.
Asset	Maintenance.AssetStatusHistory	AssetStatusHistoryConfiguration.cs	Append-only state changes for a maintainable asset.
Condition	Maintenance.AssetCondition	AssetConditionConfiguration.cs	Point-in-time condition assessment, health score and useful-life estimate.
Condition	Maintenance.AssetConditionMeasurement	AssetConditionMeasurementConfiguration.cs	Measurement evidence used by a condition assessment.
Condition	Maintenance.DegradationRecord	DegradationRecordConfiguration.cs	Active or historical degradation case for an asset.
Condition	Maintenance.DegradationTrendPoint	DegradationTrendPointConfiguration.cs	Trend points for degradation rate, thickness loss, vibration growth or other condition metric.
Inventory	Maintenance.Supplier	SupplierConfiguration.cs	External supplier or vendor used for spares, services and documents.
Inventory	Maintenance.SparePart	SparePartConfiguration.cs	Spare part catalogue: part number, unit of issue, manufacturer and description.
Inventory	Maintenance.SupplierPart	SupplierPartConfiguration.cs	Supplier-specific part number, lead time and commercial reference.
Inventory	Maintenance.SparePartStock	SparePartStockConfiguration.cs	On-hand, reserved and reorder inventory quantities.
Inventory	Maintenance.AssetSparePart	AssetSparePartConfiguration.cs	Mapping of which spare parts are suitable for which assets.
Plan	Maintenance.MaintenancePlan	MaintenancePlanConfiguration.cs	Preventive, predictive or condition-based plan for an asset or unit.
Plan	Maintenance.MaintenancePlanTask	MaintenancePlanTaskConfiguration.cs	Reusable tasks that become work-order tasks when the plan is scheduled.
Plan	Maintenance.MaintenanceSchedule	MaintenanceScheduleConfiguration.cs	Rule and next-due information for a maintenance plan.
Plan	Maintenance.MaintenanceScheduleOccurrence	MaintenanceScheduleOccurrenceConfiguration.cs	Planned occurrence generated from a schedule before a work order exists.

GROUP	SQL TABLE	CONFIGURATION FILE	PURPOSE
Plan	Maintenance.MaintenanceWindow	MaintenanceWindowConfiguration.cs	Allowed or planned time window for maintenance work.
Work	Maintenance.WorkOrder	WorkOrderConfiguration.cs	Executable work package created from a plan, event, incident action or manual request.
Work	Maintenance.WorkOrderStatusHistory	WorkOrderStatusHistoryConfiguration.cs	Append-only lifecycle history for a work order.
Work	Maintenance.WorkOrderTask	WorkOrderTaskConfiguration.cs	Task list inside a work order.
Work	Maintenance.WorkOrderLabour	WorkOrderLabourConfiguration.cs	Estimated or actual labour assignment and hours.
Work	Maintenance.WorkOrderMaterial	WorkOrderMaterialConfiguration.cs	Planned, reserved, issued or consumed material for a work order.
Work	Maintenance.WorkOrderEventLink	WorkOrderEventLinkConfiguration.cs	Why a work order was opened, connected to operational events and incident actions.
Inspection	Maintenance.Inspection	InspectionConfiguration.cs	An inspection performed or planned for an asset.
Inspection	Maintenance.InspectionFinding	InspectionFindingConfiguration.cs	Finding, nonconformance or observation produced by an inspection.
Evidence	Maintenance.MaintenanceDocument	MaintenanceDocumentConfiguration.cs	Metadata for procedures, manuals, photos, certificates and work-package files.
Evidence	Maintenance.MaintenanceApproval	MaintenanceApprovalConfiguration.cs	Approval record for work orders, plans, documents or inspection close-out.

14.3 Group map

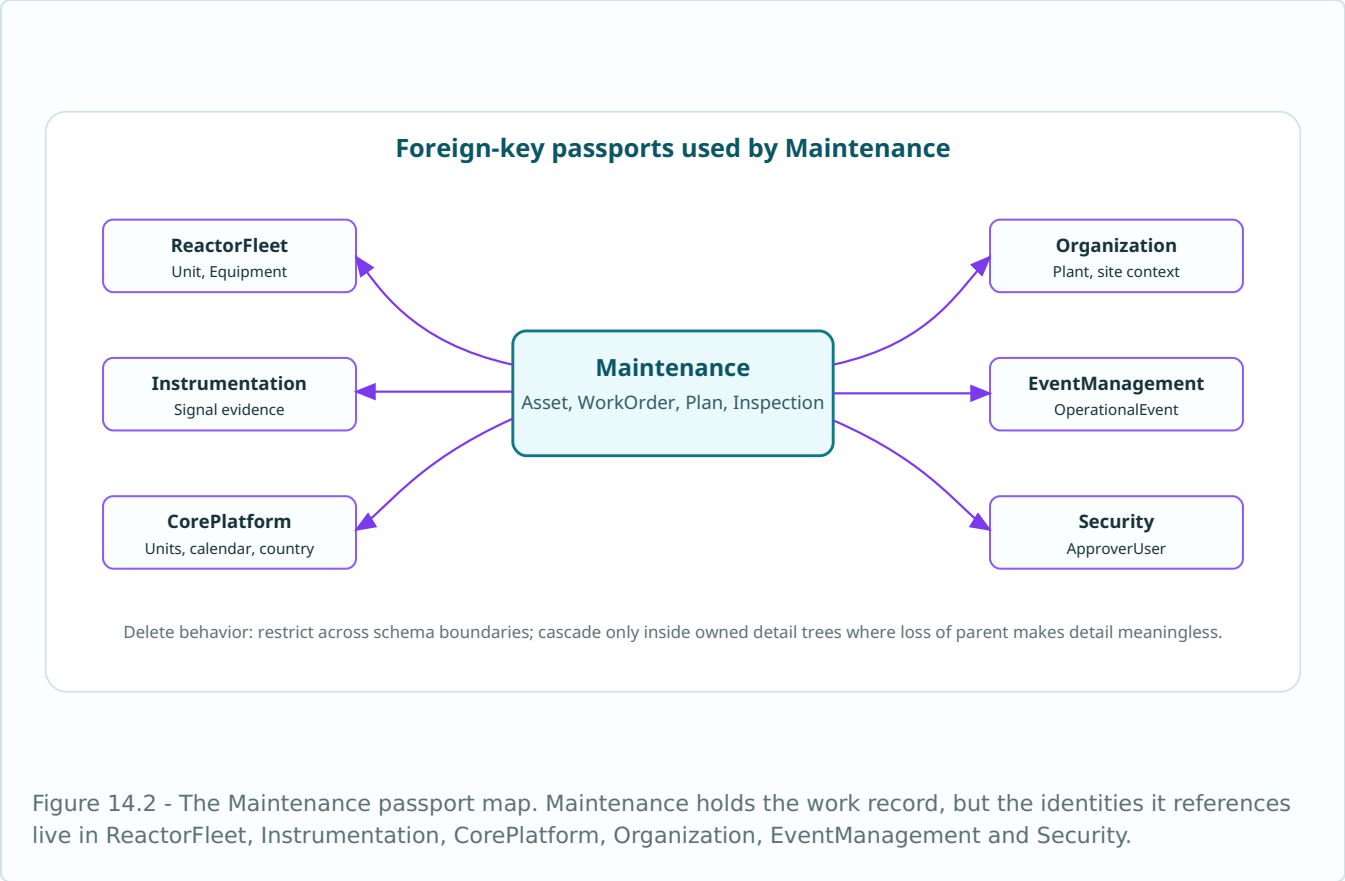
Lookup 18 configurations AssetCategory, AssetStatus, AssetCriticality, ConditionGrade, DegradationMechanism ...	Asset 3 configurations Asset, AssetComponent, AssetStatusHistory	Condition 4 configurations AssetCondition, AssetConditionMeasurement, DegradationRecord, DegradationTrendPoint
Inventory 5 configurations Supplier, SparePart, SupplierPart, SparePartStock, AssetSparePart	Plan 5 configurations MaintenancePlan, MaintenancePlanTask, MaintenanceSchedule, MaintenanceScheduleOccurrence, MaintenanceWindow	Work 6 configurations WorkOrder, WorkOrderStatusHistory, WorkOrderTask, WorkOrderLabour, WorkOrderMaterial ...
Inspection 2 configurations Inspection, InspectionFinding	Evidence 2 configurations MaintenanceDocument, MaintenanceApproval	

The group map is not only documentation. It is also the recommended folder structure. Each group becomes a subfolder under `Configurations/Maintenance` when the schema grows large enough to need that separation.



14.4 Passport map

Maintenance uses more cross-schema passports than most sectors because work is attached to the physical plant, evidence, calendars, users and operational events. The EF Core mapping must make those crossings explicit and should normally use `DeleteBehavior.Restrict` across schema boundaries.



14.5 Folder structure

```
Nexus.Infrastructure/
  Persistence/
    Configurations/
      Maintenance/
        Lookups/
          AssetCategoryConfiguration.cs
          AssetStatusConfiguration.cs
          WorkOrderStatusConfiguration.cs
          ...
        Assets/
          AssetConfiguration.cs
          AssetComponentConfiguration.cs
          AssetStatusHistoryConfiguration.cs
        Condition/
          AssetConditionConfiguration.cs
          AssetConditionMeasurementConfiguration.cs
          DegradationRecordConfiguration.cs
          DegradationTrendPointConfiguration.cs
        Inventory/
          SupplierConfiguration.cs
          SparePartConfiguration.cs
          SupplierPartConfiguration.cs
          SparePartStockConfiguration.cs
          AssetSparePartConfiguration.cs
        Plans/
          MaintenancePlanConfiguration.cs
          MaintenanceScheduleConfiguration.cs
          MaintenanceWindowConfiguration.cs
        Work/
          WorkOrderConfiguration.cs
          WorkOrderTaskConfiguration.cs
          WorkOrderMaterialConfiguration.cs
          WorkOrderEventLinkConfiguration.cs
        Inspection/
          InspectionConfiguration.cs
          InspectionFindingConfiguration.cs
        Evidence/
          MaintenanceDocumentConfiguration.cs
          MaintenanceApprovalConfiguration.cs
```

14.6 Shared Maintenance conventions

The Maintenance schema has many typed lookup tables. A shared base configuration keeps them uniform without hiding each table's explicit key and table name.

```
namespace Nexus.Infrastructure.Persistence.Configurations;

internal static partial class NexusSql
{
    public const string Maintenance = "Maintenance";
    public const string CorePlatform = "CorePlatform";
    public const string ReactorFleet = "ReactorFleet";
    public const string Organization = "Organization";
    public const string Instrumentation = "Instrumentation";
    public const string EventManagement = "EventManagement";
    public const string Security = "Security";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class MaintenanceLookupConfiguration<TEntity>
    : IEntityConfiguration<TEntity>
    where TEntity : MaintenanceLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(150)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
```

```

        .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

internal static class MaintenanceMappingExtensions
{
    public static PropertyBuilder<string> HasMaintenanceCode(
        this PropertyBuilder<string> property, int length = 80) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);

    public static PropertyBuilder<decimal?> HasMetric(
        this PropertyBuilder<decimal?> property) => property
        .HasColumnType("decimal(18,6)");
}

```

14.7 Lookup configuration examples

The examples below show the repeated lookup shape. In the full source tree each lookup has its own small file, but the pattern stays identical: table name, primary key name, shared base, unique code index.

```
public sealed class AssetCategoryConfiguration
    : MaintenanceLookupConfiguration<AssetCategory>
{
    public override void Configure(EntityTypeBuilder<AssetCategory> b)
    {
        b.ToTable("AssetCategory", NexusSql.Maintenance);
        b.HasKey(e => e.AssetId)
            .HasName("PK_Maintenance_AssetId");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Maintenance_AssetId_Code");
    }
}

public sealed class WorkOrderStatusConfiguration
    : MaintenanceLookupConfiguration<WorkOrderStatus>
{
    public override void Configure(EntityTypeBuilder<WorkOrderStatus> b)
    {
        b.ToTable("WorkOrderStatus", NexusSql.Maintenance);
        b.HasKey(e => e.WorkOrderId)
            .HasName("PK_Maintenance_WorkOrderId");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Maintenance_WorkOrderId_Code");
    }
}

public sealed class DegradationMechanismConfiguration
    : MaintenanceLookupConfiguration<DegradationMechanism>
{
    public override void Configure(EntityTypeBuilder<DegradationMechanism> b)
    {
        b.ToTable("DegradationMechanism", NexusSql.Maintenance);
        b.HasKey(e => e.DegradationMechanismId)
            .HasName("PK_Maintenance_DegradationMechanismId");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_Maintenance_DegradationMechanism_Code");
    }
}
```

14.8 Asset mapping

Asset is the root of the Maintenance sector. It connects the maintainable record to the physical unit and equipment passports from ReactorFleet and to the typed asset category lookup.

```
public sealed class AssetConfiguration : IEntityTypeConfiguration<Asset>
{
    public void Configure(EntityTypeBuilder<Asset> b)
    {
        b.ToTable("Asset", NexusSql.Maintenance);

        b.HasKey(e => e.AssetId)
            .HasName("PK_Maintenance_Asset");

        b.Property(e => e.AssetId).HasMaintenanceCode(80);
        b.Property(e => e.Name).HasMaxLength(180).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1000);
        b.Property(e => e.SerialNumber).HasMaxLength(100);
        b.Property(e => e.Manufacturer).HasMaxLength(150);
        b.Property(e => e.ModelNumber).HasMaxLength(120);
        b.Property(e => e.InstalledAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.AssetId).IsUnique()
            .HasDatabaseName("UQ_Maintenance_Asset_AssetId");
        b.HasIndex(e => new { e.UnitId, e.AssetCategoryId })
            .HasDatabaseName("IX_Maintenance_Asset_Unit_Category");
        b.HasIndex(e => e.EquipmentId)
            .HasDatabaseName("IX_Maintenance_Asset_EquipmentId");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Maintenance_Asset_ReactorFleet_Unit");

        b.HasOne(e => e.Equipment)
            .WithMany()
            .HasForeignKey(e => e.EquipmentId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Maintenance_Asset_ReactorFleet_Equipment");

        b.HasOne(e => e.AssetCategory)
            .WithMany()
            .HasForeignKey(e => e.AssetCategoryId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Maintenance_Asset_AssetCategory");
    }
}
```

14.9 Condition and degradation mapping

Condition records are time-oriented facts. They should be indexed by asset and assessment time. Degradation records add the engineering mechanism and trend points that make long-term ageing queryable.

```
public sealed class AssetConditionConfiguration
    : IEntityTypeConfiguration<AssetCondition>
{
    public void Configure(EntityTypeBuilder<AssetCondition> b)
    {
        b.ToTable("AssetCondition", NexusSql.Maintenance);

        b.HasKey(e => e.AssetId)
            .HasName("PK_Maintenance_AssetCondition");

        b.Property(e => e.AssessedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.HealthScore).HasColumnType("decimal(5,2)");
        b.Property(e => e.RemainingUsefulLifeDays).HasColumnType("decimal(12,2)");
        b.Property(e => e.AssessmentNote).HasMaxLength(1200);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.AssetId, e.AssessedAtUtc })
            .HasDatabaseName("IX_Maintenance_AssetCondition_Asset_AssessedAt");
        b.HasIndex(e => new { e.ConditionGradeId, e.AssessedAtUtc })
            .HasDatabaseName("IX_Maintenance_AssetCondition_Grade_AssessedAt");

        b.HasOne(e => e.Asset)
            .WithMany(e => e.Conditions)
            .HasForeignKey(e => e.AssetId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_Maintenance_AssetCondition_Asset");

        b.HasOne(e => e.ConditionGrade)
            .WithMany()
            .HasForeignKey(e => e.ConditionGradeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Maintenance_AssetCondition_ConditionGrade");
    }
}

public sealed class DegradationRecordConfiguration
    : IEntityTypeConfiguration<DegradationRecord>
{
    public void Configure(EntityTypeBuilder<DegradationRecord> b)
    {
        b.ToTable("DegradationRecord", NexusSql.Maintenance);
        b.HasKey(e => e.DegradationRecordId)
            .HasName("PK_Maintenance_DegradationRecord");

        b.Property(e => e.RecordCode).HasMaintenanceCode(80);
        b.Property(e => e.DetectedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ClosedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CurrentSeverityScore).HasColumnType("decimal(6,3)");
        b.Property(e => e.Description).HasMaxLength(1600);

        b.HasIndex(e => e.RecordCode).IsUnique()
            .HasDatabaseName("UQ_Maintenance_DegradationRecord_RecordCode");
        b.HasIndex(e => new { e.AssetId, e.DegradationMechanismId, e.DetectedAtUtc })
            .HasDatabaseName("IX_Maintenance_DegradationRecord_Asset_Mechanism_DetectedAt");
    }
}
```

14.10 Inventory mapping

Inventory mapping keeps the supplier catalogue, spare-part catalogue, supplier-specific part numbers and plant stock separate. The unique index on `SparePartStock` prevents two stock rows for the same part at the same plant.

```
public sealed class SupplierConfiguration : IEntityTypeConfiguration<Supplier>
{
    public void Configure(EntityTypeBuilder<Supplier> b)
    {
        b.ToTable("Supplier", NexusSql.Maintenance);
        b.HasKey(e => e.SupplierId)
            .HasName("PK_Maintenance_Supplier");

        b.Property(e => e.SupplierCode).HasMaintenanceCode(60);
        b.Property(e => e.Name).HasMaxLength(200).IsRequired();
        b.Property(e => e.ContactEmail).HasMaxLength(254);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.SupplierCode).IsUnique()
            .HasDatabaseName("UQ_Maintenance_Supplier_SupplierCode");
        b.HasIndex(e => new { e.CountryId, e.SupplierStatusId })
            .HasDatabaseName("IX_Maintenance_Supplier_Country_Status");
    }
}

public sealed class SparePartConfiguration : IEntityTypeConfiguration<SparePart>
{
    public void Configure(EntityTypeBuilder<SparePart> b)
    {
        b.ToTable("SparePart", NexusSql.Maintenance);
        b.HasKey(e => e.SparePartId)
            .HasName("PK_Maintenance_SparePart");

        b.Property(e => e.PartNumber).HasMaintenanceCode(120);
        b.Property(e => e.Name).HasMaxLength(200).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1000);
        b.Property(e => e.StandardUnitQuantity).HasColumnType("decimal(18,6)");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.PartNumber).IsUnique()
            .HasDatabaseName("UQ_Maintenance_SparePart_PartNumber");
        b.HasIndex(e => e.EngineeringUnitId)
            .HasDatabaseName("IX_Maintenance_SparePart_EngineeringUnitId");
    }
}

public sealed class SparePartStockConfiguration
    : IEntityTypeConfiguration<SparePartStock>
{
    public void Configure(EntityTypeBuilder<SparePartStock> b)
    {
        b.ToTable("SparePartStock", NexusSql.Maintenance);
        b.HasKey(e => e.SparePartStockId)
            .HasName("PK_Maintenance_SparePartStock");

        b.Property(e => e.OnHandQuantity).HasColumnType("decimal(18,6)");
        b.Property(e => e.ReservedQuantity).HasColumnType("decimal(18,6)");
        b.Property(e => e.ReorderPoint).HasColumnType("decimal(18,6)");

        b.HasIndex(e => new { e.SparePartId, e.PlantId }).IsUnique()
            .HasDatabaseName("UQ_Maintenance_SparePartStock_Part_Plant");
    }
}
```


14.11 Plan and schedule mapping

Plans describe reusable maintenance intent. Schedules decide when that intent becomes due. Work orders are execution records and should not be overloaded with recurrence rules.

```
public sealed class MaintenancePlanConfiguration
    : IEntityTypeConfiguration<MaintenancePlan>
{
    public void Configure(EntityTypeBuilder<MaintenancePlan> b)
    {
        b.ToTable("MaintenancePlan", NexusSql.Maintenance);
        b.HasKey(e => e.MaintenancePlanId)
            .HasName("PK_Maintenance_MaintenancePlan");

        b.Property(e => e.PlanCode).HasMaintenanceCode(80);
        b.Property(e => e.Name).HasMaxLength(200).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1500);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.PlanCode).IsUnique()
            .HasDatabaseName("UQ_Maintenance_MaintenancePlan_PlanCode");
        b.HasIndex(e => new { e.AssetId, e.MaintenancePlanTypeId, e.IsActive })
            .HasDatabaseName("IX_Maintenance_MaintenancePlan_Asset_Type_Active");
    }
}

public sealed class MaintenanceScheduleConfiguration
    : IEntityTypeConfiguration<MaintenanceSchedule>
{
    public void Configure(EntityTypeBuilder<MaintenanceSchedule> b)
    {
        b.ToTable("MaintenanceSchedule", NexusSql.Maintenance);
        b.HasKey(e => e.MaintenanceScheduleId)
            .HasName("PK_Maintenance_MaintenanceSchedule");

        b.Property(e => e.ScheduleCode).HasMaintenanceCode(80);
        b.Property(e => e.IntervalValue).HasColumnType("decimal(18,6)");
        b.Property(e => e.NextDueAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.IsEnabled).HasDefaultValue(true);

        b.HasIndex(e => e.ScheduleCode).IsUnique()
            .HasDatabaseName("UQ_Maintenance_MaintenanceSchedule_ScheduleCode");
        b.HasIndex(e => new { e.MaintenancePlanId, e.NextDueAtUtc })
            .HasDatabaseName("IX_Maintenance_MaintenanceSchedule_Plan_NextDueAt");
    }
}
```

14.12 Work order mapping

The work order is the operational centre of the schema. It owns tasks, labour lines, material lines, status history, event links and close-out evidence. Its indexes are designed around planner queries: by unit, status, requested time and asset.

```
public sealed class WorkOrderConfiguration : IEntityTypeConfiguration<WorkOrder>
{
    public void Configure(EntityTypeBuilder<WorkOrder> b)
    {
        b.ToTable("WorkOrder", NexusSql.Maintenance);

        b.HasKey(e => e.WorkOrderId)
            .HasName("PK_Maintenance_WorkOrder");

        b.Property(e => e.WorkOrderNumber).HasMaintenanceCode(80);
        b.Property(e => e.Title).HasMaxLength(250).IsRequired();
        b.Property(e => e.Description).HasMaxLength(2000);
        b.Property(e => e.RequestedAtUtc).HasUtcDefault();
        b.Property(e => e.PlannedStartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.PlannedFinishUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ActualStartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ActualFinishUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.WorkOrderNumber).IsUnique()
            .HasDatabaseName("UQ_Maintenance_WorkOrder_WorkOrderNumber");
        b.HasIndex(e => new { e.UnitId, e.WorkOrderStatusId, e.RequestedAtUtc })
            .HasDatabaseName("IX_Maintenance_WorkOrder_Unit_Status_RequestedAt");
        b.HasIndex(e => new { e.AssetId, e.WorkOrderStatusId })
            .HasDatabaseName("IX_Maintenance_WorkOrder_Asset_Status");

        b.HasOne(e => e.Asset)
            .WithMany(e => e.WorkOrders)
            .HasForeignKey(e => e.AssetId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Maintenance_WorkOrder_Asset");

        b.HasOne(e => e.Status)
            .WithMany()
            .HasForeignKey(e => e.WorkOrderStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Maintenance_WorkOrder_WorkOrderStatus");
    }
}

public sealed class WorkOrderEventLinkConfiguration
    : IEntityTypeConfiguration<WorkOrderEventLink>
{
    public void Configure(EntityTypeBuilder<WorkOrderEventLink> b)
    {
        b.ToTable("WorkOrderEventLink", NexusSql.Maintenance);
        b.HasKey(e => e.WorkOrderEventLinkId)
            .HasName("PK_Maintenance_WorkOrderEventLink");

        b.Property(e => e.LinkReason).HasMaxLength(500);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasIndex(e => new { e.WorkOrderId, e.OperationalEventId }).IsUnique()
            .HasDatabaseName("UQ_Maintenance_WorkOrderEventLink_WorkOrder_Event");
    }
}
```

14.13 Foreign-key map

The table below is a compact index of the main mapped passports and internal references. It is intentionally repetitive: repetition is the price of making the model reviewable.

SOURCE TABLE	FK PROPERTY	TARGET	KIND
Maintenance.Asset	UnitId	ReactorFleet.Unit	passport
Maintenance.Asset	EquipmentId	ReactorFleet.Equipment	passport
Maintenance.Asset	AssetCategoryId	Maintenance.AssetCategory	internal
Maintenance.AssetComponent	AssetId	Maintenance / cross-schema target	internal
Maintenance.AssetComponent	ParentAssetComponentId	Maintenance.AssetComponent	internal
Maintenance.AssetStatusHistory	AssetId	Maintenance / cross-schema target	internal
Maintenance.AssetStatusHistory	AssetStatusId	Maintenance.AssetStatus	internal
Maintenance.AssetCondition	AssetId	Maintenance / cross-schema target	internal
Maintenance.AssetCondition	ConditionGradeId	Maintenance.ConditionGrade	internal
Maintenance.AssetConditionMeasurement	AssetConditionId	Maintenance / cross-schema target	internal
Maintenance.AssetConditionMeasurement	SignalId	Instrumentation.Signal	passport
Maintenance.DegradationRecord	AssetId	Maintenance / cross-schema target	internal
Maintenance.DegradationRecord	DegradationMechanismId	Maintenance.DegradationMechanism	internal
Maintenance.DegradationTrendPoint	DegradationRecordId	Maintenance / cross-schema target	internal
Maintenance.DegradationTrendPoint	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
Maintenance.Supplier	CountryId	CorePlatform.Country	passport
Maintenance.Supplier	SupplierStatusId	Maintenance.SupplierStatus	internal
Maintenance.SparePart	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
Maintenance.SupplierPart	SupplierId	Maintenance.Supplier	internal
Maintenance.SupplierPart	SparePartId	Maintenance.SparePart	internal
Maintenance.SparePartStock	SparePartId	Maintenance.SparePart	internal
Maintenance.SparePartStock	PlantId	Organization.Plant	passport
Maintenance.AssetSparePart	AssetId	Maintenance / cross-schema target	internal
Maintenance.AssetSparePart	SparePartId	Maintenance.SparePart	internal
Maintenance.MaintenancePlan	AssetId	Maintenance / cross-schema target	internal

SOURCE TABLE	FK PROPERTY	TARGET	KIND
Maintenance.MaintenancePlan	MaintenancePlanTypeId	Maintenance.MaintenancePlanType	internal
Maintenance.MaintenancePlanTask	MaintenancePlanId	Maintenance.MaintenancePlan	internal
Maintenance.MaintenanceSchedule	MaintenancePlanId	Maintenance.MaintenancePlan	internal
Maintenance.MaintenanceSchedule	ScheduleBasisId	Maintenance.ScheduleBasis	internal
Maintenance.MaintenanceScheduleOccurrence	MaintenanceScheduleId	Maintenance.MaintenanceSchedule	internal
Maintenance.MaintenanceWindow	UnitId	ReactorFleet.Unit	passport
Maintenance.MaintenanceWindow	CalendarId	CorePlatform.Calendar	passport
Maintenance.WorkOrder	UnitId	ReactorFleet.Unit	passport
Maintenance.WorkOrder	AssetId	Maintenance / cross-schema target	internal
Maintenance.WorkOrder	WorkOrderStatusId	Maintenance.WorkOrderStatus	internal
Maintenance.WorkOrderStatusHistory	WorkOrderId	Maintenance.WorkOrder	internal
Maintenance.WorkOrderStatusHistory	WorkOrderStatusId	Maintenance.WorkOrderStatus	internal
Maintenance.WorkOrderTask	WorkOrderId	Maintenance.WorkOrder	internal
Maintenance.WorkOrderTask	TaskStatusId	Maintenance.TaskStatus	internal
Maintenance.WorkOrderLabour	WorkOrderId	Maintenance.WorkOrder	internal
Maintenance.WorkOrderLabour	LabourRoleId	Maintenance.LabourRole	internal
Maintenance.WorkOrderMaterial	WorkOrderId	Maintenance.WorkOrder	internal
Maintenance.WorkOrderMaterial	SparePartId	Maintenance.SparePart	internal
Maintenance.WorkOrderEventLink	WorkOrderId	Maintenance.WorkOrder	internal
Maintenance.WorkOrderEventLink	OperationalEventId	EventManagement.OperationalEvent	passport
Maintenance.Inspection	AssetId	Maintenance / cross-schema target	internal
Maintenance.Inspection	InspectionTypeId	Maintenance.InspectionType	internal
Maintenance.Inspection	WorkOrderId	Maintenance.WorkOrder	internal
Maintenance.InspectionFinding	InspectionId	Maintenance.Inspection	internal
Maintenance.InspectionFinding	FindingSeverityId	Maintenance.FindingSeverity	internal
Maintenance.MaintenanceDocument	AssetId	Maintenance / cross-schema target	internal
Maintenance.MaintenanceDocument	WorkOrderId	Maintenance.WorkOrder	internal
Maintenance.MaintenanceDocument	DocumentTypeId	Maintenance.DocumentType	internal
Maintenance.MaintenanceApproval	ApprovalStatusId	Maintenance.ApprovalStatus	internal
Maintenance.MaintenanceApproval	ApproverUserId	Security.ApplicationUser	passport

14.14 Migration and verification notes

Migration order

Lookups first, then Asset, then condition/inventory/plan roots, then work orders, then inspection/evidence tables.

Delete behavior

Use restrict for passports to other schemas.
Use cascade only for owned children such as work-order task lines when parent removal is valid in test data.

Concurrency

Mutable maintenance records should carry **RowVersion**. Append-only history rows do not need the same update surface.

Precision

Quantities, health scores, degradation rates and stock amounts should be decimal, not floating point.

```
// One line in NexusContext keeps the atlas discoverable.
protected override void OnModelCreating(ModelBuilder m)
{
    base.OnModelCreating(m);
    m.ApplyConfigurationsFromAssembly(typeof(NexusContext).Assembly);
}

// Fast smoke check during development.
var configuredTables = context.Model.GetEntityTypes()
    .Where(e => e.GetSchema() == "Maintenance")
    .Select(e => e.GetTableName())
    .OrderBy(x => x)
    .ToList();

// A representative query the mappings must produce cleanly.
var openWork = await context.WorkOrders
    .Where(w => w.Unit.Code == "NX1-U1" && w.Status.Code != "CLOSED")
    .OrderByDescending(w => w.RequestedAtUtc)
    .Select(w => new
    {
        w.WorkOrderNumber,
        Asset = w.Asset.AssetName,
        Status = w.Status.Code,
        MaterialLines = w.Materials.Count,
        OpenEvents = w.EventLinks.Count
    })
    .ToListAsync();
```

14.15 Configuration index

The following index is the programmer's checklist for this schema chapter. If every file exists and `ApplyConfigurationsFromAssembly` is active, the context can discover the whole sector without a giant `OnModelCreating`.

AssetCategoryConfiguration.cs Lookup	DegradationRecordConfiguration.cs Condition
AssetStatusConfiguration.cs Lookup	DegradationTrendPointConfiguration.cs Condition
AssetCriticalityConfiguration.cs Lookup	SupplierConfiguration.cs Inventory
ConditionGradeConfiguration.cs Lookup	SparePartConfiguration.cs Inventory
DegradationMechanismConfiguration.cs Lookup	SupplierPartConfiguration.cs Inventory
InspectionTypeConfiguration.cs Lookup	SparePartStockConfiguration.cs Inventory
FindingSeverityConfiguration.cs Lookup	AssetSparePartConfiguration.cs Inventory
WorkOrderTypeConfiguration.cs Lookup	MaintenancePlanConfiguration.cs Plan
WorkOrderStatusConfiguration.cs Lookup	MaintenancePlanTaskConfiguration.cs Plan
WorkPriorityConfiguration.cs Lookup	MaintenanceScheduleConfiguration.cs Plan
TaskStatusConfiguration.cs Lookup	MaintenanceScheduleOccurrenceConfiguration.cs Plan
MaintenancePlanTypeConfiguration.cs Lookup	MaintenanceWindowConfiguration.cs Plan
ScheduleBasisConfiguration.cs Lookup	WorkOrderConfiguration.cs Work
MaterialUsageTypeConfiguration.cs Lookup	WorkOrderStatusHistoryConfiguration.cs Work
ApprovalStatusConfiguration.cs Lookup	WorkOrderTaskConfiguration.cs Work
SupplierStatusConfiguration.cs Lookup	WorkOrderLabourConfiguration.cs Work
DocumentTypeConfiguration.cs Lookup	WorkOrderMaterialConfiguration.cs Work
LabourRoleConfiguration.cs Lookup	WorkOrderEventLinkConfiguration.cs Work
AssetConfiguration.cs Asset	InspectionConfiguration.cs Inspection
AssetComponentConfiguration.cs Asset	InspectionFindingConfiguration.cs Inspection
AssetStatusHistoryConfiguration.cs Asset	MaintenanceDocumentConfiguration.cs Evidence
AssetConditionConfiguration.cs Condition	MaintenanceApprovalConfiguration.cs Evidence
AssetConditionMeasurementConfiguration.cs Condition	

Epilogue for this schema step

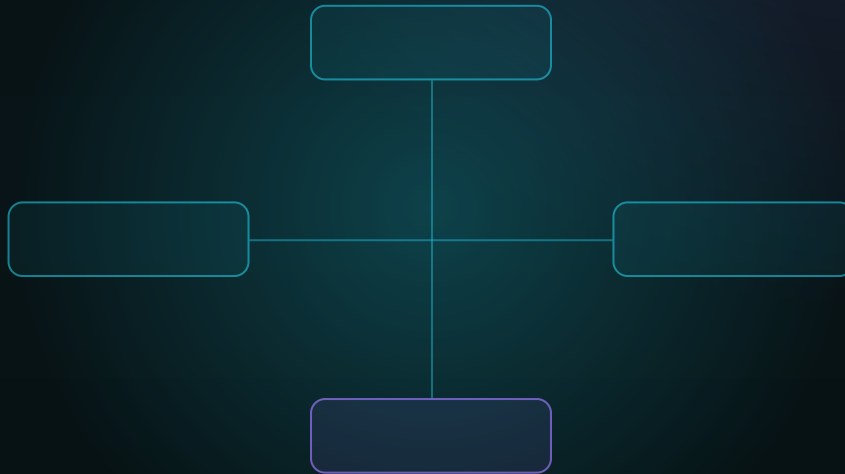
Maintenance is where the digital twin stops being only a mirror and becomes a memory of work. The EF Core configuration files keep that memory disciplined: every asset, condition assessment, spare part, plan, work order, inspection and approval is mapped with stable names, stable keys and reviewable relationships.

Honest boundary. This chapter is an enterprise-style mapping atlas for the NEXUS-1 demonstrator. It is not a certified maintenance-management system and not a substitute for plant maintenance procedures, quality assurance rules or regulated asset-management software.

FROM ENTITY TO CONTEXT

Chapter 15 - RootCause Configurations

The EF Core mapping layer for deterministic, grounded and auditable root-cause analysis.



SCHEMA CONFIGURATION COMPANION

Causal graph - analysis context - witnesses - conclusion

RootCause maps the graph, the node test, the grounding triangle and the evidence chain into disciplined EF Core configuration files.

Chapter 15 - RootCause Configurations

A professional EF Core configuration atlas for the NEXUS-1 RootCause schema. This chapter maps graph structure, grounding context, candidates, hypotheses, evidence witnesses, model outputs and audit trail with named keys, named indexes, explicit delete behavior and stable cross-schema passports.

The root-cause engine decides from structure and evidence. The mapping layer makes that structure queryable, versioned and auditable.

Standing rule. The RootCause schema must record both what the engine selected and what it rejected. Configuration classes therefore protect append-only evidence, explicit rejection records and deterministic naming. The language model, if used downstream, never defines the cause; it only explains a grounded result.

15.1 Purpose of This Schema Chapter

The RootCause schema is the EF Core mapping layer for a causal reasoning system. It is not a single result table. It contains the versioned causal graph, the alarm flood under analysis, the sealed grounding context, the candidates tested by the node test, the hypotheses built from them, the telemetry/registry/document witnesses, and the final conclusion or abstention.

Graph as code

The graph tables are configured as versioned, named, immutable structures. Nodes and edges are not anonymous rows; they are addressable facts.

Evidence as rows

The configuration files separate candidates, hypotheses, witnesses and conclusions so a reviewer can replay the analysis after the fact.

Passports, not copies

Units, equipment, signals, alarms, events and users are referenced by foreign keys. RootCause does not duplicate the platform.

Configuration file catalogue

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	<code>RootCause.CausalNodeType</code>	<code>CausalNodeTypeConfiguration.cs</code>	Kinds of node in the causal graph: component, condition, signal, protection action, external stressor or artefact.
Lookup	<code>RootCause.CausalEdgeType</code>	<code>CausalEdgeTypeConfiguration.cs</code>	Directed causal relation: physical propagation, control influence, common cause, artefact, rejected candidate.
Lookup	<code>RootCause.EdgeProvenanceType</code>	<code>EdgeProvenanceTypeConfiguration.cs</code>	Why an edge exists: fault-tree backbone, topology, engineering review, learned weight, proposed, rejected.
Lookup	<code>RootCause.EdgeStatus</code>	<code>EdgeStatusConfiguration.cs</code>	Lifecycle state of a causal edge: draft, active, superseded, rejected, retired.
Lookup	<code>RootCause.AnalysisStatus</code>	<code>AnalysisStatusConfiguration.cs</code>	Lifecycle of a root-cause analysis: opened, context built, running, concluded, abstained, closed.
Lookup	<code>RootCause.CandidateRole</code>	<code>CandidateRoleConfiguration.cs</code>	Candidate classification: origin, proximate, parallel, contributing, artefact, ruled out.
Lookup	<code>RootCause.CandidateStatus</code>	<code>CandidateStatusConfiguration.cs</code>	Candidate state: proposed, tested, supported, weakened, rejected, selected.
Lookup	<code>RootCause.HypothesisStatus</code>	<code>HypothesisStatusConfiguration.cs</code>	Hypothesis lifecycle: generated, grounded, challenged, accepted, rejected, abstained.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	RootCause.EvidenceType	EvidenceTypeConfiguration.cs	Evidence kind: alarm, telemetry, registry, document, maintenance, event, model output, operator note.
Lookup	RootCause.WitnessType	WitnessTypeConfiguration.cs	Witness class for the grounding triangle: telemetry, registry, documentation, maintenance or event.
Lookup	RootCause.ConfidenceBand	ConfidenceBandConfiguration.cs	Human-readable confidence band: low, medium, high, grounded, insufficient evidence.
Lookup	RootCause.GroundingSourceType	GroundingSourceTypeConfiguration.cs	Source type included in the unified context: alarm flood, telemetry window, registry slice, corpus document.
Lookup	RootCause.ValidationResult	ValidationResultConfiguration.cs	Validation outcome: passed, failed, warning, not applicable, not enough evidence.
Lookup	RootCause.AbstentionReason	AbstentionReasonConfiguration.cs	Reason the engine refuses to assert a cause: missing graph, weak grounding, stale context, conflicting witnesses.
Graph	RootCause.CausalGraph	CausalGraphConfiguration.cs	Named causal graph for a unit, system or reusable graph family.
Graph	RootCause.CausalGraphVersion	CausalGraphVersionConfiguration.cs	Versioned immutable graph release used by analyses.
Graph	RootCause.CausalNode	CausalNodeConfiguration.cs	A component, condition, signal or artefact node in a versioned causal graph.
Graph	RootCause.CausalNodeEquipmentLink	CausalNodeEquipmentLinkConfiguration.cs	Binds a causal node to physical equipment in ReactorFleet.
Graph	RootCause.CausalNodeSignalLink	CausalNodeSignalLinkConfiguration.cs	Binds a causal node to one or more instrumentation signals.
Graph	RootCause.CausalEdge	CausalEdgeConfiguration.cs	Directed causal influence carrying type, provenance, status, weight and delay.
Graph	RootCause.CausalEdgeDelayWindow	CausalEdgeDelayWindowConfiguration.cs	Expected propagation window used by the node test and path fit.
Graph	RootCause.CausalEdgeEvidence	CausalEdgeEvidenceConfiguration.cs	Evidence or document reference that justifies a graph edge.
Graph	RootCause.CausalGraphChangeRequest	CausalGraphChangeRequestConfiguration.cs	Versioned workflow for proposing graph changes.
Graph	RootCause.CausalGraphChangeReview	CausalGraphChangeReviewConfiguration.cs	Engineering review and approval/rejection of a graph change request.
Analysis	RootCause.RootCauseAnalysis	RootCauseAnalysisConfiguration.cs	One analysis run against an alarm flood, event, incident or manual request.
Analysis	RootCause.AnalysisEventLink	AnalysisEventLinkConfiguration.cs	Connects an analysis to operational events and incidents.
Analysis	RootCause.AnalysisAlarm	AnalysisAlarmConfiguration.cs	Alarm events included in the flood being analysed.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Analysis	RootCause.GroundingContext	GroundingContextConfiguration.cs	Sealed unified context: registry slice, telemetry window, alarm flood and retrieved documents.
Analysis	RootCause.GroundingContextSignalWindow	GroundingContextSignalWindowConfiguration.cs	Telemetry window boundaries for each signal included in the context.
Analysis	RootCause.GroundingContextDocument	GroundingContextDocumentConfiguration.cs	Retrieved document or corpus fragment included in the context.
Analysis	RootCause.AnalysisCandidate	AnalysisCandidateConfiguration.cs	Node tested as a possible root, proximate cause, artefact or rejected candidate.
Analysis	RootCause.AnalysisHypothesis	AnalysisHypothesisConfiguration.cs	Structured hypothesis derived from a candidate after graph and grounding checks.
Analysis	RootCause.HypothesisEvidence	HypothesisEvidenceConfiguration.cs	Typed evidence item attached to a hypothesis.
Analysis	RootCause.RejectedCandidate	RejectedCandidateConfiguration.cs	Explicit rejection record so ruled-out candidates remain auditable.
Analysis	RootCause.CausalPath	CausalPathConfiguration.cs	Forward path explained by a candidate.
Analysis	RootCause.CausalPathStep	CausalPathStepConfiguration.cs	One edge traversal in an explained path, including delay-fit result.
Witness	RootCause.TelemetryWitness	TelemetryWitnessConfiguration.cs	Telemetry corroboration or contradiction for a hypothesis.
Witness	RootCause.RegistryWitness	RegistryWitnessConfiguration.cs	Registry and maintenance facts that support or weaken a hypothesis.
Witness	RootCause.DocumentWitness	DocumentWitnessConfiguration.cs	Document/corpus evidence retrieved for the explanation.
Output	RootCause.AnalysisConclusion	AnalysisConclusionConfiguration.cs	Final conclusion, selected hypothesis or explicit abstention.
Output	RootCause.AnalysisModelRun	AnalysisModelRunConfiguration.cs	Deterministic model or downstream LLM explanation run metadata.
Output	RootCause.AnalysisModelOutput	AnalysisModelOutputConfiguration.cs	Structured machine-readable output, prompt hash, response hash and citations.
Output	RootCause.AnalysisValidationCheck	AnalysisValidationCheckConfiguration.cs	H1-H10 style validation checks applied to grounding, schema and evidence.
Output	RootCause.AnalysisAuditTrail	AnalysisAuditTrailConfiguration.cs	Append-only lifecycle trace for analysis events.
Output	RootCause.AnalysisComment	AnalysisCommentConfiguration.cs	Human review comment attached to an analysis.
Output	RootCause.AnalysisTag	AnalysisTagConfiguration.cs	Reusable tag for grouping analyses: feedwater, EMI, training, regression, validation.
Output	RootCause.AnalysisTagAssignment	AnalysisTagAssignmentConfiguration.cs	Many-to-many assignment of tags to analyses.

15.2 Group Map and Folder Structure

The folder structure mirrors the schema groups: lookup vocabulary, graph, analysis, witnesses and output. The goal is that a developer can open the `RootCause` configuration folder and immediately see the same architecture as the SQL schema.

Lookup

14 configurations

CausalNodeType, CausalEdgeType, EdgeProvenanceType, EdgeStatus, AnalysisStatus ...

Graph

10 configurations

CausalGraph, CausalGraphVersion, CausalNode, CausalNodeEquipmentLink, CausalNodeSignalLink ...

Analysis

12 configurations

RootCauseAnalysis, AnalysisEventLink, AnalysisAlarm, GroundingContext, GroundingContextSignalWindow ...

Witness

3 configurations

TelemetryWitness, RegistryWitness, DocumentWitness

Output

8 configurations

AnalysisConclusion, AnalysisModelRun, AnalysisModelOutput, AnalysisValidationCheck, AnalysisAuditTrail ...

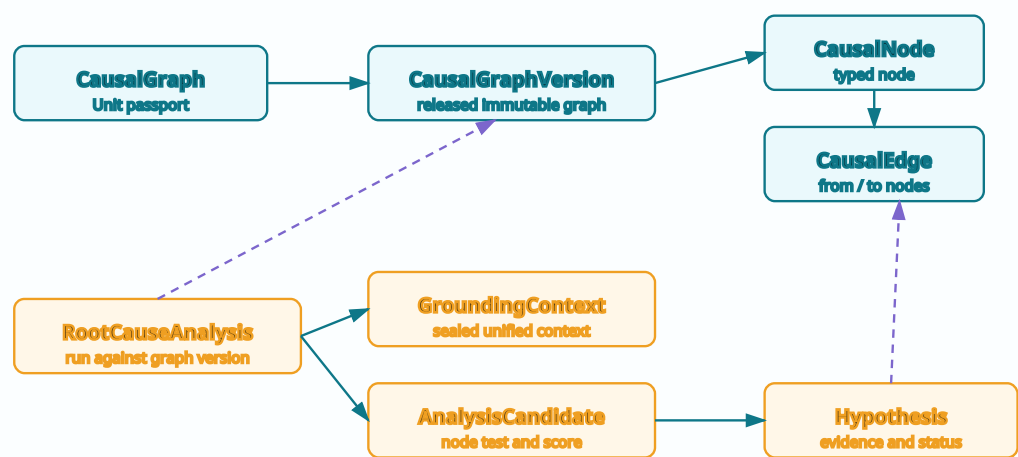
Recommended folder layout

```
src/Nexus.Infrastructure/Persistence/Configurations/
RootCause/
  Lookups/
    CausalNodeTypeConfiguration.cs
    CausalEdgeTypeConfiguration.cs
    EdgeProvenanceTypeConfiguration.cs
    AnalysisStatusConfiguration.cs
    CandidateRoleConfiguration.cs
    EvidenceTypeConfiguration.cs
    AbstentionReasonConfiguration.cs
  Graph/
    CausalGraphConfiguration.cs
    CausalGraphVersionConfiguration.cs
    CausalNodeConfiguration.cs
    CausalEdgeConfiguration.cs
    CausalEdgeDelayWindowConfiguration.cs
    CausalGraphChangeRequestConfiguration.cs
  Analysis/
    RootCauseAnalysisConfiguration.cs
    AnalysisAlarmConfiguration.cs
    GroundingContextConfiguration.cs
    AnalysisCandidateConfiguration.cs
    AnalysisHypothesisConfiguration.cs
    RejectedCandidateConfiguration.cs
    CausalPathConfiguration.cs
  Witnesses/
    TelemetryWitnessConfiguration.cs
    RegistryWitnessConfiguration.cs
    DocumentWitnessConfiguration.cs
  Output/
    AnalysisConclusionConfiguration.cs
    AnalysisModelRunConfiguration.cs
    AnalysisModelOutputConfiguration.cs
    AnalysisValidationCheckConfiguration.cs
    AnalysisAuditTrailConfiguration.cs
```

Why this matters. Root-cause code becomes unreadable when graph edges, analysis runs and evidence witnesses are mixed in one flat folder. The physical folder structure is a teaching device and a maintenance device: it forces the team to ask which part of the reasoning chain a configuration belongs to.

15.3 EF Core Dependency Diagram

EF Core dependency shape - RootCause sector

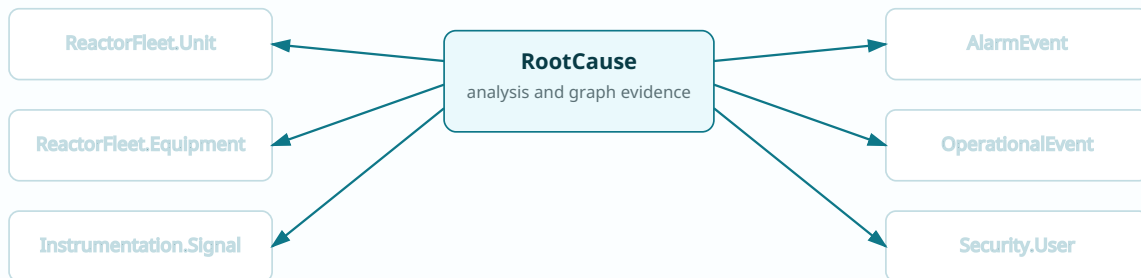


The configuration files keep the graph immutable once released, while analyses, contexts, candidates, witnesses and conclusions remain append-only evidence.

Figure 15.1 - RootCause EF Core dependency shape. The graph side is versioned and reusable; the analysis side is incident-specific and evidence bearing.

Cross-schema passport map

Cross-schema passports configured by RootCause



DeleteBehavior.Restrict at the boundary

RootCause records evidence about the outside world; it should not delete units, alarms, events, signals or users.

Internal graph and analysis details may cascade from the owning analysis; cross-schema passports stay restrict/no-action.

Figure 15.2 - RootCause passport map. External facts are referenced, not copied, and delete behavior is restrictive across schema boundaries.

15.4 Shared Lookup and Mapping Conventions

RootCause has a rich typed vocabulary: node type, edge type, provenance, status, candidate role, evidence type, witness type, confidence band, validation result and abstention reason. A shared base configuration prevents fifty tiny differences from becoming accidental behaviour.

```
namespace Nexus.Infrastructure.Persistence.Configurations.RootCause;

internal static partial class NexusSql
{
    public const string RootCause = "RootCause";
    public const string ReactorFleet = "ReactorFleet";
    public const string Instrumentation = "Instrumentation";
    public const string AlarmManagement = "AlarmManagement";
    public const string EventManagement = "EventManagement";
    public const string Security = "Security";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class RootCauseLookupConfiguration<TEntity>
    : IEntityTypeConfiguration<TEntity>
    where TEntity : RootCauseLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(150)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

internal static class RootCauseMappingExtensions
{
    public static PropertyBuilder<string> HasRootCauseCode(
        this PropertyBuilder<string> property, int length = 80) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);

    public static PropertyBuilder<decimal?> HasScore(
        this PropertyBuilder<decimal?> property) => property
        .HasColumnType("decimal(9,6)");
}
```

Lookup configuration examples

```
public sealed class CausalNodeTypeConfiguration
    : RootCauseLookupConfiguration<CausalNodeType>
{
    public override void Configure(EntityTypeBuilder<CausalNodeType> b)
```

```

    {
        b.ToTable("CausalNodeType", NexusSql.RootCause);
        b.HasKey(e => e.CausalNodeId)
            .HasName("PK_RootCause_CausalNodeType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_RootCause_CausalNodeType_Code");
    }
}

public sealed class EvidenceTypeConfiguration
    : RootCauseLookupConfiguration<EvidenceType>
{
    public override void Configure(EntityTypeBuilder<EvidenceType> b)
    {
        b.ToTable("EvidenceType", NexusSql.RootCause);
        b.HasKey(e => e.EvidenceTypeId)
            .HasName("PK_RootCause_EvidenceType");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_RootCause_EvidenceType_Code");
    }
}

public sealed class AbstentionReasonConfiguration
    : RootCauseLookupConfiguration<AbstentionReason>
{
    public override void Configure(EntityTypeBuilder<AbstentionReason> b)
    {
        b.ToTable("AbstentionReason", NexusSql.RootCause);
        b.HasKey(e => e.AbstentionReasonId)
            .HasName("PK_RootCause_AbstentionReason");
        base.Configure(b);
        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_RootCause_AbstentionReason_Code");
    }
}

```


15.5 Graph Configuration Files

The graph configuration files are the heart of the schema. They keep the graph versioned, make node identity unique inside a version, prevent duplicate edges, and name the self-referencing relationships so that migrations stay readable.

```
public sealed class CausalGraphConfiguration
    : IEntityTypeConfiguration<CausalGraph>
{
    public void Configure(EntityTypeBuilder<CausalGraph> b)
    {
        b.ToTable("CausalGraph", NexusSql.RootCause);
        b.HasKey(e => e.CausalGraphId)
            .HasName("PK_RootCause_CausalGraph");

        b.Property(e => e.Code).HasRootCauseCode(80);
        b.Property(e => e.Name).HasMaxLength(200).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1600);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique()
            .HasDatabaseName("UQ_RootCause_CausalGraph_Code");
        b.HasIndex(e => new { e.UnitId, e.IsActive })
            .HasDatabaseName("IX_RootCause_CausalGraph_Unit_Active");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RootCause_CausalGraph_ReactorFleet_Unit");
    }
}

public sealed class CausalGraphVersionConfiguration
    : IEntityTypeConfiguration<CausalGraphVersion>
{
    public void Configure(EntityTypeBuilder<CausalGraphVersion> b)
    {
        b.ToTable("CausalGraphVersion", NexusSql.RootCause);
        b.HasKey(e => e.CausalGraphVersionId)
            .HasName("PK_RootCause_CausalGraphVersion");

        b.Property(e => e.VersionNumber).HasMaxLength(30).IsRequired();
        b.Property(e => e.ReleasedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.IsReleased).HasDefaultValue(false);
        b.Property(e => e.Hash).HasMaxLength(128);

        b.HasIndex(e => new { e.CausalGraphId, e.VersionNumber }).IsUnique()
            .HasDatabaseName("UQ_RootCause_CausalGraphVersion_Graph_Version");

        b.HasOne(e => e.CausalGraph)
            .WithMany(e => e.Versions)
            .HasForeignKey(e => e.CausalGraphId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RootCause_CausalGraphVersion_CausalGraph");
    }
}
```

```
public sealed class CausalNodeConfiguration
    : IEntityTypeConfiguration<CausalNode>
{
    public void Configure(EntityTypeBuilder<CausalNode> b)
    {
        b.ToTable("CausalNode", NexusSql.RootCause);
        b.HasKey(e => e.CausalNodeId)
            .HasName("PK_RootCause_CausalNode");

        b.Property(e => e.NodeCode).HasRootCauseCode(100);
        b.Property(e => e.DisplayName).HasMaxLength(200).IsRequired();
        b.Property(e => e.Statement).HasMaxLength(1600);
        b.Property(e => e.DefaultWeight).HasScore();
        b.Property(e => e.IsArtefactCandidate).HasDefaultValue(false);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.CausalGraphVersionId, e.NodeCode }).IsUnique()
            .HasDatabaseName("UQ_RootCause_CausalNode_Version_NodeCode");
        b.HasIndex(e => new { e.CausalGraphVersionId, e.CausalNodeTypeId })
            .HasDatabaseName("IX_RootCause_CausalNode_Version_Type");

        b.HasOne(e => e.CausalGraphVersion)
            .WithMany(e => e.Nodes)
            .HasForeignKey(e => e.CausalGraphVersionId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RootCause_CausalNode_CausalGraphVersion");
    }
}
```

```

        b.HasOne(e => e.CausalNodeType)
            .WithMany()
            .HasForeignKey(e => e.CausalNodeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RootCause_CausalNode_CausalNodeType");
    }
}

public sealed class CausalEdgeConfiguration
    : IEntityTypeConfiguration<CausalEdge>
{
    public void Configure(EntityTypeBuilder<CausalEdge> b)
    {
        b.ToTable("CausalEdge", NexusSql.RootCause);
        b.HasKey(e => e.CausalEdgeId)
            .HasName("PK_RootCause_CausalEdge");

        b.Property(e => e.EdgeCode).HasRootCauseCode(120);
        b.Property(e => e.Weight).HasColumnType("decimal(9,6)");
        b.Property(e => e.MinDelaySeconds).HasColumnType("decimal(12,3)");
        b.Property(e => e.MaxDelaySeconds).HasColumnType("decimal(12,3)");
        b.Property(e => e.Rationale).HasMaxLength(2000);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.FromCausalNodeId, e.ToCausalNodeId }).IsUnique()
            .HasDatabaseName("UQ_RootCause_CausalEdge_From_To");
        b.HasIndex(e => new { e.EdgeStatusId, e.EdgeProvenanceTypeId })
            .HasDatabaseName("IX_RootCause_CausalEdge_Status_Provenance");

        b.HasOne(e => e.FromNode)
            .WithMany(e => e.OutgoingEdges)
            .HasForeignKey(e => e.FromCausalNodeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RootCause_CausalEdge_FromNode");

        b.HasOne(e => e.ToNode)
            .WithMany(e => e.IncomingEdges)
            .HasForeignKey(e => e.ToCausalNodeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RootCause_CausalEdge_ToNode");
    }
}

```

15.6 Analysis and Grounding Configurations

The analysis tables are the runtime side of RootCause. An analysis is opened against a unit and a graph version; it includes alarms, builds a sealed context, tests candidates, constructs hypotheses and records the path of reasoning.

```
public sealed class RootCauseAnalysisConfiguration
    : IEntityTypeConfiguration<RootCauseAnalysis>
{
    public void Configure(EntityTypeBuilder<RootCauseAnalysis> b)
    {
        b.ToTable("RootCauseAnalysis", NexusSql.RootCause);
        b.HasKey(e => e.RootCauseAnalysisId)
            .HasName("PK_RootCause_RootCauseAnalysis");

        b.Property(e => e.AnalysisCode).HasRootCauseCode(80);
        b.Property(e => e.Title).HasMaxLength(250).IsRequired();
        b.Property(e => e.OpenedAtUtc).HasUtcDefault();
        b.Property(e => e.ClosedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.TriggerSummary).HasMaxLength(2000);
        b.Property(e => e.ContextHash).HasMaxLength(128);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.AnalysisCode).IsUnique()
            .HasDatabaseName("UQ_RootCause_RootCauseAnalysis_AnalysisCode");
        b.HasIndex(e => new { e.UnitId, e.OpenedAtUtc })
            .HasDatabaseName("IX_RootCause_RootCauseAnalysis_Unit_OpenedAt");
        b.HasIndex(e => new { e.AnalysisStatusId, e.OpenedAtUtc })
            .HasDatabaseName("IX_RootCause_RootCauseAnalysis_Status_OpenedAt");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RootCause_RootCauseAnalysis_ReactorFleet_Unit");

        b.HasOne(e => e.CausalGraphVersion)
            .WithMany()
            .HasForeignKey(e => e.CausalGraphVersionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RootCause_RootCauseAnalysis_CausalGraphVersion");
    }
}

public sealed class AnalysisAlarmConfiguration
    : IEntityTypeConfiguration<AnalysisAlarm>
{
    public void Configure(EntityTypeBuilder<AnalysisAlarm> b)
    {
        b.ToTable("AnalysisAlarm", NexusSql.RootCause);
        b.HasKey(e => e.AnalysisAlarmId)
            .HasName("PK_RootCause_AnalysisAlarm");

        b.Property(e => e.SequenceNumber).IsRequired();
        b.Property(e => e.IncludedAtUtc).HasUtcDefault();
        b.Property(e => e.Note).HasMaxLength(800);

        b.HasIndex(e => new { e.RootCauseAnalysisId, e.AlarmEventId }).IsUnique()
            .HasDatabaseName("UQ_RootCause_AnalysisAlarm_Analysis_Alarm");

        b.HasOne(e => e.RootCauseAnalysis)
            .WithMany(e => e.Alarms)
            .HasForeignKey(e => e.RootCauseAnalysisId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RootCause_AnalysisAlarm_RootCauseAnalysis");

        b.HasOne(e => e.AlarmEvent)
            .WithMany()
            .HasForeignKey(e => e.AlarmEventId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RootCause_AnalysisAlarm_AlarmManagement_AlarmEvent");
    }
}
```

Grounding context

```
public sealed class GroundingContextConfiguration
    : IEntityTypeConfiguration<GroundingContext>
{
    public void Configure(EntityTypeBuilder<GroundingContext> b)
    {
        b.ToTable("GroundingContext", NexusSql.RootCause);
        b.HasKey(e => e.GroundingContextId)
            .HasName("PK_RootCause_GroundingContext");

        b.Property(e => e.ContextCode).HasRootCauseCode(80);
        b.Property(e => e.BuiltAtUtc).HasUtcDefault();
        b.Property(e => e.WindowStartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.WindowEndUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ContextJson).HasColumnType("nvarchar(max)");
        b.Property(e => e.ContextHash).HasMaxLength(128).IsRequired();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.RootCauseAnalysisId, e.ContextHash }).IsUnique()
            .HasDatabaseName("UQ_RootCause_GroundingContext_Analysis_Hash");

        b.HasOne(e => e.RootCauseAnalysis)
            .WithMany(e => e.GroundingContexts)
            .HasForeignKey(e => e.RootCauseAnalysisId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RootCause_GroundingContext_RootCauseAnalysis");
    }
}

public sealed class GroundingContextSignalWindowConfiguration
    : IEntityTypeConfiguration<GroundingContextSignalWindow>
{
    public void Configure(EntityTypeBuilder<GroundingContextSignalWindow> b)
    {
        b.ToTable("GroundingContextSignalWindow", NexusSql.RootCause);
        b.HasKey(e => e.GroundingContextSignalWindowId)
            .HasName("PK_RootCause_GroundingContextSignalWindow");

        b.Property(e => e.StartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.EndUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.SampleCount).HasDefaultValue(0);
        b.Property(e => e.QualitySummary).HasMaxLength(800);

        b.HasIndex(e => new { e.GroundingContextId, e.SignalId }).IsUnique()
            .HasDatabaseName("UQ_RootCause_GroundingContextSignalWindow_Context_Signal");
    }
}
```

15.7 Candidates, Hypotheses and Witnesses

A professional RCA model must show what it tested, not only what it selected. Candidates and hypotheses are therefore separate. A candidate says: this graph node was tested. A hypothesis says: this candidate became a structured claim. Witness rows then explain whether telemetry, registry or documents supported the claim.

```
public sealed class AnalysisCandidateConfiguration
    : IEntityTypeConfiguration<AnalysisCandidate>
{
    public void Configure(EntityTypeBuilder<AnalysisCandidate> b)
    {
        b.ToTable("AnalysisCandidate", NexusSql.RootCause);
        b.HasKey(e => e.AnalysisCandidateId)
            .HasName("PK_RootCause_AnalysisCandidate");

        b.Property(e => e.CoverageScore).HasScore();
        b.Property(e => e.TimingFitScore).HasScore();
        b.Property(e => e.GroundingScore).HasScore();
        b.Property(e => e.OverallScore).HasScore();
        b.Property(e => e.Explanation).HasMaxLength(2000);
        b.Property(e => e.TestedAtUtc).HasColumnType("datetime2(7)");

        b.HasIndex(e => new { e.RootCauseAnalysisId, e.CausalNodeId }).IsUnique()
            .HasDatabaseName("UQ_RootCause_AnalysisCandidate_Analysis_Node");
        b.HasIndex(e => new { e.RootCauseAnalysisId, e.OverallScore })
            .HasDatabaseName("IX_RootCause_AnalysisCandidate_Analysis_Score");

        b.HasOne(e => e.RootCauseAnalysis)
            .WithMany(e => e.Candidates)
            .HasForeignKey(e => e.RootCauseAnalysisId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RootCause_AnalysisCandidate_RootCauseAnalysis");

        b.HasOne(e => e.CausalNode)
            .WithMany()
            .HasForeignKey(e => e.CausalNodeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RootCause_AnalysisCandidate_CausalNode");
    }
}

public sealed class AnalysisHypothesisConfiguration
    : IEntityTypeConfiguration<AnalysisHypothesis>
{
    public void Configure(EntityTypeBuilder<AnalysisHypothesis> b)
    {
        b.ToTable("AnalysisHypothesis", NexusSql.RootCause);
        b.HasKey(e => e.AnalysisHypothesisId)
            .HasName("PK_RootCause_AnalysisHypothesis");

        b.Property(e => e.HypothesisText).HasMaxLength(2000).IsRequired();
        b.Property(e => e.ConfidenceScore).HasScore();
        b.Property(e => e.GeneratedAtUtc).HasUtcDefault();
        b.Property(e => e.StructuredJson).HasColumnType("nvarchar(max)");

        b.HasIndex(e => new { e.AnalysisCandidateId, e.HypothesisStatusId })
            .HasDatabaseName("IX_RootCause_AnalysisHypothesis_Candidate_Status");
    }
}
```

```
public sealed class TelemetryWitnessConfiguration
    : IEntityTypeConfiguration<TelemetryWitness>
{
    public void Configure(EntityTypeBuilder<TelemetryWitness> b)
    {
        b.ToTable("TelemetryWitness", NexusSql.RootCause);
        b.HasKey(e => e.TelemetryWitnessId)
            .HasName("PK_RootCause_TelemetryWitness");

        b.Property(e => e.ObservedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.SupportScore).HasScore();
        b.Property(e => e.QualityFlag).HasMaxLength(50);
        b.Property(e => e.Summary).HasMaxLength(1200);

        b.HasIndex(e => new { e.AnalysisHypothesisId, e.SignalId })
            .HasDatabaseName("IX_RootCause_TelemetryWitness_Hypothesis_Signal");
    }
}

public sealed class RegistryWitnessConfiguration
    : IEntityTypeConfiguration<RegistryWitness>
```

```

{
    public void Configure(EntityTypeBuilder<RegistryWitness> b)
    {
        b.ToTable("RegistryWitness", NexusSql.RootCause);
        b.HasKey(e => e.RegistryWitnessId)
            .HasName("PK_RootCause_RegistryWitness");

        b.Property(e => e.SupportScore).HasScore();
        b.Property(e => e.RegistryFact).HasMaxLength(1200).IsRequired();
        b.Property(e => e.CapturedAtUtc).HasUtcDefault();

        b.HasIndex(e => new { e.AnalysisHypothesisId, e.EquipmentId })
            .HasDatabaseName("IX_RootCause_RegistryWitness_Hypothesis_Equipment");
    }
}

public sealed class DocumentWitnessConfiguration
    : IEntityTypeConfiguration<DocumentWitness>
{
    public void Configure(EntityTypeBuilder<DocumentWitness> b)
    {
        b.ToTable("DocumentWitness", NexusSql.RootCause);
        b.HasKey(e => e.DocumentWitnessId)
            .HasName("PK_RootCause_DocumentWitness");

        b.Property(e => e.DocumentReference).HasMaxLength(400).IsRequired();
        b.Property(e => e.FragmentHash).HasMaxLength(128);
        b.Property(e => e.RelevanceScore).HasScore();
        b.Property(e => e.Excerpt).HasMaxLength(2000);
    }
}

```

15.8 Output, Validation and Audit Trail

The output tables complete the chain: the selected conclusion or abstention, the deterministic or downstream model run, structured model output, validation checks and lifecycle trace. The configuration pattern is explicit about hashes, timestamps and named indexes because these rows are the audit face of the engine.

```
public sealed class AnalysisConclusionConfiguration
    : IEntityTypeConfiguration<AnalysisConclusion>
{
    public void Configure(EntityTypeBuilder<AnalysisConclusion> b)
    {
        b.ToTable("AnalysisConclusion", NexusSql.RootCause);
        b.HasKey(e => e.AnalysisConclusionId)
            .HasName("PK_RootCause_AnalysisConclusion");

        b.Property(e => e.ConcludedAtUtc).HasUtcDefault();
        b.Property(e => e.ConclusionText).HasMaxLength(3000).IsRequired();
        b.Property(e => e.ConfidenceScore).HasScore();
        b.Property(e => e.IsAbstention).HasDefaultValue(false);
        b.Property(e => e.ConclusionHash).HasMaxLength(128);

        b.HasIndex(e => e.RootCauseAnalysisId).IsUnique()
            .HasDatabaseName("UQ_RootCause_AnalysisConclusion_Analysis");
    }
}

public sealed class AnalysisModelRunConfiguration
    : IEntityTypeConfiguration<AnalysisModelRun>
{
    public void Configure(EntityTypeBuilder<AnalysisModelRun> b)
    {
        b.ToTable("AnalysisModelRun", NexusSql.RootCause);
        b.HasKey(e => e.AnalysisModelRunId)
            .HasName("PK_RootCause_AnalysisModelRun");

        b.Property(e => e.ModelName).HasMaxLength(160).IsRequired();
        b.Property(e => e.ModelVersion).HasMaxLength(80).IsRequired();
        b.Property(e => e.StartedAtUtc).HasUtcDefault();
        b.Property(e => e.CompletedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.PromptHash).HasMaxLength(128);
        b.Property(e => e.ResponseHash).HasMaxLength(128);
        b.Property(e => e.WasGrounded).HasDefaultValue(false);

        b.HasIndex(e => new { e.RootCauseAnalysisId, e.StartedAtUtc })
            .HasDatabaseName("IX_RootCause_AnalysisModelRun_Analysis_StartedAt");
    }
}

public sealed class AnalysisValidationCheckConfiguration
    : IEntityTypeConfiguration<AnalysisValidationCheck>
{
    public void Configure(EntityTypeBuilder<AnalysisValidationCheck> b)
    {
        b.ToTable("AnalysisValidationCheck", NexusSql.RootCause);
        b.HasKey(e => e.AnalysisValidationCheckId)
            .HasName("PK_RootCause_AnalysisValidationCheck");

        b.Property(e => e.CheckCode).HasRootCauseCode(30);
        b.Property(e => e.CheckName).HasMaxLength(150).IsRequired();
        b.Property(e => e.CheckedAtUtc).HasUtcDefault();
        b.Property(e => e.Details).HasMaxLength(2000);

        b.HasIndex(e => new { e.RootCauseAnalysisId, e.CheckCode }).IsUnique()
            .HasDatabaseName("UQ_RootCause_AnalysisValidationCheck_Analysis_CheckCode");
    }
}
```

Migration and verification notes

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(NexusContext).Assembly);
}

// Safety query after migration:
// There should be no default EF-generated index names in this schema.
SELECT i.name
FROM sys.indexes AS i
JOIN sys.tables AS t ON t.object_id = i.object_id
JOIN sys.schemas AS s ON s.schema_id = t.schema_id
```

```
WHERE s.name = 'RootCause'
      AND i.name LIKE 'IX_%[_][0-9A-F][0-9A-F]';

// Operational check:
// Every analysis has exactly one released graph version.
SELECT rca.AnalysisCode, cgv.VersionNumber, cg.Code AS GraphCode
FROM RootCause.RootCauseAnalysis AS rca
JOIN RootCause.CausalGraphVersion AS cgv
      ON cgv.CausalGraphVersionId = rca.CausalGraphVersionId
JOIN RootCause.CausalGraph AS cg
      ON cg.CausalGraphId = cgv.CausalGraphId;
```


15.9 Foreign-Key Mapping

The following mapping is the EF Core checklist for `HasOne`, `WithMany`, `HasForeignKey`, `OnDelete` and `HasConstraintName`. Internal `RootCause` relationships may cascade from their owner. Cross-schema passports should normally use `DeleteBehavior.Restrict` or `NoAction`.

CONFIGURATION	FK PROPERTY	TARGET	KIND
<code>RootCause.CausalGraph</code>	<code>UnitId</code>	<code>ReactorFleet.Unit</code>	passport
<code>RootCause.CausalGraphVersion</code>	<code>CausalGraphId</code>	<code>RootCause.CausalGraph</code>	internal
<code>RootCause.CausalNode</code>	<code>CausalGraphVersionId</code>	<code>RootCause.CausalGraphVersion</code>	internal
<code>RootCause.CausalNode</code>	<code>CausalNodeId</code>	<code>RootCause.CausalNode</code>	internal
<code>RootCause.CausalNodeEquipmentLink</code>	<code>CausalNodeId</code>	<code>RootCause.CausalNode</code>	internal
<code>RootCause.CausalNodeEquipmentLink</code>	<code>EquipmentId</code>	<code>ReactorFleet.Equipment</code>	passport
<code>RootCause.CausalNodeSignalLink</code>	<code>CausalNodeId</code>	<code>RootCause.CausalNode</code>	internal
<code>RootCause.CausalNodeSignalLink</code>	<code>SignalId</code>	<code>Instrumentation.Signal</code>	passport
<code>RootCause.CausalEdge</code>	<code>FromCausalNodeId</code>	<code>RootCause.CausalNode</code>	internal
<code>RootCause.CausalEdge</code>	<code>ToCausalNodeId</code>	<code>RootCause.CausalNode</code>	internal
<code>RootCause.CausalEdgeDelayWindow</code>	<code>CausalEdgeId</code>	<code>RootCause.CausalEdge</code>	internal
<code>RootCause.CausalEdgeEvidence</code>	<code>CausalEdgeId</code>	<code>RootCause.CausalEdge</code>	internal
<code>RootCause.CausalEdgeEvidence</code>	<code>EvidenceTypeId</code>	<code>RootCause.EvidenceType</code>	internal
<code>RootCause.CausalGraphChangeRequest</code>	<code>CausalGraphId</code>	<code>RootCause.CausalGraph</code>	internal
<code>RootCause.CausalGraphChangeRequest</code>	<code>RequestedByUserId</code>	<code>Security.ApplicationUser</code>	passport
<code>RootCause.CausalGraphChangeReview</code>	<code>CausalGraphChangeRequestId</code>	<code>RootCause / cross-schema target</code>	passport
<code>RootCause.RootCauseAnalysis</code>	<code>UnitId</code>	<code>ReactorFleet.Unit</code>	passport
<code>RootCause.RootCauseAnalysis</code>	<code>CausalGraphVersionId</code>	<code>RootCause.CausalGraphVersion</code>	internal
<code>RootCause.AnalysisEventLink</code>	<code>RootCauseAnalysisId</code>	<code>RootCause.RootCauseAnalysis</code>	internal
<code>RootCause.AnalysisEventLink</code>	<code>OperationalEventId</code>	<code>EventManagement.OperationalEvent</code>	passport
<code>RootCause.AnalysisAlarm</code>	<code>RootCauseAnalysisId</code>	<code>RootCause.RootCauseAnalysis</code>	internal
<code>RootCause.AnalysisAlarm</code>	<code>AlarmEventId</code>	<code>AlarmManagement.AlarmEvent</code>	passport
<code>RootCause.GroundingContext</code>	<code>RootCauseAnalysisId</code>	<code>RootCause.RootCauseAnalysis</code>	internal
<code>RootCause.GroundingContextSignalWindow</code>	<code>GroundingContextId</code>	<code>RootCause.GroundingContext</code>	internal
<code>RootCause.GroundingContextSignalWindow</code>	<code>SignalId</code>	<code>Instrumentation.Signal</code>	passport
<code>RootCause.GroundingContextDocument</code>	<code>GroundingContextId</code>	<code>RootCause.GroundingContext</code>	internal
<code>RootCause.AnalysisCandidate</code>	<code>RootCauseAnalysisId</code>	<code>RootCause.RootCauseAnalysis</code>	internal
<code>RootCause.AnalysisCandidate</code>	<code>CausalNodeId</code>	<code>RootCause.CausalNode</code>	internal
<code>RootCause.AnalysisHypothesis</code>	<code>AnalysisCandidateId</code>	<code>RootCause.AnalysisCandidate</code>	internal
<code>RootCause.HypothesisEvidence</code>	<code>AnalysisHypothesisId</code>	<code>RootCause.AnalysisHypothesis</code>	internal

CONFIGURATION	FK PROPERTY	TARGET	KIND
RootCause.HypothesisEvidence	EvidenceTypeId	RootCause.EvidenceType	internal
RootCause.RejectedCandidate	AnalysisCandidateId	RootCause.AnalysisCandidate	internal
RootCause.CausalPath	AnalysisCandidateId	RootCause.AnalysisCandidate	internal
RootCause.CausalPathStep	CausalPathId	RootCause.CausalPath	internal
RootCause.CausalPathStep	CausalEdgeId	RootCause.CausalEdge	internal
RootCause.TelemetryWitness	AnalysisHypothesisId	RootCause.AnalysisHypothesis	internal
RootCause.TelemetryWitness	SignalId	Instrumentation.Signal	passport
RootCause.RegistryWitness	AnalysisHypothesisId	RootCause.AnalysisHypothesis	internal
RootCause.RegistryWitness	EquipmentId	ReactorFleet.Equipment	passport
RootCause.DocumentWitness	AnalysisHypothesisId	RootCause.AnalysisHypothesis	internal
RootCause.AnalysisConclusion	RootCauseAnalysisId	RootCause.RootCauseAnalysis	internal
RootCause.AnalysisConclusion	AnalysisHypothesisId	RootCause.AnalysisHypothesis	internal
RootCause.AnalysisModelRun	RootCauseAnalysisId	RootCause.RootCauseAnalysis	internal
RootCause.AnalysisModelOutput	AnalysisModelRunId	RootCause.AnalysisModelRun	internal
RootCause.AnalysisValidationCheck	RootCauseAnalysisId	RootCause.RootCauseAnalysis	internal
RootCause.AnalysisAuditTrail	RootCauseAnalysisId	RootCause.RootCauseAnalysis	internal
RootCause.AnalysisComment	RootCauseAnalysisId	RootCause.RootCauseAnalysis	internal
RootCause.AnalysisTagAssignment	RootCauseAnalysisId	RootCause.RootCauseAnalysis	internal
RootCause.AnalysisTagAssignment	AnalysisTagId	RootCause.AnalysisTag	internal

15.10 Chapter Close

RootCause is where the NEXUS-1 database stops being only a storage layer and becomes an argument that can be inspected. The EF Core configuration layer must therefore do more than map properties. It must preserve causality, provenance, grounding, rejection and abstention as first-class rows.

Professional rule. No anonymous constraints, no hidden cascade across schema boundaries, no unbounded strings where a code should exist, and no conclusion without a stored context and evidence path.

Configuration file index

CausalNodeTypeConfiguration.cs Lookup	RootCauseAnalysisConfiguration.cs Analysis
CausalEdgeTypeConfiguration.cs Lookup	AnalysisEventLinkConfiguration.cs Analysis
EdgeProvenanceTypeConfiguration.cs Lookup	AnalysisAlarmConfiguration.cs Analysis
EdgeStatusConfiguration.cs Lookup	GroundingContextConfiguration.cs Analysis
AnalysisStatusConfiguration.cs Lookup	GroundingContextSignalWindowConfiguration.cs Analysis
CandidateRoleConfiguration.cs Lookup	GroundingContextDocumentConfiguration.cs Analysis
CandidateStatusConfiguration.cs Lookup	AnalysisCandidateConfiguration.cs Analysis
HypothesisStatusConfiguration.cs Lookup	AnalysisHypothesisConfiguration.cs Analysis
EvidenceTypeConfiguration.cs Lookup	HypothesisEvidenceConfiguration.cs Analysis
WitnessTypeConfiguration.cs Lookup	RejectedCandidateConfiguration.cs Analysis
ConfidenceBandConfiguration.cs Lookup	CausalPathConfiguration.cs Analysis
GroundingSourceTypeConfiguration.cs Lookup	CausalPathStepConfiguration.cs Analysis
ValidationResultConfiguration.cs Lookup	TelemetryWitnessConfiguration.cs Witness
AbstentionReasonConfiguration.cs Lookup	RegistryWitnessConfiguration.cs Witness
CausalGraphConfiguration.cs Graph	DocumentWitnessConfiguration.cs Witness
CausalGraphVersionConfiguration.cs Graph	AnalysisConclusionConfiguration.cs Output
CausalNodeConfiguration.cs Graph	AnalysisModelRunConfiguration.cs Output
CausalNodeEquipmentLinkConfiguration.cs Graph	AnalysisModelOutputConfiguration.cs Output
CausalNodeSignalLinkConfiguration.cs Graph	AnalysisValidationCheckConfiguration.cs Output
CausalEdgeConfiguration.cs Graph	AnalysisAuditTrailConfiguration.cs Output
CausalEdgeDelayWindowConfiguration.cs Graph	AnalysisCommentConfiguration.cs Output
CausalEdgeEvidenceConfiguration.cs Graph	AnalysisTagConfiguration.cs Output
CausalGraphChangeRequestConfiguration.cs Graph	AnalysisTagAssignmentConfiguration.cs Output
CausalGraphChangeReviewConfiguration.cs Graph	

FROM ENTITY TO CONTEXT

Chapter 16 - ReinforcementLearning Configurations
The readable policy mapped as enterprise EF Core configuration



CONFIGURATION COMPANION

**Learning runs, Q-tables, policies, clamps and
advisory recommendations - fully mapped.**

Chapter 16

ReinforcementLearning Configurations

The EF Core configuration atlas for the learning sector: environment models, state/action spaces, training episodes, Q-table entries, policies, evaluations, deployments, safety clamps and advisory output.

The reinforcement-learning schema is where *From Trial to Policy* becomes persistent software. The agent's brain is not a hidden network; it is a table. This chapter maps that table, the runs that produced it, and the advisory path that reads it.

Standing rule: learning happens only in a safe environment model, policies are readable, and deployment remains advisory. EF Core mapping must preserve that boundary rather than blur it.

Chapter 16 - ReinforcementLearning Configurations

This chapter continues the schema-by-schema configuration atlas. The goal is not to teach reinforcement learning again, but to show how a serious .NET project maps the data structures that make the agent auditable: the state space, action space, reward function, training run, episode log, Q-table, policy, safety clamp and recommendation history.

Where this chapter sits. Chapter 15 mapped RootCause. Chapter 16 maps ReinforcementLearning. In the full book this is the second reasoning/learning schema and the point where the EF Core layer preserves the advisory-only philosophy of the NEXUS-1 optimization engine.

Lookup

12 configurations

LearningAlgorithm,
EnvironmentModelType,
TrainingRunStatus, EpisodeStatus,
StateSpaceType ...

Model

8 configurations

EnvironmentModel, StateSpace,
StateDefinition, ActionSpace,
ActionDefinition ...

Training

7 configurations

TrainingRun, Episode, EpisodeStep,
QTable, QTableEntry ...

Policy

5 configurations

Policy, PolicyEntry, PolicyEvaluation,
PolicyEvaluationMetric, PolicyDeployment

Advisory

5 configurations

SafetyClamp, SafetyClampRule,
AdvisorySession,
AdvisoryRecommendation,
AdvisoryRecommendationReason

16.1 Configuration File Catalogue

Every table in the `ReinforcementLearning` schema receives its own configuration class. Lookups are still individual classes, but they inherit the shared lookup base so that code, name, display order, active flag and row version never drift from table to table.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	<code>ReinforcementLearning.LearningAlgorithm</code>	<code>LearningAlgorithmConfiguration.cs</code>	Algorithm catalogue: tabular Q-learning now, future SARSA or actor-critic variants later.
Lookup	<code>ReinforcementLearning.EnvironmentModelType</code>	<code>EnvironmentModelTypeConfiguration.cs</code>	Kind of environment model used during training: surrogate, point-kinetics twin, replay, or offline dataset.
Lookup	<code>ReinforcementLearning.TrainingRunStatus</code>	<code>TrainingRunStatusConfiguration.cs</code>	Lifecycle of a training run: queued, running, completed, failed, cancelled, archived.
Lookup	<code>ReinforcementLearning.EpisodeStatus</code>	<code>EpisodeStatusConfiguration.cs</code>	Lifecycle and outcome of one episode.
Lookup	<code>ReinforcementLearning.StateSpaceType</code>	<code>StateSpaceTypeConfiguration.cs</code>	State representation: discrete bins, continuous vector, replay-only, hybrid.
Lookup	<code>ReinforcementLearning.ActionSpaceType</code>	<code>ActionSpaceTypeConfiguration.cs</code>	Action representation: discrete rod nudges, continuous actuator values, advisory labels.
Lookup	<code>ReinforcementLearning.RewardFunctionType</code>	<code>RewardFunctionTypeConfiguration.cs</code>	Reward family: squared error plus movement cost, custom SQL, external function, frozen formula.
Lookup	<code>ReinforcementLearning.PolicyStatus</code>	<code>PolicyStatusConfiguration.cs</code>	Policy lifecycle: draft, training, candidate, validated, deployed, retired.
Lookup	<code>ReinforcementLearning.AdvisoryMode</code>	<code>AdvisoryModeConfiguration.cs</code>	How advice is used: simulation, training, shadow, advisory, disabled.
Lookup	<code>ReinforcementLearning.RecommendationStatus</code>	<code>RecommendationStatusConfiguration.cs</code>	Recommendation lifecycle: proposed, clamped, shown, accepted, rejected, expired.
Lookup	<code>ReinforcementLearning.SafetyClampType</code>	<code>SafetyClampTypeConfiguration.cs</code>	Clamp category: action bounds, state coverage, confidence floor, rate limit, protected-mode block.
Lookup	<code>ReinforcementLearning.EvaluationMetricType</code>	<code>EvaluationMetricTypeConfiguration.cs</code>	Evaluation metric catalogue: average reward, target error, movement count, violation rate, coverage.
Model	<code>ReinforcementLearning.EnvironmentModel</code>	<code>EnvironmentModelConfiguration.cs</code>	Named training environment: the safe world in which the agent is allowed to fail.
Model	<code>ReinforcementLearning.StateSpace</code>	<code>StateSpaceConfiguration.cs</code>	Definition of a state space, such as power deviation x trend bins.
Model	<code>ReinforcementLearning.StateDefinition</code>	<code>StateDefinitionConfiguration.cs</code>	One discrete state row, with bin boundaries and a stable state index.

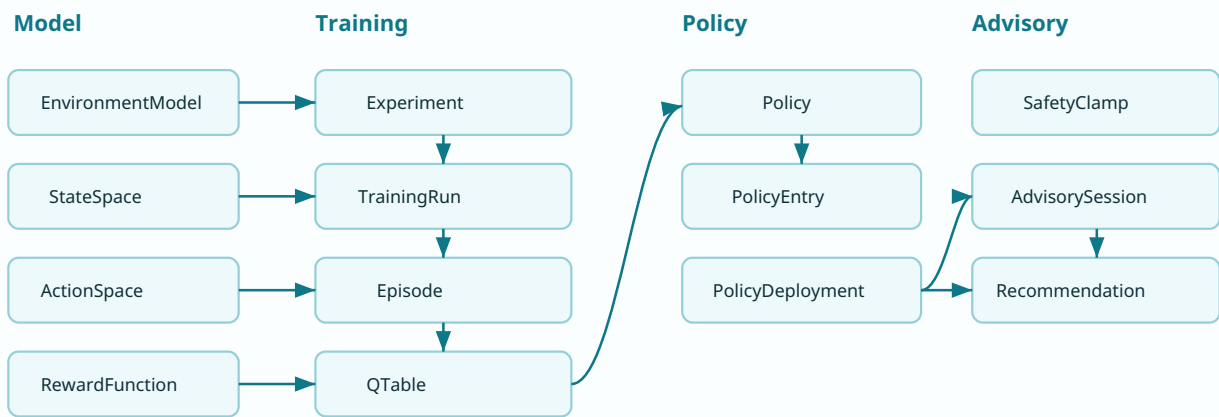
GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Model	ReinforcementLearning.ActionSpace	ActionSpaceConfiguration.cs	Definition of available actions for an agent.
Model	ReinforcementLearning.ActionDefinition	ActionDefinitionConfiguration.cs	One action, such as insert hard, insert gently, hold, withdraw gently, withdraw hard.
Model	ReinforcementLearning.RewardFunction	RewardFunctionConfiguration.cs	Stored reward formula and coefficients.
Model	ReinforcementLearning.HyperparameterSet	HyperparameterSetConfiguration.cs	Learning rate, discount factor, epsilon schedule and training horizon.
Model	ReinforcementLearning.Experiment	ExperimentConfiguration.cs	Named experiment grouping runs, policies and evaluation results.
Training	ReinforcementLearning.TrainingRun	TrainingRunConfiguration.cs	One deterministic training run with pinned environment, reward, states, actions and hyperparameters.
Training	ReinforcementLearning.Episode	EpisodeConfiguration.cs	One rollout against the environment.
Training	ReinforcementLearning.EpisodeStep	EpisodeStepConfiguration.cs	One transition tuple: state, action, reward, next state and done flag.
Training	ReinforcementLearning.QTable	QTableConfiguration.cs	A complete Q-table snapshot produced by a training run.
Training	ReinforcementLearning.QTableEntry	QTableEntryConfiguration.cs	One Q-value for one state-action pair.
Training	ReinforcementLearning.ModelArtifact	ModelArtifactConfiguration.cs	Saved artifact hashes and paths for reproducibility.
Training	ReinforcementLearning.TrainingRunAuditTrail	TrainingRunAuditTrailConfiguration.cs	Append-only lifecycle and diagnostic trace of training.
Policy	ReinforcementLearning.Policy	PolicyConfiguration.cs	Readable policy extracted from the Q-table: best action per state.
Policy	ReinforcementLearning.PolicyEntry	PolicyEntryConfiguration.cs	One policy decision per state with Q-value and margin.
Policy	ReinforcementLearning.PolicyEvaluation	PolicyEvaluationConfiguration.cs	Evaluation run for a frozen policy.
Policy	ReinforcementLearning.PolicyEvaluationMetric	PolicyEvaluationMetricConfiguration.cs	One metric result for a policy evaluation.
Policy	ReinforcementLearning.PolicyDeployment	PolicyDeploymentConfiguration.cs	Controlled deployment of a policy into a simulation, shadow or advisory context.
Advisory	ReinforcementLearning.SafetyClamp	SafetyClampConfiguration.cs	Named safety envelope that can block or alter an advisory recommendation.
Advisory	ReinforcementLearning.SafetyClampRule	SafetyClampRuleConfiguration.cs	One rule in a clamp: max nudge, confidence floor, protected mode block, state coverage check.
Advisory	ReinforcementLearning.AdvisorySession	AdvisorySessionConfiguration.cs	A session in which a deployed policy is asked for advisory recommendations.
Advisory	ReinforcementLearning.AdvisoryRecommendation	AdvisoryRecommendationConfiguration.cs	One recommendation returned by a policy, including original and clamped action.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Advisory	ReinforcementLearning.AdvisoryRecommendationReason	AdvisoryRecommendationReasonConfiguration.cs	Readable explanation bars: Q-values and why-this-action reasons.

16.2 Folder Structure

The folder layout follows the schema, then separates lookup configuration from model, training, policy and advisory configuration. This keeps the `DbContext` clean and lets `ApplyConfigurationsFromAssembly` discover everything without a giant switch statement.

```
Nexus.Infrastructure/  
  Persistence/  
    NexusContext.cs  
  Configurations/  
    ReinforcementLearning/  
      Lookups/  
        LearningAlgorithmConfiguration.cs  
        EnvironmentModelTypeConfiguration.cs  
        TrainingRunStatusConfiguration.cs  
        EpisodeStatusConfiguration.cs  
        StateSpaceTypeConfiguration.cs  
        ActionSpaceTypeConfiguration.cs  
        RewardFunctionTypeConfiguration.cs  
        PolicyStatusConfiguration.cs  
        AdvisoryModeConfiguration.cs  
        RecommendationStatusConfiguration.cs  
        SafetyClampTypeConfiguration.cs  
        EvaluationMetricTypeConfiguration.cs  
      Model/  
        EnvironmentModelConfiguration.cs  
        StateSpaceConfiguration.cs  
        StateDefinitionConfiguration.cs  
        ActionSpaceConfiguration.cs  
        ActionDefinitionConfiguration.cs  
        RewardFunctionConfiguration.cs  
        HyperparameterSetConfiguration.cs  
        ExperimentConfiguration.cs  
      Training/  
        TrainingRunConfiguration.cs  
        EpisodeConfiguration.cs  
        EpisodeStepConfiguration.cs  
        QTableConfiguration.cs  
        QTableEntryConfiguration.cs  
      Policy/  
        PolicyConfiguration.cs  
        PolicyEntryConfiguration.cs  
        PolicyEvaluationConfiguration.cs  
        PolicyDeploymentConfiguration.cs  
      Advisory/  
        SafetyClampConfiguration.cs  
        AdvisorySessionConfiguration.cs  
        AdvisoryRecommendationConfiguration.cs
```

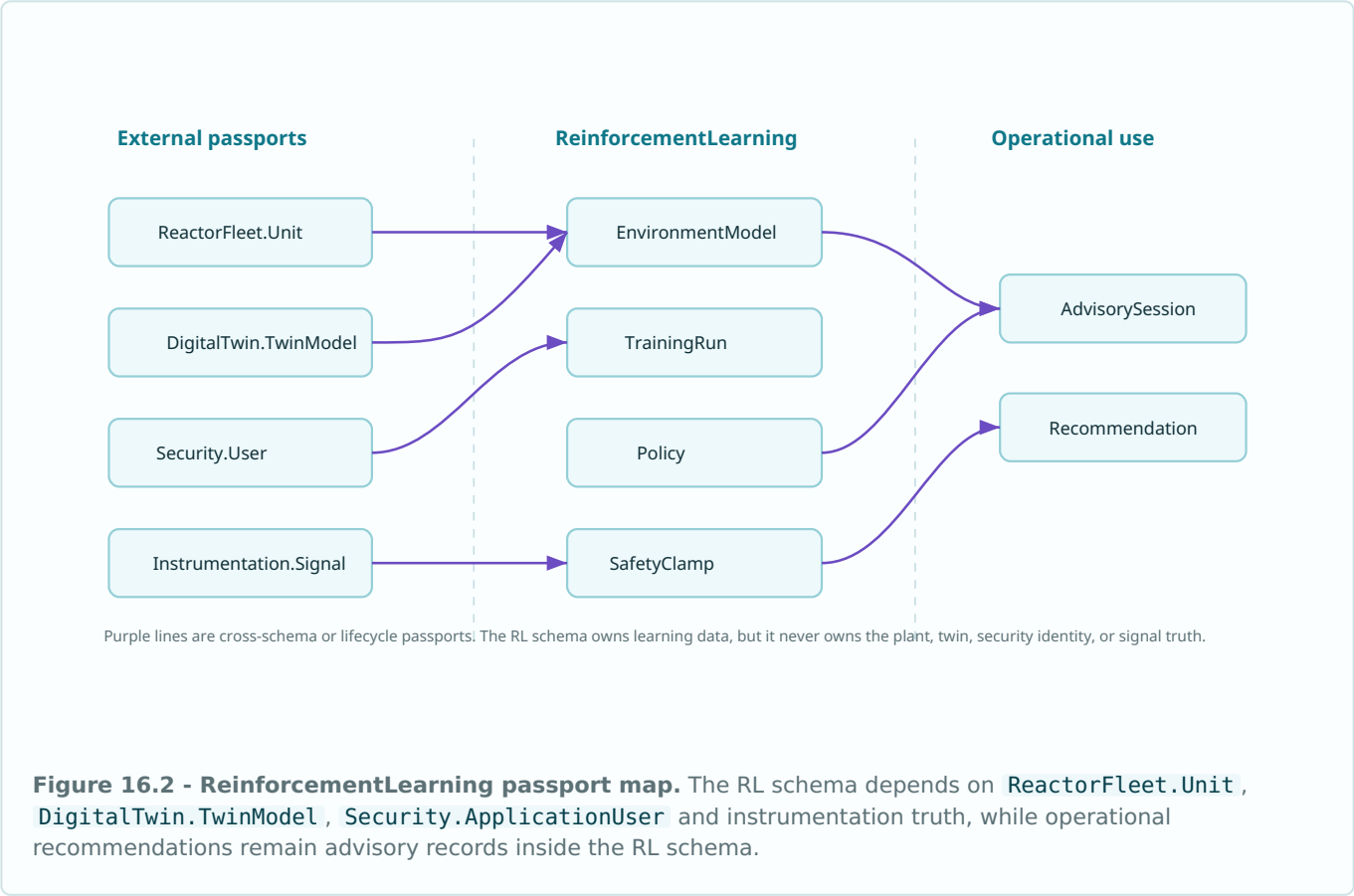


The mapping mirrors the runtime loop: model definition feeds training; training produces a Q-table; a policy is extracted; deployment only serves advisory recommendations

Figure 16.1 - EF Core dependency diagram. The configuration structure mirrors the learning lifecycle: model definition -> training -> Q-table -> policy -> advisory output.

16.3 Passport Map

The learning sector owns training data, but it does not own the physical unit, the digital-twin model, users or signal truth. Those remain passports into other schemas. The configuration files make each crossing explicit with a named foreign-key constraint and restrictive delete behaviour.



16.4 Shared Mapping Base

Lookup tables are typed. They are not enums hidden in code and not strings scattered through the application. The base class below gives every RL lookup the same discipline while still allowing each lookup table to own its table name and key.

```
namespace Nexus.Infrastructure.Persistence.Configurations.ReinforcementLearning;

internal static partial class NexusSql
{
    public const string ReinforcementLearning = "ReinforcementLearning";
    public const string ReactorFleet = "ReactorFleet";
    public const string DigitalTwin = "DigitalTwin";
    public const string Instrumentation = "Instrumentation";
    public const string Security = "Security";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class ReinforcementLearningLookupConfiguration<TEntity>
    : IEntityTypeConfiguration<TEntity>
    where TEntity : ReinforcementLearningLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(150)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

internal static class ReinforcementLearningMappingExtensions
{
    public static PropertyBuilder<string> HasRLCode(
        this PropertyBuilder<string> property, int length = 80) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);

    public static PropertyBuilder<decimal> HasRewardOrValue(
        this PropertyBuilder<decimal> property) => property
        .HasColumnType("decimal(18,8)");

    public static PropertyBuilder<decimal?> HasOptionalScore(
        this PropertyBuilder<decimal?> property) => property
        .HasColumnType("decimal(18,8)");
}
```

16.5 Environment Model Configuration

`EnvironmentModel` is the safe world in which the agent may fail. It points to the physical unit and the digital-twin model, but it remains a training artefact, not a control path.

```
public sealed class EnvironmentModelConfiguration
    : IEntityTypeConfiguration<EnvironmentModel>
{
    public void Configure(EntityTypeBuilder<EnvironmentModel> b)
    {
        b.ToTable("EnvironmentModel", NexusSql.ReinforcementLearning);
        b.HasKey(e => e.EnvironmentModelId)
            .HasName("PK_RL_EnvironmentModel");

        b.Property(e => e.Code).HasRtCode(80);
        b.Property(e => e.Name).HasMaxLength(180).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1000);
        b.Property(e => e.ModelVersion).HasMaxLength(40).IsRequired();
        b.Property(e => e.ConfigurationJson).HasColumnType("nvarchar(max)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.EnvironmentModelType)
            .WithMany()
            .HasForeignKey(e => e.EnvironmentModelTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_EnvironmentModel_ModelType");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_EnvironmentModel_Unit");

        b.HasOne(e => e.TwinModel)
            .WithMany()
            .HasForeignKey(e => e.TwinModelId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_EnvironmentModel_TwinModel");

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_RL_EnvironmentModel_Code");

        b.HasIndex(e => new { e.UnitId, e.TwinModelId, e.ModelVersion })
            .HasDatabaseName("IX_RL_EnvironmentModel_Unit_Twin_Version");
    }
}
```

16.6 State and Action Definitions

Tabular Q-learning is readable because the state and action spaces are discrete and named. The configurations enforce stable indices inside each space. A state/action pair should not change meaning after a policy has been trained.

```
public sealed class StateDefinitionConfiguration
    : IEntityTypeConfiguration<StateDefinition>
{
    public void Configure(EntityTypeBuilder<StateDefinition> b)
    {
        b.ToTable("StateDefinition", NexusSql.ReinforcementLearning);
        b.HasKey(e => e.StateDefinitionId)
            .HasName("PK_RL_StateDefinition");

        b.Property(e => e.StateIndex).IsRequired();
        b.Property(e => e.Code).HasRLCode(80);
        b.Property(e => e.Label).HasMaxLength(180).IsRequired();
        b.Property(e => e.LowerBound).HasColumnType("decimal(18,8)");
        b.Property(e => e.UpperBound).HasColumnType("decimal(18,8)");
        b.Property(e => e.TrendBucket).HasMaxLength(40);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.StateSpace)
            .WithMany(e => e.StateDefinitions)
            .HasForeignKey(e => e.StateSpaceId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RL_StateDefinition_StateSpace");

        b.HasIndex(e => new { e.StateSpaceId, e.StateIndex })
            .IsUnique()
            .HasDatabaseName("UQ_RL_StateDefinition_Space_Index");
    }
}

public sealed class ActionDefinitionConfiguration
    : IEntityTypeConfiguration<ActionDefinition>
{
    public void Configure(EntityTypeBuilder<ActionDefinition> b)
    {
        b.ToTable("ActionDefinition", NexusSql.ReinforcementLearning);
        b.HasKey(e => e.ActionDefinitionId)
            .HasName("PK_RL_ActionDefinition");

        b.Property(e => e.ActionIndex).IsRequired();
        b.Property(e => e.Code).HasRLCode(80);
        b.Property(e => e.Label).HasMaxLength(180).IsRequired();
        b.Property(e => e.ActionValue).HasColumnType("decimal(18,8)");
        b.Property(e => e.ActionUnit).HasMaxLength(40).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.ActionSpace)
            .WithMany(e => e.ActionDefinitions)
            .HasForeignKey(e => e.ActionSpaceId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RL_ActionDefinition_ActionSpace");

        b.HasIndex(e => new { e.ActionSpaceId, e.ActionIndex })
            .IsUnique()
            .HasDatabaseName("UQ_RL_ActionDefinition_Space_Index");
    }
}
```

16.7 Episode Steps and Q-Table Entries

The episode step is the audit row for learning: state, action, reward, next state and terminal flag. The Q-table entry is the learned value for one state-action pair. The unique index on `QTableId + StateDefinitionId + ActionDefinitionId` is the database equivalent of a two-dimensional table cell.

```
public sealed class EpisodeStepConfiguration
    : IEntityTypeConfiguration<EpisodeStep>
{
    public void Configure(EntityTypeBuilder<EpisodeStep> b)
    {
        b.ToTable("EpisodeStep", NexusSql.ReinforcementLearning);
        b.HasKey(e => e.EpisodeStepId)
            .HasName("PK_RL_EpisodeStep");

        b.Property(e => e.StepIndex).IsRequired();
        b.Property(e => e.Reward).HasColumnType("decimal(18,8)");
        b.Property(e => e.PowerPercent).HasOptionalScore();
        b.Property(e => e.TargetPercent).HasOptionalScore();
        b.Property(e => e.IsTerminal).HasDefaultValue(false);
        b.Property(e => e.ObservedAtUtc).HasUtcDefault();

        b.HasOne(e => e.Episode)
            .WithMany(e => e.Steps)
            .HasForeignKey(e => e.EpisodeId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RL_EpisodeStep_Episode");

        b.HasOne(e => e.StateDefinition)
            .WithMany()
            .HasForeignKey(e => e.StateDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_EpisodeStep_State");

        b.HasOne(e => e.ActionDefinition)
            .WithMany()
            .HasForeignKey(e => e.ActionDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_EpisodeStep_Action");

        b.HasOne(e => e.NextStateDefinition)
            .WithMany()
            .HasForeignKey(e => e.NextStateDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_EpisodeStep_NextState");

        b.HasIndex(e => new { e.EpisodeId, e.StepIndex })
            .IsUnique()
            .HasDatabaseName("UQ_RL_EpisodeStep_Episode_Step");
    }
}

public sealed class QTableEntryConfiguration
    : IEntityTypeConfiguration<QTableEntry>
{
    public void Configure(EntityTypeBuilder<QTableEntry> b)
    {
        b.ToTable("QTableEntry", NexusSql.ReinforcementLearning);
        b.HasKey(e => e.QTableEntryId)
            .HasName("PK_RL_QTableEntry");

        b.Property(e => e.QValue).HasRewardOrValue();
        b.Property(e => e.VisitCount).HasDefaultValue(0);
        b.Property(e => e.UpdatedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.QTable)
            .WithMany(e => e.Entries)
            .HasForeignKey(e => e.QTableId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RL_QTableEntry_QTable");

        b.HasOne(e => e.StateDefinition)
            .WithMany()
            .HasForeignKey(e => e.StateDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_QTableEntry_State");

        b.HasOne(e => e.ActionDefinition)
            .WithMany()
            .HasForeignKey(e => e.ActionDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_QTableEntry_Action");

        b.HasIndex(e => new { e.QTableId, e.StateDefinitionId, e.ActionDefinitionId })
            .IsUnique()
            .HasDatabaseName("UQ_RL_QTableEntry_State_Action");
    }
}
```


16.8 Policy Deployment and Advisory Recommendation

A policy is not a command. It can be deployed into simulation, shadow or advisory mode, then used to create a recommendation record. The recommendation records both original and clamped actions so that the safety envelope is visible, not implied.

```
public sealed class PolicyDeploymentConfiguration
    : IEntityTypeConfiguration<PolicyDeployment>
{
    public void Configure(EntityTypeBuilder<PolicyDeployment> b)
    {
        b.ToTable("PolicyDeployment", NexusSql.ReinforcementLearning);
        b.HasKey(e => e.PolicyDeploymentId)
            .HasName("PK_RL_PolicyDeployment");

        b.Property(e => e.Code).HasRLCode(80);
        b.Property(e => e.DeploymentName).HasMaxLength(180).IsRequired();
        b.Property(e => e.DeployedAtUtc).HasUtcDefault();
        b.Property(e => e.RetiredAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.IsActive).HasDefaultValue(true);

        b.HasOne(e => e.Policy)
            .WithMany(e => e.Deployments)
            .HasForeignKey(e => e.PolicyId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_PolicyDeployment_Policy");

        b.HasOne(e => e.AdvisoryMode)
            .WithMany()
            .HasForeignKey(e => e.AdvisoryModeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_PolicyDeployment_AdvisoryMode");

        b.HasIndex(e => new { e.PolicyId, e.AdvisoryModeId, e.IsActive })
            .HasDatabaseName("IX_RL_PolicyDeployment_Policy_Mode_Active");
    }
}

public sealed class AdvisoryRecommendationConfiguration
    : IEntityTypeConfiguration<AdvisoryRecommendation>
{
    public void Configure(EntityTypeBuilder<AdvisoryRecommendation> b)
    {
        b.ToTable("AdvisoryRecommendation", NexusSql.ReinforcementLearning);
        b.HasKey(e => e.AdvisoryRecommendationId)
            .HasName("PK_RL_AdvisoryRecommendation");

        b.Property(e => e.RecommendedAtUtc).HasUtcDefault();
        b.Property(e => e.OriginalQValue).HasOptionalScore();
        b.Property(e => e.ClampedQValue).HasOptionalScore();
        b.Property(e => e.Explanation).HasMaxLength(2000);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.AdvisorySession)
            .WithMany(e => e.Recommendations)
            .HasForeignKey(e => e.AdvisorySessionId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RL_Recommendation_Session");

        b.HasOne(e => e.StateDefinition)
            .WithMany()
            .HasForeignKey(e => e.StateDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_Recommendation_State");

        b.HasOne(e => e.RecommendationStatus)
            .WithMany()
            .HasForeignKey(e => e.RecommendationStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_Recommendation_Status");

        b.HasOne(e => e.OriginalActionDefinition)
            .WithMany()
            .HasForeignKey(e => e.OriginalActionDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_Recommendation_OriginalAction");

        b.HasOne(e => e.ClampedActionDefinition)
            .WithMany()
            .HasForeignKey(e => e.ClampedActionDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RL_Recommendation_ClampedAction");

        b.HasIndex(e => new { e.AdvisorySessionId, e.RecommendedAtUtc })
            .HasDatabaseName("IX_RL_Recommendation_Session_Time");
    }
}
```

16.9 Foreign-Key Mapping

The table below lists the main relationships extracted from the schema catalogue. Internal relationships remain inside `ReinforcementLearning`. Passport relationships cross to the plant, twin, instrumentation and security sectors.

FROM TABLE	FK COLUMN	TARGET	KIND
<code>ReinforcementLearning.EnvironmentModel</code>	<code>UnitId</code>	<code>ReactorFleet.Unit</code>	passport
<code>ReinforcementLearning.EnvironmentModel</code>	<code>TwinModelId</code>	<code>DigitalTwin.TwinModel</code>	passport
<code>ReinforcementLearning.StateSpace</code>	<code>StateSpaceTypeId</code>	<code>ReinforcementLearning.StateSpaceType</code>	internal
<code>ReinforcementLearning.StateDefinition</code>	<code>StateSpaceId</code>	<code>ReinforcementLearning.StateSpace</code>	internal
<code>ReinforcementLearning.ActionSpace</code>	<code>ActionSpaceTypeId</code>	<code>ReinforcementLearning.ActionSpaceType</code>	internal
<code>ReinforcementLearning.ActionDefinition</code>	<code>ActionSpaceId</code>	<code>ReinforcementLearning.ActionSpace</code>	internal
<code>ReinforcementLearning.RewardFunction</code>	<code>RewardFunctionTypeId</code>	<code>ReinforcementLearning.RewardFunctionType</code>	internal
<code>ReinforcementLearning.Experiment</code>	<code>UnitId</code>	<code>ReactorFleet.Unit</code>	passport
<code>ReinforcementLearning.TrainingRun</code>	<code>ExperimentId</code>	<code>ReinforcementLearning.Experiment</code>	internal
<code>ReinforcementLearning.TrainingRun</code>	<code>EnvironmentModelId</code>	<code>ReinforcementLearning.EnvironmentModel</code>	internal
<code>ReinforcementLearning.Episode</code>	<code>TrainingRunId</code>	<code>ReinforcementLearning.TrainingRun</code>	internal
<code>ReinforcementLearning.EpisodeStep</code>	<code>EpisodeId</code>	<code>ReinforcementLearning.Episode</code>	internal
<code>ReinforcementLearning.EpisodeStep</code>	<code>StateDefinitionId</code>	<code>ReinforcementLearning.StateDefinition</code>	internal
<code>ReinforcementLearning.EpisodeStep</code>	<code>ActionDefinitionId</code>	<code>ReinforcementLearning.ActionDefinition</code>	internal
<code>ReinforcementLearning.QTable</code>	<code>TrainingRunId</code>	<code>ReinforcementLearning.TrainingRun</code>	internal
<code>ReinforcementLearning.QTable</code>	<code>StateSpaceId</code>	<code>ReinforcementLearning.StateSpace</code>	internal
<code>ReinforcementLearning.QTable</code>	<code>ActionSpaceId</code>	<code>ReinforcementLearning.ActionSpace</code>	internal
<code>ReinforcementLearning.QTableEntry</code>	<code>QTableId</code>	<code>ReinforcementLearning.QTable</code>	internal
<code>ReinforcementLearning.QTableEntry</code>	<code>StateDefinitionId</code>	<code>ReinforcementLearning.StateDefinition</code>	internal
<code>ReinforcementLearning.QTableEntry</code>	<code>ActionDefinitionId</code>	<code>ReinforcementLearning.ActionDefinition</code>	internal
<code>ReinforcementLearning.ModelArtifact</code>	<code>TrainingRunId</code>	<code>ReinforcementLearning.TrainingRun</code>	internal
<code>ReinforcementLearning.TrainingRunAuditTrail</code>	<code>TrainingRunId</code>	<code>ReinforcementLearning.TrainingRun</code>	internal

FROM TABLE	FK COLUMN	TARGET	KIND
ReinforcementLearning.Policy	QTableId	ReinforcementLearning.QTable	internal
ReinforcementLearning.Policy	PolicyStatusId	ReinforcementLearning.PolicyStatus	internal
ReinforcementLearning.PolicyEntry	PolicyId	ReinforcementLearning.Policy	internal
ReinforcementLearning.PolicyEntry	StateDefinitionId	ReinforcementLearning.StateDefinition	internal
ReinforcementLearning.PolicyEntry	BestActionDefinitionId	ReinforcementLearning.ActionDefinition	internal
ReinforcementLearning.PolicyEvaluation	PolicyId	ReinforcementLearning.Policy	internal
ReinforcementLearning.PolicyEvaluation	EnvironmentModelId	ReinforcementLearning.EnvironmentModel	internal
ReinforcementLearning.PolicyEvaluationMetric	PolicyEvaluationId	ReinforcementLearning.PolicyEvaluation	internal
ReinforcementLearning.PolicyEvaluationMetric	EvaluationMetricTypeId	ReinforcementLearning.EvaluationMetricType	internal
ReinforcementLearning.PolicyDeployment	PolicyId	ReinforcementLearning.Policy	internal
ReinforcementLearning.PolicyDeployment	AdvisoryModeId	ReinforcementLearning.AdvisoryMode	internal
ReinforcementLearning.SafetyClamp	UnitId	ReactorFleet.Unit	passport
ReinforcementLearning.SafetyClampRule	SafetyClampId	ReinforcementLearning.SafetyClamp	internal
ReinforcementLearning.SafetyClampRule	SafetyClampTypeId	ReinforcementLearning.SafetyClampType	internal
ReinforcementLearning.AdvisorySession	PolicyDeploymentId	ReinforcementLearning.PolicyDeployment	internal
ReinforcementLearning.AdvisorySession	UnitId	ReactorFleet.Unit	passport
ReinforcementLearning.AdvisoryRecommendation	AdvisorySessionId	ReinforcementLearning.AdvisorySession	internal
ReinforcementLearning.AdvisoryRecommendation	StateDefinitionId	ReinforcementLearning.StateDefinition	internal
ReinforcementLearning.AdvisoryRecommendationReason	AdvisoryRecommendationId	ReinforcementLearning.AdvisoryRecommendation	internal

16.10 Migration and Verification Notes

1. Create lookup tables first.

Algorithm, state-space, action-space, policy, advisory and clamp lookups must exist before the model and training tables reference them.

2. Freeze state/action indices.

Once a training run has produced a Q-table, state/action indices become part of the model contract. Add new spaces instead of reusing old indices.

3. Keep training append-heavy.

Episode and step data are history. Avoid cascade deletes except inside the training-run aggregate where reproducibility remains intact.

4. Keep deployment advisory.

Policy deployments and recommendations store advice. No EF Core relationship should point from recommendations into a command path.

```
// In NexusContext.OnModelCreating:
modelBuilder.ApplyConfigurationsFromAssembly(typeof(NexusContext).Assembly);

// Verification after migration:
SELECT COUNT(*) AS rl_tables
FROM sys.tables t
JOIN sys.schemas s ON s.schema_id = t.schema_id
WHERE s.name = 'ReinforcementLearning';

SELECT i.name, OBJECT_SCHEMA_NAME(i.object_id) AS [schema], OBJECT_NAME(i.object_id) AS [table]
FROM sys.indexes i
WHERE i.name LIKE 'UQ_RL_%' OR i.name LIKE 'IX_RL_%'
ORDER BY [table], i.name;

SELECT fk.name, OBJECT_SCHEMA_NAME(fk.parent_object_id) AS [from_schema],
       OBJECT_NAME(fk.parent_object_id) AS [from_table],
       OBJECT_SCHEMA_NAME(fk.referenced_object_id) AS [to_schema],
       OBJECT_NAME(fk.referenced_object_id) AS [to_table]
FROM sys.foreign_keys fk
WHERE OBJECT_SCHEMA_NAME(fk.parent_object_id) = 'ReinforcementLearning'
ORDER BY [from_table], fk.name;
```

16.11 What Programmers Should Notice

- `ApplyConfigurationsFromAssembly` scales because every entity has one file and one responsibility.
- Named indexes such as `UQ_RL_QTableEntry_State_Action` replace opaque migration-generated names.
- The Q-table is not a blob. It is first-class relational data with state/action foreign keys.
- Safety clamps are modelled as data so the advisory layer can be audited after the fact.
- Cross-schema relationships use `DeleteBehavior.Restrict` because training records should not delete plant, twin or user truth.

Honest boundary. These mappings make an interpretable learning agent persistent and reviewable. They do not make the agent operationally certified, and they do not grant it control authority. The schema stores a policy and its advice; it does not operate a plant.

16.12 Configuration File Index

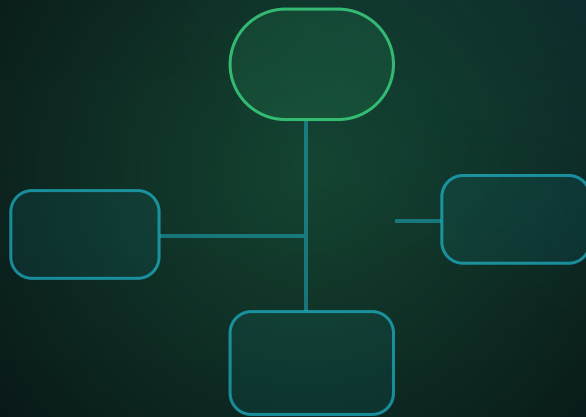
LearningAlgorithmConfiguration.cs Lookup
EnvironmentModelTypeConfiguration.cs Lookup
TrainingRunStatusConfiguration.cs Lookup
EpisodeStatusConfiguration.cs Lookup
StateSpaceTypeConfiguration.cs Lookup
ActionSpaceTypeConfiguration.cs Lookup
RewardFunctionTypeConfiguration.cs Lookup
PolicyStatusConfiguration.cs Lookup
AdvisoryModeConfiguration.cs Lookup
RecommendationStatusConfiguration.cs Lookup
SafetyClampTypeConfiguration.cs Lookup
EvaluationMetricTypeConfiguration.cs Lookup
EnvironmentModelConfiguration.cs Model
StateSpaceConfiguration.cs Model
StateDefinitionConfiguration.cs Model
ActionSpaceConfiguration.cs Model
ActionDefinitionConfiguration.cs Model
RewardFunctionConfiguration.cs Model
HyperparameterSetConfiguration.cs Model

ExperimentConfiguration.cs Model
TrainingRunConfiguration.cs Training
EpisodeConfiguration.cs Training
EpisodeStepConfiguration.cs Training
QTableConfiguration.cs Training
QTableEntryConfiguration.cs Training
ModelArtifactConfiguration.cs Training
TrainingRunAuditTrailConfiguration.cs Training
PolicyConfiguration.cs Policy
PolicyEntryConfiguration.cs Policy
PolicyEvaluationConfiguration.cs Policy
PolicyEvaluationMetricConfiguration.cs Policy
PolicyDeploymentConfiguration.cs Policy
SafetyClampConfiguration.cs Advisory
SafetyClampRuleConfiguration.cs Advisory
AdvisorySessionConfiguration.cs Advisory
AdvisoryRecommendationConfiguration.cs Advisory
AdvisoryRecommendationReasonConfiguration.cs Advisory

FROM ENTITY TO CONTEXT

Chapter 17 - Robotics Configurations

Mapping robots, missions, telemetry, docking, readiness and operational links.



SCHEMA CONFIGURATION COMPANION

Robotics as mapped code, not hidden conventions.

Each table receives an explicit configuration class with named keys, indexes, precision, defaults and passport relationships.

Chapter 17 - Robotics Configurations

The EF Core configuration atlas for the NEXUS-1 Robotics schema: robot fleet, payloads, sensors, missions, routes, telemetry, docking, readiness gates and integration links.

This chapter continues the same pattern as the earlier schema chapters: every substantive table in the Robotics sector gets a visible mapping strategy, and the code keeps the `DbContext` clean by using `IEntityTypeConfiguration<TEntity>`.

Standing rule: the model builder does not become a dumping ground. The Robotics sector is mapped by small, named files that can be reviewed, tested and migrated independently.

What this chapter adds

Robotics is the first sector where moving assets, telemetry streams and mission execution meet in one bounded context. The EF Core mappings therefore need three habits at once: current-state rows for the fleet, append-only rows for telemetry and mission events, and restrictive passports into the plant, instrumentation, maintenance and event schemas.

Fleet state

Robots, models, payloads, capabilities, work envelopes and communication links are mostly current-state records. They use unique codes, row versions and stable lookup FKs.

Mission execution

Missions, tasks, routes, waypoints and assignments are hierarchical. The mission owns its child rows, while robots and lookup rows are referenced with restrict behavior.

Operational evidence

Telemetry, location history and mission events are append-only evidence. They are indexed by robot, mission and UTC timestamp for playback, diagnosis and audit.

Professional habit. The chapter keeps every index and foreign key name explicit. This avoids unstable generated names and makes migrations readable during code review.

Configuration file catalogue

The Robotics schema contains **41** mapped tables in this chapter. Each table is represented by one configuration file. Lookup mappings may inherit from a common base, but they still remain separate files so the atlas is navigable.

Lookup 15 configurations RobotType, RobotStatus, RobotCapabilityType, PayloadType, MissionType ...	Fleet 8 configurations RobotModel, Robot, RobotCapability, RobotCapabilityAssignment, RobotPayload ...	Telemetry 3 configurations RobotTelemetry, RobotLocationHistory, RobotHealthSnapshot
Docking 2 configurations DockingStation, ChargingSession	Mission 7 configurations Mission, MissionRobotAssignment, MissionTask, MissionRoute, MissionRouteWaypoint ...	Readiness 4 configurations MissionChecklist, MissionChecklistItem, MissionReadinessAssessment, MissionReadinessItem
Integration 2 configurations RobotMaintenanceLink, MissionOperationalEventLink		

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	Robotics.RobotType	RobotTypeConfiguration.cs	Robot category: crawler, drone, manipulator, rover, inspection arm or support vehicle.
Lookup	Robotics.RobotStatus	RobotStatusConfiguration.cs	Operational state of a robot: available, charging, deployed, faulted, maintenance or retired.
Lookup	Robotics.RobotCapabilityType	RobotCapabilityTypeConfiguration.cs	Capability catalogue: visual inspection, radiation survey, thermal imaging, sample pickup, valve manipulation.
Lookup	Robotics.PayloadType	PayloadTypeConfiguration.cs	Payload category: camera, dosimeter, thermal sensor, manipulator, sample container, LiDAR.
Lookup	Robotics.MissionType	MissionTypeConfiguration.cs	Mission category: inspection, radiation survey, patrol, delivery, recovery, training drill.
Lookup	Robotics.MissionStatus	MissionStatusConfiguration.cs	Mission lifecycle: planned, approved, active, paused, completed, aborted, cancelled.
Lookup	Robotics.MissionPriority	MissionPriorityConfiguration.cs	Priority tier used by dispatch and readiness views.
Lookup	Robotics.MissionTaskStatus	MissionTaskStatusConfiguration.cs	Task lifecycle inside a mission: pending, active, completed, skipped, failed.
Lookup	Robotics.RouteType	RouteTypeConfiguration.cs	Route family: predefined, operator drawn, autonomous, return-to-base, emergency extraction.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	Robotics.WaypointType	WaypointTypeConfiguration.cs	Waypoint purpose: start, transit, inspection, sample, hold, charging, exit.
Lookup	Robotics.BatteryStatus	BatteryStatusConfiguration.cs	Battery status used by fleet cards and readiness gates.
Lookup	Robotics.CommunicationStatus	CommunicationStatusConfiguration.cs	Radio/network condition: nominal, degraded, lost, local-only, simulated.
Lookup	Robotics.ReadinessStatus	ReadinessStatusConfiguration.cs	Readiness assessment outcome: ready, conditional, blocked, expired, unknown.
Lookup	Robotics.DockingStationStatus	DockingStationStatusConfiguration.cs	Docking station state: available, occupied, charging, faulted, isolated.
Lookup	Robotics.TelemetryQuality	TelemetryQualityConfiguration.cs	Quality of robot telemetry samples: good, estimated, stale, lost, simulated.
Fleet	Robotics.RobotModel	RobotModelConfiguration.cs	Manufacturer/model record with physical limits, battery capacity and navigation profile.
Fleet	Robotics.Robot	RobotConfiguration.cs	One physical or simulated robotic asset in the plant demonstrator.
Fleet	Robotics.RobotCapability	RobotCapabilityConfiguration.cs	Governed capability definition, optionally measured in an engineering unit.
Fleet	Robotics.RobotCapabilityAssignment	RobotCapabilityAssignmentConfiguration.cs	Which robot can perform which capability, with rating and validation date.
Fleet	Robotics.RobotPayload	RobotPayloadConfiguration.cs	Payload mounted on a robot: camera, detector, manipulator or sample tool.
Fleet	Robotics.RobotSensorBinding	RobotSensorBindingConfiguration.cs	Binds robot-mounted sensors to the Instrumentation signal catalogue.
Fleet	Robotics.RobotCommunicationLink	RobotCommunicationLinkConfiguration.cs	Current or configured communication channel for a robot.
Fleet	Robotics.RobotWorkEnvelope	RobotWorkEnvelopeConfiguration.cs	Where the robot is authorised to operate inside a unit or location.
Telemetry	Robotics.RobotTelemetry	RobotTelemetryConfiguration.cs	High-volume telemetry metric: battery %, dose rate, temperature, speed, link quality.
Telemetry	Robotics.RobotLocationHistory	RobotLocationHistoryConfiguration.cs	Timestamped location trace for missions and playback.
Telemetry	Robotics.RobotHealthSnapshot	RobotHealthSnapshotConfiguration.cs	Periodic health summary used by mission-readiness screens.
Docking	Robotics.DockingStation	DockingStationConfiguration.cs	Charging/docking point in the demonstrator plant.
Docking	Robotics.ChargingSession	ChargingSessionConfiguration.cs	Charging interval, energy delivered and battery state before/after.
Mission	Robotics.Mission	MissionConfiguration.cs	Mission header: purpose, scope, priority, requested user, target unit and schedule.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Mission	<code>Robotics.MissionRobotAssignment</code>	<code>MissionRobotAssignmentConfiguration.cs</code>	Robot assigned to a mission with role and planned start/end.
Mission	<code>Robotics.MissionTask</code>	<code>MissionTaskConfiguration.cs</code>	Task list inside a mission: inspect, measure, capture image, sample, return.
Mission	<code>Robotics.MissionRoute</code>	<code>MissionRouteConfiguration.cs</code>	Route plan for a mission.
Mission	<code>Robotics.MissionRouteWaypoint</code>	<code>MissionRouteWaypointConfiguration.cs</code>	One ordered waypoint in a mission route.
Mission	<code>Robotics.MissionPayloadUsage</code>	<code>MissionPayloadUsageConfiguration.cs</code>	Payload used in a mission and the operational purpose for which it was used.
Mission	<code>Robotics.MissionEvent</code>	<code>MissionEventConfiguration.cs</code>	Append-only mission timeline: dispatch, waypoint reached, measurement captured, fault, return.
Readiness	<code>Robotics.MissionChecklist</code>	<code>MissionChecklistConfiguration.cs</code>	Checklist template for a mission type.
Readiness	<code>Robotics.MissionChecklistItem</code>	<code>MissionChecklistItemConfiguration.cs</code>	One checklist rule, such as battery threshold or required capability.
Readiness	<code>Robotics.MissionReadinessAssessment</code>	<code>MissionReadinessAssessmentConfiguration.cs</code>	Readiness assessment for a mission before dispatch.
Readiness	<code>Robotics.MissionReadinessItem</code>	<code>MissionReadinessItemConfiguration.cs</code>	One pass/fail/conditional readiness check.
Integration	<code>Robotics.RobotMaintenanceLink</code>	<code>RobotMaintenanceLinkConfiguration.cs</code>	Links robot faults or service requirements to Maintenance work orders.
Integration	<code>Robotics.MissionOperationalEventLink</code>	<code>MissionOperationalEventLinkConfiguration.cs</code>	Connects a robotic mission to an EventManagement operational event.

Folder structure

The folder structure follows the database schema and then splits by purpose. This keeps robotic fleet state separate from mission execution and high-volume telemetry.

```
Nexus.Infrastructure/  
  Persistence/  
    Configurations/  
      Robotics/  
        Lookups/  
          RobotTypeConfiguration.cs  
          RobotStatusConfiguration.cs  
          MissionStatusConfiguration.cs  
          TelemetryQualityConfiguration.cs  
        Fleet/  
          RobotModelConfiguration.cs  
          RobotConfiguration.cs  
          RobotCapabilityConfiguration.cs  
          RobotCapabilityAssignmentConfiguration.cs  
          RobotPayloadConfiguration.cs  
          RobotSensorBindingConfiguration.cs  
          RobotCommunicationLinkConfiguration.cs  
          RobotWorkEnvelopeConfiguration.cs  
        Telemetry/  
          RobotTelemetryConfiguration.cs  
          RobotLocationHistoryConfiguration.cs  
          RobotHealthSnapshotConfiguration.cs  
      Docking/  
        DockingStationConfiguration.cs  
        ChargingSessionConfiguration.cs  
      Missions/  
        MissionConfiguration.cs  
        MissionRobotAssignmentConfiguration.cs  
        MissionTaskConfiguration.cs  
        MissionRouteConfiguration.cs  
        MissionRouteWaypointConfiguration.cs  
        MissionPayloadUsageConfiguration.cs  
        MissionEventConfiguration.cs  
      Readiness/  
        MissionChecklistConfiguration.cs  
        MissionChecklistItemConfiguration.cs  
        MissionReadinessAssessmentConfiguration.cs  
        MissionReadinessItemConfiguration.cs  
      Integration/  
        RobotMaintenanceLinkConfiguration.cs  
        MissionOperationalEventLinkConfiguration.cs
```

Figure 17.1 - Robotics mapping dependency diagram

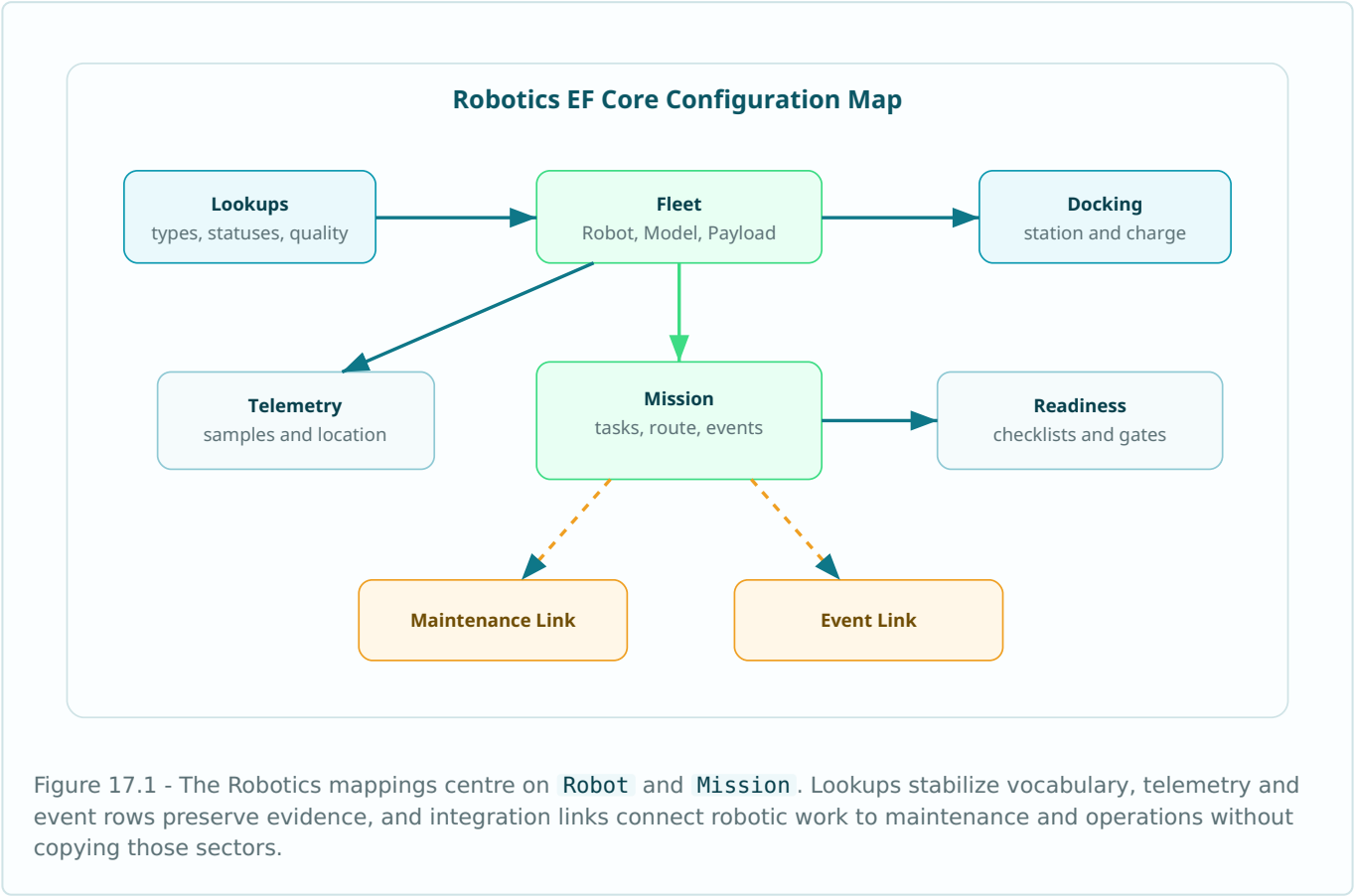
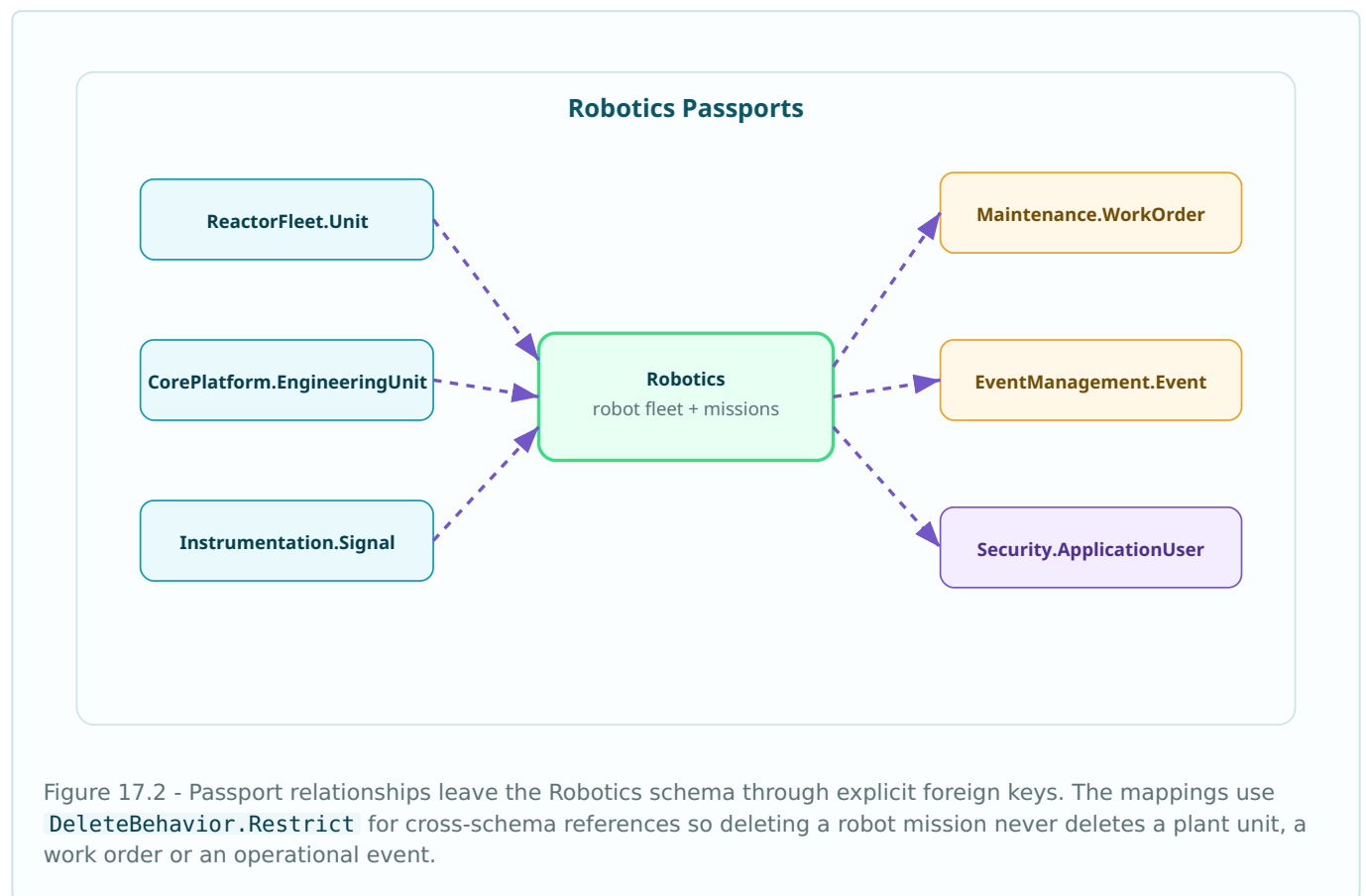


Figure 17.2 - Cross-schema passport map



Shared mapping code

Every lookup table in Robotics has the same basic shape: code, name, description, display order, active flag, audit timestamps and row version. The shared base keeps the convention in one place but still allows each lookup to name its table, key and unique index explicitly.

```
namespace Nexus.Infrastructure.Persistence.Configurations.Robotics;

internal static partial class NexusSql
{
    public const string Robotics = "Robotics";
    public const string ReactorFleet = "ReactorFleet";
    public const string CorePlatform = "CorePlatform";
    public const string Instrumentation = "Instrumentation";
    public const string Maintenance = "Maintenance";
    public const string EventManagement = "EventManagement";
    public const string Security = "Security";
    public const string UtcNow = "SYSUTCDATETIME()";
}

public abstract class RoboticsLookupConfiguration<TEntity>
    : IEntityConfiguration<TEntity>
    where TEntity : RoboticsLookupEntity
{
    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.Property(e => e.Code)
            .HasMaxLength(50)
            .IsRequired();

        b.Property(e => e.Name)
            .HasMaxLength(150)
            .IsRequired();

        b.Property(e => e.Description)
            .HasMaxLength(500);

        b.Property(e => e.DisplayOrder)
            .HasDefaultValue(0);

        b.Property(e => e.IsActive)
            .HasDefaultValue(true);

        b.Property(e => e.CreatedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.ModifiedAtUtc)
            .HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.Property(e => e.RowVersion)
            .IsRowVersion();
    }
}

internal static class RoboticsMappingExtensions
{
    public static PropertyBuilder<string> HasRobotCode(
        this PropertyBuilder<string> property, int length = 80) => property
        .HasMaxLength(length)
        .IsRequired();

    public static PropertyBuilder<DateTime> HasUtcDefault(
        this PropertyBuilder<DateTime> property) => property
        .HasColumnType("datetime2(7)")
        .HasDefaultValueSql(NexusSql.UtcNow);

    public static PropertyBuilder<decimal> HasMetricPrecision(
        this PropertyBuilder<decimal> property) => property
        .HasPrecision(18, 6);
}
```


Lookup examples

```
public sealed class RobotTypeConfiguration
    : RoboticsLookupConfiguration<RobotType>
{
    public override void Configure(EntityTypeBuilder<RobotType> b)
    {
        b.ToTable("RobotType", NexusSql.Robotics);
        b.HasKey(e => e.RobotTypeId);

        base.Configure(b);

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_RobotType_Code");
    }
}

public sealed class MissionStatusConfiguration
    : RoboticsLookupConfiguration<MissionStatus>
{
    public override void Configure(EntityTypeBuilder<MissionStatus> b)
    {
        b.ToTable("MissionStatus", NexusSql.Robotics);
        b.HasKey(e => e.MissionStatusId);

        base.Configure(b);

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_MissionStatus_Code");
    }
}
```

Fleet configuration files

The fleet mappings protect identity. A robot has a stable code, a model, a unit passport and a status. The robot row describes the asset; telemetry and mission evidence are separate tables.

RobotModelConfiguration.cs

```
public sealed class RobotModelConfiguration : IEntityTypeConfiguration<RobotModel>
{
    public void Configure(EntityTypeBuilder<RobotModel> b)
    {
        b.ToTable("RobotModel", NexusSql.Robotics);
        b.HasKey(e => e.RobotModelId);

        b.Property(e => e.Manufacturer)
            .HasMaxLength(120)
            .IsRequired();

        b.Property(e => e.ModelName)
            .HasMaxLength(120)
            .IsRequired();

        b.Property(e => e.ModelCode)
            .HasRobotCode(60);

        b.Property(e => e.MaxPayloadKg).HasPrecision(10, 3);
        b.Property(e => e.MaxSpeedMetersPerSecond).HasPrecision(10, 3);
        b.Property(e => e.BatteryCapacityWh).HasPrecision(12, 3);

        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.RobotType)
            .WithMany()
            .HasForeignKey(e => e.RobotTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RobotModel_RobotType");

        b.HasIndex(e => new { e.Manufacturer, e.ModelCode })
            .IsUnique()
            .HasDatabaseName("UQ_RobotModel_Manufacturer_ModelCode");
    }
}
```

RobotConfiguration.cs

```
public sealed class RobotConfiguration : IEntityTypeConfiguration<Robot>
{
    public void Configure(EntityTypeBuilder<Robot> b)
    {
        b.ToTable("Robot", NexusSql.Robotics);
        b.HasKey(e => e.RobotId);

        b.Property(e => e.Code).HasRobotCode(50);
        b.Property(e => e.DisplayName).HasMaxLength(160).IsRequired();
        b.Property(e => e.SerialNumber).HasMaxLength(120);
        b.Property(e => e.CommissionedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.RobotModel)
            .WithMany(m => m.Robots)
            .HasForeignKey(e => e.RobotModelId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Robot_RobotModel");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Robot_Unit");

        b.HasOne(e => e.RobotStatus)
            .WithMany()
            .HasForeignKey(e => e.RobotStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Robot_RobotStatus");

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_Robot_Code");

        b.HasIndex(e => new { e.UnitId, e.RobotStatusId })
            .HasDatabaseName("IX_Robot_Unit_Status");
    }
}
```

Capabilities, payloads and sensor bindings

Robotic capability is not a free-text list on the robot. Capabilities are governed records and assignments are separate rows. This lets a readiness gate ask whether a robot has a validated capability before a mission is dispatched.

```
public sealed class RobotCapabilityConfiguration
    : IEntityConfiguration<RobotCapability>
{
    public void Configure(EntityTypeBuilder<RobotCapability> b)
    {
        b.ToTable("RobotCapability", NexusSql.Robotics);
        b.HasKey(e => e.RobotCapabilityId);

        b.Property(e => e.Code).HasRobotCode(60);
        b.Property(e => e.Name).HasMaxLength(160).IsRequired();
        b.Property(e => e.MinimumRating).HasPrecision(18, 6);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();

        b.HasOne(e => e.RobotCapabilityType)
            .WithMany()
            .HasForeignKey(e => e.RobotCapabilityTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RobotCapability_RobotCapabilityType");

        b.HasOne(e => e.EngineeringUnit)
            .WithMany()
            .HasForeignKey(e => e.EngineeringUnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RobotCapability_EngineeringUnit");

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_RobotCapability_Code");
    }
}

public sealed class RobotCapabilityAssignmentConfiguration
    : IEntityConfiguration<RobotCapabilityAssignment>
{
    public void Configure(EntityTypeBuilder<RobotCapabilityAssignment> b)
    {
        b.ToTable("RobotCapabilityAssignment", NexusSql.Robotics);
        b.HasKey(e => e.RobotCapabilityAssignmentId);

        b.Property(e => e.Rating).HasPrecision(18, 6);
        b.Property(e => e.ValidatedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasOne(e => e.Robot)
            .WithMany(r => r.CapabilityAssignments)
            .HasForeignKey(e => e.RobotId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RobotCapabilityAssignment_Robot");

        b.HasOne(e => e.RobotCapability)
            .WithMany()
            .HasForeignKey(e => e.RobotCapabilityId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RobotCapabilityAssignment_RobotCapability");

        b.HasIndex(e => new { e.RobotId, e.RobotCapabilityId })
            .IsUnique()
            .HasDatabaseName("UQ_RobotCapabilityAssignment_Robot_Capability");
    }
}
```

Mapping rule. Capability and payload lookups are restrictive references. Assignment rows may be cascaded from the owning robot in the demonstrator, but operational systems normally archive instead of delete.

Telemetry and location history

Telemetry rows are high-volume facts. They use UTC timestamps, numeric precision and time-oriented indexes. The mapping avoids navigation-heavy loading by making playback queries explicit.

```
public sealed class RobotTelemetryConfiguration
    : IEntityTypeConfiguration<RobotTelemetry>
{
    public void Configure(EntityTypeBuilder<RobotTelemetry> b)
    {
        b.ToTable("RobotTelemetry", NexusSql.Robotics);
        b.HasKey(e => e.RobotTelemetryId);

        b.Property(e => e.RobotTelemetryId)
            .ValueGeneratedOnAdd();

        b.Property(e => e.MetricCode)
            .HasMaxLength(80)
            .IsRequired();

        b.Property(e => e.SampledAtUtc)
            .HasColumnType("datetime2(7)")
            .IsRequired();

        b.Property(e => e.ValueNumeric).HasPrecision(18, 6);
        b.Property(e => e.ValueText).HasMaxLength(500);

        b.HasOne(e => e.Robot)
            .WithMany(r => r.Telemetry)
            .HasForeignKey(e => e.RobotId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RobotTelemetry_Robot");

        b.HasOne(e => e.EngineeringUnit)
            .WithMany()
            .HasForeignKey(e => e.EngineeringUnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RobotTelemetry_EngineeringUnit");

        b.HasOne(e => e.TelemetryQuality)
            .WithMany()
            .HasForeignKey(e => e.TelemetryQualityId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RobotTelemetry_TelemetryQuality");

        b.HasIndex(e => new { e.RobotId, e.MetricCode, e.SampledAtUtc })
            .HasDatabaseName("IX_RobotTelemetry_Robot_Metric_Time");
    }
}

public sealed class RobotLocationHistoryConfiguration
    : IEntityTypeConfiguration<RobotLocationHistory>
{
    public void Configure(EntityTypeBuilder<RobotLocationHistory> b)
    {
        b.ToTable("RobotLocationHistory", NexusSql.Robotics);
        b.HasKey(e => e.RobotLocationHistoryId);

        b.Property(e => e.RecordedAtUtc).HasColumnType("datetime2(7)").IsRequired();
        b.Property(e => e.X).HasPrecision(18, 6);
        b.Property(e => e.Y).HasPrecision(18, 6);
        b.Property(e => e.Z).HasPrecision(18, 6);
        b.Property(e => e.HeadingDegrees).HasPrecision(9, 3);

        b.HasOne(e => e.Robot)
            .WithMany(r => r.LocationHistory)
            .HasForeignKey(e => e.RobotId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RobotLocationHistory_Robot");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RobotLocationHistory_Unit");

        b.HasIndex(e => new { e.RobotId, e.RecordedAtUtc })
            .HasDatabaseName("IX_RobotLocationHistory_Robot_Time");
    }
}
```

Docking and charging

Docking stations belong to a unit and may point at a plant location. Charging sessions reference both robot and station, but they do not own either side.

```
public sealed class DockingStationConfiguration
    : IEntityTypeConfiguration<DockingStation>
{
    public void Configure(EntityTypeBuilder<DockingStation> b)
    {
        b.ToTable("DockingStation", NexusSql.Robotics);
        b.HasKey(e => e.DockingStationId);

        b.Property(e => e.Code).HasRobotCode(60);
        b.Property(e => e.Name).HasMaxLength(160).IsRequired();
        b.Property(e => e.MaxChargingPowerKw).HasPrecision(10, 3);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DockingStation_Unit");

        b.HasOne(e => e.DockingStationStatus)
            .WithMany()
            .HasForeignKey(e => e.DockingStationStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DockingStation_DockingStationStatus");

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_DockingStation_Code");
    }
}

public sealed class ChargingSessionConfiguration
    : IEntityTypeConfiguration<ChargingSession>
{
    public void Configure(EntityTypeBuilder<ChargingSession> b)
    {
        b.ToTable("ChargingSession", NexusSql.Robotics);
        b.HasKey(e => e.ChargingSessionId);

        b.Property(e => e.StartedAtUtc).HasColumnType("datetime2(7)").IsRequired();
        b.Property(e => e.EndedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.EnergyDeliveredKwh).HasPrecision(12, 4);
        b.Property(e => e.BatteryPercentBefore).HasPrecision(5, 2);
        b.Property(e => e.BatteryPercentAfter).HasPrecision(5, 2);

        b.HasOne(e => e.Robot)
            .WithMany()
            .HasForeignKey(e => e.RobotId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ChargingSession_Robot");

        b.HasOne(e => e.DockingStation)
            .WithMany()
            .HasForeignKey(e => e.DockingStationId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ChargingSession_DockingStation");

        b.HasIndex(e => new { e.RobotId, e.StartedAtUtc })
            .HasDatabaseName("IX_ChargingSession_Robot_Start");
    }
}
```

Mission header and robot assignments

A mission is the aggregate root for route, tasks, payload usage and mission events. The robot is referenced restrictively: a mission assignment cannot delete the robot it used.

```
public sealed class MissionConfiguration : IEntityTypeConfiguration<Mission>
{
    public void Configure(EntityTypeBuilder<Mission> b)
    {
        b.ToTable("Mission", NexusSql.Robotics);
        b.HasKey(e => e.MissionId);

        b.Property(e => e.MissionCode).HasRobotCode(60);
        b.Property(e => e.Title).HasMaxLength(220).IsRequired();
        b.Property(e => e.Description).HasMaxLength(2000);
        b.Property(e => e.PlannedStartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.PlannedEndUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ActualStartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ActualEndUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();
        b.Property(e => e.ModifiedAtUtc).HasUtcDefault();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Mission_Unit");

        b.HasOne(e => e.MissionType)
            .WithMany()
            .HasForeignKey(e => e.MissionTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Mission_MissionType");

        b.HasOne(e => e.MissionStatus)
            .WithMany()
            .HasForeignKey(e => e.MissionStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Mission_MissionStatus");

        b.HasIndex(e => e.MissionCode)
            .IsUnique()
            .HasDatabaseName("UQ_Mission_MissionCode");

        b.HasIndex(e => new { e.UnitId, e.MissionStatusId, e.PlannedStartUtc })
            .HasDatabaseName("IX_Mission_Unit_Status_PlannedStart");
    }
}

public sealed class MissionRobotAssignmentConfiguration
    : IEntityTypeConfiguration<MissionRobotAssignment>
{
    public void Configure(EntityTypeBuilder<MissionRobotAssignment> b)
    {
        b.ToTable("MissionRobotAssignment", NexusSql.Robotics);
        b.HasKey(e => e.MissionRobotAssignmentId);

        b.Property(e => e.Role).HasMaxLength(80).IsRequired();
        b.Property(e => e.PlannedStartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.PlannedEndUtc).HasColumnType("datetime2(7)");

        b.HasOne(e => e.Mission)
            .WithMany(m => m.RobotAssignments)
            .HasForeignKey(e => e.MissionId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_MissionRobotAssignment_Mission");

        b.HasOne(e => e.Robot)
            .WithMany()
            .HasForeignKey(e => e.RobotId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MissionRobotAssignment_Robot");

        b.HasIndex(e => new { e.MissionId, e.RobotId })
            .IsUnique()
            .HasDatabaseName("UQ_MissionRobotAssignment_Mission_Robot");
    }
}
```

Routes and waypoints

The route owns its ordered waypoints. The unique index on `(MissionRouteId, SequenceNumber)` turns route order into a database invariant rather than a UI convention.

```
public sealed class MissionRouteConfiguration
    : IEntityTypeConfiguration<MissionRoute>
{
    public void Configure(EntityTypeBuilder<MissionRoute> b)
    {
        b.ToTable("MissionRoute", NexusSql.Robotics);
        b.HasKey(e => e.MissionRouteId);

        b.Property(e => e.RouteName).HasMaxLength(160).IsRequired();
        b.Property(e => e.EstimatedDistanceMeters).HasPrecision(12, 3);
        b.Property(e => e.EstimatedDurationSeconds).HasPrecision(12, 3);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasOne(e => e.Mission)
            .WithMany(m => m.Routes)
            .HasForeignKey(e => e.MissionId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_MissionRoute_Mission");

        b.HasOne(e => e.RouteType)
            .WithMany()
            .HasForeignKey(e => e.RouteTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MissionRoute_RouteType");

        b.HasIndex(e => new { e.MissionId, e.RouteName })
            .IsUnique()
            .HasDatabaseName("UQ_MissionRoute_Mission_Name");
    }
}

public sealed class MissionRouteWaypointConfiguration
    : IEntityTypeConfiguration<MissionRouteWaypoint>
{
    public void Configure(EntityTypeBuilder<MissionRouteWaypoint> b)
    {
        b.ToTable("MissionRouteWaypoint", NexusSql.Robotics);
        b.HasKey(e => e.MissionRouteWaypointId);

        b.Property(e => e.SequenceNumber).IsRequired();
        b.Property(e => e.X).HasPrecision(18, 6);
        b.Property(e => e.Y).HasPrecision(18, 6);
        b.Property(e => e.Z).HasPrecision(18, 6);
        b.Property(e => e.Instruction).HasMaxLength(500);

        b.HasOne(e => e.MissionRoute)
            .WithMany(r => r.Waypoints)
            .HasForeignKey(e => e.MissionRouteId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_MissionRouteWaypoint_MissionRoute");

        b.HasOne(e => e.WaypointType)
            .WithMany()
            .HasForeignKey(e => e.WaypointTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MissionRouteWaypoint_WaypointType");

        b.HasIndex(e => new { e.MissionRouteId, e.SequenceNumber })
            .IsUnique()
            .HasDatabaseName("UQ_MissionRouteWaypoint_Route_Sequence");
    }
}
```

Mission tasks and timeline events

Tasks describe planned work; mission events describe what actually happened. Events are append-only operational evidence and should be indexed for replay by mission and time.

```
public sealed class MissionTaskConfiguration : IEntityTypeConfiguration<MissionTask>
{
    public void Configure(EntityTypeBuilder<MissionTask> b)
    {
        b.ToTable("MissionTask", NexusSql.Robotics);
        b.HasKey(e => e.MissionTaskId);

        b.Property(e => e.TaskCode).HasRobotCode(80);
        b.Property(e => e.Title).HasMaxLength(220).IsRequired();
        b.Property(e => e.SequenceNumber).IsRequired();
        b.Property(e => e.Instruction).HasMaxLength(1000);
        b.Property(e => e.StartedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CompletedAtUtc).HasColumnType("datetime2(7)");

        b.HasOne(e => e.Mission)
            .WithMany(m => m.Tasks)
            .HasForeignKey(e => e.MissionId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_MissionTask_Mission");

        b.HasOne(e => e.MissionTaskStatus)
            .WithMany()
            .HasForeignKey(e => e.MissionTaskStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MissionTask_MissionTaskStatus");

        b.HasIndex(e => new { e.MissionId, e.SequenceNumber })
            .IsUnique()
            .HasDatabaseName("UQ_MissionTask_Mission_Sequence");
    }
}

public sealed class MissionEventConfiguration : IEntityTypeConfiguration<MissionEvent>
{
    public void Configure(EntityTypeBuilder<MissionEvent> b)
    {
        b.ToTable("MissionEvent", NexusSql.Robotics);
        b.HasKey(e => e.MissionEventId);

        b.Property(e => e.OccurredAtUtc).HasColumnType("datetime2(7)").IsRequired();
        b.Property(e => e.EventCode).HasMaxLength(80).IsRequired();
        b.Property(e => e.Summary).HasMaxLength(500).IsRequired();
        b.Property(e => e.PayloadJson).HasColumnType("nvarchar(max)");

        b.HasOne(e => e.Mission)
            .WithMany(m => m.Events)
            .HasForeignKey(e => e.MissionId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_MissionEvent_Mission");

        b.HasOne(e => e.Robot)
            .WithMany()
            .HasForeignKey(e => e.RobotId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MissionEvent_Robot");

        b.HasIndex(e => new { e.MissionId, e.OccurredAtUtc })
            .HasDatabaseName("IX_MissionEvent_Mission_Time");
    }
}
```


Readiness gates

Readiness maps the pre-dispatch checklist: is the battery high enough, is the communications path good, does the selected robot have the required capability, and is the target area allowed?

```
public sealed class MissionChecklistConfiguration
    : IEntityTypeConfiguration<MissionChecklist>
{
    public void Configure(EntityTypeBuilder<MissionChecklist> b)
    {
        b.ToTable("MissionChecklist", NexusSql.Robotics);
        b.HasKey(e => e.MissionChecklistId);

        b.Property(e => e.Code).HasRobotCode(80);
        b.Property(e => e.Name).HasMaxLength(180).IsRequired();
        b.Property(e => e.Version).HasMaxLength(40).IsRequired();
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasOne(e => e.MissionType)
            .WithMany()
            .HasForeignKey(e => e.MissionTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MissionChecklist_MissionType");

        b.HasIndex(e => new { e.Code, e.Version })
            .IsUnique()
            .HasDatabaseName("UQ_MissionChecklist_Code_Version");
    }
}

public sealed class MissionReadinessAssessmentConfiguration
    : IEntityTypeConfiguration<MissionReadinessAssessment>
{
    public void Configure(EntityTypeBuilder<MissionReadinessAssessment> b)
    {
        b.ToTable("MissionReadinessAssessment", NexusSql.Robotics);
        b.HasKey(e => e.MissionReadinessAssessmentId);

        b.Property(e => e.AssessedAtUtc).HasColumnType("datetime2(7)").IsRequired();
        b.Property(e => e.Summary).HasMaxLength(1000);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.Mission)
            .WithMany(m => m.ReadinessAssessments)
            .HasForeignKey(e => e.MissionId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_MissionReadinessAssessment_Mission");

        b.HasOne(e => e.ReadinessStatus)
            .WithMany()
            .HasForeignKey(e => e.ReadinessStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MissionReadinessAssessment_ReadinessStatus");

        b.HasIndex(e => new { e.MissionId, e.AssessedAtUtc })
            .HasDatabaseName("IX_MissionReadinessAssessment_Mission_Time");
    }
}
```

Integration link configurations

Robotics connects to Maintenance and EventManagement with link tables instead of copying work-order or event fields. These links keep the operational chain queryable without merging bounded contexts.

```
public sealed class RobotMaintenanceLinkConfiguration
    : IEntityTypeConfiguration<RobotMaintenanceLink>
{
    public void Configure(EntityTypeBuilder<RobotMaintenanceLink> b)
    {
        b.ToTable("RobotMaintenanceLink", NexusSql.Robotics);
        b.HasKey(e => e.RobotMaintenanceLinkId);

        b.Property(e => e.LinkReason).HasMaxLength(500).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasOne(e => e.Robot)
            .WithMany()
            .HasForeignKey(e => e.RobotId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RobotMaintenanceLink_Robot");

        b.HasOne(e => e.WorkOrder)
            .WithMany()
            .HasForeignKey(e => e.WorkOrderId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RobotMaintenanceLink_WorkOrder");

        b.HasIndex(e => new { e.RobotId, e.WorkOrderId })
            .IsUnique()
            .HasDatabaseName("UQ_RobotMaintenanceLink_Robot_WorkOrder");
    }
}

public sealed class MissionOperationalEventLinkConfiguration
    : IEntityTypeConfiguration<MissionOperationalEventLink>
{
    public void Configure(EntityTypeBuilder<MissionOperationalEventLink> b)
    {
        b.ToTable("MissionOperationalEventLink", NexusSql.Robotics);
        b.HasKey(e => e.MissionOperationalEventLinkId);

        b.Property(e => e.LinkReason).HasMaxLength(500).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasUtcDefault();

        b.HasOne(e => e.Mission)
            .WithMany()
            .HasForeignKey(e => e.MissionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MissionOperationalEventLink_Mission");

        b.HasOne(e => e.OperationalEvent)
            .WithMany()
            .HasForeignKey(e => e.OperationalEventId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MissionOperationalEventLink_OperationalEvent");

        b.HasIndex(e => new { e.MissionId, e.OperationalEventId })
            .IsUnique()
            .HasDatabaseName("UQ_MissionOperationalEventLink_Mission_Event");
    }
}
```

DbContext discovery

The context stays clean. The Robotics mapping files are discovered by assembly scanning in exactly the same way as the earlier sectors.

```
public sealed class NexusContext : DbContext
{
    public DbSet<Robot> Robots => Set<Robot>();
    public DbSet<RobotModel> RobotModels => Set<RobotModel>();
    public DbSet<RobotTelemetry> RobotTelemetry => Set<RobotTelemetry>();
    public DbSet<Mission> Missions => Set<Mission>();
    public DbSet<MissionEvent> MissionEvents => Set<MissionEvent>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(NexusContext).Assembly);
    }
}
```

Do not inline this sector in OnModelCreating. With more than forty Robotics mappings, inline configuration would hide the domain structure and make review of mission, telemetry and readiness rules much harder.

Foreign-key mapping

TABLE	FK PROPERTY	TARGET	KIND
Robotics.RobotModel	RobotTypeId	Robotics.RobotType	internal
Robotics.Robot	RobotModelId	Robotics.RobotModel	internal
Robotics.Robot	UnitId	ReactorFleet.Unit	passport
Robotics.Robot	RobotStatusId	Robotics.RobotStatus	internal
Robotics.RobotCapability	RobotCapabilityTypeId	Robotics.RobotCapabilityType	internal
Robotics.RobotCapability	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
Robotics.RobotCapabilityAssignment	RobotId	Robotics.Robot	internal
Robotics.RobotCapabilityAssignment	RobotCapabilityId	Robotics.RobotCapability	internal
Robotics.RobotPayload	RobotId	Robotics.Robot	internal
Robotics.RobotPayload	PayloadTypeId	Robotics.PayloadType	internal
Robotics.RobotSensorBinding	RobotId	Robotics.Robot	internal
Robotics.RobotSensorBinding	SignalId	Instrumentation.Signal	passport
Robotics.RobotCommunicationLink	RobotId	Robotics.Robot	internal
Robotics.RobotCommunicationLink	CommunicationStatusId	Robotics.CommunicationStatus	internal
Robotics.RobotWorkEnvelope	RobotId	Robotics.Robot	internal
Robotics.RobotWorkEnvelope	UnitId	ReactorFleet.Unit	passport
Robotics.RobotWorkEnvelope	EquipmentLocationId	ReactorFleet.EquipmentLocation	passport
Robotics.RobotTelemetry	RobotId	Robotics.Robot	internal
Robotics.RobotTelemetry	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
Robotics.RobotTelemetry	TelemetryQualityId	Robotics.TelemetryQuality	internal
Robotics.RobotLocationHistory	RobotId	Robotics.Robot	internal
Robotics.RobotLocationHistory	UnitId	ReactorFleet.Unit	passport
Robotics.RobotLocationHistory	EquipmentLocationId	ReactorFleet.EquipmentLocation	passport
Robotics.RobotHealthSnapshot	RobotId	Robotics.Robot	internal
Robotics.RobotHealthSnapshot	BatteryStatusId	Robotics.BatteryStatus	internal
Robotics.RobotHealthSnapshot	CommunicationStatusId	Robotics.CommunicationStatus	internal
Robotics.DockingStation	UnitId	ReactorFleet.Unit	passport
Robotics.DockingStation	EquipmentLocationId	ReactorFleet.EquipmentLocation	passport
Robotics.DockingStation	DockingStationStatusId	Robotics.DockingStationStatus	internal
Robotics.ChargingSession	RobotId	Robotics.Robot	internal
Robotics.ChargingSession	DockingStationId	Robotics.DockingStation	internal
Robotics.Mission	UnitId	ReactorFleet.Unit	passport
Robotics.Mission	MissionTypeId	Robotics.MissionType	internal
Robotics.Mission	MissionStatusId	Robotics.MissionStatus	internal

TABLE	FK PROPERTY	TARGET	KIND
Robotics.MissionRobotAssignment	MissionId	Robotics.Mission	internal
Robotics.MissionRobotAssignment	RobotId	Robotics.Robot	internal
Robotics.MissionTask	MissionId	Robotics.Mission	internal
Robotics.MissionTask	MissionTaskStatusId	Robotics.MissionTaskStatus	internal
Robotics.MissionRoute	MissionId	Robotics.Mission	internal
Robotics.MissionRoute	RouteTypeId	Robotics.RouteType	internal
Robotics.MissionRouteWaypoint	MissionRouteId	Robotics.MissionRoute	internal
Robotics.MissionRouteWaypoint	WaypointTypeId	Robotics.WaypointType	internal
Robotics.MissionPayloadUsage	MissionId	Robotics.Mission	internal
Robotics.MissionPayloadUsage	RobotPayloadId	Robotics.RobotPayload	internal
Robotics.MissionEvent	MissionId	Robotics.Mission	internal
Robotics.MissionEvent	RobotId	Robotics.Robot	internal
Robotics.MissionChecklist	MissionTypeId	Robotics.MissionType	internal
Robotics.MissionChecklistItem	MissionChecklistId	Robotics.MissionChecklist	internal
Robotics.MissionReadinessAssessment	MissionId	Robotics.Mission	internal
Robotics.MissionReadinessAssessment	ReadinessStatusId	Robotics.ReadinessStatus	internal
Robotics.MissionReadinessItem	MissionReadinessAssessmentId	Robotics.MissionReadinessAssessment	internal
Robotics.MissionReadinessItem	ReadinessStatusId	Robotics.ReadinessStatus	internal
Robotics.RobotMaintenanceLink	RobotId	Robotics.Robot	internal
Robotics.RobotMaintenanceLink	WorkOrderId	Maintenance.WorkOrder	passport
Robotics.MissionOperationalEventLink	MissionId	Robotics.Mission	internal
Robotics.MissionOperationalEventLink	OperationalEventId	EventManagement.OperationalEvent	passport

Migration and verification notes

Migration order

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.EnsureSchema(name: "Robotics");

    // Lookups first.
    // Fleet and docking next.
    // Mission headers before mission details.
    // Telemetry and append-only timelines last.
}

// Recommended delete behavior policy:
// - lookup and passport FKs: Restrict
// - mission child rows: Cascade from Mission
// - telemetry rows: Cascade from Robot only in disposable demonstrator builds;
//   operational installations usually archive instead of delete.
```

Metadata verification

```
-- Configuration classes discovered by assembly scan.
var entityTypes = context.Model.GetEntityTypes()
    .Where(e => e.GetSchema() == "Robotics")
    .Select(e => new { Table = e.GetTableName(), Schema = e.GetSchema() })
    .OrderBy(e => e.Table)
    .ToList();

// High-value assertions for the Robotics sector.
Assert.Contains(entityTypes, e => e.Table == "Robot" && e.Schema == "Robotics");
Assert.Contains(entityTypes, e => e.Table == "Mission" && e.Schema == "Robotics");

var robot = context.Model.FindEntityType(typeof(Robot));
Assert.NotNull(robot.FindIndex(robot.FindProperty(nameof(Robot.Code))));
Assert.Equal("Robotics", robot.GetSchema());
Assert.Equal("Robot", robot.GetTableName());

var telemetry = context.Model.FindEntityType(typeof(RobotTelemetry));
Assert.Equal("decimal(18,6)", telemetry
    .FindProperty(nameof(RobotTelemetry.ValueNumeric))!
    .GetColumnType());
```

Verification habit. The test does not need to connect to a plant. It checks the EF Core model metadata: table names, schemas, unique indexes, column types and foreign-key delete behavior.

Professional notes

Use explicit constraint names

A readable migration should contain `FK_Mission_Unit`, not a generated identifier nobody can recognise during a production incident.

Do not hide units

Any measured robotic value points at `CorePlatform.EngineeringUnit`. Do not store `"%"`, `"m/s"` or `"mSv/h"` as uncontrolled strings.

Separate planned from observed

`MissionTask` is planned work.
`MissionEvent` is observed evidence. The model remains audit-ready because those concepts do not collapse into one row.

Keep passports restrictive

Robotics may reference units, signals, work orders and operational events. It must never own their lifecycle.

Honest boundary

The Robotics schema in this atlas is a software-engineering reference for the NEXUS-1 demonstrator. It is not a certified robotics control system, not a navigation safety case and not a real plant integration. The EF Core mappings show how the data backbone can be made explicit and reviewable.

Index of Robotics configuration files

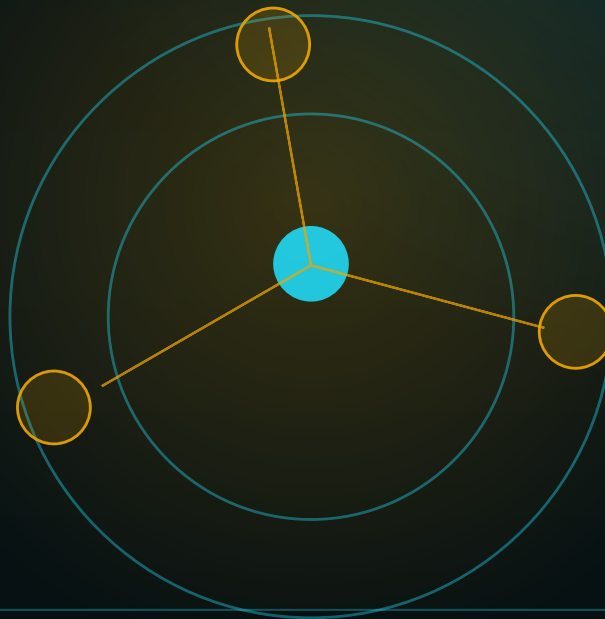
RobotTypeConfiguration.cs Lookup
RobotStatusConfiguration.cs Lookup
RobotCapabilityTypeConfiguration.cs Lookup
PayloadTypeConfiguration.cs Lookup
MissionTypeConfiguration.cs Lookup
MissionStatusConfiguration.cs Lookup
MissionPriorityConfiguration.cs Lookup
MissionTaskStatusConfiguration.cs Lookup
RouteTypeConfiguration.cs Lookup
WaypointTypeConfiguration.cs Lookup
BatteryStatusConfiguration.cs Lookup
CommunicationStatusConfiguration.cs Lookup
ReadinessStatusConfiguration.cs Lookup
DockingStationStatusConfiguration.cs Lookup
TelemetryQualityConfiguration.cs Lookup
RobotModelConfiguration.cs Fleet
RobotConfiguration.cs Fleet
RobotCapabilityConfiguration.cs Fleet
RobotCapabilityAssignmentConfiguration.cs Fleet
RobotPayloadConfiguration.cs Fleet
RobotSensorBindingConfiguration.cs Fleet
RobotCommunicationLinkConfiguration.cs Fleet

RobotWorkEnvelopeConfiguration.cs Fleet
RobotTelemetryConfiguration.cs Telemetry
RobotLocationHistoryConfiguration.cs Telemetry
RobotHealthSnapshotConfiguration.cs Telemetry
DockingStationConfiguration.cs Docking
ChargingSessionConfiguration.cs Docking
MissionConfiguration.cs Mission
MissionRobotAssignmentConfiguration.cs Mission
MissionTaskConfiguration.cs Mission
MissionRouteConfiguration.cs Mission
MissionRouteWaypointConfiguration.cs Mission
MissionPayloadUsageConfiguration.cs Mission
MissionEventConfiguration.cs Mission
MissionChecklistConfiguration.cs Readiness
MissionChecklistItemConfiguration.cs Readiness
MissionReadinessAssessmentConfiguration.cs Readiness
MissionReadinessItemConfiguration.cs Readiness
RobotMaintenanceLinkConfiguration.cs Integration
MissionOperationalEventLinkConfiguration.cs Integration

FROM ENTITY TO CONTEXT

Chapter 18 - RadiationMonitoring Configurations

The EF Core mapping atlas for zones, monitors, readings, dosimetry, surveys, permits and radiological evidence.



TECHNICAL REFERENCE COMPANION

Zone, dose, survey, permit and evidence mappings

SQL Server / EF Core configuration files for the NEXUS-1 RadiationMonitoring schema.

Chapter 18 - RadiationMonitoring Configurations

The EF Core configuration atlas for the NEXUS-1 RadiationMonitoring schema: controlled zones, fixed and portable monitors, time-series readings, dosimetry, surveys, environmental samples, radiation work permits and integration links.

This chapter continues the schema-by-schema configuration atlas. The goal is not to hide radiological data behind a large `DbContext`, but to make every mapping file explicit, reviewable and stable.

Standing rule: radiation data is operational evidence. Current-state entities use row versions; readings, samples and dose records are append-only facts indexed by time, unit, monitor, zone and person.

What this chapter adds

RadiationMonitoring is where physical plant context, personal dose, environmental evidence and work authorization meet. The EF Core mappings therefore need four habits: explicit passports into plant and people, precise numeric columns, time-oriented indexes, and restrictive delete behavior across schema boundaries.

Zones and monitors

Zones describe where radiological rules apply. Monitors bind those zones to instruments and signals without duplicating instrumentation metadata.

Dose and survey evidence

Dose readings, survey readings, contamination results and environmental results are timestamped facts, not mutable notes.

Permits and integration

Radiation work permits connect people, zones and dose records. Event and robot links preserve the operational chain without merging schemas.

Professional habit. Every measured value maps a decimal precision and an `EngineeringUnitId`. The unit is never a free-text string.

Configuration file catalogue

The RadiationMonitoring schema contains **46** mapped tables in this chapter. Each table has one configuration file. Lookup mappings share a base class, but every lookup still has its own named configuration class.

Lookup 17 configurations RadiationZoneType, RadiationZoneStatus, RadiationAreaClassification, MonitorType ...	Zone 3 configurations RadiationZone, RadiationZoneBoundaryPoint, RadiationZoneAccessRequirement	Monitor 4 configurations RadiationMonitor, RadiationMonitorSignalBinding, RadiationMonitorCalibration, RadiationReading
Dosimetry 6 configurations Dosimeter, PersonDosimeterAssignment, PersonDoseReading, DoseLimit ...	Survey 4 configurations RadiologicalSurvey, RadiologicalSurveyReading, ContaminationSample, ContaminationSampleResult	Environment 3 configurations EnvironmentalSamplingLocation, EnvironmentalSample, EnvironmentalSampleResult
Inventory 1 configurations IsotopeInventory	Snapshot 1 configurations ZoneDoseSnapshot	Permit 4 configurations RadiationWorkPermit, RadiationWorkPermitZone, RadiationWorkPermitPerson, RadiationWorkPermitDoseRecord
Integration 2 configurations RadiationEventLink, RadiationRobotMissionLink	Reporting 1 configurations RadiationTrendSummary	

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	RadiationMonitoring.RadiationZoneType	RadiationZoneTypeConfiguration.cs	Zone category: controlled area, supervised area, exclusion zone, hot cell, waste store or environmental station.
Lookup	RadiationMonitoring.RadiationZoneStatus	RadiationZoneStatusConfiguration.cs	Operational status of a radiation zone: open, restricted, locked out, decontamination, retired.
Lookup	RadiationMonitoring.RadiationAreaClassification	RadiationAreaClassificationConfiguration.cs	Classification label used by access rules and dose planning.
Lookup	RadiationMonitoring.MonitorType	MonitorTypeConfiguration.cs	Fixed, portable, airborne, area gamma, neutron, contamination or effluent monitor.
Lookup	RadiationMonitoring.MonitorStatus	MonitorStatusConfiguration.cs	Monitor lifecycle: in service, calibration due, faulted, isolated, retired.
Lookup	RadiationMonitoring.MeasurementType	MeasurementTypeConfiguration.cs	Measured quantity: dose rate, contamination, count rate, airborne activity, release concentration.
Lookup	RadiationMonitoring.MeasurementQuality	MeasurementQualityConfiguration.cs	Quality flag for radiation readings: good, estimated, suspect, stale, simulated.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Lookup	RadiationMonitoring.DoseType	DoseTypeConfiguration.cs	Dose category: whole body, extremity, lens, committed dose, operational dose.
Lookup	RadiationMonitoring.DosimeterType	DosimeterTypeConfiguration.cs	Dosimeter category: electronic, TLD, OSL, neutron badge, area badge.
Lookup	RadiationMonitoring.DosimeterStatus	DosimeterStatusConfiguration.cs	Dosimeter lifecycle: available, assigned, returned, reading pending, lost, retired.
Lookup	RadiationMonitoring.SurveyType	SurveyTypeConfiguration.cs	Survey category: routine, post-work, release, contamination, emergency, drill.
Lookup	RadiationMonitoring.SurveyStatus	SurveyStatusConfiguration.cs	Survey lifecycle: planned, active, completed, reviewed, rejected, cancelled.
Lookup	RadiationMonitoring.ContaminationType	ContaminationTypeConfiguration.cs	Contamination family: alpha, beta, gamma, removable, fixed, airborne.
Lookup	RadiationMonitoring.SampleType	SampleTypeConfiguration.cs	Sample category: smear, air filter, water, soil, effluent, surface.
Lookup	RadiationMonitoring.LimitType	LimitTypeConfiguration.cs	Limit family: administrative, regulatory, job-specific, warning, alarm, stop-work.
Lookup	RadiationMonitoring.AlertStatus	AlertStatusConfiguration.cs	Alert lifecycle: open, acknowledged, under review, closed, false positive.
Lookup	RadiationMonitoring.ExposureCategory	ExposureCategoryConfiguration.cs	Worker or task exposure category used by planning and reporting.
Zone	RadiationMonitoring.RadiationZone	RadiationZoneConfiguration.cs	One controlled or monitored radiological area in a unit, plant location or environmental station.
Zone	RadiationMonitoring.RadiationZoneBoundaryPoint	RadiationZoneBoundaryPointConfiguration.cs	Ordered geometry points for drawing a zone boundary in 2-D maps or 3-D plant views.
Zone	RadiationMonitoring.RadiationZoneAccessRequirement	RadiationZoneAccessRequirementConfiguration.cs	Dose or survey conditions that must be satisfied before entering a zone.
Monitor	RadiationMonitoring.RadiationMonitor	RadiationMonitorConfiguration.cs	One installed or portable radiation monitor.
Monitor	RadiationMonitoring.RadiationMonitorSignalBinding	RadiationMonitorSignalBindingConfiguration.cs	Binds a radiation monitor channel to the common Instrumentation signal catalogue.
Monitor	RadiationMonitoring.RadiationMonitorCalibration	RadiationMonitorCalibrationConfiguration.cs	Calibration record, due date and pass/fail result for a monitor.
Monitor	RadiationMonitoring.RadiationReading	RadiationReadingConfiguration.cs	High-volume timestamped radiation measurement from a monitor.
Dosimetry	RadiationMonitoring.Dosimeter	DosimeterConfiguration.cs	One dosimetry device or badge.
Dosimetry	RadiationMonitoring.PersonDosimeterAssignment	PersonDosimeterAssignmentConfiguration.cs	Assignment of a dosimeter to a person for a period.
Dosimetry	RadiationMonitoring.PersonDoseReading	PersonDoseReadingConfiguration.cs	Dose reading collected from a person assignment.
Dosimetry	RadiationMonitoring.DoseLimit	DoseLimitConfiguration.cs	Administrative or regulatory dose threshold.
Dosimetry	RadiationMonitoring.DoseLimitAssignment	DoseLimitAssignmentConfiguration.cs	Applies a dose limit to a person or exposure category over a validity window.

GROUP	TABLE	CONFIGURATION FILE	PURPOSE
Dosimetry	RadiationMonitoring.DoseAlert	DoseAlertConfiguration.cs	Alert raised when a personal dose reading approaches or exceeds a limit.
Survey	RadiationMonitoring.RadiologicalSurvey	RadiologicalSurveyConfiguration.cs	Survey header: purpose, location, status, assigned person and review status.
Survey	RadiationMonitoring.RadiologicalSurveyReading	RadiologicalSurveyReadingConfiguration.cs	Individual point or grid reading captured during a radiological survey.
Survey	RadiationMonitoring.ContaminationSample	ContaminationSampleConfiguration.cs	Sample collected during a survey.
Survey	RadiationMonitoring.ContaminationSampleResult	ContaminationSampleResultConfiguration.cs	Analytical result for a contamination sample.
Environment	RadiationMonitoring.EnvironmentalSamplingLocation	EnvironmentalSamplingLocationConfiguration.cs	External or boundary location used for environmental radiological samples.
Environment	RadiationMonitoring.EnvironmentalSample	EnvironmentalSampleConfiguration.cs	Environmental sample header.
Environment	RadiationMonitoring.EnvironmentalSampleResult	EnvironmentalSampleResultConfiguration.cs	Measured environmental sample result.
Inventory	RadiationMonitoring.IsotopeInventory	IsotopeInventoryConfiguration.cs	Simple demonstrator-grade isotope inventory record for sources, waste or sample context.
Snapshot	RadiationMonitoring.ZoneDoseSnapshot	ZoneDoseSnapshotConfiguration.cs	Periodic aggregate zone dose-rate snapshot for trends and dashboards.
Permit	RadiationMonitoring.RadiationWorkPermit	RadiationWorkPermitConfiguration.cs	Radiation work permit for planned access or work in controlled areas.
Permit	RadiationMonitoring.RadiationWorkPermitZone	RadiationWorkPermitZoneConfiguration.cs	Zones covered by a radiation work permit.
Permit	RadiationMonitoring.RadiationWorkPermitPerson	RadiationWorkPermitPersonConfiguration.cs	People authorized on a radiation work permit.
Permit	RadiationMonitoring.RadiationWorkPermitDoseRecord	RadiationWorkPermitDoseRecordConfiguration.cs	Dose accumulated against a permit participant.
Integration	RadiationMonitoring.RadiationEventLink	RadiationEventLinkConfiguration.cs	Links radiological data to an operational event or incident.
Integration	RadiationMonitoring.RadiationRobotMissionLink	RadiationRobotMissionLinkConfiguration.cs	Links a robotics mission to radiological monitoring records.
Reporting	RadiationMonitoring.RadiationTrendSummary	RadiationTrendSummaryConfiguration.cs	Precomputed trend bucket for dashboards and reporting.

Folder structure

The folder structure separates static vocabulary, monitored places, measuring devices, evidence rows and authorization records.

```
Nexus.Infrastructure/  
  Persistence/  
    Configurations/  
      RadiationMonitoring/  
        Lookups/  
          RadiationZoneTypeConfiguration.cs  
          RadiationZoneStatusConfiguration.cs  
          MonitorTypeConfiguration.cs  
          MeasurementQualityConfiguration.cs  
          DoseTypeConfiguration.cs  
          SurveyTypeConfiguration.cs  
          AlertStatusConfiguration.cs  
        Zones/  
          RadiationZoneConfiguration.cs  
          RadiationZoneBoundaryPointConfiguration.cs  
          RadiationZoneAccessRequirementConfiguration.cs  
        Monitors/  
          RadiationMonitorConfiguration.cs  
          RadiationMonitorSignalBindingConfiguration.cs  
          RadiationMonitorCalibrationConfiguration.cs  
          RadiationReadingConfiguration.cs  
        Dosimetry/  
          DosimeterConfiguration.cs  
          PersonDosimeterAssignmentConfiguration.cs  
          PersonDoseReadingConfiguration.cs  
          DoseLimitConfiguration.cs  
          DoseAlertConfiguration.cs  
        Surveys/  
          RadiologicalSurveyConfiguration.cs  
          RadiologicalSurveyReadingConfiguration.cs  
          ContaminationSampleConfiguration.cs  
          ContaminationSampleResultConfiguration.cs  
        Environment/  
          EnvironmentalSamplingLocationConfiguration.cs  
          EnvironmentalSampleConfiguration.cs  
          EnvironmentalSampleResultConfiguration.cs  
        Permits/  
          RadiationWorkPermitConfiguration.cs  
          RadiationWorkPermitZoneConfiguration.cs  
          RadiationWorkPermitPersonConfiguration.cs  
          RadiationWorkPermitDoseRecordConfiguration.cs  
        Integration/  
          RadiationEventLinkConfiguration.cs  
          RadiationRobotMissionLinkConfiguration.cs  
        Reporting/  
          RadiationTrendSummaryConfiguration.cs
```

Figure 18.1 - RadiationMonitoring mapping dependency diagram

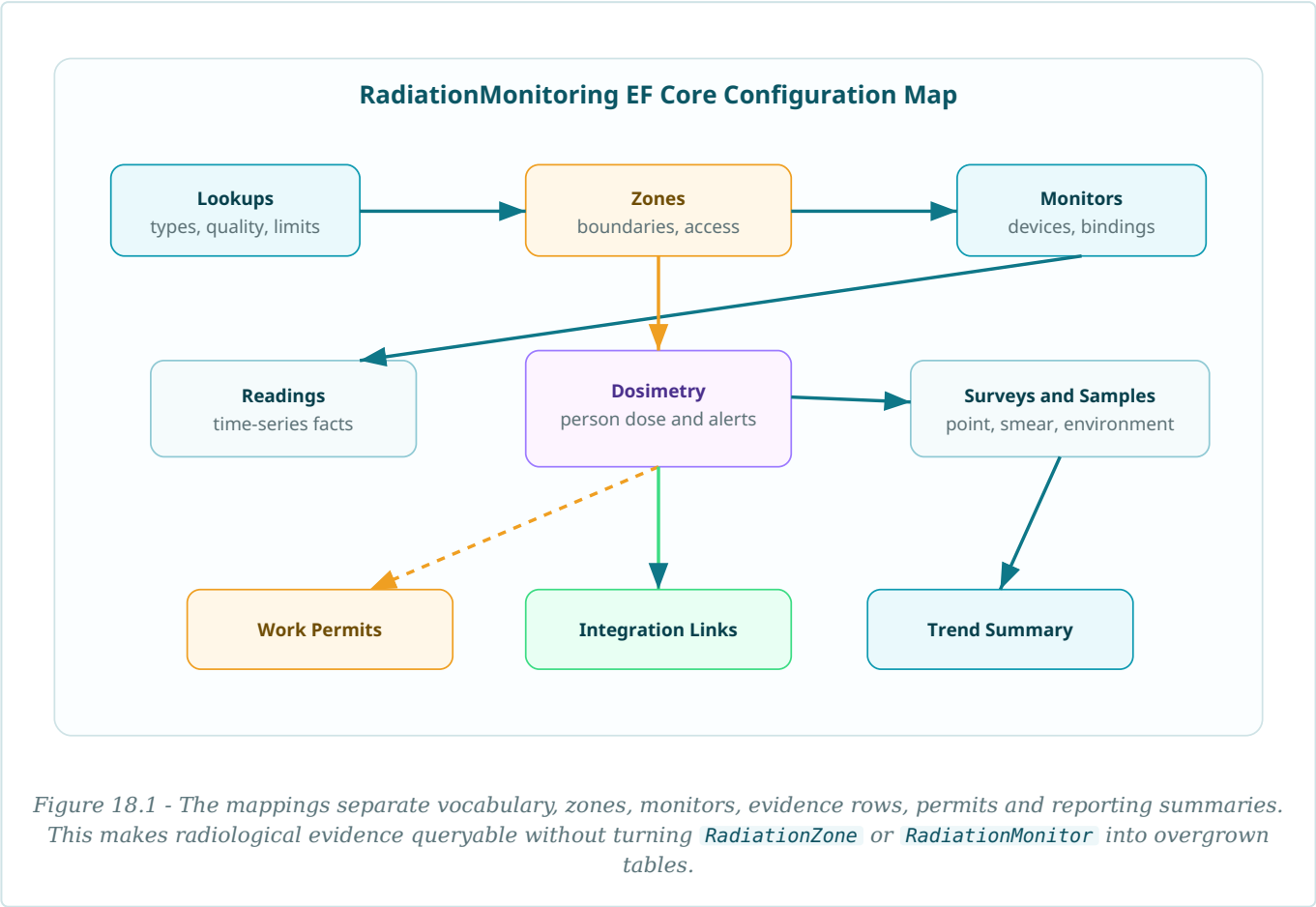
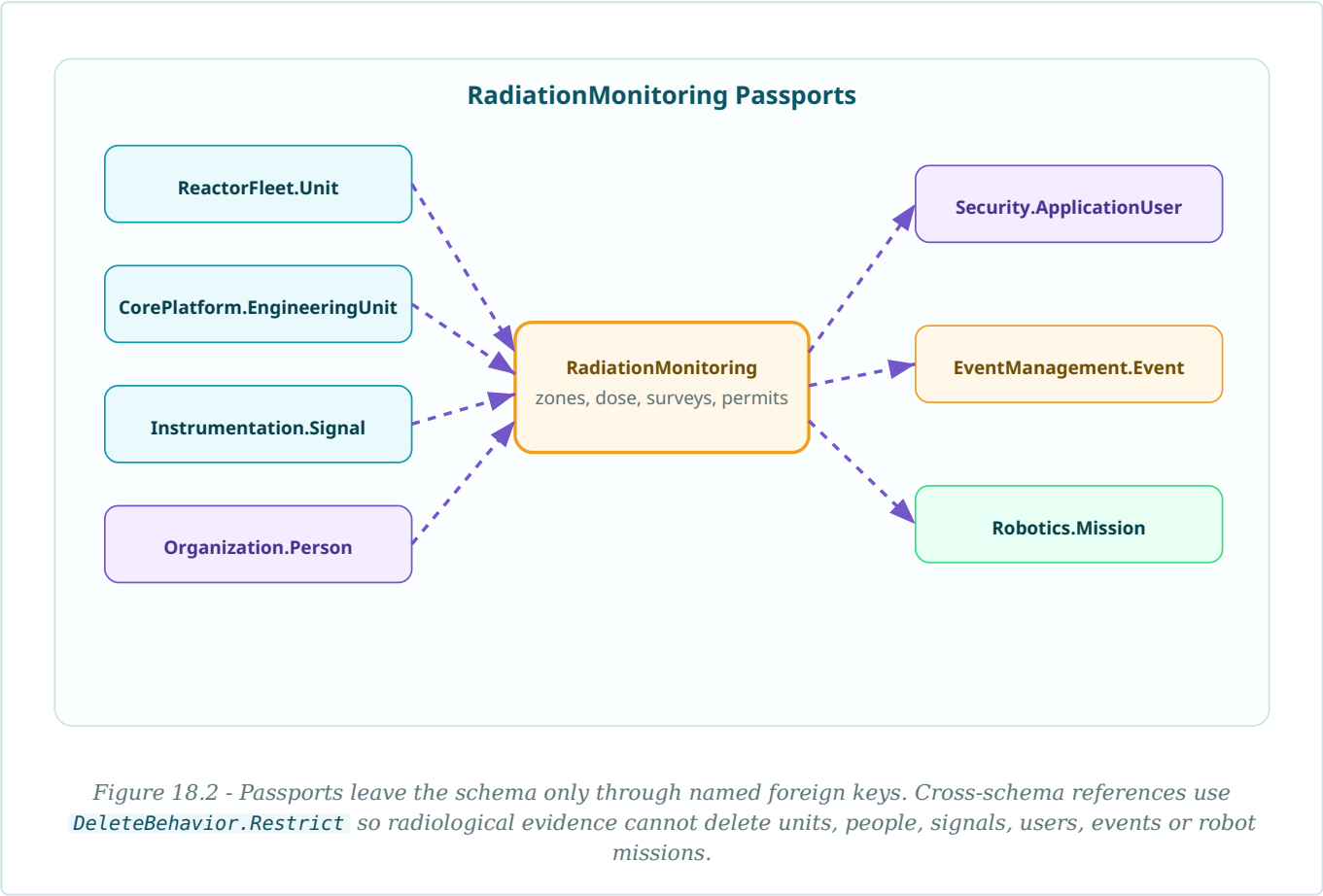


Figure 18.2 - Cross-schema passport map



Shared lookup mapping

Radiation vocabulary is broad, but the shape is stable. The base class keeps code, display order, active flag, timestamps and row version consistent across all lookup tables.

```
public abstract class RadiationMonitoringLookupConfiguration<TEntity>
    : IEntityTypeConfiguration<TEntity>
    where TEntity : RadiationMonitoringLookup
{
    protected abstract string TableName { get; }
    protected abstract string KeyName { get; }
    protected virtual string CodeIndexName => $"UQ_{TableName}_Code";

    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.ToTable(TableName, "RadiationMonitoring");
        b.HasKey(e => e.Id).HasName($"PK_{TableName}");
        b.Property(e => e.Id).HasColumnName(KeyName);
        b.Property(e => e.Code).HasMaxLength(40).IsRequired();
        b.Property(e => e.Name).HasMaxLength(160).IsRequired();
        b.Property(e => e.Description).HasMaxLength(800);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();
        b.HasIndex(e => e.Code).IsUnique().HasDatabaseName(CodeIndexName);
    }
}
```

Lookup configuration examples

```
public sealed class MonitorTypeConfiguration
    : RadiationMonitoringLookupConfiguration<MonitorType>
{
    protected override string TableName => "MonitorType";
    protected override string KeyName => "MonitorTypeId";
}

public sealed class DoseTypeConfiguration
    : RadiationMonitoringLookupConfiguration<DoseType>
{
    protected override string TableName => "DoseType";
    protected override string KeyName => "DoseTypeId";
}

public sealed class MeasurementQualityConfiguration
    : RadiationMonitoringLookupConfiguration<MeasurementQuality>
{
    protected override string TableName => "MeasurementQuality";
    protected override string KeyName => "MeasurementQualityId";
}
```

Zone configuration files

Zones are the controlled areas where access rules, survey requirements and dose planning become concrete rows.

```
public sealed class RadiationZoneConfiguration : IEntityTypeConfiguration<RadiationZone>
{
    public void Configure(EntityTypeBuilder<RadiationZone> b)
    {
        b.ToTable("RadiationZone", "RadiationMonitoring");
        b.HasKey(e => e.RadiationZoneId).HasName("PK_RadiationZone");

        b.Property(e => e.Code).HasMaxLength(40).IsRequired();
        b.Property(e => e.Name).HasMaxLength(200).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1000);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique().HasDatabaseName("UQ_RadiationZone_Code");
        b.HasIndex(e => new { e.UnitId, e.RadiationZoneTypeId })
            .HasDatabaseName("IX_RadiationZone_Unit_Type");
        b.HasIndex(e => e.RadiationZoneStatusId)
            .HasDatabaseName("IX_RadiationZone_Status");

        b.HasOne(e => e.Unit)
            .WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationZone_Unit");

        b.HasOne(e => e.EquipmentLocation)
            .WithMany()
            .HasForeignKey(e => e.EquipmentLocationId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationZone_EquipmentLocation");

        b.HasOne(e => e.ZoneType)
            .WithMany()
            .HasForeignKey(e => e.RadiationZoneTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationZone_RadiationZoneType");
    }
}

public sealed class RadiationZoneBoundaryPointConfiguration
    : IEntityTypeConfiguration<RadiationZoneBoundaryPoint>
{
    public void Configure(EntityTypeBuilder<RadiationZoneBoundaryPoint> b)
    {
        b.ToTable("RadiationZoneBoundaryPoint", "RadiationMonitoring");
        b.HasKey(e => e.RadiationZoneBoundaryPointId)
            .HasName("PK_RadiationZoneBoundaryPoint");
        b.Property(e => e.X).HasPrecision(18, 6);
        b.Property(e => e.Y).HasPrecision(18, 6);
        b.Property(e => e.Z).HasPrecision(18, 6);
        b.HasIndex(e => new { e.RadiationZoneId, e.SequenceNumber })
            .IsUnique()
            .HasDatabaseName("UQ_RadiationZoneBoundaryPoint_Zone_Seq");
        b.HasOne(e => e.RadiationZone).WithMany(e => e.BoundaryPoints)
            .HasForeignKey(e => e.RadiationZoneId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RadiationZoneBoundaryPoint_RadiationZone");
    }
}
```

Monitor configuration files

Monitors bind radiological devices to plant units, zones and instrumentation signals. A monitor owns its local bindings but not the signal catalogue.

```
public sealed class RadiationMonitorConfiguration : IEntityTypeConfiguration<RadiationMonitor>
{
    public void Configure(EntityTypeBuilder<RadiationMonitor> b)
    {
        b.ToTable("RadiationMonitor", "RadiationMonitoring");
        b.HasKey(e => e.RadiationMonitorId).HasName("PK_RadiationMonitor");
        b.Property(e => e.Code).HasMaxLength(60).IsRequired();
        b.Property(e => e.Tag).HasMaxLength(80);
        b.Property(e => e.SerialNumber).HasMaxLength(80);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique().HasDatabaseName("UQ_RadiationMonitor_Code");
        b.HasIndex(e => new { e.UnitId, e.RadiationZoneId })
            .HasDatabaseName("IX_RadiationMonitor_Unit_Zone");
        b.HasIndex(e => e.MonitorStatusId).HasDatabaseName("IX_RadiationMonitor_Status");

        b.HasOne(e => e.Unit).WithMany()
            .HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationMonitor_Unit");
        b.HasOne(e => e.RadiationZone).WithMany(e => e.Monitors)
            .HasForeignKey(e => e.RadiationZoneId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationMonitor_RadiationZone");
        b.HasOne(e => e.MonitorType).WithMany()
            .HasForeignKey(e => e.MonitorTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationMonitor_MonitorType");
    }
}

public sealed class RadiationMonitorSignalBindingConfiguration
    : IEntityTypeConfiguration<RadiationMonitorSignalBinding>
{
    public void Configure(EntityTypeBuilder<RadiationMonitorSignalBinding> b)
    {
        b.ToTable("RadiationMonitorSignalBinding", "RadiationMonitoring");
        b.HasKey(e => e.RadiationMonitorSignalBindingId)
            .HasName("PK_RadiationMonitorSignalBinding");
        b.Property(e => e.ChannelName).HasMaxLength(80).IsRequired();
        b.HasIndex(e => new { e.RadiationMonitorId, e.SignalId })
            .IsUnique()
            .HasDatabaseName("UQ_RadiationMonitorSignalBinding_Monitor_Signal");
        b.HasOne(e => e.RadiationMonitor).WithMany(e => e.SignalBindings)
            .HasForeignKey(e => e.RadiationMonitorId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RadiationMonitorSignalBinding_Monitor");
        b.HasOne(e => e.Signal).WithMany()
            .HasForeignKey(e => e.SignalId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationMonitorSignalBinding_Signal");
    }
}
```

Radiation readings

Radiation readings are high-volume facts. The mapping favours time-series access paths and explicit units.

```
public sealed class RadiationReadingConfiguration : IEntityTypeConfiguration<RadiationReading>
{
    public void Configure(EntityTypeBuilder<RadiationReading> b)
    {
        b.ToTable("RadiationReading", "RadiationMonitoring");
        b.HasKey(e => e.RadiationReadingId).HasName("PK_RadiationReading");
        b.Property(e => e.MeasuredAtUtc).HasColumnType("datetime2(7)").IsRequired();
        b.Property(e => e.Value).HasPrecision(18, 8).IsRequired();
        b.Property(e => e.BackgroundValue).HasPrecision(18, 8);
        b.Property(e => e.Comment).HasMaxLength(800);

        b.HasIndex(e => new { e.RadiationMonitorId, e.MeasuredAtUtc })
            .HasDatabaseName("IX_RadiationReading_Monitor_Time");
        b.HasIndex(e => new { e.MeasurementTypeId, e.MeasuredAtUtc })
            .HasDatabaseName("IX_RadiationReading_Type_Time");
        b.HasIndex(e => e.MeasurementQualityId)
            .HasDatabaseName("IX_RadiationReading_Quality");

        b.HasOne(e => e.RadiationMonitor).WithMany(e => e.Readings)
            .HasForeignKey(e => e.RadiationMonitorId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationReading_RadiationMonitor");
        b.HasOne(e => e.EngineeringUnit).WithMany()
            .HasForeignKey(e => e.EngineeringUnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationReading_EngineeringUnit");
        b.HasOne(e => e.MeasurementQuality).WithMany()
            .HasForeignKey(e => e.MeasurementQualityId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationReading_MeasurementQuality");
    }
}
```

Dosimetry configuration files

Dosimetry maps devices, assignments and readings separately, so a person, a dosimeter and a reading do not collapse into one mutable row.

```
public sealed class DosimeterConfiguration : IEntityTypeConfiguration<Dosimeter>
{
    public void Configure(EntityTypeBuilder<Dosimeter> b)
    {
        b.ToTable("Dosimeter", "RadiationMonitoring");
        b.HasKey(e => e.DosimeterId).HasName("PK_Dosimeter");
        b.Property(e => e.SerialNumber).HasMaxLength(80).IsRequired();
        b.Property(e => e.CalibrationDueUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RowVersion).IsRowVersion();
        b.HasIndex(e => e.SerialNumber).IsUnique().HasDatabaseName("UQ_Dosimeter_SerialNumber");
        b.HasOne(e => e.DosimeterType).WithMany()
            .HasForeignKey(e => e.DosimeterTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Dosimeter_DosimeterType");
        b.HasOne(e => e.DosimeterStatus).WithMany()
            .HasForeignKey(e => e.DosimeterStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Dosimeter_DosimeterStatus");
    }
}

public sealed class PersonDosimeterAssignmentConfiguration
    : IEntityTypeConfiguration<PersonDosimeterAssignment>
{
    public void Configure(EntityTypeBuilder<PersonDosimeterAssignment> b)
    {
        b.ToTable("PersonDosimeterAssignment", "RadiationMonitoring");
        b.HasKey(e => e.PersonDosimeterAssignmentId)
            .HasName("PK_PersonDosimeterAssignment");
        b.Property(e => e.AssignedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ReturnedAtUtc).HasColumnType("datetime2(7)");
        b.HasIndex(e => new { e.PersonId, e.AssignedAtUtc })
            .HasDatabaseName("IX_PersonDosimeterAssignment_Person_Time");
        b.HasIndex(e => new { e.DosimeterId, e.AssignedAtUtc })
            .HasDatabaseName("IX_PersonDosimeterAssignment_Dosimeter_Time");
        b.HasOne(e => e.Person).WithMany()
            .HasForeignKey(e => e.PersonId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_PersonDosimeterAssignment_Person");
        b.HasOne(e => e.Dosimeter).WithMany(e => e.Assignments)
            .HasForeignKey(e => e.DosimeterId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_PersonDosimeterAssignment_Dosimeter");
    }
}

public sealed class PersonDoseReadingConfiguration : IEntityTypeConfiguration<PersonDoseReading>
{
    public void Configure(EntityTypeBuilder<PersonDoseReading> b)
    {
        b.ToTable("PersonDoseReading", "RadiationMonitoring");
        b.HasKey(e => e.PersonDoseReadingId).HasName("PK_PersonDoseReading");
        b.Property(e => e.ReadAtUtc).HasColumnType("datetime2(7)").IsRequired();
        b.Property(e => e.Value).HasPrecision(18, 8).IsRequired();
        b.HasIndex(e => new { e.PersonDosimeterAssignmentId, e.ReadAtUtc })
            .HasDatabaseName("IX_PersonDoseReading_Assignment_Time");
        b.HasOne(e => e.Assignment).WithMany(e => e.Readings)
            .HasForeignKey(e => e.PersonDosimeterAssignmentId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_PersonDoseReading_Assignment");
        b.HasOne(e => e.EngineeringUnit).WithMany()
            .HasForeignKey(e => e.EngineeringUnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_PersonDoseReading_EngineeringUnit");
    }
}
```

Dose limits and alerts

Dose limits turn administrative or regulatory thresholds into rows. Alerts point to the reading that triggered them and keep their own lifecycle.

```
public sealed class DoseLimitConfiguration : IEntityTypeConfiguration<DoseLimit>
{
    public void Configure(EntityTypeBuilder<DoseLimit> b)
    {
        b.ToTable("DoseLimit", "RadiationMonitoring");
        b.HasKey(e => e.DoseLimitId).HasName("PK_DoseLimit");
        b.Property(e => e.Code).HasMaxLength(60).IsRequired();
        b.Property(e => e.LimitValue).HasPrecision(18, 8).IsRequired();
        b.Property(e => e.ValidFromUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ValidToUtc).HasColumnType("datetime2(7)");
        b.HasIndex(e => e.Code).IsUnique().HasDatabaseName("UQ_DoseLimit_Code");
        b.HasIndex(e => new { e.DoseTypeId, e.LimitTypeId }).HasDatabaseName("IX_DoseLimit_Type_Limit");
        b.HasOne(e => e.EngineeringUnit).WithMany().HasForeignKey(e => e.EngineeringUnitId)
            .OnDelete(DeleteBehavior.Restrict).HasConstraintName("FK_DoseLimit_EngineeringUnit");
    }
}

public sealed class DoseAlertConfiguration : IEntityTypeConfiguration<DoseAlert>
{
    public void Configure(EntityTypeBuilder<DoseAlert> b)
    {
        b.ToTable("DoseAlert", "RadiationMonitoring");
        b.HasKey(e => e.DoseAlertId).HasName("PK_DoseAlert");
        b.Property(e => e.RaisedAtUtc).HasColumnType("datetime2(7)").IsRequired();
        b.Property(e => e.AcknowledgedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ClosedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.Message).HasMaxLength(1000).IsRequired();
        b.HasIndex(e => new { e.AlertStatusId, e.RaisedAtUtc }).HasDatabaseName("IX_DoseAlert_Status_Time");
        b.HasOne(e => e.DoseLimit).WithMany().HasForeignKey(e => e.DoseLimitId)
            .OnDelete(DeleteBehavior.Restrict).HasConstraintName("FK_DoseAlert_DoseLimit");
        b.HasOne(e => e.PersonDoseReading).WithMany().HasForeignKey(e => e.PersonDoseReadingId)
            .OnDelete(DeleteBehavior.Restrict).HasConstraintName("FK_DoseAlert_PersonDoseReading");
    }
}
```

Surveys and contamination samples

Survey headers, point readings and sample results are separate entities. The database can replay where a reading was taken and which analytical result came back later.

```
public sealed class RadiologicalSurveyConfiguration : IEntityTypeConfiguration<RadiologicalSurvey>
{
    public void Configure(EntityTypeBuilder<RadiologicalSurvey> b)
    {
        b.ToTable("RadiologicalSurvey", "RadiationMonitoring");
        b.HasKey(e => e.RadiologicalSurveyId).HasName("PK_RadiologicalSurvey");
        b.Property(e => e.SurveyNumber).HasMaxLength(60).IsRequired();
        b.Property(e => e.StartedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CompletedAtUtc).HasColumnType("datetime2(7)");
        b.HasIndex(e => e.SurveyNumber).IsUnique().HasDatabaseName("UQ_RadiologicalSurvey_Number");
        b.HasIndex(e => new { e.UnitId, e.RadiationZoneId, e.StartedAtUtc })
            .HasDatabaseName("IX_RadiologicalSurvey_Unit_Zone_Time");
        b.HasOne(e => e.RadiationZone).WithMany(e => e.Surveys)
            .HasForeignKey(e => e.RadiationZoneId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiologicalSurvey_RadiationZone");
    }
}

public sealed class RadiologicalSurveyReadingConfiguration
    : IEntityTypeConfiguration<RadiologicalSurveyReading>
{
    public void Configure(EntityTypeBuilder<RadiologicalSurveyReading> b)
    {
        b.ToTable("RadiologicalSurveyReading", "RadiationMonitoring");
        b.HasKey(e => e.RadiologicalSurveyReadingId)
            .HasName("PK_RadiologicalSurveyReading");
        b.Property(e => e.PointCode).HasMaxLength(60).IsRequired();
        b.Property(e => e.Value).HasPrecision(18, 8).IsRequired();
        b.HasIndex(e => new { e.RadiologicalSurveyId, e.PointCode })
            .IsUnique()
            .HasDatabaseName("UQ_RadiologicalSurveyReading_Survey_Point");
        b.HasOne(e => e.RadiologicalSurvey).WithMany(e => e.Readings)
            .HasForeignKey(e => e.RadiologicalSurveyId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RadiologicalSurveyReading_Survey");
        b.HasOne(e => e.EngineeringUnit).WithMany()
            .HasForeignKey(e => e.EngineeringUnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiologicalSurveyReading_EngineeringUnit");
    }
}

public sealed class ContaminationSampleResultConfiguration
    : IEntityTypeConfiguration<ContaminationSampleResult>
{
    public void Configure(EntityTypeBuilder<ContaminationSampleResult> b)
    {
        b.ToTable("ContaminationSampleResult", "RadiationMonitoring");
        b.HasKey(e => e.ContaminationSampleResultId)
            .HasName("PK_ContaminationSampleResult");
        b.Property(e => e.ActivityValue).HasPrecision(18, 8).IsRequired();
        b.Property(e => e.AnalysedAtUtc).HasColumnType("datetime2(7)");
        b.HasIndex(e => new { e.ContaminationSampleId, e.AnalysedAtUtc })
            .HasDatabaseName("IX_ContaminationSampleResult_Sample_Time");
    }
}
```


Environmental monitoring

Environmental locations are reference points outside the immediate plant unit. Their samples and results form a slower evidence stream for reporting and trend analysis.

```
public sealed class EnvironmentalSamplingLocationConfiguration
    : IEntityTypeConfiguration<EnvironmentalSamplingLocation>
{
    public void Configure(EntityTypeBuilder<EnvironmentalSamplingLocation> b)
    {
        b.ToTable("EnvironmentalSamplingLocation", "RadiationMonitoring");
        b.HasKey(e => e.EnvironmentalSamplingLocationId)
            .HasName("PK_EnvironmentalSamplingLocation");
        b.Property(e => e.Code).HasMaxLength(60).IsRequired();
        b.Property(e => e.Name).HasMaxLength(200).IsRequired();
        b.Property(e => e.Latitude).HasPrecision(10, 7);
        b.Property(e => e.Longitude).HasPrecision(10, 7);
        b.HasIndex(e => e.Code).IsUnique().HasDatabaseName("UQ_EnvironmentalSamplingLocation_Code");
        b.HasOne(e => e.Country).WithMany().HasForeignKey(e => e.CountryId)
            .onDelete(DeleteBehavior.Restrict).HasConstraintName("FK_EnvironmentalSamplingLocation_Country");
    }
}

public sealed class EnvironmentalSampleResultConfiguration
    : IEntityTypeConfiguration<EnvironmentalSampleResult>
{
    public void Configure(EntityTypeBuilder<EnvironmentalSampleResult> b)
    {
        b.ToTable("EnvironmentalSampleResult", "RadiationMonitoring");
        b.HasKey(e => e.EnvironmentalSampleResultId).HasName("PK_EnvironmentalSampleResult");
        b.Property(e => e.ResultValue).HasPrecision(18, 8).IsRequired();
        b.Property(e => e.AnalysedAtUtc).HasColumnType("datetime2(7)");
        b.HasIndex(e => new { e.EnvironmentalSampleId, e.AnalysedAtUtc })
            .HasDatabaseName("IX_EnvironmentalSampleResult_Sample_Time");
    }
}
```

Radiation work permits

Work permits connect planned access, authorized people, affected zones and dose records. They reference users and people restrictively, because approval history is evidence.

```
public sealed class RadiationWorkPermitConfiguration : IEntityTypeConfiguration<RadiationWorkPermit>
{
    public void Configure(EntityTypeBuilder<RadiationWorkPermit> b)
    {
        b.ToTable("RadiationWorkPermit", "RadiationMonitoring");
        b.HasKey(e => e.RadiationWorkPermitId).HasName("PK_RadiationWorkPermit");
        b.Property(e => e.PermitNumber).HasMaxLength(60).IsRequired();
        b.Property(e => e.ValidFromUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ValidToUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.WorkDescription).HasMaxLength(1200).IsRequired();
        b.HasIndex(e => e.PermitNumber).IsUnique().HasDatabaseName("UQ_RadiationWorkPermit_Number");
        b.HasIndex(e => new { e.UnitId, e.ValidFromUtc }).HasDatabaseName("IX_RadiationWorkPermit_Unit_Time");
        b.HasOne(e => e.Unit).WithMany().HasForeignKey(e => e.UnitId)
            .OnDelete(DeleteBehavior.Restrict).HasConstraintName("FK_RadiationWorkPermit_Unit");
        b.HasOne(e => e.RequestedByUser).WithMany().HasForeignKey(e => e.RequestedById)
            .OnDelete(DeleteBehavior.Restrict).HasConstraintName("FK_RadiationWorkPermit_RequestedByUser");
        b.HasOne(e => e.ApprovedByUser).WithMany().HasForeignKey(e => e.ApprovedById)
            .OnDelete(DeleteBehavior.Restrict).HasConstraintName("FK_RadiationWorkPermit_ApprovedByUser");
    }
}

public sealed class RadiationWorkPermitPersonConfiguration
    : IEntityTypeConfiguration<RadiationWorkPermitPerson>
{
    public void Configure(EntityTypeBuilder<RadiationWorkPermitPerson> b)
    {
        b.ToTable("RadiationWorkPermitPerson", "RadiationMonitoring");
        b.HasKey(e => e.RadiationWorkPermitPersonId).HasName("PK_RadiationWorkPermitPerson");
        b.HasIndex(e => new { e.RadiationWorkPermitId, e.PersonId })
            .IsUnique()
            .HasDatabaseName("UQ_RadiationWorkPermitPerson_Permit_Person");
        b.HasOne(e => e.RadiationWorkPermit).WithMany(e => e.People)
            .HasForeignKey(e => e.RadiationWorkPermitId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_RadiationWorkPermitPerson_Permit");
        b.HasOne(e => e.Person).WithMany()
            .HasForeignKey(e => e.PersonId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationWorkPermitPerson_Person");
    }
}
```

Integration link configurations

Radiological data often becomes part of an operational event or a robot mission. Link tables keep the evidence queryable without copying event or mission fields.

```
public sealed class RadiationEventLinkConfiguration : IEntityTypeConfiguration<RadiationEventLink>
{
    public void Configure(EntityTypeBuilder<RadiationEventLink> b)
    {
        b.ToTable("RadiationEventLink", "RadiationMonitoring");
        b.HasKey(e => e.RadiationEventLinkId).HasName("PK_RadiationEventLink");
        b.Property(e => e.LinkRole).HasMaxLength(80).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)").HasDefaultValueSql(NexusSql.UtcNow);
        b.HasIndex(e => e.OperationalEventId).HasDatabaseName("IX_RadiationEventLink_OperationalEvent");
        b.HasOne(e => e.OperationalEvent).WithMany()
            .HasForeignKey(e => e.OperationalEventId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationEventLink_OperationalEvent");
    }
}

public sealed class RadiationRobotMissionLinkConfiguration
    : IEntityTypeConfiguration<RadiationRobotMissionLink>
{
    public void Configure(EntityTypeBuilder<RadiationRobotMissionLink> b)
    {
        b.ToTable("RadiationRobotMissionLink", "RadiationMonitoring");
        b.HasKey(e => e.RadiationRobotMissionLinkId).HasName("PK_RadiationRobotMissionLink");
        b.Property(e => e.LinkRole).HasMaxLength(80).IsRequired();
        b.HasIndex(e => new { e.MissionId, e.RadiationZoneId })
            .HasDatabaseName("IX_RadiationRobotMissionLink_Mission_Zone");
        b.HasOne(e => e.Mission).WithMany()
            .HasForeignKey(e => e.MissionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationRobotMissionLink_Mission");
    }
}
```

Trend summary configuration

Trend summaries are precomputed buckets for reporting. They are derived from measurements, but still mapped explicitly so dashboards can query them cheaply.

```
public sealed class RadiationTrendSummaryConfiguration
    : IEntityTypeConfiguration<RadiationTrendSummary>
{
    public void Configure(EntityTypeBuilder<RadiationTrendSummary> b)
    {
        b.ToTable("RadiationTrendSummary", "RadiationMonitoring");
        b.HasKey(e => e.RadiationTrendSummaryId).HasName("PK_RadiationTrendSummary");
        b.Property(e => e.BucketStartUtc).HasColumnType("datetime2(7)").IsRequired();
        b.Property(e => e.BucketEndUtc).HasColumnType("datetime2(7)").IsRequired();
        b.Property(e => e.MinimumValue).HasPrecision(18, 8);
        b.Property(e => e.MaximumValue).HasPrecision(18, 8);
        b.Property(e => e.AverageValue).HasPrecision(18, 8);
        b.HasIndex(e => new { e.UnitId, e.RadiationZoneId, e.BucketStartUtc })
            .HasDatabaseName("IX_RadiationTrendSummary_Unit_Zone_Bucket");
        b.HasOne(e => e.EngineeringUnit).WithMany()
            .HasForeignKey(e => e.EngineeringUnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RadiationTrendSummary_EngineeringUnit");
    }
}
```

DbContext discovery

The context continues to stay clean. The RadiationMonitoring files are discovered from the assembly, not pasted into a long `OnModelCreating` method.

```
public sealed class NexusContext : DbContext
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(NexusContext).Assembly,
            type => type.Namespace != null &&
                type.Namespace.Contains("Configurations.RadiationMonitoring"));
    }
}
```

Do not cascade across passports. Zone readings may disappear only by an explicit archive or retention action, never because a referenced person, signal or unit was changed.

Foreign-key mapping

TABLE	FK PROPERTY	TARGET	KIND
RadiationMonitoring.RadiationZone	UnitId	ReactorFleet.Unit	passport
RadiationMonitoring.RadiationZone	EquipmentLocationId	ReactorFleet.EquipmentLocation	passport
RadiationMonitoring.RadiationZoneBoundaryPoint	RadiationZoneId	RadiationMonitoring.RadiationZone	internal
RadiationMonitoring.RadiationZoneAccessRequirement	RadiationZoneId	RadiationMonitoring.RadiationZone	internal
RadiationMonitoring.RadiationZoneAccessRequirement	DoseTypeId	RadiationMonitoring.DoseType	internal
RadiationMonitoring.RadiationZoneAccessRequirement	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
RadiationMonitoring.RadiationMonitor	UnitId	ReactorFleet.Unit	passport
RadiationMonitoring.RadiationMonitor	EquipmentId	ReactorFleet.Equipment	passport
RadiationMonitoring.RadiationMonitor	RadiationZoneId	RadiationMonitoring.RadiationZone	internal
RadiationMonitoring.RadiationMonitorSignalBinding	RadiationMonitorId	RadiationMonitoring.RadiationMonitor	internal
RadiationMonitoring.RadiationMonitorSignalBinding	SignalId	Instrumentation.Signal	passport
RadiationMonitoring.RadiationMonitorCalibration	RadiationMonitorId	RadiationMonitoring.RadiationMonitor	internal
RadiationMonitoring.RadiationMonitorCalibration	ApplicationUserId	Security.ApplicationUser	passport
RadiationMonitoring.RadiationReading	RadiationMonitorId	RadiationMonitoring.RadiationMonitor	internal
RadiationMonitoring.RadiationReading	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
RadiationMonitoring.Dosimeter	DosimeterTypeId	RadiationMonitoring.DosimeterType	internal
RadiationMonitoring.Dosimeter	DosimeterStatusId	RadiationMonitoring.DosimeterStatus	internal
RadiationMonitoring.PersonDosimeterAssignment	PersonId	Organization.Person	passport
RadiationMonitoring.PersonDosimeterAssignment	DosimeterId	RadiationMonitoring.Dosimeter	internal
RadiationMonitoring.PersonDoseReading	PersonDosimeterAssignmentId	RadiationMonitoring.PersonDosimeterAssignment	internal
RadiationMonitoring.PersonDoseReading	DoseTypeId	RadiationMonitoring.DoseType	internal
RadiationMonitoring.PersonDoseReading	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
RadiationMonitoring.DoseLimit	DoseTypeId	RadiationMonitoring.DoseType	internal
RadiationMonitoring.DoseLimit	LimitTypeId	RadiationMonitoring.LimitType	internal
RadiationMonitoring.DoseLimit	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
RadiationMonitoring.DoseLimitAssignment	DoseLimitId	RadiationMonitoring.DoseLimit	internal
RadiationMonitoring.DoseLimitAssignment	PersonId	Organization.Person	passport

TABLE	FK PROPERTY	TARGET	KIND
RadiationMonitoring.DoseLimitAssignment	ExposureCategoryId	RadiationMonitoring.ExposureCategory	internal
RadiationMonitoring.DoseAlert	DoseLimitId	RadiationMonitoring.DoseLimit	internal
RadiationMonitoring.DoseAlert	PersonDoseReadingId	RadiationMonitoring.PersonDoseReading	internal
RadiationMonitoring.DoseAlert	AlertStatusId	RadiationMonitoring.AlertStatus	internal
RadiationMonitoring.RadiologicalSurvey	UnitId	ReactorFleet.Unit	passport
RadiationMonitoring.RadiologicalSurvey	RadiationZoneId	RadiationMonitoring.RadiationZone	internal
RadiationMonitoring.RadiologicalSurvey	SurveyTypeId	RadiationMonitoring.SurveyType	internal
RadiationMonitoring.RadiologicalSurvey	SurveyStatusId	RadiationMonitoring.SurveyStatus	internal
RadiationMonitoring.RadiologicalSurveyReading	RadiologicalSurveyId	RadiationMonitoring.RadiologicalSurvey	internal
RadiationMonitoring.RadiologicalSurveyReading	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
RadiationMonitoring.ContaminationSample	RadiologicalSurveyId	RadiationMonitoring.RadiologicalSurvey	internal
RadiationMonitoring.ContaminationSample	SampleTypeId	RadiationMonitoring.SampleType	internal
RadiationMonitoring.ContaminationSample	ContaminationTypeId	RadiationMonitoring.ContaminationType	internal
RadiationMonitoring.ContaminationSampleResult	ContaminationSampleId	RadiationMonitoring.ContaminationSample	internal
RadiationMonitoring.ContaminationSampleResult	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
RadiationMonitoring.EnvironmentalSamplingLocation	PlantId	ReactorFleet.Plant	passport
RadiationMonitoring.EnvironmentalSamplingLocation	CountryId	CorePlatform.Country	passport
RadiationMonitoring.EnvironmentalSamplingLocation	RegionId	CorePlatform.Region	passport
RadiationMonitoring.EnvironmentalSample	EnvironmentalSamplingLocationId	RadiationMonitoring.EnvironmentalSamplingLocation	internal
RadiationMonitoring.EnvironmentalSample	SampleTypeId	RadiationMonitoring.SampleType	internal
RadiationMonitoring.EnvironmentalSampleResult	EnvironmentalSampleId	RadiationMonitoring.EnvironmentalSample	internal
RadiationMonitoring.EnvironmentalSampleResult	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
RadiationMonitoring.IsotopeInventory	UnitId	ReactorFleet.Unit	passport
RadiationMonitoring.IsotopeInventory	EquipmentId	ReactorFleet.Equipment	passport
RadiationMonitoring.IsotopeInventory	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
RadiationMonitoring.ZoneDoseSnapshot	RadiationZoneId	RadiationMonitoring.RadiationZone	internal
RadiationMonitoring.ZoneDoseSnapshot	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
RadiationMonitoring.RadiationWorkPermit	UnitId	ReactorFleet.Unit	passport
RadiationMonitoring.RadiationWorkPermit	RequestedByUserId	Security.ApplicationUser	passport
RadiationMonitoring.RadiationWorkPermit	ApprovedByUserId	Security.ApplicationUser	passport

TABLE	FK PROPERTY	TARGET	KIND
RadiationMonitoring.RadiationWorkPermitZone	RadiationWorkPermitId	RadiationMonitoring.RadiationWorkPermit	internal
RadiationMonitoring.RadiationWorkPermitZone	RadiationZoneId	RadiationMonitoring.RadiationZone	internal
RadiationMonitoring.RadiationWorkPermitPerson	RadiationWorkPermitId	RadiationMonitoring.RadiationWorkPermit	internal
RadiationMonitoring.RadiationWorkPermitPerson	PersonId	Organization.Person	passport
RadiationMonitoring.RadiationWorkPermitDoseRecord	RadiationWorkPermitPersonId	RadiationMonitoring.RadiationWorkPermitPerson	internal
RadiationMonitoring.RadiationWorkPermitDoseRecord	DoseTypeId	RadiationMonitoring.DoseType	internal
RadiationMonitoring.RadiationWorkPermitDoseRecord	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
RadiationMonitoring.RadiationEventLink	OperationalEventId	EventManagement.OperationalEvent	passport
RadiationMonitoring.RadiationEventLink	RadiationZoneId	RadiationMonitoring.RadiationZone	internal
RadiationMonitoring.RadiationEventLink	RadiationReadingId	RadiationMonitoring / cross-schema target	passport
RadiationMonitoring.RadiationEventLink	RadiologicalSurveyId	RadiationMonitoring.RadiologicalSurvey	internal
RadiationMonitoring.RadiationRobotMissionLink	MissionId	Robotics.Mission	passport
RadiationMonitoring.RadiationRobotMissionLink	RadiationZoneId	RadiationMonitoring.RadiationZone	internal
RadiationMonitoring.RadiationRobotMissionLink	RadiationReadingId	RadiationMonitoring / cross-schema target	passport
RadiationMonitoring.RadiationRobotMissionLink	RadiologicalSurveyId	RadiationMonitoring.RadiologicalSurvey	internal
RadiationMonitoring.RadiationTrendSummary	UnitId	ReactorFleet.Unit	passport
RadiationMonitoring.RadiationTrendSummary	RadiationZoneId	RadiationMonitoring.RadiationZone	internal
RadiationMonitoring.RadiationTrendSummary	EngineeringUnitId	CorePlatform.EngineeringUnit	passport

Migration and verification notes

Migration order

```
-- Suggested schema order for this chapter:  
-- 1. CorePlatform, Security, Organization, ReactorFleet and Instrumentation already exist.  
-- 2. Create RadiationMonitoring lookup tables.  
-- 3. Create zones and monitor master data.  
-- 4. Create readings, surveys, dosimetry, permits and integration links.  
-- 5. Add operational indexes for time-series and permit queries.
```

Metadata verification

```
[Fact]  
public void RadiationReading_has_time_series_index_and_restrict_passports()  
{  
    using var db = new NexusContext(_options);  
    var entity = db.Model.FindEntityType(typeof(RadiationReading))!;  
  
    entity.GetTableName().Should().Be("RadiationReading");  
    entity.GetSchema().Should().Be("RadiationMonitoring");  
  
    entity.GetIndexes()  
        .Should().Contain(i => i.GetDatabaseName() == "IX_RadiationReading_Monitor_Time");  
  
    entity.GetForeignKeys()  
        .Where(fk => fk.PrincipalEntityType.GetSchema() != "RadiationMonitoring")  
        .Should().OnlyContain(fk => fk.DeleteBehavior == DeleteBehavior.Restrict);  
}
```

Verification habit. The test checks the EF Core model metadata. It does not need real radiation data and it does not connect to a plant.

Professional notes

Use decimal precision

Radiological values must not be mapped as uncontrolled floating-point fields when they are reported and audited.

Keep dose as evidence

Personal dose records are append-only facts. Corrections should add rows or audit records, not overwrite history silently.

Separate access from measurement

A radiation zone defines rules. A monitor reading proves a measurement. A work permit authorizes people. The model keeps them separate.

Keep external links restrictive

Events, robot missions, users, people and signals are passports, not owned children.

Honest boundary

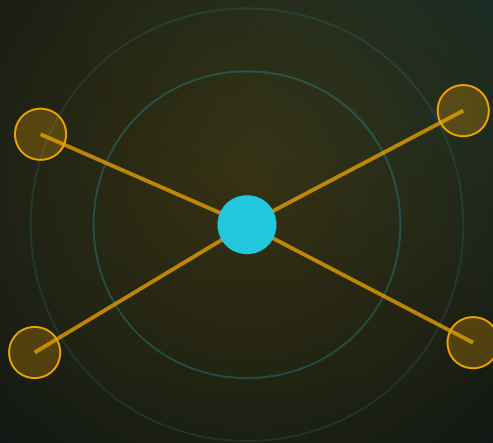
The RadiationMonitoring configuration atlas is a software-engineering reference for the NEXUS-1 demonstrator. It is not a radiological protection program, not a licensed dose record system, not a safety case and not connected to any real facility. It shows how such a bounded context can be mapped clearly in EF Core.

Index of RadiationMonitoring configuration files

RadiationZoneTypeConfiguration.cs	Lookup	RadiationReadingConfiguration.cs	Monitor
RadiationZoneStatusConfiguration.cs	Lookup	DosimeterConfiguration.cs	Dosimetry
RadiationAreaClassificationConfiguration.cs	Lookup	PersonDosimeterAssignmentConfiguration.cs	Dosimetry
MonitorTypeConfiguration.cs	Lookup	PersonDoseReadingConfiguration.cs	Dosimetry
MonitorStatusConfiguration.cs	Lookup	DoseLimitConfiguration.cs	Dosimetry
MeasurementTypeConfiguration.cs	Lookup	DoseLimitAssignmentConfiguration.cs	Dosimetry
MeasurementQualityConfiguration.cs	Lookup	DoseAlertConfiguration.cs	Dosimetry
DoseTypeConfiguration.cs	Lookup	RadiologicalSurveyConfiguration.cs	Survey
DosimeterTypeConfiguration.cs	Lookup	RadiologicalSurveyReadingConfiguration.cs	Survey
DosimeterStatusConfiguration.cs	Lookup	ContaminationSampleConfiguration.cs	Survey
SurveyTypeConfiguration.cs	Lookup	ContaminationSampleResultConfiguration.cs	Survey
SurveyStatusConfiguration.cs	Lookup	EnvironmentalSamplingLocationConfiguration.cs	Environment
ContaminationTypeConfiguration.cs	Lookup	EnvironmentalSampleConfiguration.cs	Environment
SampleTypeConfiguration.cs	Lookup	EnvironmentalSampleResultConfiguration.cs	Environment
LimitTypeConfiguration.cs	Lookup	IsotopeInventoryConfiguration.cs	Inventory
AlertStatusConfiguration.cs	Lookup	ZoneDoseSnapshotConfiguration.cs	Snapshot
ExposureCategoryConfiguration.cs	Lookup	RadiationWorkPermitConfiguration.cs	Permit
RadiationZoneConfiguration.cs	Zone	RadiationWorkPermitZoneConfiguration.cs	Permit
RadiationZoneBoundaryPointConfiguration.cs	Zone	RadiationWorkPermitPersonConfiguration.cs	Permit
RadiationZoneAccessRequirementConfiguration.cs	Zone	RadiationWorkPermitDoseRecordConfiguration.cs	Permit
RadiationMonitorConfiguration.cs	Monitor	RadiationEventLinkConfiguration.cs	Integration
RadiationMonitorSignalBindingConfiguration.cs	Monitor	RadiationRobotMissionLinkConfiguration.cs	Integration
RadiationMonitorCalibrationConfiguration.cs	Monitor	RadiationTrendSummaryConfiguration.cs	Reporting

FROM ENTITY TO CONTEXT

Chapter 19 - EmergencyPreparedness Configurations
Plans, exercises, routes, muster, protective actions and response resources.



SCHEMA CHAPTER

Emergency preparedness as configuration code

One configuration file per entity, explicit indexes, clear passports, and no emergency plan hidden inside the DbContext.

Chapter 19 - EmergencyPreparedness Configurations

The EF Core configuration atlas for emergency plans, classifications, response teams, exercises, evacuation routes, muster records, protective actions, notifications, resources and integration links.

This chapter continues the second technical reference companion: **From Entity to Context**. It maps the SQL Server emergency-preparedness sector into professional EF Core configuration classes, keeping the model explicit, discoverable and migration-friendly.

Emergency preparedness is not a single table called Emergency. It is a chain: plan, revision, scenario, exercise, route, communication, resource, action, evidence, and audit-facing links.

Standing EF Core rule. The DbContext discovers these files by assembly scanning. The configuration classes own table names, key names, indexes, defaults, delete behavior and foreign-key constraint names.

19.1 What this schema configures

The EmergencyPreparedness sector is the part of NEXUS-1 that turns response planning into structured data. It records which plan is valid, which revision was approved, which scenario is being exercised, who participated, where people must muster, which route is open, which resource is ready, and which external sectors were touched by the event.

Planning

Plan and revision configuration keep the authoritative response text versioned, owned and statused.

Exercises

Exercise configuration binds scenarios, injects, observations, evaluations and participants into one testable event.

Routes and muster

Route and muster configuration makes evacuation and accountability queryable by plant, zone, person and assembly point.

Communications

Notification template and communication entities separate reusable wording from outbound message instances.

Resources

Resources and readiness checks model equipment, PPE, vehicles, communications and monitoring kits.

Integration links

Plan-to-incident, plan-to-alarm and plan-to-mission links keep the response chain replayable.

19.2 Configuration file catalogue

Every row below represents one EF Core configuration file. Lookup files are not treated as second-class code: they carry unique codes, display order, active flags, rowversion and stable names just like the rest of the platform.

GROUP	ENTITY TABLE	CONFIGURATION FILE	PURPOSE
Planning	EmergencyPreparedness.EmergencyPlan	EmergencyPlanConfiguration.cs	The named emergency plan for a site or plant, with status, owner and validity window.
Planning	EmergencyPreparedness.EmergencyPlanRevision	EmergencyPlanRevisionConfiguration.cs	Versioned plan text and approval metadata. The active plan points to the current revision number.
Classification	EmergencyPreparedness.EmergencyClassification	EmergencyClassificationConfiguration.cs	A declared emergency class for an actual event, unit, incident or exercise boundary.
Teams	EmergencyPreparedness.EmergencyRole	EmergencyRoleConfiguration.cs	Role catalogue for command, technical, radiological, logistics and communications duties.
Teams	EmergencyPreparedness.ResponseTeam	ResponseTeamConfiguration.cs	Emergency response team aligned with organization team records and plant scope.
Teams	EmergencyPreparedness.ResponseTeamMember	ResponseTeamMemberConfiguration.cs	Person-to-team assignment with emergency role and validity window.
Scenario	EmergencyPreparedness.EmergencyScenario	EmergencyScenarioConfiguration.cs	Reusable scenario library entry for exercises, training and preparedness analysis.
Exercise	EmergencyPreparedness.Exercise	ExerciseConfiguration.cs	A planned or completed emergency exercise with schedule, coordinator and scope.
Exercise	EmergencyPreparedness.ExerciseParticipant	ExerciseParticipantConfiguration.cs	Who took part in the exercise, in what capacity, and whether they attended.
Exercise	EmergencyPreparedness.ExerciseInject	ExerciseInjectConfiguration.cs	Scripted exercise injects: alarms, messages, field reports and simulated failures.
Exercise	EmergencyPreparedness.ExerciseObservation	ExerciseObservationConfiguration.cs	Observed strengths, gaps and deficiencies captured during an exercise.
Exercise	EmergencyPreparedness.ExerciseEvaluation	ExerciseEvaluationConfiguration.cs	Final evaluation, lessons learned, recommendations and overall score.
Routes	EmergencyPreparedness.AssemblyPoint	AssemblyPointConfiguration.cs	Muster or assembly location, optionally tied to a radiological zone.
Routes	EmergencyPreparedness.EvacuationRoute	EvacuationRouteConfiguration.cs	Named evacuation route from an area to an assembly point.
Routes	EmergencyPreparedness.EvacuationRouteZone	EvacuationRouteZoneConfiguration.cs	Radiological zones intersected by a route and whether the route should be avoided when alarmed.
Muster	EmergencyPreparedness.MusterRecord	MusterRecordConfiguration.cs	Accountability record for people during an exercise or real event.
Actions	EmergencyPreparedness.ProtectiveAction	ProtectiveActionConfiguration.cs	Ordered protective actions such as evacuation, sheltering or access restriction.

GROUP	ENTITY TABLE	CONFIGURATION FILE	PURPOSE
Comms	EmergencyPreparedness.NotificationTemplate	NotificationTemplateConfiguration.cs	Reusable notification wording by channel and language.
Comms	EmergencyPreparedness.EmergencyCommunication	EmergencyCommunicationConfiguration.cs	Outbound message instance, recipient, status and failure reason.
Resources	EmergencyPreparedness.EmergencyResource	EmergencyResourceConfiguration.cs	Vehicles, communications equipment, PPE, monitoring kits and other emergency resources.
Resources	EmergencyPreparedness.ResourceReadinessCheck	ResourceReadinessCheckConfiguration.cs	Periodic readiness check for a resource.
Integration	EmergencyPreparedness.PlanIncidentLink	PlanIncidentLinkConfiguration.cs	Connects an emergency plan to an incident that used, tested or changed it.
Integration	EmergencyPreparedness.PlanAlarmLink	PlanAlarmLinkConfiguration.cs	Connects an emergency plan to an alarm trigger or historical alarm event.
Integration	EmergencyPreparedness.PlanRobotMissionLink	PlanRobotMissionLinkConfiguration.cs	Connects emergency plans to robotics missions used for survey or access support.
Lookup	EmergencyPreparedness.PlanStatus	PlanStatusConfiguration.cs	Referenced by EmergencyPlan and PlanRevision.
Lookup	EmergencyPreparedness.EmergencyClassType	EmergencyClassTypeConfiguration.cs	Referenced by EmergencyClassification and scenarios.
Lookup	EmergencyPreparedness.ScenarioStatus	ScenarioStatusConfiguration.cs	Referenced by EmergencyScenario.
Lookup	EmergencyPreparedness.ExerciseType	ExerciseTypeConfiguration.cs	Referenced by Exercise.
Lookup	EmergencyPreparedness.ExerciseStatus	ExerciseStatusConfiguration.cs	Referenced by Exercise.
Lookup	EmergencyPreparedness.ParticipantStatus	ParticipantStatusConfiguration.cs	Referenced by ExerciseParticipant.
Lookup	EmergencyPreparedness.InjectType	InjectTypeConfiguration.cs	Referenced by ExerciseInject.
Lookup	EmergencyPreparedness.ObservationSeverity	ObservationSeverityConfiguration.cs	Referenced by ExerciseObservation.
Lookup	EmergencyPreparedness.EvaluationStatus	EvaluationStatusConfiguration.cs	Referenced by ExerciseEvaluation.
Lookup	EmergencyPreparedness.ProtectiveActionType	ProtectiveActionTypeConfiguration.cs	Referenced by ProtectiveAction.
Lookup	EmergencyPreparedness.ProtectiveActionStatus	ProtectiveActionStatusConfiguration.cs	Referenced by ProtectiveAction.
Lookup	EmergencyPreparedness.ResourceType	ResourceTypeConfiguration.cs	Referenced by EmergencyResource.
Lookup	EmergencyPreparedness.ResourceStatus	ResourceStatusConfiguration.cs	Referenced by EmergencyResource.
Lookup	EmergencyPreparedness.ReadinessStatus	ReadinessStatusConfiguration.cs	Referenced by ResourceReadinessCheck.
Lookup	EmergencyPreparedness.RouteStatus	RouteStatusConfiguration.cs	Referenced by EvacuationRoute.
Lookup	EmergencyPreparedness.MusterStatus	MusterStatusConfiguration.cs	Referenced by MusterRecord.
Lookup	EmergencyPreparedness.NotificationChannel	NotificationChannelConfiguration.cs	Referenced by templates and communications.

GROUP	ENTITY TABLE	CONFIGURATION FILE	PURPOSE
Lookup	EmergencyPreparedness.CommunicationStatus	CommunicationStatusConfiguration.cs	Referenced by EmergencyCommunication.

19.3 Group map

Planning 2 configurations EmergencyPlan, EmergencyPlanRevision	Classification 1 configurations EmergencyClassification	Teams 3 configurations EmergencyRole, ResponseTeam, ResponseTeamMember
Scenario 1 configurations EmergencyScenario	Exercise 5 configurations Exercise, ExerciseParticipant, ExerciseInject, ExerciseObservation ...	Routes 3 configurations AssemblyPoint, EvacuationRoute, EvacuationRouteZone
Muster 1 configurations MusterRecord	Actions 1 configurations ProtectiveAction	Comms 2 configurations NotificationTemplate, EmergencyCommunication
Resources 2 configurations EmergencyResource, ResourceReadinessCheck	Integration 3 configurations PlanIncidentLink, PlanAlarmLink, PlanRobotMissionLink	Lookup 18 configurations PlanStatus, EmergencyClassType, ScenarioStatus, ExerciseType ...

Why a group map helps. In a large EF Core model, the human problem is navigation. Grouping configuration files by planning, exercise, routes, communications and resources lets a developer find the right mapping without searching inside the DbContext.

19.4 Lookup tables

The lookup tables give the emergency domain typed vocabulary. The application should not compare raw strings such as `FULL_SCALE` or `ACCOUNTED_FOR` in scattered code; it should reference stable rows through foreign keys.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
EmergencyPreparedness.PlanStatus	DRAFT, ACTIVE, UNDER_REVIEW, SUPERSEDED, RETIRED	Referenced by EmergencyPlan and PlanRevision.
EmergencyPreparedness.EmergencyClassType	ALERT, SITE_AREA, GENERAL, SECURITY, RADIOLOGICAL	Referenced by EmergencyClassification and scenarios.
EmergencyPreparedness.ScenarioStatus	DRAFT, VALIDATED, IN_TRAINING, RETIRED	Referenced by EmergencyScenario.
EmergencyPreparedness.ExerciseType	TABLETOP, FUNCTIONAL, FULL_SCALE, DRILL, COMMUNICATIONS	Referenced by Exercise.
EmergencyPreparedness.ExerciseStatus	PLANNED, ACTIVE, COMPLETED, EVALUATED, CANCELLED	Referenced by Exercise.
EmergencyPreparedness.ParticipantStatus	INVITED, CONFIRMED, PRESENT, ABSENT, EXCUSED	Referenced by ExerciseParticipant.
EmergencyPreparedness.InjectType	ALARM, MESSAGE, FIELD_REPORT, RAD_READING, RESOURCE_FAILURE	Referenced by ExerciseInject.

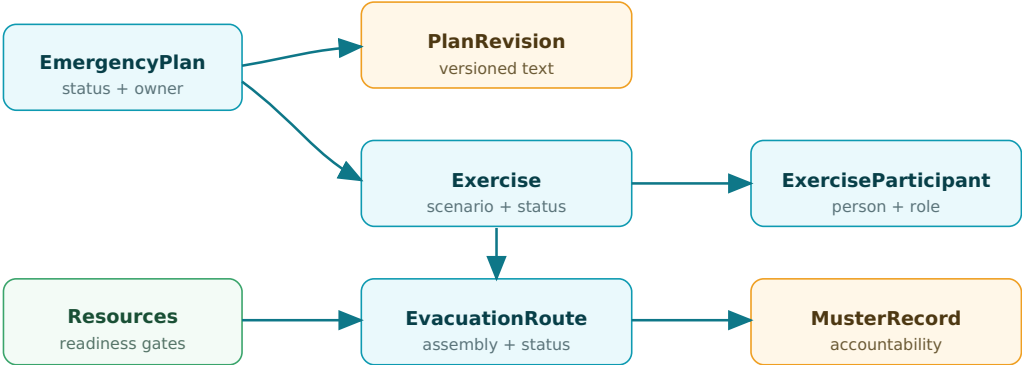
LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
EmergencyPreparedness.ObservationSeverity	INFO, MINOR, MAJOR, CRITICAL	Referenced by ExerciseObservation.
EmergencyPreparedness.EvaluationStatus	DRAFT, REVIEWED, APPROVED, REWORK_REQUIRED	Referenced by ExerciseEvaluation.
EmergencyPreparedness.ProtectiveActionType	EVACUATE, SHELTER, IODINE, RESTRICT_ACCESS, STOP_WORK	Referenced by ProtectiveAction.
EmergencyPreparedness.ProtectiveActionStatus	ORDERED, IN_PROGRESS, COMPLETED, CANCELLED, FAILED	Referenced by ProtectiveAction.
EmergencyPreparedness.ResourceType	VEHICLE, COMMUNICATION, PPE, MEDICAL, MONITORING, POWER	Referenced by EmergencyResource.
EmergencyPreparedness.ResourceStatus	AVAILABLE, ASSIGNED, OUT_OF_SERVICE, CALIBRATION_DUE, RETIRED	Referenced by EmergencyResource.
EmergencyPreparedness.ReadinessStatus	READY, DEGRADED, NOT_READY, UNKNOWN	Referenced by ResourceReadinessCheck.
EmergencyPreparedness.RouteStatus	OPEN, RESTRICTED, BLOCKED, UNDER_REVIEW, RETIRED	Referenced by EvacuationRoute.
EmergencyPreparedness.MusterStatus	EXPECTED, ACCOUNTED_FOR, MISSING, RELEASED, MEDICAL	Referenced by MusterRecord.
EmergencyPreparedness.NotificationChannel	EMAIL, SMS, RADIO, PA, PHONE, SECURE_APP	Referenced by templates and communications.
EmergencyPreparedness.CommunicationStatus	QUEUED, SENT, DELIVERED, FAILED, ACKNOWLEDGED	Referenced by EmergencyCommunication.

19.5 Folder structure

```
Nexus.Infrastructure.Persistence/  
  Configurations/  
    EmergencyPreparedness/  
      EmergencyPreparednessLookupConfiguration.cs  
      Lookups/  
        PlanStatusConfiguration.cs  
        EmergencyClassTypeConfiguration.cs  
        ScenarioStatusConfiguration.cs  
        ExerciseTypeConfiguration.cs  
        ExerciseStatusConfiguration.cs  
        ParticipantStatusConfiguration.cs  
        InjectTypeConfiguration.cs  
        ObservationSeverityConfiguration.cs  
        EvaluationStatusConfiguration.cs  
        ProtectiveActionTypeConfiguration.cs  
        ProtectiveActionStatusConfiguration.cs  
        ResourceTypeConfiguration.cs  
        ResourceStatusConfiguration.cs  
        ReadinessStatusConfiguration.cs  
        RouteStatusConfiguration.cs  
        MusterStatusConfiguration.cs  
        NotificationChannelConfiguration.cs  
        CommunicationStatusConfiguration.cs  
      Planning/  
        EmergencyPlanConfiguration.cs  
        EmergencyPlanRevisionConfiguration.cs  
        EmergencyClassificationConfiguration.cs  
      Teams/  
        EmergencyRoleConfiguration.cs  
        ResponseTeamConfiguration.cs  
        ResponseTeamMemberConfiguration.cs  
      Exercises/  
        EmergencyScenarioConfiguration.cs  
        ExerciseConfiguration.cs  
        ExerciseParticipantConfiguration.cs  
        ExerciseInjectConfiguration.cs  
        ExerciseObservationConfiguration.cs  
        ExerciseEvaluationConfiguration.cs  
      Routes/  
        AssemblyPointConfiguration.cs  
        EvacuationRouteConfiguration.cs  
        EvacuationRouteZoneConfiguration.cs  
        MusterRecordConfiguration.cs  
      ActionsAndComms/  
        ProtectiveActionConfiguration.cs  
        NotificationTemplateConfiguration.cs  
        EmergencyCommunicationConfiguration.cs  
      Resources/  
        EmergencyResourceConfiguration.cs  
        ResourceReadinessCheckConfiguration.cs  
      Integration/  
        PlanIncidentLinkConfiguration.cs  
        PlanAlarmLinkConfiguration.cs  
        PlanRobotMissionLinkConfiguration.cs
```

19.6 Dependency and passport diagrams

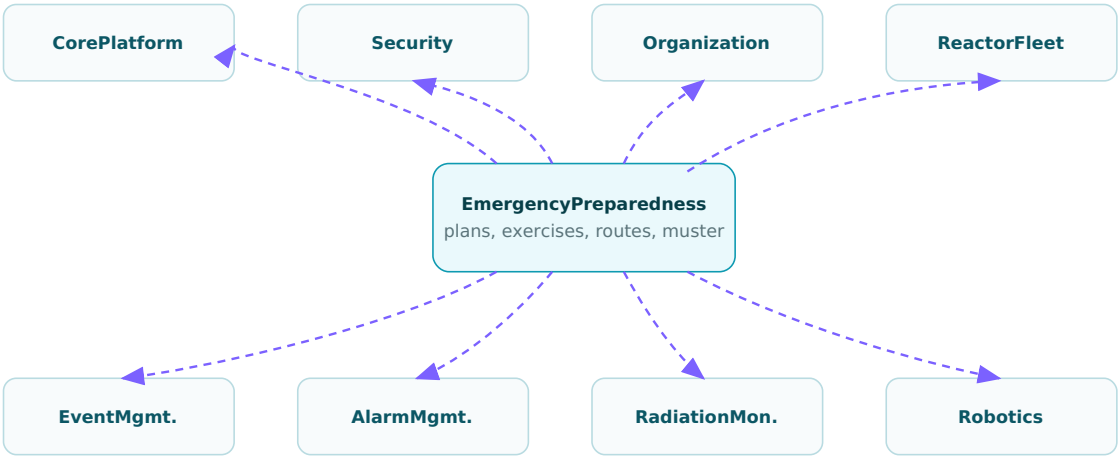
EmergencyPreparedness EF Core dependency focus



Lookup configurations sit behind the main entities. Cross-schema passports stay explicit and restricted.

Figure 19.1 - EF Core dependency focus for EmergencyPreparedness. The diagram shows the spine from plan and exercise to routes, muster and resources.

Passport map - where emergency preparedness reaches outward



Languageld, EngineeringUnitId - owner/preparer/approver users - site/person/team - plant/unit - incident/event/alarm - radiation zones - missions.

Figure 19.2 - Cross-schema passports used by emergency preparedness. External identities remain explicit and restricted.

19.7 Shared lookup configuration

All emergency lookup tables use the same base configuration. The base class keeps the code/name/display-order pattern stable and prevents each lookup file from drifting.

```
public abstract class EmergencyPreparednessLookupConfiguration<TEntity>
    : IEntityConfiguration<TEntity>
    where TEntity : EmergencyPreparednessLookup
{
    protected abstract string TableName { get; }
    protected abstract string KeyName { get; }

    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.ToTable(TableName, "EmergencyPreparedness");
        b.HasKey(e => e.Id).HasName($"PK_{TableName}");
        b.Property(e => e.Id).HasColumnName(KeyName);
        b.Property(e => e.Code).HasMaxLength(40).IsRequired();
        b.Property(e => e.Name).HasMaxLength(160).IsRequired();
        b.Property(e => e.Description).HasMaxLength(800);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.ModifiedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();
        b.HasIndex(e => e.Code).IsUnique().HasDatabaseName($"UQ_{TableName}_Code");
    }
}
```

Lookup configuration examples

```
public sealed class EmergencyClassTypeConfiguration
    : EmergencyPreparednessLookupConfiguration<EmergencyClassType>
{
    protected override string TableName => "EmergencyClassType";
    protected override string KeyName => "EmergencyClassTypeId";
}

public sealed class NotificationChannelConfiguration
    : EmergencyPreparednessLookupConfiguration<NotificationChannel>
{
    protected override string TableName => "NotificationChannel";
    protected override string KeyName => "NotificationChannelId";
}
```

19.8 Planning configurations

The plan is the root of the sector. It is keyed by a stable code, scoped to site and plant, statused through a lookup, and owned by a security user. Revisions are children of the plan and cascade because a revision has no independent life outside its plan.

```
public sealed class EmergencyPlanConfiguration : IEntityTypeConfiguration<EmergencyPlan>
{
    public void Configure(EntityTypeBuilder<EmergencyPlan> b)
    {
        b.ToTable("EmergencyPlan", "EmergencyPreparedness");
        b.HasKey(e => e.EmergencyPlanId).HasName("PK_EmergencyPlan");

        b.Property(e => e.Code).HasMaxLength(40).IsRequired();
        b.Property(e => e.Name).HasMaxLength(220).IsRequired();
        b.Property(e => e.ScopeSummary).HasMaxLength(1000);
        b.Property(e => e.ValidFromUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ValidToUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique().HasDatabaseName("UQ_EmergencyPlan_Code");
        b.HasIndex(e => new { e.SiteId, e.PlantId, e.PlanStatusId })
            .HasDatabaseName("IX_EmergencyPlan_Scope_Status");

        b.HasOne(e => e.PlanStatus)
            .WithMany()
            .HasForeignKey(e => e.PlanStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EmergencyPlan_PlanStatus");

        b.HasOne(e => e.Site)
            .WithMany()
            .HasForeignKey(e => e.SiteId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EmergencyPlan_Site");

        b.HasOne(e => e.Plant)
            .WithMany()
            .HasForeignKey(e => e.PlantId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EmergencyPlan_Plant");

        b.HasOne(e => e.OwnerUser)
            .WithMany()
            .HasForeignKey(e => e.OwnerUserId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EmergencyPlan_OwnerUser");
    }
}
```

```
public sealed class EmergencyPlanRevisionConfiguration
: IEntityTypeConfiguration<EmergencyPlanRevision>
{
    public void Configure(EntityTypeBuilder<EmergencyPlanRevision> b)
    {
        b.ToTable("EmergencyPlanRevision", "EmergencyPreparedness");
        b.HasKey(e => e.EmergencyPlanRevisionId)
            .HasName("PK_EmergencyPlanRevision");

        b.Property(e => e.RevisionNumber).IsRequired();
        b.Property(e => e.Title).HasMaxLength(220).IsRequired();
        b.Property(e => e.BodyMarkdown).IsRequired();
        b.Property(e => e.PreparedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ApprovedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.EmergencyPlanId, e.RevisionNumber })
            .IsUnique()
            .HasDatabaseName("UQ_EmergencyPlanRevision_Plan_Revision");
    }
}
```

```
b.HasOne(e => e.EmergencyPlan)
    .WithMany(p => p.Revisions)
    .HasForeignKey(e => e.EmergencyPlanId)
    .OnDelete(DeleteBehavior.Cascade)
    .HasConstraintName("FK_EmergencyPlanRevision_EmergencyPlan");

b.HasOne(e => e.PlanStatus)
    .WithMany()
    .HasForeignKey(e => e.PlanStatusId)
    .OnDelete(DeleteBehavior.Restrict)
    .HasConstraintName("FK_EmergencyPlanRevision_PlanStatus");
}
```


19.9 Exercise configurations

Exercise configuration connects scenario, type, status and plant scope. Participant configuration then makes attendance queryable by exercise and person with a unique key.

```
public sealed class ExerciseConfiguration : IEntityTypeConfiguration<Exercise>
{
    public void Configure(EntityTypeBuilder<Exercise> b)
    {
        b.ToTable("Exercise", "EmergencyPreparedness");
        b.HasKey(e => e.ExerciseId).HasName("PK_Exercise");

        b.Property(e => e.Code).HasMaxLength(40).IsRequired();
        b.Property(e => e.Name).HasMaxLength(220).IsRequired();
        b.Property(e => e.ScheduledStartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ScheduledEndUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ActualStartUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.ActualEndUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CreatedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique().HasDatabaseName("UQ_Exercise_Code");
        b.HasIndex(e => new { e.PlantId, e.ExerciseStatusId, e.ScheduledStartUtc })
            .HasDatabaseName("IX_Exercise_Plant_Status_Start");

        b.HasOne(e => e.ExerciseType).WithMany()
            .HasForeignKey(e => e.ExerciseTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Exercise_ExerciseType");
        b.HasOne(e => e.ExerciseStatus).WithMany()
            .HasForeignKey(e => e.ExerciseStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Exercise_ExerciseStatus");
        b.HasOne(e => e.EmergencyScenario).WithMany()
            .HasForeignKey(e => e.EmergencyScenarioId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Exercise_EmergencyScenario");
    }
}
```

```
public sealed class ExerciseParticipantConfiguration
    : IEntityTypeConfiguration<ExerciseParticipant>
{
    public void Configure(EntityTypeBuilder<ExerciseParticipant> b)
    {
        b.ToTable("ExerciseParticipant", "EmergencyPreparedness");
        b.HasKey(e => e.ExerciseParticipantId).HasName("PK_ExerciseParticipant");

        b.Property(e => e.AttendedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.Remarks).HasMaxLength(1000);

        b.HasIndex(e => new { e.ExerciseId, e.PersonId })
            .IsUnique()
            .HasDatabaseName("UQ_ExerciseParticipant_Exercise_Person");

        b.HasOne(e => e.Exercise)
            .WithMany(x => x.Participants)
            .HasForeignKey(e => e.ExerciseId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_ExerciseParticipant_Exercise");

        b.HasOne(e => e.Person).WithMany()
            .HasForeignKey(e => e.PersonId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ExerciseParticipant_Person");
    }
}
```

19.10 Routes, muster and protective action

Evacuation routes, muster records and protective actions are high-value operational data. Their mappings use strict delete behavior so a plan or exercise cannot accidentally delete people, sites, units or historical event records.

```
public sealed class EvacuationRouteConfiguration
    : IEntityTypeConfiguration<EvacuationRoute>
{
    public void Configure(EntityTypeBuilder<EvacuationRoute> b)
    {
        b.ToTable("EvacuationRoute", "EmergencyPreparedness");
        b.HasKey(e => e.EvacuationRouteId).HasName("PK_EvacuationRoute");

        b.Property(e => e.Code).HasMaxLength(40).IsRequired();
        b.Property(e => e.Name).HasMaxLength(220).IsRequired();
        b.Property(e => e.RouteDescription).HasMaxLength(2000);
        b.Property(e => e.EstimatedMinutes).HasPrecision(9, 2);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique().HasDatabaseName("UQ_EvacuationRoute_Code");
        b.HasIndex(e => new { e.PlantId, e.RouteStatusId })
            .HasDatabaseName("IX_EvacuationRoute_Plant_Status");

        b.HasOne(e => e.AssemblyPoint)
            .WithMany()
            .HasForeignKey(e => e.AssemblyPointId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EvacuationRoute_AssemblyPoint");
        b.HasOne(e => e.RouteStatus).WithMany()
            .HasForeignKey(e => e.RouteStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EvacuationRoute_RouteStatus");
    }
}
```

```
public sealed class MusterRecordConfiguration : IEntityTypeConfiguration<MusterRecord>
{
    public void Configure(EntityTypeBuilder<MusterRecord> b)
    {
        b.ToTable("MusterRecord", "EmergencyPreparedness");
        b.HasKey(e => e.MusterRecordId).HasName("PK_MusterRecord");

        b.Property(e => e.RecordedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.Comment).HasMaxLength(1000);

        b.HasIndex(e => new { e.ExerciseId, e.PersonId, e.RecordedAtUtc })
            .HasDatabaseName("IX_MusterRecord_Exercise_Person_Time");
        b.HasIndex(e => new { e.AssemblyPointId, e.MusterStatusId })
            .HasDatabaseName("IX_MusterRecord_AssemblyPoint_Status");

        b.HasOne(e => e.Exercise).WithMany()
            .HasForeignKey(e => e.ExerciseId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MusterRecord_Exercise");
        b.HasOne(e => e.Person).WithMany()
            .HasForeignKey(e => e.PersonId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MusterRecord_Person");
        b.HasOne(e => e.MusterStatus).WithMany()
            .HasForeignKey(e => e.MusterStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_MusterRecord_MusterStatus");
    }
}
```

```

public sealed class ProtectiveActionConfiguration
    : IEntityConfiguration<ProtectiveAction>
{
    public void Configure(EntityTypeBuilder<ProtectiveAction> b)
    {
        b.ToTable("ProtectiveAction", "EmergencyPreparedness");
        b.HasKey(e => e.ProtectiveActionId).HasName("PK_ProtectiveAction");

        b.Property(e => e.ActionText).HasMaxLength(1200).IsRequired();
        b.Property(e => e.OrderedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.CompletedAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => new { e.EmergencyClassificationId, e.ProtectiveActionStatusId })
            .HasDatabaseName("IX_ProtectiveAction_Class_Status");

        b.HasOne(e => e.EmergencyClassification).WithMany()
            .HasForeignKey(e => e.EmergencyClassificationId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ProtectiveAction_EmergencyClassification");
        b.HasOne(e => e.ProtectiveActionType).WithMany()
            .HasForeignKey(e => e.ProtectiveActionTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ProtectiveAction_ProtectiveActionType");
    }
}

```

19.11 Communications, resources and integration links

Communications separate the reusable template from the message instance. Resource mappings add readiness and quantity precision. Integration links connect plans to incidents, alarms and robotics missions without hiding those relationships in free text.

```
public sealed class EmergencyCommunicationConfiguration
    : IEntityTypeConfiguration<EmergencyCommunication>
{
    public void Configure(EntityTypeBuilder<EmergencyCommunication> b)
    {
        b.ToTable("EmergencyCommunication", "EmergencyPreparedness");
        b.HasKey(e => e.EmergencyCommunicationId).HasName("PK_EmergencyCommunication");

        b.Property(e => e.Recipient).HasMaxLength(320).IsRequired();
        b.Property(e => e.Subject).HasMaxLength(300);
        b.Property(e => e.Body).HasMaxLength(4000);
        b.Property(e => e.QueuedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);
        b.Property(e => e.SentAtUtc).HasColumnType("datetime2(7)");
        b.Property(e => e.FailureReason).HasMaxLength(1000);

        b.HasIndex(e => new { e.NotificationChannelId, e.CommunicationStatusId, e.QueuedAtUtc })
            .HasDatabaseName("IX_EmergencyCommunication_Channel_Status_Time");

        b.HasOne(e => e.NotificationTemplate).WithMany()
            .HasForeignKey(e => e.NotificationTemplateId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EmergencyCommunication_NotificationTemplate");
    }
}
```

```
public sealed class EmergencyResourceConfiguration
    : IEntityTypeConfiguration<EmergencyResource>
{
    public void Configure(EntityTypeBuilder<EmergencyResource> b)
    {
        b.ToTable("EmergencyResource", "EmergencyPreparedness");
        b.HasKey(e => e.EmergencyResourceId).HasName("PK_EmergencyResource");

        b.Property(e => e.Code).HasMaxLength(40).IsRequired();
        b.Property(e => e.Name).HasMaxLength(220).IsRequired();
        b.Property(e => e.Quantity).HasPrecision(18, 4);
        b.Property(e => e.LocationNote).HasMaxLength(500);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasIndex(e => e.Code).IsUnique().HasDatabaseName("UQ_EmergencyResource_Code");
        b.HasIndex(e => new { e.PlantId, e.ResourceTypeId, e.ResourceStatusId })
            .HasDatabaseName("IX_EmergencyResource_Plant_Type_Status");

        b.HasOne(e => e.ResourceType).WithMany()
            .HasForeignKey(e => e.ResourceTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EmergencyResource_ResourceType");
        b.HasOne(e => e.EngineeringUnit).WithMany()
            .HasForeignKey(e => e.EngineeringUnitId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EmergencyResource_EngineeringUnit");
    }
}
```

```
public sealed class PlanRobotMissionLinkConfiguration
    : IEntityTypeConfiguration<PlanRobotMissionLink>
{
    public void Configure(EntityTypeBuilder<PlanRobotMissionLink> b)
    {
        b.ToTable("PlanRobotMissionLink", "EmergencyPreparedness");
```

```

        b.HasKey(e => e.PlanRobotMissionLinkId)
            .HasName("PK_PlanRobotMissionLink");

        b.Property(e => e.LinkReason).HasMaxLength(700);
        b.Property(e => e.LinkedAtUtc).HasColumnType("datetime2(7)")
            .HasDefaultValueSql(NexusSql.UtcNow);

        b.HasIndex(e => new { e.EmergencyPlanId, e.MissionId })
            .IsUnique()
            .HasDatabaseName("UQ_PlanRobotMissionLink_Plan_Mission");

        b.HasOne(e => e.EmergencyPlan).WithMany()
            .HasForeignKey(e => e.EmergencyPlanId)
            .OnDelete(DeleteBehavior.Cascade)
            .HasConstraintName("FK_PlanRobotMissionLink_EmergencyPlan");
        b.HasOne(e => e.Mission).WithMany()
            .HasForeignKey(e => e.MissionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_PlanRobotMissionLink_Mission");
    }
}

```

19.12 Foreign-key mapping

The table below is the EF Core passport checklist for the sector. Internal links stay inside `EmergencyPreparedness` ; passport links cross to `CorePlatform`, `Security`, `Organization`, `ReactorFleet`, `EventManager`, `AlarmManagement`, `RadiationMonitoring` and `Robotics`.

ENTITY TABLE	FOREIGN KEY	TARGET	KIND
<code>EmergencyPreparedness.EmergencyPlan</code>	<code>PlanStatusId</code>	<code>EmergencyPreparedness.PlanStatus</code>	internal
<code>EmergencyPreparedness.EmergencyPlan</code>	<code>SiteId</code>	<code>Organization.Site</code>	passport
<code>EmergencyPreparedness.EmergencyPlan</code>	<code>PlantId</code>	<code>ReactorFleet.Plant</code>	passport
<code>EmergencyPreparedness.EmergencyPlan</code>	<code>OwnerUserId</code>	<code>Security.ApplicationUser</code>	passport
<code>EmergencyPreparedness.EmergencyPlanRevision</code>	<code>EmergencyPlanId</code>	<code>EmergencyPreparedness.EmergencyPlan</code>	internal
<code>EmergencyPreparedness.EmergencyPlanRevision</code>	<code>PlanStatusId</code>	<code>EmergencyPreparedness.PlanStatus</code>	internal
<code>EmergencyPreparedness.EmergencyPlanRevision</code>	<code>PreparedByUserId</code>	<code>Security.ApplicationUser</code>	passport
<code>EmergencyPreparedness.EmergencyPlanRevision</code>	<code>ApprovedByUserId</code>	<code>Security.ApplicationUser</code>	passport
<code>EmergencyPreparedness.EmergencyClassification</code>	<code>EmergencyClassTypeId</code>	<code>EmergencyPreparedness.EmergencyClassType</code>	internal
<code>EmergencyPreparedness.EmergencyClassification</code>	<code>UnitId</code>	<code>ReactorFleet.Unit</code>	passport
<code>EmergencyPreparedness.EmergencyClassification</code>	<code>OperationalEventId</code>	<code>EventManager.OperationalEvent</code>	passport
<code>EmergencyPreparedness.EmergencyClassification</code>	<code>IncidentId</code>	<code>EventManager.Incident</code>	passport
<code>EmergencyPreparedness.ResponseTeam</code>	<code>SiteId</code>	<code>Organization.Site</code>	passport
<code>EmergencyPreparedness.ResponseTeam</code>	<code>PlantId</code>	<code>ReactorFleet.Plant</code>	passport
<code>EmergencyPreparedness.ResponseTeam</code>	<code>TeamId</code>	<code>Organization.Team</code>	passport
<code>EmergencyPreparedness.ResponseTeamMember</code>	<code>ResponseTeamId</code>	<code>EmergencyPreparedness.ResponseTeam</code>	internal
<code>EmergencyPreparedness.ResponseTeamMember</code>	<code>PersonId</code>	<code>Organization.Person</code>	passport
<code>EmergencyPreparedness.ResponseTeamMember</code>	<code>EmergencyRoleId</code>	<code>EmergencyPreparedness.EmergencyRole</code>	internal
<code>EmergencyPreparedness.EmergencyScenario</code>	<code>EmergencyClassTypeId</code>	<code>EmergencyPreparedness.EmergencyClassType</code>	internal
<code>EmergencyPreparedness.EmergencyScenario</code>	<code>ScenarioStatusId</code>	<code>EmergencyPreparedness.ScenarioStatus</code>	internal
<code>EmergencyPreparedness.EmergencyScenario</code>	<code>PrimaryUnitId</code>	<code>ReactorFleet.Unit</code>	passport
<code>EmergencyPreparedness.Exercise</code>	<code>ExerciseTypeId</code>	<code>EmergencyPreparedness.ExerciseType</code>	internal
<code>EmergencyPreparedness.Exercise</code>	<code>ExerciseStatusId</code>	<code>EmergencyPreparedness.ExerciseStatus</code>	internal
<code>EmergencyPreparedness.Exercise</code>	<code>EmergencyScenarioId</code>	<code>EmergencyPreparedness.EmergencyScenario</code>	internal
<code>EmergencyPreparedness.Exercise</code>	<code>SiteId</code>	<code>Organization.Site</code>	passport
<code>EmergencyPreparedness.Exercise</code>	<code>PlantId</code>	<code>ReactorFleet.Plant</code>	passport

ENTITY TABLE	FOREIGN KEY	TARGET	KIND
EmergencyPreparedness.ExerciseParticipant	ExerciseId	EmergencyPreparedness.Exercise	internal
EmergencyPreparedness.ExerciseParticipant	PersonId	Organization.Person	passport
EmergencyPreparedness.ExerciseParticipant	ResponseTeamId	EmergencyPreparedness.ResponseTeam	internal
EmergencyPreparedness.ExerciseParticipant	EmergencyRoleId	EmergencyPreparedness.EmergencyRole	internal
EmergencyPreparedness.ExerciseParticipant	ParticipantStatusId	EmergencyPreparedness.ParticipantStatus	internal
EmergencyPreparedness.ExerciseInject	ExerciseId	EmergencyPreparedness.Exercise	internal
EmergencyPreparedness.ExerciseInject	InjectTypeId	EmergencyPreparedness.InjectType	internal
EmergencyPreparedness.ExerciseInject	DeliveredByUserId	Security.ApplicationUser	passport
EmergencyPreparedness.ExerciseObservation	ExerciseId	EmergencyPreparedness.Exercise	internal
EmergencyPreparedness.ExerciseObservation	ExerciseInjectId	EmergencyPreparedness.ExerciseInject	internal
EmergencyPreparedness.ExerciseObservation	ObservationSeverityId	EmergencyPreparedness.ObservationSeverity	internal
EmergencyPreparedness.ExerciseObservation	ObservedByUserId	Security.ApplicationUser	passport
EmergencyPreparedness.ExerciseEvaluation	ExerciseId	EmergencyPreparedness.Exercise	internal
EmergencyPreparedness.ExerciseEvaluation	EvaluationStatusId	EmergencyPreparedness.EvaluationStatus	internal
EmergencyPreparedness.ExerciseEvaluation	EvaluatedByUserId	Security.ApplicationUser	passport
EmergencyPreparedness.AssemblyPoint	SiteId	Organization.Site	passport
EmergencyPreparedness.AssemblyPoint	PlantId	ReactorFleet.Plant	passport
EmergencyPreparedness.AssemblyPoint	RadiationZoneId	RadiationMonitoring.RadiationZone	passport
EmergencyPreparedness.EvacuationRoute	SiteId	Organization.Site	passport
EmergencyPreparedness.EvacuationRoute	PlantId	ReactorFleet.Plant	passport
EmergencyPreparedness.EvacuationRoute	AssemblyPointId	EmergencyPreparedness.AssemblyPoint	internal
EmergencyPreparedness.EvacuationRoute	RouteStatusId	EmergencyPreparedness.RouteStatus	internal
EmergencyPreparedness.EvacuationRouteZone	EvacuationRouteId	EmergencyPreparedness.EvacuationRoute	internal
EmergencyPreparedness.EvacuationRouteZone	RadiationZoneId	RadiationMonitoring.RadiationZone	passport
EmergencyPreparedness.MusterRecord	ExerciseId	EmergencyPreparedness.Exercise	internal
EmergencyPreparedness.MusterRecord	OperationalEventId	EventManagement.OperationalEvent	passport
EmergencyPreparedness.MusterRecord	AssemblyPointId	EmergencyPreparedness.AssemblyPoint	internal
EmergencyPreparedness.MusterRecord	PersonId	Organization.Person	passport
EmergencyPreparedness.MusterRecord	MusterStatusId	EmergencyPreparedness.MusterStatus	internal
EmergencyPreparedness.ProtectiveAction	ProtectiveActionTypeId	EmergencyPreparedness.ProtectiveActionType	internal
EmergencyPreparedness.ProtectiveAction	ProtectiveActionStatusId	EmergencyPreparedness.ProtectiveActionStatus	internal

ENTITY TABLE	FOREIGN KEY	TARGET	KIND
EmergencyPreparedness.ProtectiveAction	EmergencyClassificationId	EmergencyPreparedness.EmergencyClassification	internal
EmergencyPreparedness.ProtectiveAction	UnitId	ReactorFleet.Unit	passport
EmergencyPreparedness.NotificationTemplate	NotificationChannelId	EmergencyPreparedness.NotificationChannel	internal
EmergencyPreparedness.NotificationTemplate	LanguageId	CorePlatform.Language	passport
EmergencyPreparedness.EmergencyCommunication	EmergencyClassificationId	EmergencyPreparedness.EmergencyClassification	internal
EmergencyPreparedness.EmergencyCommunication	ExerciseId	EmergencyPreparedness.Exercise	internal
EmergencyPreparedness.EmergencyCommunication	NotificationTemplateId	EmergencyPreparedness / cross-schema target	passport
EmergencyPreparedness.EmergencyCommunication	NotificationChannelId	EmergencyPreparedness.NotificationChannel	internal
EmergencyPreparedness.EmergencyResource	ResourceTypeId	EmergencyPreparedness.ResourceType	internal
EmergencyPreparedness.EmergencyResource	ResourceStatusId	EmergencyPreparedness.ResourceStatus	internal
EmergencyPreparedness.EmergencyResource	SiteId	Organization.Site	passport
EmergencyPreparedness.EmergencyResource	PlantId	ReactorFleet.Plant	passport
EmergencyPreparedness.EmergencyResource	OwnerTeamId	Organization.Team	passport
EmergencyPreparedness.EmergencyResource	EngineeringUnitId	CorePlatform.EngineeringUnit	passport
EmergencyPreparedness.ResourceReadinessCheck	EmergencyResourceId	EmergencyPreparedness.EmergencyResource	internal
EmergencyPreparedness.ResourceReadinessCheck	ReadinessStatusId	EmergencyPreparedness.ReadinessStatus	internal
EmergencyPreparedness.ResourceReadinessCheck	CheckedByUserId	Security.ApplicationUser	passport
EmergencyPreparedness.PlanIncidentLink	EmergencyPlanId	EmergencyPreparedness.EmergencyPlan	internal
EmergencyPreparedness.PlanIncidentLink	IncidentId	EventManager.Incident	passport
EmergencyPreparedness.PlanIncidentLink	LinkedByUserId	Security.ApplicationUser	passport
EmergencyPreparedness.PlanAlarmLink	EmergencyPlanId	EmergencyPreparedness.EmergencyPlan	internal
EmergencyPreparedness.PlanAlarmLink	AlarmEventId	AlarmManagement.AlarmEvent	passport
EmergencyPreparedness.PlanAlarmLink	LinkedByUserId	Security.ApplicationUser	passport
EmergencyPreparedness.PlanRobotMissionLink	EmergencyPlanId	EmergencyPreparedness.EmergencyPlan	internal
EmergencyPreparedness.PlanRobotMissionLink	MissionId	Robotics.Mission	passport
EmergencyPreparedness.PlanRobotMissionLink	LinkedByUserId	Security.ApplicationUser	passport

19.13 Migration and verification notes

The migration order mirrors the runtime dependency order: lookups first, independent roots second, dependent children third, cross-schema integration links last.

```
// DbContext discovery remains one line. Every class in this chapter is found automatically.
protected override void OnModelCreating(ModelBuilder m)
{
    m.ApplyConfigurationsFromAssembly(typeof(NexusContext).Assembly);
}

// Emergency preparedness migration order:
// 1. lookup tables
// 2. EmergencyPlan / EmergencyRole / EmergencyScenario
// 3. ResponseTeam and ResponseTeamMember
// 4. Exercise, participants, injects, observations and evaluations
// 5. Assembly points, evacuation routes and muster records
// 6. protective actions, communications and resources
// 7. integration links to incident, alarm and robotics sectors
```

Verification queries

```
-- Configuration classes discovered by convention: one file per entity.
SELECT COUNT(*) AS emergency_preparedness_tables
FROM sys.tables t
JOIN sys.schemas s ON s.schema_id = t.schema_id
WHERE s.name = 'EmergencyPreparedness';

-- Lookup codes must be unique and human readable.
SELECT Code, COUNT(*) AS duplicates
FROM EmergencyPreparedness.PlanStatus
GROUP BY Code
HAVING COUNT(*) > 1;

-- Active plans must have a status and owner.
SELECT p.EmergencyPlanId, p.Code, ps.Code AS status_code
FROM EmergencyPreparedness.EmergencyPlan p
JOIN EmergencyPreparedness.PlanStatus ps ON ps.PlanStatusId = p.PlanStatusId
WHERE p.OwnerUserId IS NULL;

-- Route readiness joins to radiation zones through a declared passport.
SELECT r.Code AS route_code, z.Code AS radiation_zone_code
FROM EmergencyPreparedness.EvacuationRouteZone rz
JOIN EmergencyPreparedness.EvacuationRoute r ON r.EvacuationRouteId = rz.EvacuationRouteId
JOIN RadiationMonitoring.RadiationZone z ON z.RadiationZoneId = rz.RadiationZoneId;
```

Honest boundary. These configurations make emergency-preparedness data consistent and queryable. They do not certify plans, validate routes, or claim operational readiness. NEXUS-1 remains a Phase-0 educational and demonstrator-grade platform.

19.14 Configuration file index

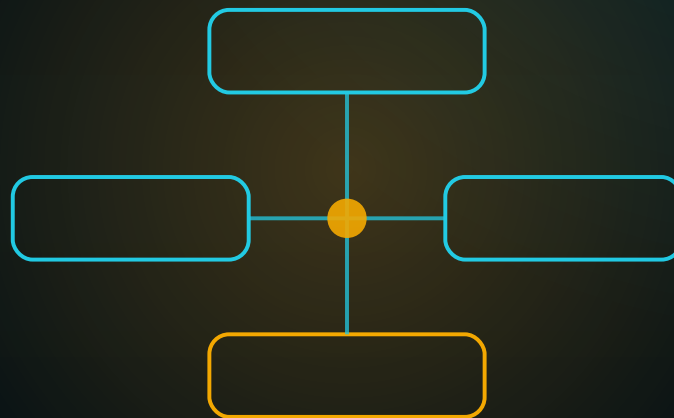
EmergencyPlanConfiguration.cs	Planning	PlanIncidentLinkConfiguration.cs	Integration
EmergencyPlanRevisionConfiguration.cs	Planning	PlanAlarmLinkConfiguration.cs	Integration
EmergencyClassificationConfiguration.cs	Classification	PlanRobotMissionLinkConfiguration.cs	Integration
EmergencyRoleConfiguration.cs	Teams	PlanStatusConfiguration.cs	Lookup
ResponseTeamConfiguration.cs	Teams	EmergencyClassTypeConfiguration.cs	Lookup
ResponseTeamMemberConfiguration.cs	Teams	ScenarioStatusConfiguration.cs	Lookup
EmergencyScenarioConfiguration.cs	Scenario	ExerciseTypeConfiguration.cs	Lookup
ExerciseConfiguration.cs	Exercise	ExerciseStatusConfiguration.cs	Lookup
ExerciseParticipantConfiguration.cs	Exercise	ParticipantStatusConfiguration.cs	Lookup
ExerciseInjectConfiguration.cs	Exercise	InjectTypeConfiguration.cs	Lookup
ExerciseObservationConfiguration.cs	Exercise	ObservationSeverityConfiguration.cs	Lookup
ExerciseEvaluationConfiguration.cs	Exercise	EvaluationStatusConfiguration.cs	Lookup
AssemblyPointConfiguration.cs	Routes	ProtectiveActionTypeConfiguration.cs	Lookup
EvacuationRouteConfiguration.cs	Routes	ProtectiveActionStatusConfiguration.cs	Lookup
EvacuationRouteZoneConfiguration.cs	Routes	ResourceTypeConfiguration.cs	Lookup
MusterRecordConfiguration.cs	Muster	ResourceStatusConfiguration.cs	Lookup
ProtectiveActionConfiguration.cs	Actions	ReadinessStatusConfiguration.cs	Lookup
NotificationTemplateConfiguration.cs	Comms	RouteStatusConfiguration.cs	Lookup
EmergencyCommunicationConfiguration.cs	Comms	MusterStatusConfiguration.cs	Lookup
EmergencyResourceConfiguration.cs	Resources	NotificationChannelConfiguration.cs	Lookup
ResourceReadinessCheckConfiguration.cs	Resources	CommunicationStatusConfiguration.cs	Lookup

CHAPTER 20

COMPLIANCE

CONFIGURATIONS

Mapping requirements, controls, audits, evidence, waivers, reviews, and corrective actions into explicit EF Core configuration files.



CONFIGURATION ATLAS STEP

One compliance model - controls, evidence, and findings - fully mapped

SQL Server / EF Core friendly mapping reference for the NEXUS-1 compliance sector.

Chapter 20 - Compliance Configurations

From Entity to Context

The EF Core Configuration Atlas of the NEXUS-1 Digital Twin

The Compliance schema is the governance layer of the platform. Its EF Core mappings are not only about table names and column lengths; they make requirements, controls, audits, evidence, waivers, and corrective actions explicit enough to survive migration, review, and audit.

Configuration principle. Compliance data must be traceable. The mapping therefore favours named keys, named indexes, restrictive deletes, row versions, stable code columns, and direct foreign-key passports to the sectors that produce evidence.

20.1 Purpose of the Compliance configuration layer

The SQL schema says what the Compliance sector stores. The EF Core configuration layer says how enterprise C# code respects that model. Every configuration file in this chapter keeps a single promise: a compliance fact should be explicit, named, constrained, and queryable.

Requirements

Regulations and atomic obligations become rows with status, priority, jurisdiction, and owner context.

Controls

Controls map to requirements through a bridge table so one control can satisfy many obligations.

Evidence

Evidence links requirements, controls, findings, actions, incidents, work orders, RCA runs, and exercises.

Boundary. This chapter is not legal advice and not a certified compliance framework. It is an EF Core mapping reference for a demonstrator-grade industrial digital-twin platform.

20.2 Configuration file catalogue

The catalogue follows the same chapter discipline as the previous configuration PDFs: one entity, one configuration class, one clear owner folder.

Lookup 24 configurations RegulationType, Jurisdiction, RequirementStatus, RequirementPriority, ControlType ...	Requirements 3 configurations RegulatoryBody, Regulation, Requirement	Controls 4 configurations Control, ControlRequirementMap, ComplianceProgram, ComplianceProgramControl
Audits 4 configurations Audit, AuditScope, AuditTeamMember, AuditFinding	Evidence and Actions 4 configurations NonConformance, CorrectiveAction, Evidence, EvidenceLink	Governance 5 configurations Attestation, Waiver, ComplianceReview, ReviewFinding, ObligationCalendar

GROUP	TABLE	CONFIGURATION FILE	MAPPING PURPOSE
Lookup	Compliance.RegulationType	RegulationTypeConfiguration.cs	Type of regulatory instrument: law, standard, guidance, licence condition.
Lookup	Compliance.Jurisdiction	JurisdictionConfiguration.cs	Jurisdiction or source boundary for requirements.
Lookup	Compliance.RequirementStatus	RequirementStatusConfiguration.cs	Lifecycle state of a requirement.
Lookup	Compliance.RequirementPriority	RequirementPriorityConfiguration.cs	Priority used when planning implementation and evidence collection.
Lookup	Compliance.ControlType	ControlTypeConfiguration.cs	Whether a control is preventive, detective, corrective, compensating, or governance.
Lookup	Compliance.ControlStatus	ControlStatusConfiguration.cs	Lifecycle state of a control.
Lookup	Compliance.ControlFrequency	ControlFrequencyConfiguration.cs	How often a control is performed or checked.
Lookup	Compliance.ComplianceProgramStatus	ComplianceProgramStatusConfiguration.cs	Lifecycle state of a compliance program.
Lookup	Compliance.AuditType	AuditTypeConfiguration.cs	Type of audit or assessment.
Lookup	Compliance.AuditStatus	AuditStatusConfiguration.cs	Lifecycle state of an audit.
Lookup	Compliance.AuditScopeType	AuditScopeTypeConfiguration.cs	What the audit scope covers.
Lookup	Compliance.AuditRoleType	AuditRoleTypeConfiguration.cs	Role of a participant in an audit team.
Lookup	Compliance.FindingSeverity	FindingSeverityConfiguration.cs	Severity classification for audit findings.
Lookup	Compliance.FindingStatus	FindingStatusConfiguration.cs	Lifecycle state for a finding.
Lookup	Compliance.CorrectiveActionStatus	CorrectiveActionStatusConfiguration.cs	Lifecycle state for a corrective action.
Lookup	Compliance.EvidenceType	EvidenceTypeConfiguration.cs	Kind of evidence held by the compliance record.
Lookup	Compliance.EvidenceStatus	EvidenceStatusConfiguration.cs	Lifecycle state for evidence.
Lookup	Compliance.AttestationStatus	AttestationStatusConfiguration.cs	State of a control attestation.

GROUP	TABLE	CONFIGURATION FILE	MAPPING PURPOSE
Lookup	Compliance.WaiverStatus	WaiverStatusConfiguration.cs	Lifecycle state for an approved exception or waiver.
Lookup	Compliance.WaiverReason	WaiverReasonConfiguration.cs	Reason category for a waiver.
Lookup	Compliance.NonConformanceType	NonConformanceTypeConfiguration.cs	Category of compliance non-conformance.
Lookup	Compliance.RiskRating	RiskRatingConfiguration.cs	Risk classification used by findings, waivers, and reviews.
Lookup	Compliance.ReviewStatus	ReviewStatusConfiguration.cs	Lifecycle state for a compliance review.
Lookup	Compliance.ObligationStatus	ObligationStatusConfiguration.cs	Lifecycle state for a scheduled obligation.
Requirements	Compliance.RegulatoryBody	RegulatoryBodyConfiguration.cs	Authority, standards body, or internal governance owner issuing compliance material.
Requirements	Compliance.Regulation	RegulationConfiguration.cs	Regulatory instrument or source document with jurisdiction, type, and body.
Requirements	Compliance.Requirement	RequirementConfiguration.cs	Atomic obligation extracted from a regulation and made queryable.
Controls	Compliance.Control	ControlConfiguration.cs	Control that implements or monitors one or more requirements.
Controls	Compliance.ControlRequirementMap	ControlRequirementMapConfiguration.cs	Many-to-many bridge between controls and requirements.
Controls	Compliance.ComplianceProgram	ComplianceProgramConfiguration.cs	Named program grouping controls, audits, and reviews for a site, plant, or unit.
Controls	Compliance.ComplianceProgramControl	ComplianceProgramControlConfiguration.cs	Bridge from compliance programs to their active controls.
Audits	Compliance.Audit	AuditConfiguration.cs	Audit or assessment event with owner, status, date range, and report metadata.
Audits	Compliance.AuditScope	AuditScopeConfiguration.cs	Declared target scope for an audit: site, plant, unit, requirement, control, or incident.
Audits	Compliance.AuditTeamMember	AuditTeamMemberConfiguration.cs	Person or user participating in an audit with a named role.
Audits	Compliance.AuditFinding	AuditFindingConfiguration.cs	Finding raised during an audit or review with severity and status.
Evidence and Actions	Compliance.NonConformance	NonConformanceConfiguration.cs	Non-conformance record connected to a finding or operational event.
Evidence and Actions	Compliance.CorrectiveAction	CorrectiveActionConfiguration.cs	Corrective action committed against a finding, non-conformance, or control gap.
Evidence and Actions	Compliance.Evidence	EvidenceConfiguration.cs	Evidence object with status, type, hash, and ownership metadata.
Evidence and Actions	Compliance.EvidenceLink	EvidenceLinkConfiguration.cs	Multi-target evidence bridge to requirements, controls, findings, actions, incidents, and exercises.
Governance	Compliance.Attestation	AttestationConfiguration.cs	Formal statement that a control operated for a period.

GROUP	TABLE	CONFIGURATION FILE	MAPPING PURPOSE
Governance	Compliance.Waiver	WaiverConfiguration.cs	Approved exception, compensating control, or time-bound deviation from a requirement.
Governance	Compliance.ComplianceReview	ComplianceReviewConfiguration.cs	Review cycle over program, site, plant, or unit compliance posture.
Governance	Compliance.ReviewFinding	ReviewFindingConfiguration.cs	Finding produced by a compliance review.
Governance	Compliance.ObligationCalendar	ObligationCalendarConfiguration.cs	Scheduled obligation with owner, due date, and completion state.

20.3 EF Core dependency diagram

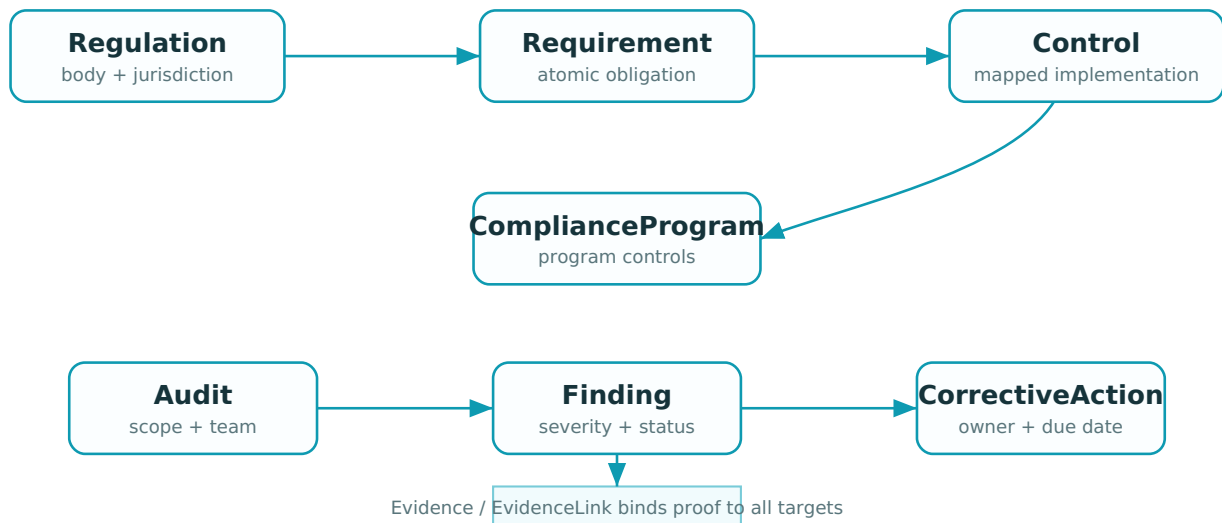


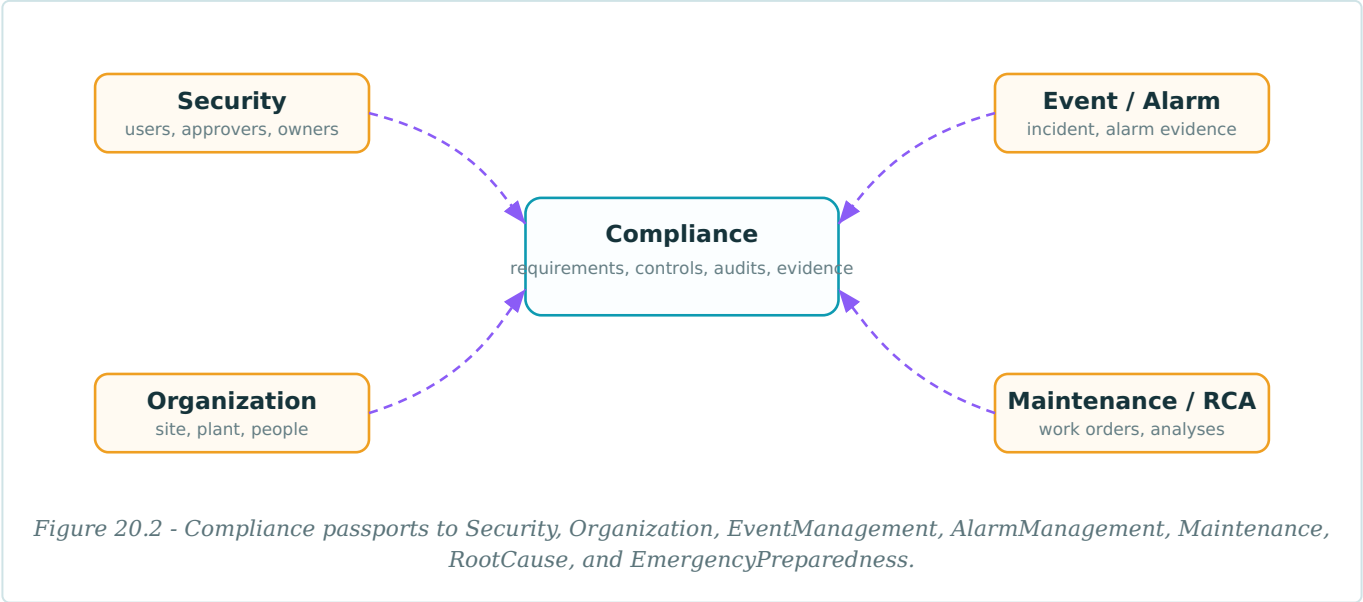
Figure 20.1 - Compliance mapping centre. Requirements map to controls; audits produce findings; findings drive corrective actions; evidence links proof to the whole chain.

Configuration folders

```
Nexus.Data/
  Configurations/
    Compliance/
      Lookups/
        RegulationTypeConfiguration.cs
        RequirementStatusConfiguration.cs
        FindingSeverityConfiguration.cs
        ...
      Requirements/
        RegulatoryBodyConfiguration.cs
        RegulationConfiguration.cs
        RequirementConfiguration.cs
      Controls/
        ControlConfiguration.cs
        ControlRequirementMapConfiguration.cs
        ComplianceProgramConfiguration.cs
      Audits/
        AuditConfiguration.cs
        AuditScopeConfiguration.cs
        AuditFindingConfiguration.cs
      EvidenceAndActions/
        EvidenceConfiguration.cs
        EvidenceLinkConfiguration.cs
        CorrectiveActionConfiguration.cs
      Governance/
        AttestationConfiguration.cs
        WaiverConfiguration.cs
        ComplianceReviewConfiguration.cs
```

20.4 Cross-schema passport map

Compliance depends on other sectors because evidence and responsibility originate elsewhere. Those relationships are passports: explicit foreign keys with restrictive delete behaviour, not copied strings.



20.5 Shared lookup configuration

The Compliance sector has many typed lookup tables. They should not be configured by repeating boilerplate in every file. The base class below gives the same shape to all lookup entities while preserving explicit table names.

```
public abstract class ComplianceLookupConfiguration<TEntity>
    : IEntityConfiguration<TEntity>
    where TEntity : ComplianceLookup
{
    protected abstract string TableName { get; }
    protected abstract string KeyName { get; }

    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.ToTable(TableName, "Compliance");
        b.HasKey(e => e.Id).HasName($"PK_Compliance_{TableName}");

        b.Property(e => e.Id).HasColumnName(KeyName);
        b.Property(e => e.Code).HasMaxLength(60).IsRequired();
        b.Property(e => e.Name).HasMaxLength(160).IsRequired();
        b.Property(e => e.Description).HasMaxLength(500);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName($"UQ_Compliance_{TableName}_Code");
    }
}
```

Use a dedicated configuration class per lookup table. The base class removes duplication; the concrete class preserves discoverability in the project tree.

Lookup catalogue

LOOKUP TABLE	REPRESENTATIVE SEED CODES	USED FOR
Compliance.RegulationType	LAW, REGULATION, STANDARD, GUIDANCE, LICENSE_CONDITION	Type of regulatory instrument: law, standard, guidance, licence condition.
Compliance.Jurisdiction	INTERNATIONAL, EU, NATIONAL, LOCAL, INTERNAL	Jurisdiction or source boundary for requirements.
Compliance.RequirementStatus	DRAFT, ACTIVE, SUPERSEDED, RETIRED, UNDER_REVIEW	Lifecycle state of a requirement.
Compliance.RequirementPriority	LOW, MEDIUM, HIGH, CRITICAL	Priority used when planning implementation and evidence collection.
Compliance.ControlType	PREVENTIVE, DETECTIVE, CORRECTIVE, COMPENSATING, GOVERNANCE	Whether a control is preventive, detective, corrective, compensating, or governance.
Compliance.ControlStatus	DRAFT, ACTIVE, INEFFECTIVE, RETIRED, UNDER_REMEDIATION	Lifecycle state of a control.
Compliance.ControlFrequency	CONTINUOUS, DAILY, WEEKLY, MONTHLY, QUARTERLY, ANNUAL, EVENT_DRIVEN	How often a control is performed or checked.
Compliance.ComplianceProgramStatus	DRAFT, ACTIVE, PAUSED, CLOSED, ARCHIVED	Lifecycle state of a compliance program.

LOOKUP TABLE	REPRESENTATIVE SEED CODES	USED FOR
Compliance.AuditType	INTERNAL, EXTERNAL, REGULATORY, SUPPLIER, FOLLOW_UP	Type of audit or assessment.
Compliance.AuditStatus	PLANNED, IN_PROGRESS, REPORTING, CLOSED, CANCELLED	Lifecycle state of an audit.
Compliance.AuditScopeType	SITE, PLANT, UNIT, SYSTEM, PROCESS, PROGRAM, SUPPLIER	What the audit scope covers.
Compliance.AuditRoleType	LEAD, AUDITOR, OBSERVER, SME, REGULATOR	Role of a participant in an audit team.
Compliance.FindingSeverity	INFO, MINOR, MAJOR, CRITICAL	Severity classification for audit findings.
Compliance.FindingStatus	OPEN, IN_REVIEW, ACTION_ASSIGNED, CLOSED, REJECTED	Lifecycle state for a finding.
Compliance.CorrectiveActionStatus	OPEN, ASSIGNED, IN_PROGRESS, BLOCKED, VERIFIED, CLOSED, CANCELLED	Lifecycle state for a corrective action.
Compliance.EvidenceType	DOCUMENT, PHOTO, SCREENSHOT, QUERY_RESULT, LOG_EXTRACT, SIGN_OFF, PROCEDURE	Kind of evidence held by the compliance record.
Compliance.EvidenceStatus	DRAFT, SUBMITTED, ACCEPTED, REJECTED, EXPIRED	Lifecycle state for evidence.
Compliance.AttestationStatus	REQUESTED, ATTESTED, REJECTED, EXPIRED, REVOKED	State of a control attestation.
Compliance.WaiverStatus	REQUESTED, APPROVED, REJECTED, EXPIRED, REVOKED	Lifecycle state for an approved exception or waiver.
Compliance.WaiverReason	TEMPORARY_CONSTRAINT, COMPENSATING_CONTROL, DESIGN_LIMITATION, AWAITING_OUTAGE, OTHER	Reason category for a waiver.
Compliance.NonConformanceType	PROCESS, EQUIPMENT, DOCUMENTATION, TRAINING, SECURITY, RADIOLOGICAL	Category of compliance non-conformance.
Compliance.RiskRating	LOW, MEDIUM, HIGH, EXTREME	Risk classification used by findings, waivers, and reviews.
Compliance.ReviewStatus	PLANNED, IN_PROGRESS, READY_FOR_APPROVAL, APPROVED, REWORK_REQUIRED, CLOSED	Lifecycle state for a compliance review.
Compliance.ObligationStatus	NOT_DUE, DUE_SOON, OVERDUE, SUBMITTED, ACCEPTED, MISSED	Lifecycle state for a scheduled obligation.

20.6 Requirement mapping

`RequirementConfiguration` is the centre of the requirement side of the model. It names the primary key, code uniqueness, row version, regulation passport, status lookup, and status-based query index.

```
public sealed class RequirementConfiguration
    : IEntityTypeConfiguration<Requirement>
{
    public void Configure(EntityTypeBuilder<Requirement> b)
    {
        b.ToTable("Requirement", "Compliance");
        b.HasKey(e => e.RequirementId)
            .HasName("PK_Compliance_Requirement");

        b.Property(e => e.Code).HasMaxLength(80).IsRequired();
        b.Property(e => e.Title).HasMaxLength(250).IsRequired();
        b.Property(e => e.RequirementText).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.ModifiedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.Regulation)
            .WithMany(e => e.Requirements)
            .HasForeignKey(e => e.RegulationId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Requirement_Regulation");

        b.HasOne(e => e.Status)
            .WithMany()
            .HasForeignKey(e => e.RequirementStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Requirement_Status");

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_Compliance_Requirement_Code");
        b.HasIndex(e => new { e.RegulationId, e.RequirementStatusId })
            .HasDatabaseName("IX_Compliance_Requirement_Regulation_Status");
    }
}
```

20.7 Control mapping

`ControlConfiguration` maps the operational control. A control has type, status, frequency, ownership, and a stable code. It can later be attached to requirements, programs, attestations, evidence, findings, and waivers.

```
public sealed class ControlConfiguration
    : IEntityTypeConfiguration<Control>
{
    public void Configure(EntityTypeBuilder<Control> b)
    {
        b.ToTable("Control", "Compliance");
        b.HasKey(e => e.ControlId).HasName("PK_Compliance_Control");

        b.Property(e => e.Code).HasMaxLength(80).IsRequired();
        b.Property(e => e.Title).HasMaxLength(250).IsRequired();
        b.Property(e => e.OwnerProcess).HasMaxLength(160);
        b.Property(e => e.CreatedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.ModifiedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.ControlType)
            .WithMany()
            .HasForeignKey(e => e.ControlTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Control_Type");

        b.HasOne(e => e.ControlFrequency)
            .WithMany()
            .HasForeignKey(e => e.ControlFrequencyId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_Control_Frequency");

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_Compliance_Control_Code");
    }
}
```

20.8 Audit finding mapping

`AuditFindingConfiguration` keeps audit findings queryable by audit and status. Finding severity and status are lookup references, not enums hidden in application code.

```
public sealed class AuditFindingConfiguration
    : IEntityTypeConfiguration<AuditFinding>
{
    public void Configure(EntityTypeBuilder<AuditFinding> b)
    {
        b.ToTable("AuditFinding", "Compliance");
        b.HasKey(e => e.AuditFindingId)
            .HasName("PK_Compliance_AuditFinding");

        b.Property(e => e.Code).HasMaxLength(80).IsRequired();
        b.Property(e => e.Title).HasMaxLength(250).IsRequired();
        b.Property(e => e.Description).IsRequired();
        b.Property(e => e.DetectedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.Audit)
            .WithMany(e => e.Findings)
            .HasForeignKey(e => e.AuditId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AuditFinding_Audit");

        b.HasOne(e => e.Severity)
            .WithMany()
            .HasForeignKey(e => e.FindingSeverityId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AuditFinding_Severity");

        b.HasIndex(e => new { e.AuditId, e.FindingStatusId })
            .HasDatabaseName("IX_Compliance_AuditFinding_Audit_Status");
    }
}
```

20.9 Evidence link mapping

Evidence is useful only when it can be attached to the object it proves. `EvidenceLink` is deliberately a multi-target bridge, because proof may belong to a requirement, control, finding, action, waiver, incident, work order, RCA analysis, or emergency exercise.

```
public sealed class EvidenceLinkConfiguration
    : IEntityTypeConfiguration<EvidenceLink>
{
    public void Configure(EntityTypeBuilder<EvidenceLink> b)
    {
        b.ToTable("EvidenceLink", "Compliance");
        b.HasKey(e => e.EvidenceLinkId)
            .HasName("PK_Compliance_EvidenceLink");

        b.Property(e => e.LinkedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.LinkRole).HasMaxLength(80).IsRequired();

        b.HasOne(e => e.Evidence)
            .WithMany(e => e.Links)
            .HasForeignKey(e => e.EvidenceId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EvidenceLink_Evidence");

        b.HasIndex(e => e.EvidenceId)
            .HasDatabaseName("IX_Compliance_EvidenceLink_EvidenceId");
        b.HasIndex(e => new { e.RequirementId, e.ControlId, e.AuditFindingId })
            .HasDatabaseName("IX_Compliance_EvidenceLink_Targets");
    }
}
```


20.10 Context discovery

The chapter keeps the same enterprise pattern used throughout the book. The `DbContext` stays clean; the assembly discovers each configuration class automatically.

```
public sealed class NexusContext : DbContext
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(NexusContext).Assembly);
    }
}
```

Why this matters. Compliance has too many relationships to hide inside a monolithic `OnModelCreating`. A separate file per entity makes review, migration debugging, and code ownership possible.

20.11 Foreign-key mapping

The table below lists the main configuration targets. Internal relationships stay inside `Compliance`; passport relationships cross into earlier sectors.

ENTITY	FK PROPERTY	TARGET	KIND
<code>Compliance.RegulatoryBody</code>	<code>JurisdictionId</code>	<code>Compliance.Jurisdiction(JurisdictionId)</code>	internal
<code>Compliance.RegulatoryBody</code>	<code>CountryId</code>	<code>CorePlatform.Country(CountryId)</code>	passport
<code>Compliance.ControlRequirementMap</code>	<code>ControlId</code>	<code>Compliance.Control(ControlId)</code>	internal
<code>Compliance.ControlRequirementMap</code>	<code>RequirementId</code>	<code>Compliance.Requirement(RequirementId)</code>	internal
<code>Compliance.ComplianceProgramControl</code>	<code>ComplianceProgramId</code>	<code>Compliance.ComplianceProgram(ComplianceProgramId)</code>	internal
<code>Compliance.ComplianceProgramControl</code>	<code>ControlId</code>	<code>Compliance.Control(ControlId)</code>	internal
<code>Compliance.AuditTeamMember</code>	<code>AuditId</code>	<code>Compliance.Audit(AuditId)</code>	internal
<code>Compliance.AuditTeamMember</code>	<code>ApplicationUserId</code>	<code>Security.ApplicationUser(ApplicationUserId)</code>	passport
<code>Compliance.AuditTeamMember</code>	<code>AuditRoleTypeId</code>	<code>Compliance.AuditRoleType(AuditRoleTypeId)</code>	internal
<code>Compliance.AuditFinding</code>	<code>AuditId</code>	<code>Compliance.Audit(AuditId)</code>	internal
<code>Compliance.AuditFinding</code>	<code>FindingSeverityId</code>	<code>Compliance.FindingSeverity(FindingSeverityId)</code>	internal
<code>Compliance.AuditFinding</code>	<code>FindingStatusId</code>	<code>Compliance.FindingStatus(FindingStatusId)</code>	internal
<code>Compliance.AuditFinding</code>	<code>RiskRatingId</code>	<code>Compliance.RiskRating(RiskRatingId)</code>	internal
<code>Compliance.AuditFinding</code>	<code>RequirementId</code>	<code>Compliance.Requirement(RequirementId)</code>	internal
<code>Compliance.AuditFinding</code>	<code>ControlId</code>	<code>Compliance.Control(ControlId)</code>	internal
<code>Compliance.AuditFinding</code>	<code>ResponsibleUserId</code>	<code>Security.ApplicationUser(ApplicationUserId)</code>	passport
<code>Compliance.EvidenceLink</code>	<code>EvidenceId</code>	<code>Compliance.Evidence(EvidenceId)</code>	internal
<code>Compliance.EvidenceLink</code>	<code>RequirementId</code>	<code>Compliance.Requirement(RequirementId)</code>	internal
<code>Compliance.EvidenceLink</code>	<code>ControlId</code>	<code>Compliance.Control(ControlId)</code>	internal
<code>Compliance.EvidenceLink</code>	<code>AuditId</code>	<code>Compliance.Audit(AuditId)</code>	internal
<code>Compliance.EvidenceLink</code>	<code>AuditFindingId</code>	<code>Compliance.AuditFinding(AuditFindingId)</code>	internal
<code>Compliance.EvidenceLink</code>	<code>CorrectiveActionId</code>	<code>Compliance.CorrectiveAction(CorrectiveActionId)</code>	internal
<code>Compliance.EvidenceLink</code>	<code>WaiverId</code>	<code>Compliance.Waiver(WaiverId)</code>	internal
<code>Compliance.EvidenceLink</code>	<code>NonConformanceId</code>	<code>Compliance.NonConformance(NonConformanceId)</code>	internal
<code>Compliance.EvidenceLink</code>	<code>IncidentId</code>	<code>EventManagement.Incident(IncidentId)</code>	passport
<code>Compliance.EvidenceLink</code>	<code>WorkOrderId</code>	<code>Maintenance.WorkOrder(WorkOrderId)</code>	passport

ENTITY	FK PROPERTY	TARGET	KIND
Compliance.EvidenceLink	RootCauseAnalysisId	RootCause.RootCauseAnalysis(RootCauseAnalysisId)	passport
Compliance.EvidenceLink	ExerciseId	EmergencyPreparedness.Exercise(ExerciseId)	passport
Compliance.EvidenceLink	LinkedByUserId	Security.ApplicationUser(ApplicationUserId)	passport
Compliance.ComplianceReview	ReviewStatusId	Compliance.ReviewStatus(ReviewStatusId)	internal
Compliance.ComplianceReview	ComplianceProgramId	Compliance.ComplianceProgram(ComplianceProgramId)	internal
Compliance.ComplianceReview	SiteId	Organization.Site(SiteId)	passport
Compliance.ComplianceReview	PlantId	Organization.Plant(PlantId)	passport
Compliance.ComplianceReview	UnitId	ReactorFleet.Unit(UnitId)	passport
Compliance.ComplianceReview	LedByUserId	Security.ApplicationUser(ApplicationUserId)	passport
Compliance.ComplianceReview	ApprovedByUserId	Security.ApplicationUser(ApplicationUserId)	passport
Compliance.ReviewFinding	ComplianceReviewId	Compliance.ComplianceReview(ComplianceReviewId)	internal
Compliance.ReviewFinding	RequirementId	Compliance.Requirement(RequirementId)	internal
Compliance.ReviewFinding	ControlId	Compliance.Control(ControlId)	internal
Compliance.ReviewFinding	AuditFindingId	Compliance.AuditFinding(AuditFindingId)	internal
Compliance.ReviewFinding	NonConformanceId	Compliance.NonConformance(NonConformanceId)	internal
Compliance.ReviewFinding	FindingSeverityId	Compliance.FindingSeverity(FindingSeverityId)	internal
Compliance.ReviewFinding	FindingStatusId	Compliance.FindingStatus(FindingStatusId)	internal
Compliance.ReviewFinding	RiskRatingId	Compliance.RiskRating(RiskRatingId)	internal
Compliance.ObligationCalendar	RequirementId	Compliance.Requirement(RequirementId)	internal
Compliance.ObligationCalendar	ObligationStatusId	Compliance.ObligationStatus(ObligationStatusId)	internal
Compliance.ObligationCalendar	ResponsibleUserId	Security.ApplicationUser(ApplicationUserId)	passport
Compliance.ObligationCalendar	SiteId	Organization.Site(SiteId)	passport
Compliance.ObligationCalendar	PlantId	Organization.Plant(PlantId)	passport
Compliance.ObligationCalendar	UnitId	ReactorFleet.Unit(UnitId)	passport

20.12 Migration and verification notes

Delete behaviour

Use `DeleteBehavior.Restrict` for compliance evidence, findings, actions, waivers, and reviews. Governance history should not disappear because a parent object is removed.

Stable codes

Every user-facing compliance object needs a unique code. It is the durable human identifier in reports, reviews, and audit conversations.

RowVersion

Mutable governance records should carry `IsRowVersion()` to prevent silent overwrite during review workflows.

Evidence hashes

Evidence should have a file hash or content hash column mapped with explicit length, so proof can be verified after export or archive.

```
// Verification query idea after migration
SELECT name
FROM sys.foreign_keys
WHERE parent_object_id = OBJECT_ID('Compliance.EvidenceLink')
ORDER BY name;

// EF Core model check idea
var entityCount = context.Model.GetEntityTypes()
    .Count(e => e.GetSchema() == "Compliance");

// Every Compliance configuration should be discovered by ApplyConfigurationsFromAssembly.
```

20.13 Configuration file index

RegulationTypeConfiguration.cs	Lookup	ReviewStatusConfiguration.cs	Lookup
JurisdictionConfiguration.cs	Lookup	ObligationStatusConfiguration.cs	Lookup
RequirementStatusConfiguration.cs	Lookup	RegulatoryBodyConfiguration.cs	Requirements
RequirementPriorityConfiguration.cs	Lookup	RegulationConfiguration.cs	Requirements
ControlTypeConfiguration.cs	Lookup	RequirementConfiguration.cs	Requirements
ControlStatusConfiguration.cs	Lookup	ControlConfiguration.cs	Controls
ControlFrequencyConfiguration.cs	Lookup	ControlRequirementMapConfiguration.cs	Controls
ComplianceProgramStatusConfiguration.cs	Lookup	ComplianceProgramConfiguration.cs	Controls
AuditTypeConfiguration.cs	Lookup	ComplianceProgramControlConfiguration.cs	Controls
AuditStatusConfiguration.cs	Lookup	AuditConfiguration.cs	Audits
AuditScopeTypeConfiguration.cs	Lookup	AuditScopeConfiguration.cs	Audits
AuditRoleTypeConfiguration.cs	Lookup	AuditTeamMemberConfiguration.cs	Audits
FindingSeverityConfiguration.cs	Lookup	AuditFindingConfiguration.cs	Audits
FindingStatusConfiguration.cs	Lookup	NonConformanceConfiguration.cs	Evidence and Actions
CorrectiveActionStatusConfiguration.cs	Lookup	CorrectiveActionConfiguration.cs	Evidence and Actions
EvidenceTypeConfiguration.cs	Lookup	EvidenceConfiguration.cs	Evidence and Actions
EvidenceStatusConfiguration.cs	Lookup	EvidenceLinkConfiguration.cs	Evidence and Actions
AttestationStatusConfiguration.cs	Lookup	AttestationConfiguration.cs	Governance
WaiverStatusConfiguration.cs	Lookup	WaiverConfiguration.cs	Governance
WaiverReasonConfiguration.cs	Lookup	ComplianceReviewConfiguration.cs	Governance
NonConformanceTypeConfiguration.cs	Lookup	ReviewFindingConfiguration.cs	Governance
RiskRatingConfiguration.cs	Lookup	ObligationCalendarConfiguration.cs	Governance

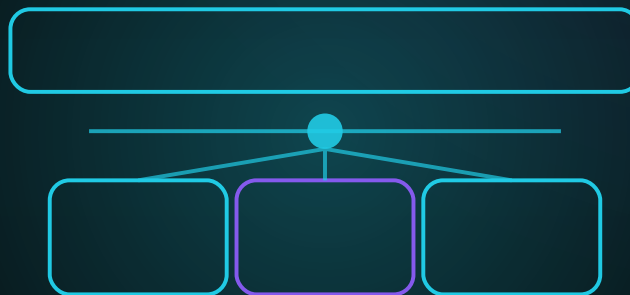
End of chapter boundary. The Compliance mapping layer makes the data backbone explicit. It does not certify compliance, interpret law, or replace a compliance officer. It gives the software a stable place to record requirements, evidence, findings, waivers, and reviews.

CHAPTER 21

REPORTING

CONFIGURATIONS

Mapping datasets, reports, dashboards, exports, KPI snapshots, subscriptions, and access grants into explicit EF Core configuration files.



CONFIGURATION ATLAS STEP

One reporting model - datasets, dashboards, exports - fully mapped

SQL Server / EF Core friendly mapping reference for the NEXUS-1 reporting sector.

Chapter 21 - Reporting Configurations

From Entity to Context

The EF Core Configuration Atlas of the NEXUS-1 Digital Twin

The Reporting schema is where operational truth becomes a readable product: reports, dashboards, exports, KPI snapshots, subscriptions, and archived snapshots. The EF Core mapping layer keeps those outputs stable, reproducible, and auditable.

Configuration principle. A report is not only a file. It is a definition, a dataset, parameters, a run, a status, an export, and often an access decision. Each of those becomes a mapped entity with named keys, indexes, and foreign-key passports.

21.1 Purpose of the Reporting configuration layer

The SQL schema defines the Reporting data backbone. The EF Core configuration layer makes that backbone readable to C# without allowing the `DbContext` to become a dumping ground. Every report, dataset, dashboard, and export has its own configuration class.

Datasets

Datasets declare source shape, columns, dependencies, and query identity before reports consume them.

Reports and exports

Definitions, parameters, runs, exports, files, and archives are mapped separately so output can be reproduced.

Dashboards and access

Widgets, KPIs, subscriptions, and access grants are explicit rows, not UI-only state.

Boundary. This chapter maps Reporting metadata and run history. It does not claim that every report is regulatory-grade; certification depends on validation, source governance, and operating procedures outside EF Core.

21.2 Configuration file catalogue

The catalogue is deliberately boring: one entity, one configuration file, one folder. That is how a large reporting model stays reviewable.

Lookup 12 configurations ReportType, ReportFormat, ReportRunStatus, ExportStatus, ScheduleFrequency ...	Data Sources 4 configurations DatasetDefinition, DatasetColumn, SavedQuery, ReportSourceDependency	Reports 5 configurations ReportDefinition, ReportParameter, ReportSchedule, ReportRun, ReportRunParameter
Exports 3 configurations ReportExport, ReportExportFile, ReportSnapshotArchive	Dashboards and KPIs 4 configurations Dashboard, KpiDefinition, DashboardWidget, KpiSnapshot	Delivery and Access 4 configurations DistributionList, DistributionListMember, ReportSubscription, ReportAccessGrant

GROUP	TABLE	CONFIGURATION FILE	MAPPING PURPOSE
Lookup	Reporting.ReportType	ReportTypeConfiguration.cs	Classifies reports: operational, compliance, executive, RCA, training, audit, and technical.
Lookup	Reporting.ReportFormat	ReportFormatConfiguration.cs	Output format for exports and deliveries: PDF, CSV, XLSX, JSON, HTML.
Lookup	Reporting.ReportRunStatus	ReportRunStatusConfiguration.cs	Lifecycle state of a report run.
Lookup	Reporting.ExportStatus	ExportStatusConfiguration.cs	Lifecycle state of an export file or package.
Lookup	Reporting.ScheduleFrequency	ScheduleFrequencyConfiguration.cs	Frequency for scheduled reporting.
Lookup	Reporting.WidgetType	WidgetTypeConfiguration.cs	Presentation type for dashboard widgets.
Lookup	Reporting.KpiCategory	KpiCategoryConfiguration.cs	Category used to group KPI definitions.
Lookup	Reporting.KpiStatus	KpiStatusConfiguration.cs	Status band used for KPI snapshots and dashboard rendering.
Lookup	Reporting.DatasetType	DatasetTypeConfiguration.cs	Classification for query datasets: SQL, view, stored procedure, materialized snapshot, or external feed.
Lookup	Reporting.AccessLevel	AccessLevelConfiguration.cs	Read/export/manage access level for a report, dashboard, or dataset.
Lookup	Reporting.SubscriptionStatus	SubscriptionStatusConfiguration.cs	Lifecycle state for report subscriptions.
Lookup	Reporting.DeliveryChannel	DeliveryChannelConfiguration.cs	Email, file drop, API callback, dashboard inbox, or manual download.
Data Sources	Reporting.DatasetDefinition	DatasetDefinitionConfiguration.cs	Named dataset exposed to reports and dashboards with source metadata and security scope.
Data Sources	Reporting.DatasetColumn	DatasetColumnConfiguration.cs	Declared columns of a dataset with type, unit, label, and display order.

GROUP	TABLE	CONFIGURATION FILE	MAPPING PURPOSE
Data Sources	Reporting.SavedQuery	SavedQueryConfiguration.cs	Versioned query text or query template used by datasets and reports.
Data Sources	Reporting.ReportSourceDependency	ReportSourceDependencyConfiguration.cs	Dependency list declaring which schemas, tables, views, or signals a report reads.
Reports	Reporting.ReportDefinition	ReportDefinitionConfiguration.cs	Report metadata: code, type, dataset, format, access model, owner, and version.
Reports	Reporting.ReportParameter	ReportParameterConfiguration.cs	Typed parameter definition for a report.
Reports	Reporting.ReportSchedule	ReportScheduleConfiguration.cs	Schedule definition attached to a report.
Reports	Reporting.ReportRun	ReportRunConfiguration.cs	One execution of a report with status, timing, actor, and parameter hash.
Reports	Reporting.ReportRunParameter	ReportRunParameterConfiguration.cs	Parameter values captured for one run.
Exports	Reporting.ReportExport	ReportExportConfiguration.cs	Export request or output package associated with a report run.
Exports	Reporting.ReportExportFile	ReportExportFileConfiguration.cs	Physical or logical file produced by an export, with hash and size metadata.
Exports	Reporting.ReportSnapshotArchive	ReportSnapshotArchiveConfiguration.cs	Archived report snapshot kept for reproducibility and audit.
Dashboards and KPIs	Reporting.Dashboard	DashboardConfiguration.cs	Dashboard definition with scope, owner, access model, and layout version.
Dashboards and KPIs	Reporting.KpiDefinition	KpiDefinitionConfiguration.cs	Reusable KPI definition with category, unit, threshold logic, and query binding.
Dashboards and KPIs	Reporting.DashboardWidget	DashboardWidgetConfiguration.cs	Widget placed on a dashboard, optionally bound to report, dataset, or KPI.
Dashboards and KPIs	Reporting.KpiSnapshot	KpiSnapshotConfiguration.cs	Point-in-time KPI value captured for trend and audit.
Delivery and Access	Reporting.DistributionList	DistributionListConfiguration.cs	Named list of users or recipients for report delivery.
Delivery and Access	Reporting.DistributionListMember	DistributionListMemberConfiguration.cs	Recipient member of a distribution list.
Delivery and Access	Reporting.ReportSubscription	ReportSubscriptionConfiguration.cs	Scheduled delivery preference for a user, list, report, or dashboard.
Delivery and Access	Reporting.ReportAccessGrant	ReportAccessGrantConfiguration.cs	Explicit access grant for reports, dashboards, datasets, exports, or saved queries.

21.3 EF Core dependency diagram

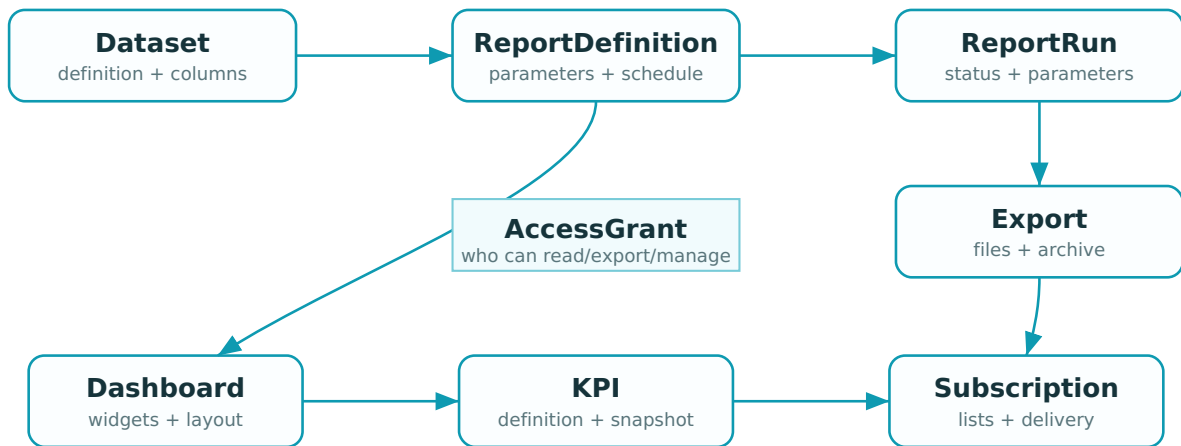


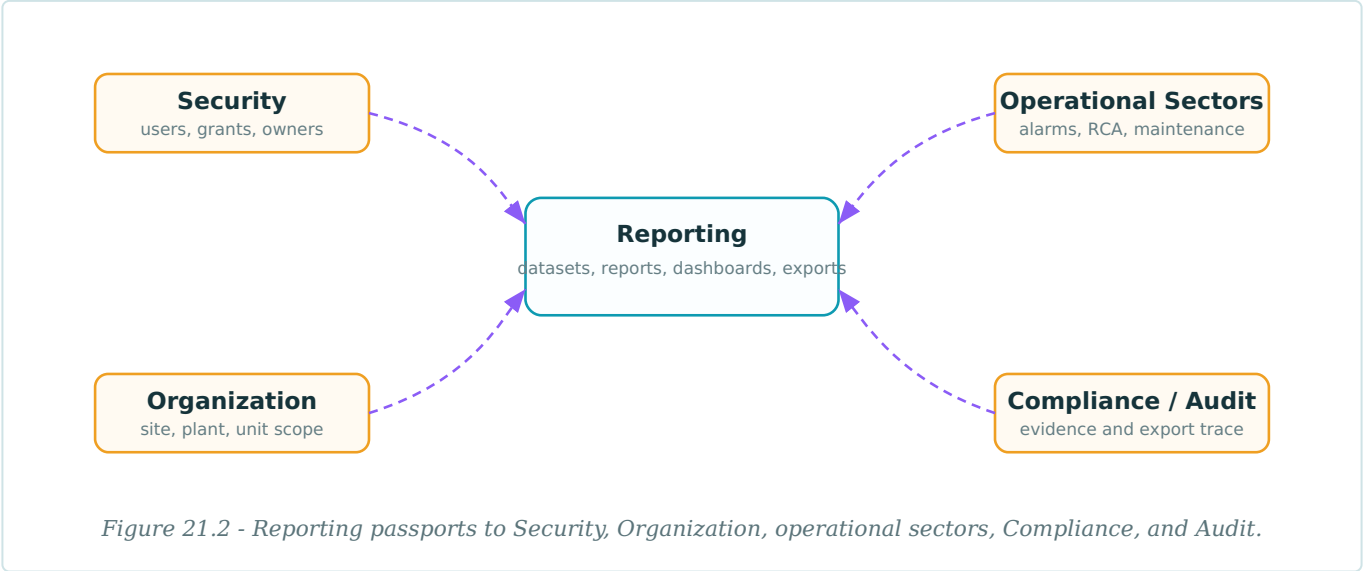
Figure 21.1 - Reporting mapping centre. Datasets feed reports; report runs produce exports and files; dashboards and KPI snapshots expose reusable output.

Configuration folders

```
Nexus.Data/
  Configurations/
    Reporting/
      Lookups/
        ReportTypeConfiguration.cs
        ReportFormatConfiguration.cs
        ReportRunStatusConfiguration.cs
        ...
      DataSources/
        DatasetDefinitionConfiguration.cs
        DatasetColumnConfiguration.cs
        SavedQueryConfiguration.cs
        ReportSourceDependencyConfiguration.cs
      Reports/
        ReportDefinitionConfiguration.cs
        ReportParameterConfiguration.cs
        ReportScheduleConfiguration.cs
        ReportRunConfiguration.cs
        ReportRunParameterConfiguration.cs
      Exports/
        ReportExportConfiguration.cs
        ReportExportFileConfiguration.cs
        ReportSnapshotArchiveConfiguration.cs
      Dashboards/
        DashboardConfiguration.cs
        KpiDefinitionConfiguration.cs
        DashboardWidgetConfiguration.cs
        KpiSnapshotConfiguration.cs
      DeliveryAndAccess/
        DistributionListConfiguration.cs
        DistributionListMemberConfiguration.cs
        ReportSubscriptionConfiguration.cs
        ReportAccessGrantConfiguration.cs
```

21.4 Cross-schema passport map

Reporting reads many sectors but should not duplicate their truth. Its configurations therefore use explicit passports for user ownership, unit/site scope, operational source dependencies, compliance evidence, and audit export trace.



21.5 Shared lookup configuration

The Reporting schema uses typed lookup tables for status, format, frequency, widget type, KPI category, access level, and delivery channel. The base class removes repetition while preserving concrete configuration files.

```
public abstract class ReportingLookupConfiguration<TEntity>
    : IEntityConfiguration<TEntity>
    where TEntity : ReportingLookup
{
    protected abstract string TableName { get; }
    protected abstract string KeyName { get; }

    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.ToTable(TableName, "Reporting");
        b.HasKey(e => e.Id).HasName($"PK_Reporting_{TableName}");
        b.Property(e => e.Id).HasColumnName(KeyName);

        b.Property(e => e.Code).HasMaxLength(60).IsRequired();
        b.Property(e => e.Name).HasMaxLength(160).IsRequired();
        b.Property(e => e.Description).HasMaxLength(500);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName($"UQ_Reporting_{TableName}_Code");
    }
}
```

Lookup catalogue

LOOKUP TABLE	REPRESENTATIVE SEED CODES	USED FOR
Reporting.ReportType	OPERATIONAL, COMPLIANCE, EXECUTIVE, ROOT_CAUSE, TRAINING, AUDIT	Classifies reports: operational, compliance, executive, RCA, training, audit, and technical.
Reporting.ReportFormat	PDF, CSV, XLSX, JSON, HTML	Output format for exports and deliveries: PDF, CSV, XLSX, JSON, HTML.
Reporting.ReportRunStatus	QUEUED, RUNNING, SUCCEEDED, FAILED, CANCELLED	Lifecycle state of a report run.
Reporting.ExportStatus	REQUESTED, GENERATING, READY, FAILED, EXPIRED	Lifecycle state of an export file or package.
Reporting.ScheduleFrequency	ONCE, HOURLY, DAILY, WEEKLY, MONTHLY, ON_DEMAND	Frequency for scheduled reporting.
Reporting.WidgetType	KPI_CARD, CHART, TABLE, TREND, MAP, STATUS_LIST	Presentation type for dashboard widgets.
Reporting.KpiCategory	SAFETY, PRODUCTION, RELIABILITY, COMPLIANCE, TRAINING	Category used to group KPI definitions.
Reporting.KpiStatus	GOOD, WATCH, WARNING, CRITICAL, UNKNOWN	Status band used for KPI snapshots and dashboard rendering.
Reporting.DatasetType	SQL, VIEW, STORED_PROCEDURE, SNAPSHOT, EXTERNAL_FEED	Classification for query datasets: SQL, view, stored procedure, materialized snapshot, or external feed.
Reporting.AccessLevel	READ, EXPORT, MANAGE, OWNER	Read/export/manage access level for a report, dashboard, or dataset.

LOOKUP TABLE	REPRESENTATIVE SEED CODES	USED FOR
Reporting.SubscriptionStatus	ACTIVE, PAUSED, FAILED, RETIRED	Lifecycle state for report subscriptions.
Reporting.DeliveryChannel	EMAIL, FILE_DROP, API_CALLBACK, DASHBOARD_INBOX, MANUAL	Email, file drop, API callback, dashboard inbox, or manual download.

21.6 Dataset mapping

`DatasetDefinitionConfiguration` is the entry point for reportable data. Reports bind to datasets, not to arbitrary ad hoc query text scattered through the application.

```
public sealed class DatasetDefinitionConfiguration
    : IEntityConfiguration<DatasetDefinition>
{
    public void Configure(EntityTypeBuilder<DatasetDefinition> b)
    {
        b.ToTable("DatasetDefinition", "Reporting");
        b.HasKey(e => e.DatasetDefinitionId)
            .HasName("PK_Reporting_DatasetDefinition");

        b.Property(e => e.Code).HasMaxLength(80).IsRequired();
        b.Property(e => e.Name).HasMaxLength(200).IsRequired();
        b.Property(e => e.SourceName).HasMaxLength(260).IsRequired();
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.DatasetType)
            .WithMany()
            .HasForeignKey(e => e.DatasetTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DatasetDefinition_DatasetType");

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_Reporting_DatasetDefinition_Code");
    }
}
```

21.7 Report definition mapping

`ReportDefinitionConfiguration` maps the stable report identity: code, title, dataset, type, format, version, and row version. It is the difference between a report as a durable contract and a report as a temporary query.

```
public sealed class ReportDefinitionConfiguration
    : IEntityConfiguration<ReportDefinition>
{
    public void Configure(EntityTypeBuilder<ReportDefinition> b)
    {
        b.ToTable("ReportDefinition", "Reporting");
        b.HasKey(e => e.ReportDefinitionId)
            .HasName("PK_Reporting_ReportDefinition");

        b.Property(e => e.Code).HasMaxLength(80).IsRequired();
        b.Property(e => e.Title).HasMaxLength(240).IsRequired();
        b.Property(e => e.Version).HasMaxLength(30).IsRequired();
        b.Property(e => e.CreatedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.DatasetDefinition)
            .WithMany(e => e.ReportDefinitions)
            .HasForeignKey(e => e.DatasetDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReportDefinition_DatasetDefinition");

        b.HasOne(e => e.ReportType)
            .WithMany()
            .HasForeignKey(e => e.ReportTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReportDefinition_ReportType");

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName("UQ_Reporting_ReportDefinition_Code");
    }
}
```


21.8 Report run mapping

`ReportRunConfiguration` records an execution. The run is where status, timing, requester, parameter hash, error text, and output chain begin.

```
public sealed class ReportRunConfiguration
    : IEntityTypeConfiguration<ReportRun>
{
    public void Configure(EntityTypeBuilder<ReportRun> b)
    {
        b.ToTable("ReportRun", "Reporting");
        b.HasKey(e => e.ReportRunId)
            .HasName("PK_Reporting_ReportRun");

        b.Property(e => e.StartedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.ParameterHash).HasMaxLength(128);
        b.Property(e => e.ErrorMessage).HasMaxLength(2000);

        b.HasOne(e => e.ReportDefinition)
            .WithMany(e => e.Runs)
            .HasForeignKey(e => e.ReportDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReportRun_ReportDefinition");

        b.HasOne(e => e.RunStatus)
            .WithMany()
            .HasForeignKey(e => e.ReportRunStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReportRun_Status");

        b.HasIndex(e => new { e.ReportDefinitionId, e.StartedAtUtc })
            .HasDatabaseName("IX_Reporting_ReportRun_Report_StartedAtUtc");
    }
}
```

21.9 Export mapping

`ReportExportConfiguration` maps the output request and its status. Export files then attach as child rows, each with hash, size, content type, and storage path.

```
public sealed class ReportExportConfiguration
    : IEntityConfiguration<ReportExport>
{
    public void Configure(EntityTypeBuilder<ReportExport> b)
    {
        b.ToTable("ReportExport", "Reporting");
        b.HasKey(e => e.ReportExportId)
            .HasName("PK_Reporting_ReportExport");

        b.Property(e => e.RequestedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.ExportName).HasMaxLength(220).IsRequired();

        b.HasOne(e => e.ReportRun)
            .WithMany(e => e.Exports)
            .HasForeignKey(e => e.ReportRunId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReportExport_ReportRun");

        b.HasOne(e => e.Format)
            .WithMany()
            .HasForeignKey(e => e.ReportFormatId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReportExport_Format");

        b.HasIndex(e => new { e.ReportRunId, e.ExportStatusId })
            .HasDatabaseName("IX_Reporting_ReportExport_Run_Status");
    }
}
```

21.10 Dashboard widget mapping

Dashboards are not only frontend layout. A widget has type, position, configuration JSON, optional dataset, report, KPI binding, and display order. That state belongs in the database.

```
public sealed class DashboardWidgetConfiguration
    : IEntityTypeConfiguration<DashboardWidget>
{
    public void Configure(EntityTypeBuilder<DashboardWidget> b)
    {
        b.ToTable("DashboardWidget", "Reporting");
        b.HasKey(e => e.DashboardWidgetId)
            .HasName("PK_Reporting_DashboardWidget");

        b.Property(e => e.Title).HasMaxLength(180).IsRequired();
        b.Property(e => e.PositionJson).IsRequired();
        b.Property(e => e.ConfigurationJson).IsRequired();

        b.HasOne(e => e.Dashboard)
            .WithMany(e => e.Widgets)
            .HasForeignKey(e => e.DashboardId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DashboardWidget_Dashboard");

        b.HasOne(e => e.WidgetType)
            .WithMany()
            .HasForeignKey(e => e.WidgetTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_DashboardWidget_WidgetType");

        b.HasIndex(e => new { e.DashboardId, e.DisplayOrder })
            .HasDatabaseName("IX_Reporting_DashboardWidget_Dashboard_Order");
    }
}
```

21.11 Subscription and access mapping

Subscriptions decide who receives a report and when. Access grants decide who can read, export, manage, or own a reporting object. Both are explicit mapped entities.

```
public sealed class ReportSubscriptionConfiguration
    : IEntityTypeConfiguration<ReportSubscription>
{
    public void Configure(EntityTypeBuilder<ReportSubscription> b)
    {
        b.ToTable("ReportSubscription", "Reporting");
        b.HasKey(e => e.ReportSubscriptionId)
            .HasName("PK_Reporting_ReportSubscription");

        b.Property(e => e.CreatedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.FilterJson).HasMaxLength(4000);

        b.HasOne(e => e.ReportDefinition)
            .WithMany(e => e.Subscriptions)
            .HasForeignKey(e => e.ReportDefinitionId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReportSubscription_ReportDefinition");

        b.HasOne(e => e.DeliveryChannel)
            .WithMany()
            .HasForeignKey(e => e.DeliveryChannelId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReportSubscription_DeliveryChannel");

        b.HasIndex(e => new { e.ReportDefinitionId, e.SubscriptionStatusId })
            .HasDatabaseName("IX_Reporting_ReportSubscription_Report_Status");
    }
}
```

```
public sealed class ReportAccessGrantConfiguration
    : IEntityTypeConfiguration<ReportAccessGrant>
{
    public void Configure(EntityTypeBuilder<ReportAccessGrant> b)
    {
        b.ToTable("ReportAccessGrant", "Reporting");
        b.HasKey(e => e.ReportAccessGrantId)
            .HasName("PK_Reporting_ReportAccessGrant");

        b.Property(e => e.TargetType).HasMaxLength(40).IsRequired();
        b.Property(e => e.TargetKey).HasMaxLength(120).IsRequired();
        b.Property(e => e.GrantedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");

        b.HasOne(e => e.AccessLevel)
            .WithMany()
            .HasForeignKey(e => e.AccessLevelId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ReportAccessGrant_AccessLevel");

        b.HasIndex(e => new { e.TargetType, e.TargetKey, e.AccessLevelId })
            .HasDatabaseName("IX_Reporting_ReportAccessGrant_Target_Access");
    }
}
```

21.12 Context discovery

The `DbContext` remains clean. Each Reporting configuration class is discovered automatically by assembly scan.

```
public sealed class NexusContext : DbContext
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(NexusContext).Assembly);
    }
}
```

Why this matters. Reporting changes often. Keeping mapping files separate prevents report evolution from turning `OnModelCreating` into an unreadable central switchboard.

21.13 Foreign-key mapping

The table below lists the main configuration targets. Internal relationships stay inside **Reporting**; passport relationships cross into earlier sectors.

ENTITY	FK PROPERTY	TARGET	KIND
Reporting.DatasetDefinition	DatasetTypeId	Reporting.DatasetType(DatasetTypeId)	internal
Reporting.DatasetDefinition	OwnerUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.DatasetColumn	DatasetDefinitionId	Reporting.DatasetDefinition(DatasetDefinitionId)	internal
Reporting.DatasetColumn	EngineeringUnitId	CorePlatform.EngineeringUnit(EngineeringUnitId)	passport
Reporting.SavedQuery	DatasetDefinitionId	Reporting.DatasetDefinition(DatasetDefinitionId)	internal
Reporting.SavedQuery	SavedByUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.ReportSourceDependency	DatasetDefinitionId	Reporting.DatasetDefinition(DatasetDefinitionId)	internal
Reporting.ReportDefinition	ReportTypeId	Reporting.ReportType(ReportTypeId)	internal
Reporting.ReportDefinition	DatasetDefinitionId	Reporting.DatasetDefinition(DatasetDefinitionId)	internal
Reporting.ReportDefinition	DefaultFormatId	Reporting.ReportFormat(ReportFormatId)	internal
Reporting.ReportDefinition	OwnerUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.ReportDefinition	ReportDefinitionId	Reporting.ReportDefinition(ReportDefinitionId)	internal
Reporting.ReportParameter	ReportDefinitionId	Reporting.ReportDefinition(ReportDefinitionId)	internal
Reporting.ReportSchedule	ReportDefinitionId	Reporting.ReportDefinition(ReportDefinitionId)	internal
Reporting.ReportSchedule	ScheduleFrequencyId	Reporting.ScheduleFrequency(ScheduleFrequencyId)	internal
Reporting.ReportSchedule	TimeZoneId	CorePlatform.TimeZone(TimeZoneId)	passport
Reporting.ReportSchedule	CreatedByUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.ReportRunParameter	ReportRunId	Reporting.ReportRun(ReportRunId)	internal
Reporting.ReportExport	ReportRunId	Reporting.ReportRun(ReportRunId)	internal
Reporting.ReportExport	ReportFormatId	Reporting.ReportFormat(ReportFormatId)	internal
Reporting.ReportExport	ExportStatusId	Reporting.ExportStatus(ExportStatusId)	internal
Reporting.ReportExport	RequestedByUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.ReportExportFile	ReportExportId	Reporting.ReportExport(ReportExportId)	internal

ENTITY	FK PROPERTY	TARGET	KIND
Reporting.ReportSnapshotArchive	ReportRunId	Reporting.ReportRun(ReportRunId)	internal
Reporting.Dashboard	OwnerUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.Dashboard	SiteId	Organization.Site(SiteId)	passport
Reporting.Dashboard	PlantId	Organization.Plant(PlantId)	passport
Reporting.Dashboard	UnitId	ReactorFleet.Unit(UnitId)	passport
Reporting.KpiDefinition	KpiCategoryId	Reporting.KpiCategory(KpiCategoryId)	internal
Reporting.KpiDefinition	DatasetDefinitionId	Reporting.DatasetDefinition(DatasetDefinitionId)	internal
Reporting.KpiDefinition	EngineeringUnitId	CorePlatform.EngineeringUnit(EngineeringUnitId)	passport
Reporting.KpiDefinition	OwnerUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.DashboardWidget	DashboardId	Reporting.Dashboard(DashboardId)	internal
Reporting.DashboardWidget	WidgetTypeId	Reporting.WidgetType(WidgetTypeId)	internal
Reporting.DashboardWidget	KpiDefinitionId	Reporting.KpiDefinition(KpiDefinitionId)	internal
Reporting.DashboardWidget	ReportDefinitionId	Reporting.ReportDefinition(ReportDefinitionId)	internal
Reporting.DashboardWidget	DatasetDefinitionId	Reporting.DatasetDefinition(DatasetDefinitionId)	internal
Reporting.KpiSnapshot	KpiDefinitionId	Reporting.KpiDefinition(KpiDefinitionId)	internal
Reporting.KpiSnapshot	KpiStatusId	Reporting.KpiStatus(KpiStatusId)	internal
Reporting.KpiSnapshot	EngineeringUnitId	CorePlatform.EngineeringUnit(EngineeringUnitId)	passport
Reporting.KpiSnapshot	SiteId	Organization.Site(SiteId)	passport
Reporting.KpiSnapshot	PlantId	Organization.Plant(PlantId)	passport
Reporting.KpiSnapshot	UnitId	ReactorFleet.Unit(UnitId)	passport
Reporting.DistributionList	OwnerUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.DistributionListMember	DistributionListId	Reporting.DistributionList(DistributionListId)	internal
Reporting.DistributionListMember	ApplicationUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.ReportSubscription	ReportDefinitionId	Reporting.ReportDefinition(ReportDefinitionId)	internal
Reporting.ReportSubscription	ApplicationUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.ReportSubscription	DistributionListId	Reporting.DistributionList(DistributionListId)	internal
Reporting.ReportSubscription	SubscriptionStatusId	Reporting.SubscriptionStatus(SubscriptionStatusId)	internal
Reporting.ReportSubscription	DeliveryChannelId	Reporting.DeliveryChannel(DeliveryChannelId)	internal

ENTITY	FK PROPERTY	TARGET	KIND
Reporting.ReportSubscription	ReportFormatId	Reporting.ReportFormat(ReportFormatId)	internal
Reporting.ReportSubscription	ScheduleFrequencyId	Reporting.ScheduleFrequency(ScheduleFrequencyId)	internal
Reporting.ReportSubscription	TimeZoneId	CorePlatform.TimeZone(TimeZoneId)	passport
Reporting.ReportAccessGrant	ReportDefinitionId	Reporting.ReportDefinition(ReportDefinitionId)	internal
Reporting.ReportAccessGrant	DashboardId	Reporting.Dashboard(DashboardId)	internal
Reporting.ReportAccessGrant	DatasetDefinitionId	Reporting.DatasetDefinition(DatasetDefinitionId)	internal
Reporting.ReportAccessGrant	AccessLevelId	Reporting.AccessLevel(AccessLevelId)	internal
Reporting.ReportAccessGrant	ApplicationUserId	Security.ApplicationUser(ApplicationUserId)	passport
Reporting.ReportAccessGrant	ApplicationRoleId	Security.ApplicationRole(ApplicationRoleId)	passport
Reporting.ReportAccessGrant	GrantedByUserId	Security.ApplicationUser(ApplicationUserId)	passport

21.14 Migration and verification notes

Restrictive deletes

Use `DeleteBehavior.Restrict` for report definitions, runs, exports, snapshots, and access grants. Output history should not vanish accidentally.

Stable report codes

Each report and dataset gets a unique code. The code is what documentation, subscriptions, and access rules can name safely.

JSON columns

Position, configuration, filter, and parameter JSON should be mapped deliberately and validated by application services.

Export reproducibility

Exports need file hashes, parameter hashes, run timestamps, and archived snapshots to support repeatability.

```
// Verification query idea after migration
SELECT name
FROM sys.indexes
WHERE object_id = OBJECT_ID('Reporting.ReportDefinition')
ORDER BY name;

// EF Core model check idea
var reportingTypes = context.Model.GetEntityType()
    .Where(e => e.GetSchema() == "Reporting")
    .Select(e => e.GetTableName())
    .OrderBy(n => n)
    .ToList();

// Every Reporting configuration should be discovered by ApplyConfigurationsFromAssembly.
```

21.15 Configuration file index

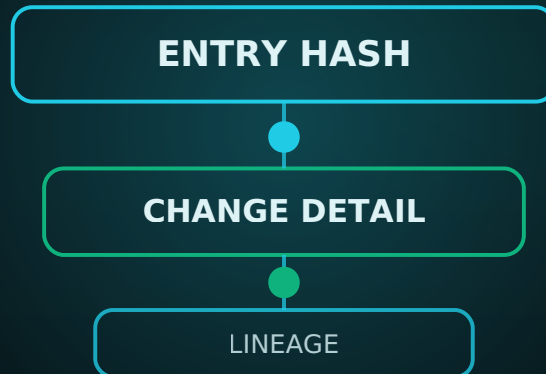
ReportTypeConfiguration.cs	Lookup	ReportDefinitionConfiguration.cs	Reports
ReportFormatConfiguration.cs	Lookup	ReportParameterConfiguration.cs	Reports
ReportRunStatusConfiguration.cs	Lookup	ReportScheduleConfiguration.cs	Reports
ExportStatusConfiguration.cs	Lookup	ReportRunConfiguration.cs	Reports
ScheduleFrequencyConfiguration.cs	Lookup	ReportRunParameterConfiguration.cs	Reports
WidgetTypeConfiguration.cs	Lookup	ReportExportConfiguration.cs	Exports
KpiCategoryConfiguration.cs	Lookup	ReportExportFileConfiguration.cs	Exports
KpiStatusConfiguration.cs	Lookup	ReportSnapshotArchiveConfiguration.cs	Exports
DatasetTypeConfiguration.cs	Lookup	DashboardConfiguration.cs	Dashboards and KPIs
AccessLevelConfiguration.cs	Lookup	KpiDefinitionConfiguration.cs	Dashboards and KPIs
SubscriptionStatusConfiguration.cs	Lookup	DashboardWidgetConfiguration.cs	Dashboards and KPIs
DeliveryChannelConfiguration.cs	Lookup	KpiSnapshotConfiguration.cs	Dashboards and KPIs
DatasetDefinitionConfiguration.cs	Data Sources	DistributionListConfiguration.cs	Delivery and Access
DatasetColumnConfiguration.cs	Data Sources	DistributionListMemberConfiguration.cs	Delivery and Access
SavedQueryConfiguration.cs	Data Sources	ReportSubscriptionConfiguration.cs	Delivery and Access
ReportSourceDependencyConfiguration.cs	Data Sources	ReportAccessGrantConfiguration.cs	Delivery and Access

End of chapter boundary. The Reporting configuration layer makes outputs explicit and reproducible. It does not guarantee the correctness of a source query; that remains the job of dataset review, test data, validation queries, and audit trace.

CHAPTER 22

AUDIT CONFIGURATIONS

Mapping immutable entries, entity changes, property changes, lineage, access checks, exports, signatures, retention, and integration trace into explicit EF Core configuration files.



CONFIGURATION ATLAS STEP

One audit chain - every change traceable - fully mapped

SQL Server / EF Core friendly mapping reference for the NEXUS-1 audit sector.

Chapter 22 - Audit Configurations

From Entity to Context

The EF Core Configuration Atlas of the NEXUS-1 Digital Twin

The Audit schema is the final mapped sector. It does not model a plant component or a report output; it models accountability. The EF Core configuration layer keeps the audit chain append-only in spirit, explicit in naming, and stable under migration.

Configuration principle. Audit mappings favour restrictive deletes, stable target identity, explicit hashes, typed lookups, and named indexes. Nothing should disappear because a parent business row was removed.

22.1 Purpose of the Audit configuration layer

The SQL schema defines the tables. The EF Core configuration layer defines how C# respects them: table names, key names, lookup patterns, relationship delete behaviour, target indexes, and hash fields. In a large system this is not decoration; it is how the audit record remains trustworthy.

Immutable record

Audit entries, changes, and property details are mapped so history is appended and reviewed, not casually overwritten.

Traceable action

Commands, access checks, exports, signatures, and retention runs all carry actor, status, and timing context.

Lineage

Data lineage and evidence links connect derived outputs back to source entities and evidence records.

Boundary. EF Core can enforce mapping discipline, but it cannot by itself make an audit regime compliant. Compliance also needs policy, access control, review process, and operational governance.

22.2 Configuration file catalogue

One entity, one configuration file. This is especially important for Audit because many tables are shared by every other sector.

Lookup 11 configurations AuditActionType, AuditEntityType, AuditSeverity, ChangeSourceType, CommandStatus ...	Audit Chain 4 configurations AuditBatch, AuditEntry, EntityChange, PropertyChange	Commands and Integration 3 configurations CommandLog, IntegrationMessage, IntegrationMessageAttempt
Lineage and Evidence 3 configurations DataLineage, LineageLink, AuditEvidenceLink	Access, Export, Signatures 3 configurations AccessAudit, ExportAudit, SignatureRecord	Retention and Temporal 3 configurations RetentionPolicy, RetentionExecution, TemporalTypeRegistry

GROUP	TABLE	CONFIGURATION FILE	MAPPING PURPOSE
Lookup	Audit.AuditActionType	AuditActionTypeConfiguration.cs	Classifies the action recorded: insert, update, delete, sign, export, login, retention, or system action.
Lookup	Audit.AuditEntityType	AuditEntityTypeConfiguration.cs	Classifies what kind of target was changed or observed: business entity, document, signal, report, policy, or configuration.
Lookup	Audit.AuditSeverity	AuditSeverityConfiguration.cs	Severity band for audit events that need attention or escalation.
Lookup	Audit.ChangeSourceType	ChangeSourceTypeConfiguration.cs	Origin of the change: user, API, job, import, system, simulation, or integration.
Lookup	Audit.CommandStatus	CommandStatusConfiguration.cs	Execution status for commands and retention runs.
Lookup	Audit.IntegrationMessageStatus	IntegrationMessageStatusConfiguration.cs	Lifecycle state for inbound or outbound integration messages.
Lookup	Audit.RetentionActionType	RetentionActionTypeConfiguration.cs	Retention action: archive, delete, anonymize, seal, or review.
Lookup	Audit.RetentionRuleStatus	RetentionRuleStatusConfiguration.cs	Lifecycle status for retention rules and policies.
Lookup	Audit.SignatureStatus	SignatureStatusConfiguration.cs	Validation state for electronic signatures.
Lookup	Audit.AccessResult	AccessResultConfiguration.cs	Allowed, denied, challenged, expired, or not applicable access decisions.
Lookup	Audit.ExportPurpose	ExportPurposeConfiguration.cs	Reason for a data export: operational, compliance, investigation, training, or support.
Audit Chain	Audit.AuditBatch	AuditBatchConfiguration.cs	Groups related audit entries into one transaction, command, job, import, or service operation.
Audit Chain	Audit.AuditEntry	AuditEntryConfiguration.cs	The central immutable audit event: actor, action, target identity, correlation, hashes, and timestamp.
Audit Chain	Audit.EntityChange	EntityChangeConfiguration.cs	Row-level change record associated with an audit entry.

GROUP	TABLE	CONFIGURATION FILE	MAPPING PURPOSE
Audit Chain	<code>Audit.PropertyChange</code>	<code>PropertyChangeConfiguration.cs</code>	Column/property-level before-and-after details associated with an entity change.
Commands and Integration	<code>Audit.CommandLog</code>	<code>CommandLogConfiguration.cs</code>	Records application commands with input/output hashes, status, timing, and actor context.
Lineage and Evidence	<code>Audit.DataLineage</code>	<code>DataLineageConfiguration.cs</code>	Captures source-to-target lineage for derived rows, reports, snapshots, and integrations.
Lineage and Evidence	<code>Audit.LineageLink</code>	<code>LineageLinkConfiguration.cs</code>	Links lineage records into chains so a derived result can be traced backward.
Commands and Integration	<code>Audit.IntegrationMessage</code>	<code>IntegrationMessageConfiguration.cs</code>	Inbound or outbound integration message envelope with correlation and status.
Commands and Integration	<code>Audit.IntegrationMessageAttempt</code>	<code>IntegrationMessageAttemptConfiguration.cs</code>	Delivery/processing attempt history for one integration message.
Access, Export, Signatures	<code>Audit.AccessAudit</code>	<code>AccessAuditConfiguration.cs</code>	Records authorization checks, allowed/denied outcomes, client context, and target resource identity.
Access, Export, Signatures	<code>Audit.ExportAudit</code>	<code>ExportAuditConfiguration.cs</code>	Records data export requests, purpose, actor, filters, row counts, and file hashes.
Access, Export, Signatures	<code>Audit.SignatureRecord</code>	<code>SignatureRecordConfiguration.cs</code>	Records a signature over a target business object, evidence item, report, or approval.
Retention and Temporal	<code>Audit.RetentionPolicy</code>	<code>RetentionPolicyConfiguration.cs</code>	Defines retention rules for schema/table targets and the action to take.
Retention and Temporal	<code>Audit.RetentionExecution</code>	<code>RetentionExecutionConfiguration.cs</code>	One execution of a retention policy with status, timing, actor, and rows affected.
Retention and Temporal	<code>Audit.TemporalTableRegistry</code>	<code>TemporalTableRegistryConfiguration.cs</code>	Registry of system-versioned tables and their history table mapping.
Lineage and Evidence	<code>Audit.AuditEvidenceLink</code>	<code>AuditEvidenceLinkConfiguration.cs</code>	Links audit entries to evidence records without pretending every possible target has the same PK type.

22.3 EF Core dependency diagram

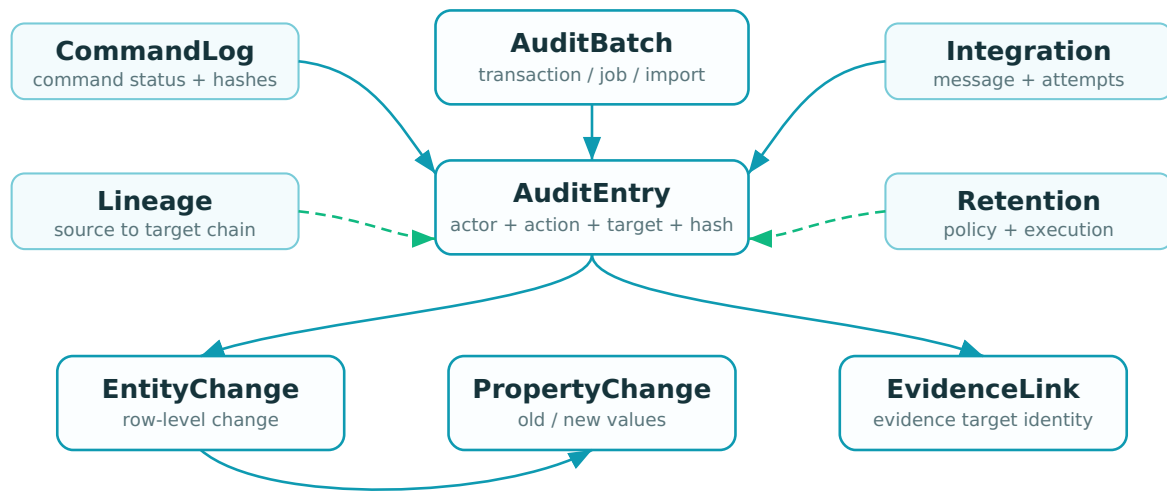


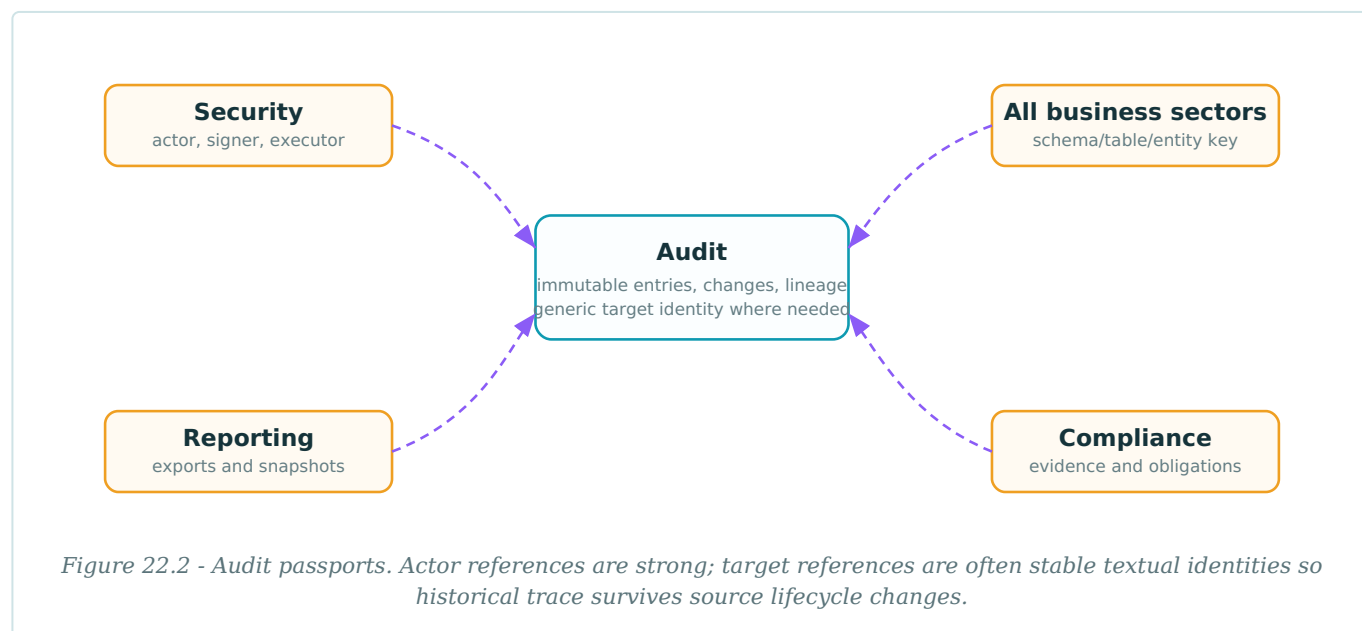
Figure 22.1 - Audit mapping centre. Batch and entry form the chain; entity and property changes record detail; evidence, commands, integration, lineage, and retention surround it.

Configuration folders

```
Nexus.Data/  
  Configurations/  
    Audit/  
      Lookups/  
        AuditActionTypeConfiguration.cs  
        AuditEntityTypeConfiguration.cs  
        AuditSeverityConfiguration.cs  
        ChangeSourceTypeConfiguration.cs  
        ...  
      AuditChain/  
        AuditBatchConfiguration.cs  
        AuditEntryConfiguration.cs  
        EntityChangeConfiguration.cs  
        PropertyChangeConfiguration.cs  
      CommandsAndIntegration/  
        CommandLogConfiguration.cs  
        IntegrationMessageConfiguration.cs  
        IntegrationMessageAttemptConfiguration.cs  
      LineageAndEvidence/  
        DataLineageConfiguration.cs  
        LineageLinkConfiguration.cs  
        AuditEvidenceLinkConfiguration.cs  
      AccessExportSignature/  
        AccessAuditConfiguration.cs  
        ExportAuditConfiguration.cs  
        SignatureRecordConfiguration.cs  
      RetentionAndTemporal/  
        RetentionPolicyConfiguration.cs  
        RetentionExecutionConfiguration.cs  
        TemporalTableRegistryConfiguration.cs
```


22.4 Cross-schema passport map

The Audit sector references users and signers strongly where the lifecycle requires it, but it does not foreign-key every audited business row. For audited targets it stores stable identity: schema name, table name, and entity key. That choice preserves history even when source rows are archived, retained, or deleted.



22.5 Shared lookup configuration

Audit lookups define action, severity, source, command status, integration status, retention status, signature status, access result, and export purpose. The base configuration keeps the lookup shape uniform.

```
public abstract class AuditLookupConfiguration<TEntity>
    : IEntityConfiguration<TEntity>
    where TEntity : AuditLookup
{
    protected abstract string TableName { get; }
    protected abstract string KeyName { get; }

    public virtual void Configure(EntityTypeBuilder<TEntity> b)
    {
        b.ToTable(TableName, "Audit");
        b.HasKey(e => e.Id).HasName($"PK_Audit_{TableName}");
        b.Property(e => e.Id).HasColumnName(KeyName);

        b.Property(e => e.Code).HasMaxLength(60).IsRequired();
        b.Property(e => e.Name).HasMaxLength(160).IsRequired();
        b.Property(e => e.Description).HasMaxLength(500);
        b.Property(e => e.IsActive).HasDefaultValue(true);
        b.Property(e => e.DisplayOrder).HasDefaultValue(0);

        b.HasIndex(e => e.Code)
            .IsUnique()
            .HasDatabaseName($"UQ_Audit_{TableName}_Code");
    }
}
```

Lookup catalogue

LOOKUP TABLE	REPRESENTATIVE SEED CODES	USED FOR
<code>Audit.AuditActionType</code>	INSERT, UPDATE, DELETE, LOGIN, LOGOUT, ACKNOWLEDGE, APPROVE, EXPORT, SIGN, RETENTION	Classifies the action recorded: insert, update, delete, sign, export, login, retention, or system action.
<code>Audit.AuditEntityType</code>	BUSINESS_ENTITY, EVENT, MEASUREMENT, DOCUMENT, REPORT, CONFIGURATION, SECURITY_OBJECT	Classifies what kind of target was changed or observed: business entity, document, signal, report, policy, or configuration.
<code>Audit.AuditSeverity</code>	INFO, NOTICE, WARNING, CRITICAL	Severity band for audit events that need attention or escalation.
<code>Audit.ChangeSourceType</code>	USER, API, JOB, IMPORT, SYSTEM, SIMULATION, INTEGRATION	Origin of the change: user, API, job, import, system, simulation, or integration.
<code>Audit.CommandStatus</code>	STARTED, SUCCEEDED, FAILED, CANCELLED, TIMED_OUT	Execution status for commands and retention runs.
<code>Audit.IntegrationMessageStatus</code>	PENDING, SENT, RECEIVED, FAILED, DEAD_LETTERED	Lifecycle state for inbound or outbound integration messages.
<code>Audit.RetentionActionType</code>	ARCHIVE, DELETE, ANONYMIZE, SEAL, REVIEW	Retention action: archive, delete, anonymize, seal, or review.
<code>Audit.RetentionRuleStatus</code>	DRAFT, ACTIVE, SUSPENDED, RETIRED	Lifecycle status for retention rules and policies.
<code>Audit.SignatureStatus</code>	PENDING, VALID, INVALID, REVOKED, EXPIRED	Validation state for electronic signatures.
<code>Audit.AccessResult</code>	ALLOWED, DENIED, CHALLENGED, EXPIRED, NOT_APPLICABLE	Allowed, denied, challenged, expired, or not applicable access decisions.

LOOKUP TABLE	REPRESENTATIVE SEED CODES	USED FOR
Audit.ExportPurpose	OPERATIONAL, COMPLIANCE, INVESTIGATION, TRAINING, SUPPORT	Reason for a data export: operational, compliance, investigation, training, or support.

22.6 Audit entry mapping

`AuditEntryConfiguration` is the centre of the chapter. It maps action, actor, target identity, correlation, request metadata, and hash fields.

```
public sealed class AuditEntryConfiguration
    : IEntityConfiguration<AuditEntry>
{
    public void Configure(EntityTypeBuilder<AuditEntry> b)
    {
        b.ToTable("AuditEntry", "Audit");
        b.HasKey(e => e.AuditEntryId)
            .HasName("PK_Audit_AuditEntry");

        b.Property(e => e.SchemaName).HasMaxLength(128).IsRequired();
        b.Property(e => e.TableName).HasMaxLength(128).IsRequired();
        b.Property(e => e.EntityKey).HasMaxLength(200).IsRequired();
        b.Property(e => e.Summary).HasMaxLength(1000);
        b.Property(e => e.OccurredAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");
        b.Property(e => e.HashAlgorithm).HasMaxLength(30).HasDefaultValue("SHA2-256");
        b.Property(e => e.PreviousEntryHash).HasMaxLength(32);
        b.Property(e => e.EntryHash).HasMaxLength(32);

        b.HasOne(e => e.AuditBatch)
            .WithMany(e => e.Entries)
            .HasForeignKey(e => e.AuditBatchId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AuditEntry_AuditBatch");

        b.HasOne(e => e.ActionType)
            .WithMany()
            .HasForeignKey(e => e.AuditActionTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AuditEntry_ActionType");

        b.HasIndex(e => new { e.SchemaName, e.TableName, e.EntityKey, e.OccurredAtUtc })
            .HasDatabaseName("IX_Audit_AuditEntry_Target_Time");

        b.HasIndex(e => e.CorrelationId)
            .HasDatabaseName("IX_Audit_AuditEntry_CorrelationId");
    }
}
```

22.7 Entity and property change mapping

Row-level and property-level changes are separated so a system can answer both questions: which row changed, and exactly which property changed?

```
public sealed class EntityChangeConfiguration
    : IEntityConfiguration<EntityChange>
{
    public void Configure(EntityTypeBuilder<EntityChange> b)
    {
        b.ToTable("EntityChange", "Audit");
        b.HasKey(e => e.EntityChangeId)
            .HasName("PK_Audit_EntityChange");

        b.Property(e => e.SchemaName).HasMaxLength(128).IsRequired();
        b.Property(e => e.TableName).HasMaxLength(128).IsRequired();
        b.Property(e => e.PrimaryKeyName).HasMaxLength(128).IsRequired();
        b.Property(e => e.PrimaryKeyValue).HasMaxLength(200).IsRequired();
        b.Property(e => e.Operation).HasMaxLength(20).IsRequired();
        b.Property(e => e.BeforeJson);
        b.Property(e => e.AfterJson);

        b.HasOne(e => e.AuditEntry)
            .WithMany(e => e.EntityChanges)
            .HasForeignKey(e => e.AuditEntryId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_EntityChange_AuditEntry");

        b.HasIndex(e => new { e.SchemaName, e.TableName, e.PrimaryKeyValue })
            .HasDatabaseName("IX_Audit_EntityChange_Target");
    }
}
```

```
public sealed class PropertyChangeConfiguration
    : IEntityConfiguration<PropertyChange>
{
    public void Configure(EntityTypeBuilder<PropertyChange> b)
    {
        b.ToTable("PropertyChange", "Audit");
        b.HasKey(e => e.PropertyChangeId)
            .HasName("PK_Audit_PropertyChange");

        b.Property(e => e.PropertyName).HasMaxLength(128).IsRequired();
        b.Property(e => e.OldValue).HasMaxLength(4000);
        b.Property(e => e.NewValue).HasMaxLength(4000);
        b.Property(e => e.ValueType).HasMaxLength(80);
        b.Property(e => e.IsSensitive).HasDefaultValue(false);

        b.HasOne(e => e.EntityChange)
            .WithMany(e => e.PropertyChanges)
            .HasForeignKey(e => e.EntityChangeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_PropertyChange_EntityChange");

        b.HasIndex(e => new { e.EntityChangeId, e.PropertyName })
            .HasDatabaseName("IX_Audit_PropertyChange_Entity_Property");
    }
}
```

22.8 Command and integration mapping

Commands describe what the application tried to do. Integration messages describe what crossed a boundary. Both are audit facts, and both deserve named mappings.

```
public sealed class CommandLogConfiguration
    : IEntityTypeConfiguration<CommandLog>
{
    public void Configure(EntityTypeBuilder<CommandLog> b)
    {
        b.ToTable("CommandLog", "Audit");
        b.HasKey(e => e.CommandLogId)
            .HasName("PK_Audit_CommandLog");

        b.Property(e => e.CommandName).HasMaxLength(180).IsRequired();
        b.Property(e => e.HandlerName).HasMaxLength(220);
        b.Property(e => e.InputHash).HasMaxLength(128);
        b.Property(e => e.OutputHash).HasMaxLength(128);
        b.Property(e => e.ErrorMessage).HasMaxLength(2000);
        b.Property(e => e.StartedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");

        b.HasOne(e => e.CommandStatus)
            .WithMany()
            .HasForeignKey(e => e.CommandStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_CommandLog_CommandStatus");

        b.HasIndex(e => new { e.CommandName, e.StartedAtUtc })
            .HasDatabaseName("IX_Audit_CommandLog_Command_Time");
    }
}
```

```
public sealed class IntegrationMessageConfiguration
    : IEntityTypeConfiguration<IntegrationMessage>
{
    public void Configure(EntityTypeBuilder<IntegrationMessage> b)
    {
        b.ToTable("IntegrationMessage", "Audit");
        b.HasKey(e => e.IntegrationMessageId)
            .HasName("PK_Audit_IntegrationMessage");

        b.Property(e => e.ExternalSystem).HasMaxLength(120).IsRequired();
        b.Property(e => e.MessageType).HasMaxLength(120).IsRequired();
        b.Property(e => e.Direction).HasMaxLength(20).IsRequired();
        b.Property(e => e.PayloadHash).HasMaxLength(128);
        b.Property(e => e.CreatedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");

        b.HasOne(e => e.Status)
            .WithMany()
            .HasForeignKey(e => e.IntegrationMessageStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_IntegrationMessage_Status");

        b.HasIndex(e => new { e.ExternalSystem, e.MessageType, e.CreatedAtUtc })
            .HasDatabaseName("IX_Audit_IntegrationMessage_System_Type_Time");
    }
}
```

22.9 Access, export, and signature mapping

Access checks, exports, and signatures often become the records people ask for after an incident. Their mappings prioritize actor, target, status, timestamp, and hashes.

```
public sealed class AccessAuditConfiguration
    : IEntityTypeConfiguration<AccessAudit>
{
    public void Configure(EntityTypeBuilder<AccessAudit> b)
    {
        b.ToTable("AccessAudit", "Audit");
        b.HasKey(e => e.AccessAuditId)
            .HasName("PK_Audit_AccessAudit");

        b.Property(e => e.ResourceSchema).HasMaxLength(128).IsRequired();
        b.Property(e => e.ResourceTable).HasMaxLength(128).IsRequired();
        b.Property(e => e.ResourceKey).HasMaxLength(200).IsRequired();
        b.Property(e => e.PermissionCode).HasMaxLength(120).IsRequired();
        b.Property(e => e.ClientIp).HasMaxLength(64);
        b.Property(e => e.OccurredAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");

        b.HasOne(e => e.Result)
            .WithMany()
            .HasForeignKey(e => e.AccessResultId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_AccessAudit_AccessResult");

        b.HasIndex(e => new { e.ActorUserId, e.OccurredAtUtc })
            .HasDatabaseName("IX_Audit_AccessAudit_Actor_Time");
    }
}
```

```
public sealed class ExportAuditConfiguration
    : IEntityTypeConfiguration<ExportAudit>
{
    public void Configure(EntityTypeBuilder<ExportAudit> b)
    {
        b.ToTable("ExportAudit", "Audit");
        b.HasKey(e => e.ExportAuditId)
            .HasName("PK_Audit_ExportAudit");

        b.Property(e => e.ExportName).HasMaxLength(220).IsRequired();
        b.Property(e => e.TargetSchema).HasMaxLength(128);
        b.Property(e => e.TargetTable).HasMaxLength(128);
        b.Property(e => e.FilterJson);
        b.Property(e => e.FileHash).HasMaxLength(128);
        b.Property(e => e.RequestedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");

        b.HasOne(e => e.Purpose)
            .WithMany()
            .HasForeignKey(e => e.ExportPurposeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_ExportAudit_ExportPurpose");

        b.HasIndex(e => new { e.RequestedByUserId, e.RequestedAtUtc })
            .HasDatabaseName("IX_Audit_ExportAudit_User_Time");
    }
}
```

```
public sealed class SignatureRecordConfiguration
    : IEntityTypeConfiguration<SignatureRecord>
{
    public void Configure(EntityTypeBuilder<SignatureRecord> b)
    {
        b.ToTable("SignatureRecord", "Audit");
        b.HasKey(e => e.SignatureRecordId)
    }
}
```

```

        .HasName("PK_Audit_SignatureRecord");

        b.Property(e => e.TargetSchema).HasMaxLength(128).IsRequired();
        b.Property(e => e.TargetTable).HasMaxLength(128).IsRequired();
        b.Property(e => e.TargetKey).HasMaxLength(200).IsRequired();
        b.Property(e => e.SignaturePurpose).HasMaxLength(160).IsRequired();
        b.Property(e => e.SignatureHash).HasMaxLength(128).IsRequired();
        b.Property(e => e.CertificateThumbprint).HasMaxLength(120);
        b.Property(e => e.SignedAtUtc).HasDefaultValueSql("SYSUTCDATETIME()");

        b.HasOne(e => e.SignatureStatus)
            .WithMany()
            .HasForeignKey(e => e.SignatureStatusId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_SignatureRecord_Status");

        b.HasIndex(e => new { e.TargetSchema, e.TargetTable, e.TargetKey })
            .HasDatabaseName("IX_Audit_SignatureRecord_Target");
    }
}

```


22.10 Retention mapping

Retention is itself auditable. A retention policy is a governed rule; a retention execution is a historical event. The mapping keeps both visible.

```
public sealed class RetentionPolicyConfiguration
    : IEntityTypeConfiguration<RetentionPolicy>
{
    public void Configure(EntityTypeBuilder<RetentionPolicy> b)
    {
        b.ToTable("RetentionPolicy", "Audit");
        b.HasKey(e => e.RetentionPolicyId)
            .HasName("PK_Audit_RetentionPolicy");

        b.Property(e => e.Code).HasMaxLength(80).IsRequired();
        b.Property(e => e.SchemaName).HasMaxLength(128).IsRequired();
        b.Property(e => e.TableName).HasMaxLength(128).IsRequired();
        b.Property(e => e.RetentionDays).IsRequired();
        b.Property(e => e.Description).HasMaxLength(1000);
        b.Property(e => e.RowVersion).IsRowVersion();

        b.HasOne(e => e.ActionType)
            .WithMany()
            .HasForeignKey(e => e.RetentionActionTypeId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RetentionPolicy_ActionType");

        b.HasIndex(e => new { e.SchemaName, e.TableName })
            .HasDatabaseName("IX_Audit_RetentionPolicy_Target");
    }
}
```

22.11 Context discovery

The audit configurations are discovered like every other sector. The `DbContext` remains clean even though Audit contains cross-cutting infrastructure.

```
public sealed class NexusContext : DbContext
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(NexusContext).Assembly);
    }
}
```

Why this matters. Audit is the easiest sector to turn into a giant special case. Keeping one configuration file per entity prevents that.

22.12 Foreign-key mapping

The table below lists the main configuration targets. Internal relationships stay inside `Audit` ; actor and signer relationships cross into `Security` .

ENTITY	FK PROPERTY	TARGET	KIND
<code>Audit.AuditBatch</code>	<code>ChangeSourceTypeId</code>	<code>Audit.ChangeSourceType(ChangeSourceTypeId)</code>	internal
<code>Audit.AuditBatch</code>	<code>StartedByUserId</code>	<code>Security.ApplicationUser(ApplicationUserId)</code>	passport
<code>Audit.AuditEntry</code>	<code>AuditBatchId</code>	<code>Audit.AuditBatch(AuditBatchId)</code>	internal
<code>Audit.AuditEntry</code>	<code>ActionTypeId</code>	<code>Audit.AuditActionType(AuditActionTypeId)</code>	internal
<code>Audit.AuditEntry</code>	<code>EntityTypeId</code>	<code>Audit.AuditEntityType(AuditEntityTypeId)</code>	internal
<code>Audit.AuditEntry</code>	<code>SeverityId</code>	<code>Audit.AuditSeverity(AuditSeverityId)</code>	internal
<code>Audit.AuditEntry</code>	<code>ChangeSourceTypeId</code>	<code>Audit.ChangeSourceType(ChangeSourceTypeId)</code>	internal
<code>Audit.AuditEntry</code>	<code>PerformedByUserId</code>	<code>Security.ApplicationUser(ApplicationUserId)</code>	passport
<code>Audit.EntityChange</code>	<code>AuditEntryId</code>	<code>Audit.AuditEntry(AuditEntryId)</code>	internal
<code>Audit.PropertyChange</code>	<code>EntityChangeId</code>	<code>Audit.EntityChange(EntityChangeId)</code>	internal
<code>Audit.CommandLog</code>	<code>CommandStatusId</code>	<code>Audit.CommandStatus(CommandStatusId)</code>	internal
<code>Audit.CommandLog</code>	<code>SourceTypeId</code>	<code>Audit.ChangeSourceType(ChangeSourceTypeId)</code>	internal
<code>Audit.CommandLog</code>	<code>RequestedByUserId</code>	<code>Security.ApplicationUser(ApplicationUserId)</code>	passport
<code>Audit.DataLineage</code>	<code>CreatedByUserId</code>	<code>Security.ApplicationUser(ApplicationUserId)</code>	passport
<code>Audit.LineageLink</code>	<code>DataLineageId</code>	<code>Audit.DataLineage(DataLineageId)</code>	internal
<code>Audit.IntegrationMessageAttempt</code>	<code>IntegrationMessageId</code>	<code>Audit.IntegrationMessage(IntegrationMessageId)</code>	internal
<code>Audit.IntegrationMessageAttempt</code>	<code>IntegrationMessageStatusId</code>	<code>Audit.IntegrationMessageStatus(IntegrationMessageStatusId)</code>	internal
<code>Audit.AccessAudit</code>	<code>ApplicationUserId</code>	<code>Security.ApplicationUser(ApplicationUserId)</code>	passport
<code>Audit.AccessAudit</code>	<code>AccessResultId</code>	<code>Audit.AccessResult(AccessResultId)</code>	internal
<code>Audit.AccessAudit</code>	<code>SourceTypeId</code>	<code>Audit.ChangeSourceType(ChangeSourceTypeId)</code>	internal
<code>Audit.ExportAudit</code>	<code>ReportExportId</code>	<code>Reporting.ReportExport(ReportExportId)</code>	passport
<code>Audit.ExportAudit</code>	<code>ExportedByUserId</code>	<code>Security.ApplicationUser(ApplicationUserId)</code>	passport
<code>Audit.ExportAudit</code>	<code>ExportPurposeId</code>	<code>Audit.ExportPurpose(ExportPurposeId)</code>	internal

ENTITY	FK PROPERTY	TARGET	KIND
Audit.SignatureRecord	SignatureStatusId	Audit.SignatureStatus(SignatureStatusId)	internal
Audit.SignatureRecord	SignedByUserId	Security.ApplicationUser(ApplicationUserId)	passport
Audit.RetentionPolicy	RetentionActionTypeId	Audit.RetentionActionType(RetentionActionTypeId)	internal
Audit.RetentionPolicy	RetentionRuleStatusId	Audit.RetentionRuleStatus(RetentionRuleStatusId)	internal
Audit.RetentionExecution	RetentionPolicyId	Audit.RetentionPolicy(RetentionPolicyId)	internal
Audit.RetentionExecution	RetentionActionTypeId	Audit.RetentionActionType(RetentionActionTypeId)	internal
Audit.RetentionExecution	ExecutedByUserId	Security.ApplicationUser(ApplicationUserId)	passport
Audit.TemporalTableRegistry	RetentionPolicyId	Audit.RetentionPolicy(RetentionPolicyId)	internal
Audit.AuditEvidenceLink	AuditEntryId	Audit.AuditEntry(AuditEntryId)	internal
Audit.AuditEvidenceLink	ComplianceEvidenceId	Compliance.Evidence(EvidenceId)	passport

22.13 Migration and verification notes

Restrictive deletes

Use `DeleteBehavior.Restrict` on audit relationships. Audit history must not be cascade-deleted by business operations.

Stable target identity

Use `SchemaName`, `TableName`, and `EntityKey` for generic targets so archived or deleted source records remain traceable.

Hashes

Map `EntryHash`, `PreviousEntryHash`, payload hashes, file hashes, and signature hashes explicitly.

Time indexes

Most audit investigations start from actor, target, correlation ID, or time window. Index accordingly.

```
// Verification query idea after migration
SELECT name
FROM sys.indexes
WHERE object_id = OBJECT_ID('Audit.AuditEntry')
ORDER BY name;

// EF Core model check idea
var auditTypes = context.Model.GetEntityTypes()
    .Where(e => e.GetSchema() == "Audit")
    .Select(e => e.GetTableName())
    .OrderBy(n => n)
    .ToList();

// Every Audit configuration should be discovered by ApplyConfigurationsFromAssembly.
```

22.14 Configuration file index

AuditActionTypeConfiguration.cs	Lookup	PropertyChangeConfiguration.cs	Audit Chain
AuditEntityTypeConfiguration.cs	Lookup	CommandLogConfiguration.cs	Commands and Integration
AuditSeverityConfiguration.cs	Lookup	DataLineageConfiguration.cs	Lineage and Evidence
ChangeSourceTypeConfiguration.cs	Lookup	LineageLinkConfiguration.cs	Lineage and Evidence
CommandStatusConfiguration.cs	Lookup	IntegrationMessageConfiguration.cs	Commands and Integration
IntegrationMessageStatusConfiguration.cs	Lookup	IntegrationMessageAttemptConfiguration.cs	Commands and Integration
RetentionActionTypeConfiguration.cs	Lookup	AccessAuditConfiguration.cs	Access, Export, Signatures
RetentionRuleStatusConfiguration.cs	Lookup	ExportAuditConfiguration.cs	Access, Export, Signatures
SignatureStatusConfiguration.cs	Lookup	SignatureRecordConfiguration.cs	Access, Export, Signatures
AccessResultConfiguration.cs	Lookup	RetentionPolicyConfiguration.cs	Retention and Temporal
ExportPurposeConfiguration.cs	Lookup	RetentionExecutionConfiguration.cs	Retention and Temporal
AuditBatchConfiguration.cs	Audit Chain	TemporalTableRegistryConfiguration.cs	Retention and Temporal
AuditEntryConfiguration.cs	Audit Chain	AuditEvidenceLinkConfiguration.cs	Lineage and Evidence
EntityChangeConfiguration.cs	Audit Chain		

End of schema atlas boundary. This chapter completes the schema-by-schema EF Core configuration atlas. The next natural step is a combined master PDF with front matter, all chapters, consolidated indexes, and a cross-schema configuration catalogue.

Epilogue

From entity to context

The book began with one small mapping class and ended with a full EF Core configuration atlas.

The lesson is deliberately simple. Large systems do not become understandable because a framework can infer enough to run. They become understandable when important rules are written down where programmers can find them, review them, test them, and migrate them safely.

In NEXUS-1, the SQL schema gives the platform its data backbone. The configuration classes give that backbone a C# shape. Between them sits the clean `DbContext`: not a monolith, not a script, but a discovery point.

Final rule: one entity, one configuration, one readable migration. When the schema grows, the discipline should grow with it.

References

- Microsoft documentation: Entity Framework Core modeling, relationships, indexes, migrations, SQL Server provider, and Fluent API.
- Microsoft documentation: ASP.NET Core Identity model customization and IdentityDbContext patterns.
- NEXUS-1 companion series: *From Grid to Core*, *From Flood to Cause*, *From Trial to Policy*, *From Table to Twin*, and *From Schema to System*.
- NEXUS-1 Phase-0 console and schema atlas materials by Grigorios Agathangelidis.
- SQL Server documentation: constraints, indexes, default values, computed columns, temporal tables, and rowversion.

Indexes

Index of Configuration Themes

ApplyConfigurationsFromAssembly - discovery pattern

Code First - C# model as authoring path

DbContext - clean persistence gateway

DeleteBehavior.Restrict - cross-schema default

Fluent API - atlas mapping language

Foreign-key passports - explicit cross-schema relationships

IEntityTypeConfiguration - one file per entity mapping

Indexes - stable names and uniqueness

Lookup configuration base - typed reference mapping pattern

Migrations - readable schema change history

Precision - physical and financial decimals

RowVersion - optimistic concurrency token

Schema folders - bounded contexts in code

SQL Server defaults - server-side created dates

Index of Schema Chapters

Chapter 6 CorePlatform

Chapter 7 Security

Chapter 8 Organization

Chapter 9 ReactorFleet

Chapter 10 Instrumentation

Chapter 11 DigitalTwin

Chapter 12 AlarmManagement

Chapter 13 EventManagement

Chapter 14 Maintenance

Chapter 15 RootCause

Chapter 16 ReinforcementLearning

Chapter 17 Robotics

Chapter 18 RadiationMonitoring

Chapter 19 EmergencyPreparedness

Chapter 20 Compliance

Chapter 21 Reporting

Chapter 22 Audit